SigmaDock and SigmaFlow

A Self-Contained Mathematical and Computational Monograph

From probability theory and Riemannian geometry to
$SE(3)$-equivariant diffusion and Riemannian flow matching
for protein–ligand docking

Technical reference document

companion to the SigmaFlow research project

August 15, 2026

Abstract

This monograph is a from-scratch, self-contained derivation and code-verified description of **SigmaDock**, an $SE(3)$-equivariant diffusion model for protein–ligand pose generation, and **SigmaFlow**, its Riemannian flow-matching counterpart. It assumes only real analysis, linear algebra, elementary probability theory, and general mathematical maturity. Every other concept — Riemannian manifolds, Lie groups, Brownian motion on manifolds, score-based diffusion, flow matching, spherical harmonics, Wigner-D matrices, Clebsch–Gordan coefficients, and the Equiformer/EquiformerV2 architecture — is introduced from first principles, motivated, and derived in full. Every mathematical construction is, wherever it is used by SigmaDock or SigmaFlow, tied explicitly to the concrete implementation: file, class, function, tensor shape, and formula. The document distinguishes throughout between (i) idealised theory, (ii) what the SigmaDock method description states, (iii) what the SigmaDock reference implementation actually computes, and (iv) what SigmaFlow changes, and flags explicitly any point at which these four do not coincide or could not be verified with certainty from the available source.

Contents

How to read this document

This document is organised as a linear course: later chapters depend on earlier ones, and no chapter uses a concept before that concept has been defined *in this document*. If you already know a topic (say, Lie groups, or diffusion models), you may skip the corresponding chapter, but you should still skim its **Notation** boxes, since notation is fixed globally and is not repeated in later chapters.

Every new mathematical construction is introduced in six steps, repeated as a pattern throughout: *(1) motivation* — why do we need this object at all; *(2) intuition* — an informal picture; *(3) formal definition*; *(4) derivation* of its key properties; *(5) a small worked example*; *(6) application* to SigmaDock or SigmaFlow. Results that are used later are stated as numbered theorems, propositions, or lemmas; all proofs that are not immediate are carried out in full, not sketched or deferred to a citation, *unless* the result is a classical one whose proof would require substantially more background than this document provides (e.g. existence/uniqueness for SDEs), in which case this is stated explicitly and a precise reference is given rather than an unstated leap.

Whenever a mathematical object is realised in the SigmaDock/SigmaFlow codebase, a grey **Code correspondence** box follows immediately, of the form

Code correspondence

Mathematical object: $v_t(x) \in T_x\mathcal{M}$
Implementation: `sigma_flow_generator.py`::`calc_vector_field`
Tensor shape: `[B x F, 3]`
Interpretation: translational velocity, one $\mathbb{R}^3$ vector per ligand fragment in the batch.

These boxes are the load-bearing content of the document in the sense that they are what makes the mathematics falsifiable against the actual program that runs on real data. Every box has been checked against the source files named in it, at the time of writing; file paths are given relative to the repository roots `SigmaDock/` (the read-only original diffusion reference) and `SigmaFlow_Development/` (the flow-matching conversion), or the checked-out reproduction clone `SigmaDock_Reproduction_JulianMueller/sigmadock/` where the two happen to coincide.

Caution 0.1 (On uncertainty). Some implementation details are genuinely ambiguous from static reading of the code (for instance, a numerical constant whose provenance is not commented, or a design choice for which no docstring states the intent). Wherever this occurs, the text says so explicitly, using the phrase *"not determinable from the code as written; interpretation:"* followed by the most plausible reading. Nothing in this document is invented to fill such a gap silently.

Global notation

Notation is introduced at first use and collected here for reference; this table is updated as new symbols are introduced and should be treated as living, not exhaustive at first read.

Symbol	Meaning
$\mathbb{R}^d$	real Euclidean space of dimension d
$\mathcal{M}$	smooth (Riemannian) manifold
$T_x\mathcal{M}$	tangent space of $\mathcal{M}$ at x
$T\mathcal{M}$	tangent bundle, $\bigsqcup_x T_x\mathcal{M}$
$g_x(\cdot,\cdot)$, $\|\cdot\|_g$	Riemannian metric and induced norm at x
$\exp_x$, $\log_x$	Riemannian exponential map and its (local) inverse at x
G, $\mathfrak{g}$	a Lie group and its Lie algebra $\mathfrak{g} = T_eG$
$SO(3)$, $\mathfrak{so}(3)$	the rotation group and its Lie algebra (skew-symmetric 3×3 matrices)
$SE(3)$	the group of rigid motions of $\mathbb{R}^3$
$(\cdot)^\wedge$, $(\cdot)^\vee$	hat and vee maps, $\mathbb{R}^3 \leftrightarrow \mathfrak{so}(3)$
$\mathcal{M}_N = SE(3)^N$	product pose manifold of N rigid fragments
p_{data}, p_{init}	data distribution and prior (source) distribution
p_t, $p_t(\cdot \mid z)$	marginal / conditional probability path at time $t \in [0,1]$
u_t, $u_t(\cdot \mid z)$	marginal / conditional (velocity) vector field
u_t^θ, s_θ	learned vector field network / learned score network
$\nabla_x \log p(x)$	(Riemannian) score of a density p
W_t	standard Wiener process (Brownian motion in $\mathbb{R}^d$)
$B_t^{\mathcal{M}}$	Brownian motion on a Riemannian manifold $\mathcal{M}$
Y_m^ℓ	real spherical harmonic of degree ℓ, order m
$D^\ell(Q)$	Wigner-D matrix of degree ℓ for $Q \in SO(3)$
$C^{(\ell,m)}_{(\ell_1,m_1)(\ell_2,m_2)}$	Clebsch–Gordan coefficient
$\otimes_{\text{cg}}$	Clebsch–Gordan (steerable) tensor product
$\mathcal{N}(\mu, \Sigma)$	Gaussian/normal distribution with mean μ, covariance Σ
$\text{IGSO}(3)(R_0, \sigma^2)$	isotropic Gaussian distribution on $SO(3)$, centred at R_0, scale σ
$\mathcal{U}(\mathcal{M})$	uniform (Haar, when $\mathcal{M}$ is a compact group) measure on $\mathcal{M}$
B, F, N, A	batch size, number of fragments, number of atoms/nodes, generic index
t	generation time, $t \in [0,1]$, with convention $p_0 = p_{\text{init}}$, $p_1 = p_{\text{data}}$
$s = 1 - t$	noising time, used only when discussing the forward/noising process

Remark 0.2 (Time convention, fixed once and for all). Throughout this document, $t \in [0,1]$ denotes *generation time*: $t = 0$ is the source/prior distribution (p_{init}, e.g. pure noise), and $t = 1$ is the data distribution (p_{data}, e.g. the true bound pose). All sampling procedures therefore integrate *forward* in t, from $t = 0$ to $t = 1$. Where a *noising* (forward diffusion) process is discussed, it runs in the reversed variable $s = 1 - t$, i.e. it starts at data ($s = 0$) and ends at noise ($s = 1$). This convention is fixed because the diffusion and flow-matching literatures disagree with each other (and sometimes with themselves) on which direction

is “forward”, and because the SigmaDock and SigmaFlow codebases must each be checked individually against it – this is done explicitly in Chapters 9 and 12.

Part I

Preliminaries

Chapter 1

The Protein–Ligand Docking Problem

Before any mathematics is introduced, we fix precisely what SigmaDock and SigmaFlow are trying to compute. This chapter is deliberately non-technical in the sense that it uses no probability theory or differential geometry yet; its purpose is to pin down the *objects* (proteins, ligands, poses, degrees of freedom) and the *task* (what must be predicted, and how success is measured) so that every later mathematical construction has an unambiguous referent.

1.1 Proteins and ligands

Definition 1.1 (Protein, for docking purposes)**.** For the purposes of this document, a *protein* is a rigid, known three-dimensional structure: a finite set of atoms $\{(y_j, \zeta_j)\}_{j=1}^{M}$, where $y_j \in \mathbb{R}^3$ is the atom's position and ζ_j collects its chemical identity (element, residue type, atomic partial charge if used). We do not model protein flexibility: y_j is held fixed throughout.

This is a modelling choice, not a physical fact — real proteins move — and it corresponds to the standard *rigid-receptor docking* setting used by essentially all methods discussed in this document, including SigmaDock. The *binding pocket* is the subset of protein atoms within some distance of the ligand; SigmaDock and SigmaFlow both assume the pocket (equivalently, a point defining its centre) is known in advance, so that the generative model only has to search over ligand poses *within* a neighbourhood of a given location, not over the protein surface.

Definition 1.2 (Ligand, for docking purposes)**.** A *ligand* is a small molecule, represented as a molecular graph $\mathcal{G}^{2D} = (\mathcal{V}, \mathcal{B})$: atoms $\mathcal{V}$ with chemical labels (element, formal charge, hybridisation, aromaticity, chirality tag), and bonds $\mathcal{B} \subseteq \mathcal{V} \times \mathcal{V}$ with labels (bond order, aromaticity, whether the bond lies in a ring, stereochemistry). A *conformer* of the ligand is an assignment of 3D coordinates $x = (x_1, \ldots, x_n) \in \mathbb{R}^{n\times 3}$, $n = |\mathcal{V}|$, to its atoms.

Crucially, the molecular graph $\mathcal{G}^{2D}$ is *given* (it is computed once, deterministically, from the ligand's chemical structure, e.g. from a SMILES string), whereas the 3D coordinates x are what a docking model must predict. This is the sense in which docking is a *structure prediction* problem: the input already specifies which atoms exist and how they are chemically connected; the output is where they sit in space.

1.2 Poses and degrees of freedom

Definition 1.3 (Pose)**.** A *pose* of a ligand with n atoms, relative to a fixed protein, is an assignment of 3D coordinates $x \in \mathbb{R}^{n\times 3}$ to its atoms, in the same coordinate frame as the protein.

Naively, a pose has $3n$ real degrees of freedom (three coordinates per atom). This naive count is almost never the right one to reason about, because it ignores that the ligand is a *chemical object*, not an arbitrary point cloud: bond lengths and bond angles are, to excellent approximation, fixed by chemistry (they fluctuate by hundredths of an Ångström around equilibrium values, an order of magnitude below the accuracy that any docking method targets), and only a small number of internal coordinates vary substantially. This observation is what licenses essentially every design decision reviewed in Chapter 9, so we make it precise now.

The internal geometry of a molecule can be decomposed into:

- **Bond lengths** $\|x_i - x_j\|$ for bonded pairs $(i, j) \in \mathcal{B}$: essentially fixed by chemistry (covalent bond lengths are determined, to first approximation, by the identities of the two bonded atoms and the bond order).
- **Bond angles** $\angle(x_i, x_j, x_k)$ for two bonds sharing atom j: also close to fixed, determined by the hybridisation of atom j.
- **Dihedral (torsion) angles** ϕ_{ijkl}, defined for four atoms i–j–k–l connected by three consecutive bonds, as the angle between the plane through (i, j, k) and the plane through (j, k, l): these are, for single (non-ring) bonds, comparatively free to rotate, since rotation about a single bond changes no bond length and no bond angle.

A bond about which the dihedral angle can vary freely without breaking the molecule (i.e. a single, non-ring, non-terminal bond) is called a *rotatable bond*. If a ligand has k rotatable bonds, and bond lengths and angles are treated as exactly fixed, then the *internal* conformation of the ligand is fully described by k angles $\boldsymbol{\phi} = (\phi_1, \ldots, \phi_k) \in \mathbb{T}^k$, where $\mathbb{T} = \mathbb{R}/2\pi\mathbb{Z}$ is the circle (the space of angles); we define and use $\mathbb{T}^k$ formally in Section 3.6. The *external* pose of the (now internally fixed) ligand is then a single rigid motion — a rotation and a translation — applied to the whole molecule, contributing $3 + 3 = 6$ further degrees of freedom (formalised as an element of the group $SE(3)$ in Chapter 3).

Degrees of freedom of a ligand pose

$$\underbrace{3}_{\text{translation}} + \underbrace{3}_{\text{global rotation}} + \underbrace{k}_{\text{torsions}} = k + 6$$

against a naive count of $3n$, where typically $n \gg k$: for a drug-like ligand with $n \approx 30$ heavy atoms one might have $k \approx 5$, so the chemically-informed parameterisation has roughly 11 degrees of freedom against a naive 90.

This factorisation — translation, rotation, torsions — is the single most important structural fact about the docking problem, and essentially every design choice in SigmaDock (and hence SigmaFlow) can be read as a decision about *how literally to take it*. Section 9.1 works out three concrete choices (all-atom, torsional, fragment-based) and the consequences of each; the mathematics needed to state those consequences precisely (Riemannian volume forms, product measures) is developed in Chapter 3.

Remark 1.4 (What is *not* varied). The receptor is rigid (Theorem 1.1). Bond lengths and bond angles are either held exactly fixed, or — as we will see is SigmaDock's actual choice — allowed to fluctuate only implicitly, by resampling a fresh low-energy conformer rather than by being explicit generative degrees of freedom. No docking model discussed here generates bond lengths or angles directly.

1.3 What a docking model predicts, and what it does not

The docking literature distinguishes several related but distinct tasks, and conflating them is a common source of confusion when reading papers or comparing methods.

Pose generation. Given $\mathcal{G}_{\text{dock}}$ (ligand graph, protein structure, binding pocket), produce one or more candidate poses x. This is the task SigmaDock and SigmaFlow solve; it is a *generative* task (a probabilistic model is used to *sample* candidates), not a search or optimisation task, though it can be built on top of either.

Pose scoring. Given a candidate pose x, output a scalar estimate of its quality (e.g. predicted binding free energy, or a proxy such as a learned or physics-based score). This is used to *rank* multiple generated poses (see the ranking heuristic in Section 9.11), but it does not by itself produce new poses.

Ranking. Given a set of candidate poses (e.g. multiple i.i.d. draws from a generative model, each called a *seed*), order them by predicted quality so that a downstream user can inspect the top-k.

Confidence estimation. A calibrated estimate of *how likely a specific prediction is to be correct*, as opposed to a relative score used only for ranking among a fixed candidate set. Chapter 12 discusses to what extent SigmaFlow's flow-matching formulation admits a principled confidence signal via the ODE log-likelihood (Section 12.9), and states explicitly which parts of this are implemented in the current codebase and which are conceptual only.

SigmaDock (Chapter 9) performs pose generation and a simple, non-learned ranking heuristic; it explicitly does *not* include a learned confidence model or post-hoc energy minimisation (Section 9.11), a deliberate design choice intended to isolate how much of its accuracy is attributable to the generative model itself, as opposed to a downstream correction step. SigmaFlow inherits this scope unless stated otherwise.

1.4 Evaluation: RMSD and structural validity

Two families of metrics are used throughout this document to talk about "how good" a predicted pose is.

Definition 1.5 (Root-mean-square deviation, RMSD)**.** For a predicted pose $\hat{x} \in \mathbb{R}^{n\times 3}$ and a reference (experimentally determined, "ground truth") pose $x^\star \in \mathbb{R}^{n\times 3}$ of the same ligand, with atoms in the same order,

$$\text{RMSD}(\hat{x}, x^\star) = \left(\frac{1}{n} \sum_{i=1}^{n} \|\hat{x}_i - x_i^\star\|_2^2 \right)^{1/2}.$$

A pose is conventionally called a *success* (in the re-docking setting of this document) if $\mathrm{RMSD}(\hat{x}, x^\star) < 2\,\text{Å}$ after optimal rigid alignment is *not* applied (i.e. RMSD is computed directly in the shared binding-site frame) — we return to this point, including the alternative of Kabsch-aligned RMSD, when we discuss the specific evaluation used for SigmaDock/SigmaFlow comparisons.

RMSD is a single number and, being an average, is insensitive to *local* chemical implausibility: a pose can have low RMSD to the reference while containing an impossibly short bond or an overlapping ring, if the error is averaged out over the rest of the molecule. This motivates a second, complementary family of metrics.

Definition 1.6 (PoseBusters-style validity checks, informal)**.** A pose $\hat{x}$ is subjected to a battery of deterministic, chemistry-based checks — e.g. are all bond lengths within a tolerance of their expected values, are all bond angles chemically reasonable, is stereochemistry preserved, do any atoms sterically clash with each other or with the protein, does the ligand's internal energy (under a cheap force field) exceed a threshold — each of which returns pass/fail. A pose *passes PoseBusters* if it passes every check in the battery.

We will use the phrase *PoseBusters pass rate* for the fraction of generated poses (over some fixed evaluation set) that pass every check, and we will occasionally need per-check pass rates (e.g. specifically the `bond_lengths` or `bond_angles` check) when discussing fragment-boundary artefacts; these are named explicitly whenever used.

Remark 1.7 (Why both metrics matter, and why neither suffices alone). RMSD measures *closeness to the correct answer*; PoseBusters measures *physical plausibility*, independent of whether the pose is close to any particular reference. A model can score well on one and poorly on the other: for instance, a model whose fragment-level rigid-body placement is accurate (low RMSD) can still produce chemically broken bonds precisely at the boundaries between rigid fragments if the relative placement of two fragments is even slightly off, since a rigid transformation exactly preserves lengths *within* a fragment but says nothing about the distance *between* fragments (see the discussion of triangulation in Section 9.4, and the analysis of transition-bond lengths at fragment boundaries in Chapter 12). PoseBusters, being a binary per-molecule pass/fail criterion, is additionally insensitive to *how far* a check is failed by, a limitation we discuss quantitatively when it becomes relevant.

This completes the non-mathematical scaffolding: we now know what a pose is, which of its coordinates are genuinely free, what a docking model is asked to produce, and how its output is judged. Chapter 2 turns to the probability theory required to make "a generative model for poses" a precise mathematical statement.

Chapter 2

Probability and Generative Modelling Foundations

This chapter collects the probabilistic vocabulary used everywhere in the rest of the document. None of it is specific to docking; it is the common language in which Chapters 5 and 10 will phrase diffusion and flow matching. Readers comfortable with densities, change-of-variables, KL divergence and maximum likelihood may skip to Section 2.5, returning here only if a later citation of a numbered result is unclear.

2.1 Densities, conditional densities, and pushforward

We work throughout with random variables taking values in $\mathbb{R}^d$ (this chapter) or, from Chapter 3 onward, in a Riemannian manifold $\mathcal{M}$; every definition below is stated for $\mathbb{R}^d$ and is repeated, without change to the underlying idea, for manifolds once the necessary geometric vocabulary (volume forms) is available.

Definition 2.1 (Density)**.** A random variable X taking values in $\mathbb{R}^d$ has *density* $p : \mathbb{R}^d \to [0, \infty)$ with respect to Lebesgue measure if

$$\mathbb{P}(X \in A) = \int_A p(x)\, dx \qquad \text{for every (measurable) } A \subseteq \mathbb{R}^d,$$

in particular $\int_{\mathbb{R}^d} p(x)\, dx = 1$.

Definition 2.2 (Conditional density)**.** For a pair (X, Z) with joint density $p(x, z)$ and marginal $p(z) = \int p(x, z)\, dx > 0$, the *conditional density* of X given $Z = z$ is $p(x \mid z) = p(x, z)/p(z)$.

The identity that we will use over and over, sometimes without comment, is the following elementary but consequential rearrangement of Theorem 2.2.

Proposition 2.3 (Marginalisation by conditioning)**.** *If $Z \sim p_{\text{data}}$ and, given $Z = z$, $X \sim p(\cdot \mid z)$, then X marginally has density*

$$p(x) = \int p(x \mid z)\, p_{\text{data}}(z)\, dz.$$

Proof. Immediate from $p(x, z) = p(x \mid z) p_{\text{data}}(z)$ and integrating z out. □

Theorem 2.3 looks almost too simple to name, but it is the single mechanism by which both diffusion models (Chapter 5) and flow matching (Chapter 10) convert an *intractable* learning problem (regress against a marginal quantity that is only available as an integral over all of p_{data}) into a *tractable* one (regress against a *conditional* quantity that is available in closed form for one data point z at a time): both frameworks first construct a family $p_t(\cdot \mid z)$ that is easy to sample and differentiate, then use Theorem 2.3 to reason about the (unavailable) marginal p_t, and finally prove — this is the content of Theorems 5.2 and 10.8 below — that a regression loss built from the conditional quantity has the *same minimiser* as one built from the marginal quantity, sidestepping the need to ever evaluate the intractable integral.

Definition 2.4 (Pushforward measure)**.** Let X have density p on $\mathbb{R}^d$ and let $\phi : \mathbb{R}^d \to \mathbb{R}^d$ be a (measurable) map. The *pushforward* of p under ϕ, written $\phi_{\#}p$, is the distribution of $Y = \phi(X)$:

$$\mathbb{P}(Y \in A) = \mathbb{P}(\phi(X) \in A) = \mathbb{P}\big(X \in \phi^{-1}(A)\big) \qquad \text{for measurable } A.$$

When ϕ is a diffeomorphism (smooth, invertible, smooth inverse), the pushforward has an explicit density, given by the classical change-of-variables formula.

Theorem 2.5 (Change of variables)**.** *Let X have density p_X and let ϕ be a diffeomorphism of an open subset of $\mathbb{R}^d$ onto its image. Then $Y = \phi(X)$ has density*

$$p_Y(y) = p_X\big(\phi^{-1}(y)\big)\ \big|\det D\phi^{-1}(y)\big| = \frac{p_X\big(\phi^{-1}(y)\big)}{\big|\det D\phi(\phi^{-1}(y))\big|},$$

where $D\phi$ is the Jacobian matrix of ϕ.

We take Theorem 2.5 as known from multivariable calculus and do not reprove it; what matters for us is how it is used. In Section 10.1 we will apply it to a map ϕ that is itself the time-t flow of an ordinary differential equation, and derive from it the *continuity equation* that governs how a probability density evolves under such a flow — the central object of flow matching.

Example 2.6 (A one-dimensional sanity check)**.** Let $X \sim \mathcal{N}(0,1)$ and $\phi(x) = 2x$ (so $Y = 2X$). Then $\phi^{-1}(y) = y/2$, $D\phi(x) = 2$, and Theorem 2.5 gives $p_Y(y) = p_X(y/2)/2 = \frac{1}{2\sqrt{2\pi}} e^{-y^2/8}$, i.e. $Y \sim \mathcal{N}(0,4)$, as expected from elementary Gaussian scaling.

2.2 Score functions

Definition 2.7 (Score)**.** Let p be a density on $\mathbb{R}^d$ that is strictly positive and differentiable on an open set $U \subseteq \mathbb{R}^d$. For $x \in U$, the *score* of p at x is

$$\nabla_x \log p(x) \;=\; \frac{\nabla_x p(x)}{p(x)} \;\in\; \mathbb{R}^d,$$

the gradient (with respect to x) of the log-density.

Caution 2.8 (Where the score is defined). The score is a *vector field on U*, not a global object: it is undefined wherever p vanishes or fails to be differentiable. This is not pedantry. Every Gaussian used in Chapter 5 is positive on all of $\mathbb{R}^d$, so the issue never arises there; but p_{data} for molecular poses is supported on a lower-dimensional set (bond lengths are essentially fixed), so $\nabla_x \log p_{\text{data}}$ does not exist in the sense above. This is precisely why diffusion models learn $\nabla_x \log p_\sigma$ for the *smoothed* densities p_σ ($\sigma > 0$), which are positive everywhere, and never the score of p_{data} itself — see the discussion following Theorem 5.2.

The score is the direction in which $\log p$, and hence p itself, increases fastest; equivalently it is the vector field that would define gradient ascent on $\log p$. Two properties of the score matter disproportionately for what follows.

Remark 2.9 (The score does not need a normalising constant). If $p(x) = \tilde{p}(x)/Z$ for an unnormalised $\tilde{p}$ and a normalising constant $Z = \int \tilde{p}$, then $\nabla_x \log p(x) = \nabla_x \log \tilde{p}(x)$, since $\nabla_x \log Z = 0$. The score is therefore computable from an unnormalised density, which ordinary likelihood evaluation is not. This is one of the two reasons (the other being Theorem 2.13 below) that score-based methods can be trained without ever evaluating Z.

Remark 2.10 (The score of a Gaussian, used repeatedly). For $p(x) = \mathcal{N}(x \mid \mu, \sigma^2 I)$,

$$\log p(x) = -\frac{\|x-\mu\|^2}{2\sigma^2} + \text{const}, \qquad \nabla_x \log p(x) = -\frac{x-\mu}{\sigma^2}.$$

This single formula is, after substitution of the appropriate μ, σ, the target of every Euclidean denoising-score-matching loss used in Chapter 5.

2.3 Why the score suffices for sampling: Langevin dynamics

It is not obvious *a priori* that knowing $\nabla_x \log p$ everywhere is enough to sample from p — after all, the score is a strictly weaker piece of information than p itself (Theorem 2.9). The classical result that resolves this is Langevin dynamics, and understanding why it works is what motivates the entire diffusion-model construction of Chapter 5.

We first need Brownian motion in $\mathbb{R}^d$; a manifold version is given in Chapter 5 once the necessary geometry is available, but the flat case can be defined immediately.

Definition 2.11 (Standard Wiener process)**.** A *standard Wiener process* (Brownian motion) $(W_\tau)_{\tau \geq 0}$ in $\mathbb{R}^d$ is a continuous-time stochastic process with $W_0 = 0$, independent increments, and $W_{\tau+h} - W_\tau \sim \mathcal{N}(0, hI_d)$ for all $\tau, h \geq 0$.

Definition 2.12 (Stochastic differential equation, informal)**.** Given a drift $f : \mathbb{R}^d \times \mathbb{R}_{\geq 0} \to \mathbb{R}^d$ and a diffusion coefficient $g : \mathbb{R}_{\geq 0} \to \mathbb{R}_{\geq 0}$, the SDE

$$dX_\tau = f(X_\tau, \tau)\, d\tau + g(\tau)\, dW_\tau$$

denotes, informally, the continuous-time limit of the discrete update $X_{\tau+h} = X_\tau + f(X_\tau, \tau)h + g(\tau)\sqrt{h}\, z$, $z \sim \mathcal{N}(0, I_d)$, as $h \to 0$. We use SDEs only as a organising language for such limits and as a source of update rules to discretise; we do not develop Itô calculus, and every fact about SDEs used below is either a discretisation recipe (which needs no further theory) or is quoted, with a precise reference, as a classical result.

Theorem 2.13 (Langevin dynamics; invariance of the target density)**.** *Let $U \in C^1(\mathbb{R}^d)$ with ∇U Lipschitz and $Z = \int e^{-U} < \infty$, and set $p = e^{-U}/Z$. Then the* overdamped Langevin SDE

$$dX_\tau = -\nabla U(X_\tau)\, d\tau + \sqrt{2}\, dW_\tau$$

has a unique strong solution, and p is a stationary *distribution: if $X_0 \sim p$ then $X_\tau \sim p$ for all $\tau \geq 0$. Because the diffusion coefficient is non-degenerate, p is moreover the* only *stationary probability distribution. Convergence* $\mathrm{Law}(X_\tau) \to p$ *as $\tau \to \infty$ from an arbitrary initial distribution requires an* additional *confinement (Lyapunov) condition, e.g. $\langle \nabla U(x), x \rangle \geq a\|x\|^2 - b$ for some $a > 0$ outside a compact set; see Pavliotis,* Stochastic Processes and Applications, *2014, Ch. 4, and Roberts–Tweedie (1996).*

Caution 2.14 (Stationarity, uniqueness and convergence are three claims). They need different hypotheses and it is easy to conflate them. Lipschitz ∇U gives *existence and uniqueness of the solution*; $\int e^{-U} < \infty$ plus the Fokker–Planck computation gives *stationarity of* p; non-degeneracy of the noise gives *uniqueness of the stationary distribution*; only the confinement condition gives *convergence from an arbitrary start*, and it also controls the rate. Nothing below depends on the rate, but the reader should not carry away the impression that integrability alone yields ergodicity.

We do not reprove Theorem 2.13 (it is a statement about the stationary distribution of a diffusion process via its Fokker–Planck equation, i.e. it requires exactly the machinery we introduce, in the special case needed for our models, in Section 5.2); we take it as a classical fact and note only the substitution that makes it useful here. Setting $U = -\log p_{\text{data}}$ gives $-\nabla U = \nabla_x \log p_{\text{data}}$, so Theorem 2.13 says that

$$dX_\tau = \nabla_x \log p_{\text{data}}(X_\tau)\, d\tau + \sqrt{2}\, dW_\tau \tag{2.1}$$

has p_{data} as its stationary distribution. Discretising (2.1) with step size ϵ gives the *unadjusted Langevin algorithm*,

$$x^{(k+1)} = x^{(k)} + \epsilon\, \nabla_x \log p_{\text{data}}\big(x^{(k)}\big) + \sqrt{2\epsilon}\, z^{(k)}, \qquad z^{(k)} \overset{\text{iid}}{\sim} \mathcal{N}(0, I_d), \tag{2.2}$$

which, run for enough steps k from an arbitrary $x^{(0)}$, produces an *approximate* sample from p_{data} using *only* the score, never the density itself. This is the precise sense in which "the score suffices for sampling", and it is why score-based diffusion models train a score network rather than a density network: Section 5.1 shows how to fit that network without ever observing p_{data} in closed form.

Caution 2.15 ("Approximate" means biased, not merely unconverged). The word *approximate* in the previous sentence is doing real work, and it is worth being explicit about what it does and does not mean. For any fixed step size $\epsilon > 0$, the Markov chain (2.2) has a stationary distribution $p^{(\epsilon)}$ that is *not* p_{data}; the discretisation error is $O(\epsilon)$ and does *not* shrink as $k \to \infty$. Running more steps removes the error from the initial condition, not the error from discretisation. Removing the latter requires either $\epsilon \to 0$ (slowly, jointly with $k \to \infty$) or a Metropolis–Hastings accept/reject correction, which turns ULA into MALA — but that correction needs pointwise evaluation of p_{data} up to a constant, which is exactly what we do not have. The bias is therefore intrinsic to the score-only setting, not an implementation shortcoming. This is one reason diffusion samplers in practice do not run Langevin dynamics at a single noise level, but anneal σ (Chapter 5).

2.4 Kullback–Leibler divergence and maximum likelihood

We record two further standard tools that recur when we discuss what a training objective is actually minimising.

Definition 2.16 (Kullback–Leibler divergence)**.** Let p and q be densities on $\mathbb{R}^d$ with respect to Lebesgue measure. The *Kullback–Leibler divergence from q to p* is

$$\mathrm{KL}(p \,\|\, q) \;=\; \int_{\{p>0\}} p(x) \log \frac{p(x)}{q(x)}\, dx \;=\; \mathbb{E}_{x\sim p}\Big[\log \frac{p(x)}{q(x)}\Big] \;\in\; [0, +\infty],$$

with the conventions $0 \log \frac{0}{0} = 0$ and $a \log \frac{a}{0} = +\infty$ for $a > 0$. In particular:

- the expectation is taken under $\mathbf{p}$, the *first* argument;
- $\mathrm{KL}(p\|q) = +\infty$ whenever p puts mass where q does not, i.e. whenever p is *not* absolutely continuous with respect to q;
- KL is always defined (possibly $+\infty$); it is finite only if $q(x) = 0 \Rightarrow p(x) = 0$ for Lebesgue-almost every x, and even then not automatically (the integral can still diverge).

The same formula with $\int \cdots dx$ replaced by $\sum_x$ defines KL for probability mass functions on a countable set.

Caution 2.17 (Absolute continuity is a conclusion, not a hypothesis). It is tempting to *assume* $p \ll q$ so that the integrand is always finite. That would hide the single most consequential feature of KL: it is $+\infty$ exactly when the second argument assigns zero probability to something the first argument considers possible. Concretely, if a model $q = p_\theta$ has support strictly smaller than p_{data}, then $\mathrm{KL}(p_{\text{data}}\|p_\theta) = +\infty$ no matter how good the model looks elsewhere. This infinite penalty is the mechanism behind the mass-covering/mode-seeking dichotomy of Theorem 2.23, and it is why models are trained on *smoothed* targets whose support is all of $\mathbb{R}^d$ (Theorem 2.8).

Remark 2.18 (KL does not depend on the reference measure — differential entropy does). This point is easy to miss in $\mathbb{R}^d$ and becomes important from Chapter 3 onward, where we work on $SO(3)$ and there is no canonical "dx".

KL is a functional of the two *measures* P, Q, not of the densities used to express them: if $p = dP/d\mu$ and $q = dQ/d\mu$ for *any* common reference measure μ dominating both, the value of $\int p \log(p/q)\, d\mu$ is the same. Changing μ rescales p and q by the same factor, which cancels inside the ratio p/q. Equivalently, KL is invariant under any diffeomorphic reparametrisation $x \mapsto \phi(x)$ applied to both arguments: the Jacobian factors from Theorem 2.5 cancel.

The quantity $-\mathbb{E}_p[\log p]$ (*differential entropy*) has *no* such invariance: it changes by $\mathbb{E}[\log|\det D\phi|]$ under reparametrisation and can be negative. So "KL = cross-entropy minus entropy" (Theorem 2.22) is an identity between three coordinate-*dependent* quantities whose combination happens to be coordinate-*independent*. When we later write KL on $SO(3)$ with respect to the Haar measure, the value does not depend on that choice; had we written an entropy, it would have.

Proposition 2.19 (Gibbs' inequality)**.** $\mathrm{KL}(p\,\|\,q) \geq 0$*, with equality if and only if* $p = q$ *Lebesgue-almost everywhere. In general* $\mathrm{KL}(p\,\|\,q) \neq \mathrm{KL}(q\,\|\,p)$*, so* KL *is* not *a metric (it also fails the triangle inequality).*

Proof. If $p \not\ll q$ then $\mathrm{KL}(p\|q) = +\infty > 0$ by Theorem 2.16, so assume $p \ll q$. Write $S = \{p > 0\}$ and apply Jensen's inequality to the strictly convex function $-\log$ under the probability measure $p\,dx$:

$$\mathrm{KL}(p\|q) = \mathbb{E}_p\Big[-\log\frac{q(x)}{p(x)}\Big] \;\geq\; -\log\mathbb{E}_p\Big[\frac{q(x)}{p(x)}\Big] \;=\; -\log\int_S q(x)\,dx \;\geq\; -\log\int_{\mathbb{R}^d} q(x)\,dx \;=\; 0.$$

The last step is an *inequality*, not an equality: $\int_S q \leq \int q = 1$, with equality iff q vanishes outside S.

Equality in the whole chain forces both steps to be tight. Jensen is tight iff q/p is p-almost-surely equal to a constant c, and then $\int_S q = c\int_S p = c$; the second step is tight iff $\int_S q = 1$, i.e. $c = 1$. Hence $q = p$ on S and $q = 0$ off S, i.e. $p = q$ almost everywhere. Conversely $p = q$ a.e. gives $\mathrm{KL} = 0$. □

Caution 2.20 (A tempting but false shortcut in the proof). One frequently sees the chain above written with $\mathbb{E}_p[q/p] = \int q\,dx = 1$, i.e. with the second $\geq$ replaced by $=$. That step is valid only if $p > 0$ almost everywhere. In general $\mathbb{E}_p[q/p] = \int_{\{p>0\}} q\,dx$, which can be strictly less than 1.

Counterexample, satisfying $p \ll q$: let p be uniform on $[0,1]$ and q uniform on $[0,2]$. Then $\mathbb{E}_p[q/p] = \frac{1}{2}$, not 1, and the correct Jensen bound is $-\log\frac{1}{2} = \log 2$ — which happens to be the exact value $\mathrm{KL}(p\|q) = \log 2$. The false version of the chain would have produced the far weaker bound 0. The stated inequality $\mathrm{KL} \geq 0$ survives, but the intermediate equality does not, and the equality case cannot be read off correctly from the flawed version.

Example 2.21 (The asymmetry, with numbers)**.** For univariate centred Gaussians, $\mathrm{KL}\big(\mathcal{N}(0,\sigma_1^2)\,\|\,\mathcal{N}(0,\sigma_2^2)\big) = \log\frac{\sigma_2}{\sigma_1} + \frac{\sigma_1^2}{2\sigma_2^2} - \frac{1}{2}$. With $\sigma_1 = 1, \sigma_2 = 2$ this is $\log 2 + \frac{1}{8} - \frac{1}{2} \approx 0.318$, whereas the reverse direction is $-\log 2 + 2 - \frac{1}{2} \approx 0.807$. The two differ by more than a factor of two, on the simplest possible pair of distributions.

Proposition 2.22 (KL = cross-entropy − entropy)**.** *Define the* differential entropy $H(p) = -\mathbb{E}_{x\sim p}[\log p(x)]$ *and the* cross-entropy $H(p,q) = -\mathbb{E}_{x\sim p}[\log q(x)]$. *Whenever $H(p)$ is finite,*

$$\mathrm{KL}(p\,\|\,q) \;=\; H(p,q) - H(p).$$

Proof. Split the logarithm: $\mathbb{E}_p[\log(p/q)] = \mathbb{E}_p[\log p] - \mathbb{E}_p[\log q]$, which is $-H(p) + H(p,q)$. The split is legitimate only when the two terms are not both infinite of the same sign, which the finiteness of $H(p)$ guarantees. □

Two consequences are used later. First, for a *fixed* p, minimising $\mathrm{KL}(p\|q)$ over q is the same optimisation as minimising the cross-entropy $H(p,q)$, because $H(p)$ is a constant — this is exactly the step taken in Theorem 2.25. Second, $H(p) \geq 0$ fails for continuous p: differential entropy can be negative (a Gaussian with $\sigma < 1/\sqrt{2\pi e}$ has $H < 0$), so no sign conclusion about KL may be drawn from this decomposition. The non-negativity of KL comes from Theorem 2.19, not from here.

Remark 2.23 (Forward versus reverse KL: which argument is which matters). Fix a data distribution p and a model family $\{q_\theta\}$. The two directions penalise different failures, and the difference follows directly from which distribution the expectation is taken under.

Forward $\mathrm{KL}(p\|q_\theta) = \mathbb{E}_p[\log(p/q_\theta)]$ averages under the *data*. Every region where p has mass is sampled, so q_θ is punished — infinitely, by Theorem 2.17 — for assigning near-zero density anywhere p is positive. Regions where q_θ is large but p is nearly zero contribute almost nothing, because they are almost never sampled. Forward KL is therefore *mass-covering*: the optimum spreads q_θ over all modes of p, accepting density in between.

Reverse $\mathrm{KL}(q_\theta\|p) = \mathbb{E}_{q_\theta}[\log(q_\theta/p)]$ averages under the *model*. Regions the model avoids cost nothing regardless of p there, but the model is punished infinitely for putting mass where p has none. Reverse KL is therefore *mode-seeking*: it prefers to concentrate on one mode and represent it well.

Maximum likelihood — and hence everything in Chapters 5 and 10 that is justified by Theorem 2.25 — optimises the *forward* direction. Variational inference in the usual ELBO formulation optimises the *reverse* direction. This document uses only the forward direction; the distinction is recorded here because the reverse direction is what most readers will have met first, under the same name.

Definition 2.24 (Maximum likelihood)**.** Given i.i.d. samples $x_1, \ldots, x_n \sim p_{\text{data}}$ and a parametric family $\{p_\theta\}$, the *maximum likelihood estimator* is $\hat{\theta} = \arg\max_\theta \sum_{i=1}^n \log p_\theta(x_i)$.

Proposition 2.25 (MLE minimises forward KL to the data distribution)**.** *Assume the differential entropy $H(p_{\text{data}})$ is finite, and that $\mathbb{E}_{p_{\text{data}}}|\log p_\theta(x)| < \infty$ for every θ. Then:*

(i) for each fixed θ, $\frac{1}{n}\sum_i \log p_\theta(x_i) \to \mathbb{E}_{x\sim p_{\text{data}}}[\log p_\theta(x)]$ almost surely as $n \to \infty$ (strong law of large numbers);

(ii) as sets,

$$\arg\max_\theta \mathbb{E}_{p_{\text{data}}}[\log p_\theta(x)] \;=\; \arg\min_\theta \mathrm{KL}\big(p_{\text{data}} \,\|\, p_\theta\big).$$

Proof. (i) is the strong law applied to the i.i.d. variables $\log p_\theta(x_i)$, which are integrable by assumption. For (ii), by Theorem 2.22, $\mathrm{KL}(p_{\text{data}}\|p_\theta) = -H(p_{\text{data}}) - \mathbb{E}_{p_{\text{data}}}[\log p_\theta(x)]$, and the first term is a finite constant independent of θ; adding a constant does not change the set of minimisers, and minimising the negative of a function is maximising the function. □

Caution 2.26 (What Theorem 2.25 does *not* say). Part (i) is pointwise in θ; it does *not* by itself imply that $\hat\theta_n \to \arg\min_\theta \mathrm{KL}(p_{\text{data}}\|p_\theta)$. Consistency of the estimator additionally requires uniform convergence over θ (e.g. a Glivenko–Cantelli condition) plus identifiability. We never rely on consistency in this document — only on the statement that the two *population* objectives in (ii) define the same optimisation — so we do not develop those conditions.

Note also the direction: it is the *forward* KL, with p_{data} in the first argument, that maximum likelihood minimises (Theorem 2.23). Finally, if the model family cannot cover the support of p_{data}, every member has infinite KL and (ii) is vacuous even though the likelihood objective still has a well-defined maximiser; this is not a pathology of the proposition but a genuine degeneracy of the setting, and it is one more reason the constructions of Chapters 5 and 10 work with everywhere-positive smoothed distributions.

Theorem 2.25 is why "fit the model by maximum likelihood" and "fit the model to minimise (forward) KL divergence to the data" are the same instruction, a fact we use once, in Section 12.9, when we discuss to what extent a flow-matching model's ODE admits an exact likelihood and what that would be useful for.

2.5 Generative modelling as distribution transport

We can now state, precisely, the problem that both diffusion models and flow matching solve, and why casting protein–ligand docking as an instance of it is the right move.

> **The generative modelling problem.** Given i.i.d. samples $x_1, \dots, x_n \sim p_{\text{data}}$, produce a mechanism that generates new samples $\hat x \sim p_{\text{data}}$ (or a good approximation thereof), *without* an explicit closed-form, evaluable expression for p_{data} ever being available.

The docking instance of this problem, following Chapter 1, sets p_{data} to be the (conditional, given the protein pocket and ligand graph) distribution of experimentally observed bound poses — a distribution about which we have samples (crystal structures) but no formula.

Rejection sampling and classical Markov chain Monte Carlo both require pointwise evaluation of p_{data} up to a known constant, which we do not have; Section 2.3 showed a way around this for MCMC-style methods (only the score is needed), but that still leaves the problem of *learning* the score from samples alone (Section 5.1). The alternative pursued by both halves of this document is:

1. Fix a *tractable* distribution p_{init} (called the *source* or *prior*) that we can sample from directly — for docking, e.g. an isotropic Gaussian for translations and the uniform (Haar) distribution for rotations (both defined precisely in Chapter 3).

2. Learn a *transport mechanism* — a stochastic process (Chapter 5) or a deterministic flow (Chapter 10) — that turns a sample from p_{init} into a sample from p_{data}.

The entire technical content of Chapters 5 and 10 is two different, and ultimately closely related (Section 5.3), answers to step 2. Both answers share one structural device, which is worth isolating before either is developed in full: both construct a *probability path*, a time-indexed family of distributions interpolating between the two endpoints.

Definition 2.27 (Probability path)**.** A *probability path* is a family of densities $(p_t)_{t\in[0,1]}$ on $\mathbb{R}^d$ (or, later, on a manifold $\mathcal{M}$) satisfying the global convention of Theorem 0.2: $p_0 = p_{\text{init}}$ and $p_1 = p_{\text{data}}$.

Diffusion models (Chapter 5) construct (p_t) implicitly, as the law of a stochastic process at time t, and characterise it via its score, $\nabla_x \log p_t(x)$; sampling then simulates a stochastic differential equation. Flow matching (Chapter 10) constructs (p_t) explicitly, via a chosen interpolation between a source point and a data point, and characterises it via a vector field u_t satisfying the continuity equation of Theorem 10.3; sampling then integrates a deterministic ordinary differential equation. Section 5.3 proves that these are not two disjoint frameworks but two different parameterisations of exactly the same underlying object, a fact that will let us describe the conversion from SigmaDock to SigmaFlow (Chapter 12) as a single, controlled substitution rather than a change of model class.

Part II

Geometry

Chapter 3

Manifolds, Groups, and Rigid Motion

Chapter 1 established that the natural coordinates of a ligand pose are not $3n$ free Cartesian numbers but a translation, a rotation, and (if fragments are used) a handful of torsion angles. None of translations, rotations, and torsion angles are elements of a vector space in the way that is needed for the calculus of Chapter 2 to apply directly: a rotation is not closed under addition (the sum of two rotation matrices is generally not a rotation matrix), and an angle is only defined modulo 2π. This chapter develops exactly the amount of differential geometry needed to do calculus, probability, and eventually diffusion and flow matching correctly on such spaces.

3.1 Manifolds and tangent spaces

Motivation. Ordinary multivariable calculus is calculus on $\mathbb{R}^n$: derivatives are linear maps $\mathbb{R}^n \to \mathbb{R}^m$, gradients are vectors in $\mathbb{R}^n$, and one can always add two points or scale a point by a real number. None of this is true, globally, on the set of rotation matrices or on a circle of angles. What *is* true is that these sets look like $\mathbb{R}^n$ *locally* — close enough together, two rotations differ by "a small rotation", and this small difference behaves, to first order, exactly like a vector. Formalising "looks like $\mathbb{R}^n$ locally, glued together globally" is the definition of a manifold.

Intuition. Picture the surface of a sphere $S^2 \subset \mathbb{R}^3$. Around any point, a small enough patch of the sphere looks flat — this is why maps of small regions of the Earth can be drawn on flat paper with negligible distortion, even though the Earth as a whole cannot. The "flat approximation at a point" is the tangent *plane* at that point; the entire curved sphere is not itself a vector space (you cannot add two points on a sphere and expect the sum to be on the sphere), but each tangent plane is.

Definition 3.1 (Smooth manifold, informal)**.** An n-dimensional *smooth manifold* $\mathcal{M}$ is a set that can be covered by patches (*charts*), each in smooth, invertible correspondence with an open subset of $\mathbb{R}^n$, such that on overlaps the two competing correspondences are related by a smooth map. (We will only ever need two concrete examples, $SO(3)$ and $\mathbb{T} = \mathbb{R}/2\pi\mathbb{Z}$, both realised concretely below as subsets of a Euclidean space; a fully general, chart-based treatment is standard differential geometry and is not needed beyond this informal level for anything in this document.)

Definition 3.2 (Tangent space, informal)**.** For $x \in \mathcal{M}$, the *tangent space* $T_x\mathcal{M}$ is the vector space of all velocity vectors $\dot\gamma(0)$ of smooth curves $\gamma : (-\varepsilon, \varepsilon) \to \mathcal{M}$ with $\gamma(0) = x$. Concretely, if $\mathcal{M} \subset \mathbb{R}^N$ is realised as a level set (as both $SO(3)$ and $\mathbb{T}$ will be), $T_x\mathcal{M}$ is the set of directions in $\mathbb{R}^N$ tangent to $\mathcal{M}$ at x, and it *is* a genuine linear subspace of $\mathbb{R}^N$: sums and scalar multiples of tangent vectors are again tangent vectors, even though sums of points on $\mathcal{M}$ are generally not points on $\mathcal{M}$.

Example 3.3. Let $\mathcal{M} = S^1 = \{x \in \mathbb{R}^2 : \|x\| = 1\}$, the unit circle, and $x = (1, 0)$. A curve $\gamma(\tau) = (\cos\tau, \sin\tau)$ has $\gamma(0) = x$ and $\dot\gamma(0) = (0, 1)$. Every tangent curve at x has velocity a multiple of $(0, 1)$ (the direction perpendicular to x), so $T_xS^1 = \{(0, c) : c \in \mathbb{R}\} \cong \mathbb{R}$: a one-dimensional vector space, matching $\dim S^1 = 1$, even though S^1 itself is not a vector space (e.g. $(1, 0) + (1, 0) = (2, 0) \notin S^1$).

3.1.1 Why a tangent space at all, and why one per point

Before the formal machinery accumulates, it is worth being explicit about the problem being solved, because the answer determines the shape of everything that follows.

The problem. In $\mathbb{R}^n$ a single vector space does two jobs at once: it holds the *points* we care about, and it holds the *directions* we move in. That coincidence is what makes $x + v$ meaningful. On a curved space the two jobs come apart. A point of S^2 is a unit vector in $\mathbb{R}^3$; a "direction to move" at that point is also a vector in $\mathbb{R}^3$, but it is a *different kind* of object, and adding it to the point takes you off the sphere.

Why not just use one global set of directions? On S^2, "north" is a perfectly good direction at the equator, but it is meaningless at the north pole. There is no way to choose, continuously and non-vanishingly, one direction at every point of the sphere — this is the hairy-ball theorem. So a single global space of directions does not exist; we are forced to attach a *separate* space of directions to each point. That family of spaces is the tangent bundle, and each member is a tangent space.

What a tangent vector *is*. The cleanest mental model, and the one Theorem 3.2 formalises, is: a tangent vector at x is the *initial velocity of a curve through* x. This is worth dwelling on, because it explains why tangent vectors can be added even though points cannot. Two curves through x have two velocities; the sum of those velocities is again realisable as the velocity of some third curve through x. Nothing about this construction requires the *points* to be addable. Velocities live in a linear space; positions do not.

The same idea, three times.

- $\mathbb{R}^n$**.** $T_x\mathbb{R}^n \cong \mathbb{R}^n$ for every x, and all these copies are canonically the same space. This is exactly the degenerate case in which the distinction between points and directions is invisible — which is why Euclidean intuition transfers so badly.
- $S^2 \subset \mathbb{R}^3$**.** $T_xS^2 = \{v \in \mathbb{R}^3 : v \cdot x = 0\}$, the plane through the origin perpendicular to x. Two dimensions, matching $\dim S^2 = 2$. Different x give genuinely different planes: there is no canonical way to compare a tangent vector at the north pole with one at the equator.
- $SO(3)$**.** A point is a rotation matrix R; a tangent vector at R turns out to be a matrix of the form $R\widehat{\omega}$ with $\omega \in \mathbb{R}^3$ — three dimensions, matching $\dim SO(3) = 3$. Section 3.3 derives this. The extra structure that $SO(3)$ has and S^2 lacks — a group operation —

will let us identify all these tangent spaces with one another, which is the content of Theorem 3.14.

Why this matters for SigmaFlow

A ligand pose contains, per rigid fragment, a rotation $R \in SO(3)$. The network must output "how this fragment should turn" — a direction of motion, not a new rotation. That direction is a tangent vector at R, and it is a 3-vector, while the point R is a 3×3 matrix with six constraints. Confusing the two is not a stylistic error: adding a 3×3 update matrix to R produces something that is not a rotation, and the pose it encodes is then not a rigid motion. Every appearance of exp and log in `so3_flow_matcher.py` exists to keep points and directions in their respective places.

3.1.2 Measuring: Riemannian metrics and geodesics

Tangent spaces let us talk about directions. To talk about *lengths*, *angles*, and hence about the shortest path between two points, we need one more ingredient: an inner product on each tangent space.

Definition 3.4 (Riemannian metric)**.** A *Riemannian metric* on $\mathcal{M}$ is a smoothly varying family of inner products $g_x : T_x\mathcal{M} \times T_x\mathcal{M} \to \mathbb{R}$ $(x \in \mathcal{M})$, inducing the norm $\|v\|_g = \sqrt{g_x(v,v)}$. The length of a smooth curve $\gamma : [0,1] \to \mathcal{M}$ is $L(\gamma) = \int_0^1 \|\dot\gamma(\tau)\|_g \, d\tau$.

Note what the metric is *not*: it is not a distance function on $\mathcal{M}$. It is a way of measuring tangent vectors, one point at a time. A distance between points is then *derived* from it, as the infimum of $L(\gamma)$ over curves joining them.

Definition 3.5 (Geodesic)**.** A *geodesic* is a curve with zero acceleration in the sense of the metric: $\nabla_{\dot\gamma}\dot\gamma = 0$, where ∇ is the Levi-Civita connection of g. Equivalently and more usefully for us: a geodesic is a curve that is *locally* length-minimising and is traversed at constant speed $\|\dot\gamma\|_g = \text{const}$.

Caution 3.6 (Geodesics are only *locally* shortest). "Geodesic" and "shortest path" are not synonyms, and the difference is not academic for us. On S^2, the great circle through two points is a geodesic in *both* directions around the sphere, but only the shorter arc is length-minimising. On $SO(3)$ the same phenomenon occurs, with the transition at a rotation angle of π: continuing a geodesic past a half-turn keeps zero acceleration but stops being the shortest route.

This is exactly why log on $SO(3)$ is conventionally taken to return the rotation vector of *smallest* angle, $\|\omega\| \le \pi$ (Theorem 3.22), and it is the geometric reason behind the numerical fragility near π that Chapter 13 documents: at exactly π there are two shortest geodesics, and the choice between them is discontinuous.

3.1.3 Replacing $x + v$ and $y - x$: the exponential and logarithmic maps

We now have directions and lengths. What we still lack is the single most used operation in all of Euclidean numerics: *take a point, take a direction, move*. In $\mathbb{R}^n$ this is $x \mapsto x + v$. On a manifold there is no "+". The replacement is forced by the following observation: in $\mathbb{R}^n$, $x + tv$ is precisely the constant-speed straight line through x with initial velocity v — that is, the geodesic. So the right generalisation of "move from x in direction v" is "follow the geodesic through x with initial velocity v, for unit time".

Definition 3.7 (Exponential and logarithmic map)**.** For $x \in \mathcal{M}$ and $v \in T_x\mathcal{M}$, let γ_v be the unique geodesic with $\gamma_v(0) = x$ and $\dot\gamma_v(0) = v$. The *exponential map* is

$$\exp_x : T_x\mathcal{M} \to \mathcal{M}, \qquad \exp_x(v) := \gamma_v(1).$$

On a neighbourhood $U \ni x$ small enough that $\exp_x$ is a diffeomorphism onto its image, its inverse is the *logarithmic map* $\log_x : U \to T_x\mathcal{M}$. The largest radius for which this holds is the *injectivity radius* at x.

Caution 3.8 ($\exp_x$ and $\log_x$ are not globally as well behaved as $+$ and $-$). Three differences from the Euclidean case, each of which shows up later:

(a) $\exp_x$ need not be injective. On $SO(3)$, rotating by θ and by $\theta + 2\pi$ about the same axis give the same element, so infinitely many tangent vectors map to the same point.
(b) $\log_x(y)$ is therefore not unique in general; one *chooses* a branch, conventionally the shortest. The injectivity radius of $SO(3)$ with the standard bi-invariant metric is π, so this choice is unambiguous except exactly at antipodal rotations.
(c) $\exp_x$ need not be surjective on a general manifold (it is on $SO(3)$, which is compact and connected).

Writing $\exp_x(v)$ "for $x + v$" is therefore a good mnemonic and a bad theorem. Whenever a statement in this document depends on invertibility, the hypothesis is stated.

Example 3.9 (Everything at once, on the circle)**.** Take $\mathcal{M} = S^1$, $x = (1, 0)$, and identify $T_xS^1 \cong \mathbb{R}$ via $c \mapsto (0, c)$ as above. The geodesics are constant-speed traversals of the circle, so $\exp_x(c) = (\cos c, \sin c)$. Then: $\exp_x$ is surjective but not injective (c and $c + 2\pi$ agree); $\log_x(y)$ is the signed angle from x to y, unique if we demand $|\log_x(y)| < \pi$; and at $y = -x$ the two choices $\pm\pi$ are equally short — the injectivity radius is π, and the antipode is where the convention has to break a tie. Every one of these features recurs verbatim on $SO(3)$, with "angle" replaced by "rotation angle".

The pair $(\exp_x, \log_x)$ is, throughout this document, the mechanism that replaces ordinary vector addition and subtraction: $\exp_x(v)$ plays the role of $x + v$, and $\log_x(y)$ plays the role of $y - x$, valid even though $\mathcal{M}$ has no addition of its own. Diffusion models (Chapter 5) will use $\exp_x$ to push a randomly sampled tangent-space perturbation onto the manifold; flow matching (Chapter 10) will use $\log_x$ to construct a geodesic interpolant between two points and $\exp_x$ to integrate the learned velocity field one step at a time.

Why this matters for SigmaFlow

The correspondence is exact and worth memorising, because it is what the sampler literally executes:

Euclidean translation channel	Rotational channel
$x_{t+dt} = x_t + dt\, v$	$R_{t+dt} = \exp_{R_t}(dt\, v) = R_t \exp(dt\, \widehat{v})$
target $u = x_1 - x_0$	target $u = \log_{R_0}(R_1) = \log(R_0^\top R_1)^\vee$
interpolant $(1-t)x_0 + tx_1$	interpolant $R_0 \exp\big(t \log(R_0^\top R_1)\big)$

The right-hand column is `so3_flow_matcher.py`; the left-hand column is `r3_flow_matcher.py`. Chapter 12 verifies each line against the source.

3.2 Groups and Lie groups

Motivation. A manifold alone tells us how to do calculus on a curved space; it says nothing about how to *compose* two elements ("first rotate by Q_1, then by Q_2"). Rigid motions have exactly this compositional structure — applying two rotations in sequence is again a rotation — and capturing it precisely is the job of group theory.

Definition 3.10 (Group)**.** A *group* is a set G with a binary operation $\circ : G \times G \to G$ such that: (i) *associativity*, $(g_1 \circ g_2) \circ g_3 = g_1 \circ (g_2 \circ g_3)$; (ii) there is an *identity* $e \in G$ with $e \circ g = g \circ e = g$ for all g; (iii) every g has an *inverse* g^{-1} with $g \circ g^{-1} = g^{-1} \circ g = e$.

Example 3.11. $(\mathbb{R}^d, +)$ is a group (identity 0, inverse $-x$); it is commutative ($g_1 \circ g_2 = g_2 \circ g_1$), i.e. *abelian*. The set of invertible $n \times n$ matrices under matrix multiplication, $GL(n, \mathbb{R})$, is a (non-abelian, for $n \geq 2$) group.

Definition 3.12 (Lie group; restated)**.** A *Lie group* is a group G that is simultaneously a smooth manifold, such that the group operations $\circ : G \times G \to G$ and $(\cdot)^{-1} : G \to G$ are smooth maps.

The point of requiring smoothness of the group operation is that it lets us transport tangent vectors from one point of the group to another using group multiplication itself — a device with no analogue on a generic manifold, and the reason Lie groups admit a canonical, group-wide notion of "the" tangent space, rather than merely a separate tangent space at every point.

Definition 3.13 (Lie algebra; restated)**.** Let G be a Lie group with identity e. Its *Lie algebra* is $\mathfrak{g} := T_eG$, the tangent space at the identity.

Definition 3.14 (Left translation and left-invariant vector fields)**.** For $g \in G$, *left translation by* g is the smooth map $L_g : G \to G$, $L_g(h) = g \circ h$. Its differential at the identity, $dL_g|_e : \mathfrak{g} = T_eG \to T_gG$, is a linear isomorphism (it has smooth inverse $dL_{g^{-1}}|_g$), so every $\xi \in \mathfrak{g}$ determines a tangent vector $dL_g|_e(\xi) \in T_gG$ at *every* point g simultaneously.

Theorem 3.14 is the mechanism, promised above, by which a single vector space $\mathfrak{g}$ suffices to describe tangent vectors everywhere on G: any $\xi \in \mathfrak{g}$ can be moved to any point g by $v = dL_g|_e(\xi) \in T_gG$. This is called *left trivialisation*, and it is not the only choice — *right* translation $R_g(h) = h \circ g$ gives an equally valid but generally *different* identification, called *right trivialisation*. Which one a given piece of code or a given formula uses is a genuine convention, not a matter of taste, because $dL_g|_e(\xi)$ and $dR_g|_e(\xi)$ are different tangent vectors at g whenever G is non-abelian; Section 3.4 below makes this concrete for $SO(3)$, and Chapter 12 traces the convention actually used by the SigmaFlow implementation with care, because — as flagged already in the introduction to this document — the two codebases are not visibly consistent with each other or with the existing thesis draft on this exact point, and getting it wrong silently flips signs in the trained model without necessarily crashing anything.

3.3 The rotation group $SO(3)$

3.3.1 Definition and dimension

Definition 3.15 (Orthogonal and special orthogonal group)**.**

$$O(3) = \{R \in \mathbb{R}^{3\times 3} : R^\top R = I\}, \qquad SO(3) = \{R \in O(3) : \det R = 1\}.$$

Every $R \in O(3)$ satisfies $\det(R^\top R) = \det(R)^2 = \det(I) = 1$, so $\det R = \pm 1$; $SO(3)$ is the subset with $\det R = +1$, and these are exactly the *orientation-preserving* orthogonal maps — rotations, as opposed to reflections ($\det R = -1$). $O(3)$ has two disjoint pieces (*connected components*), separated by the sign of the determinant, and $SO(3)$ is precisely the piece containing the identity, i.e. the piece reachable from I by a continuous path within $O(3)$. Physically: a continuous, physical rotation of a rigid body can never turn it into its mirror image, which is why $SO(3)$, not $O(3)$, is the correct group for orientations of physical objects, and why the distinction resurfaces, non-optionally, when we discuss chirality of molecules in Theorem 3.42.

Proposition 3.16 (Rotations preserve length and orientation)**.** *For $R \in SO(3)$ and $x, y \in \mathbb{R}^3$: $\|Rx\| = \|x\|$, $\langle Rx, Ry\rangle = \langle x, y\rangle$, and $(Rx) \times (Ry) = R(x \times y)$.*

Proof. $\langle Rx, Ry\rangle = x^\top R^\top R y = x^\top y = \langle x, y\rangle$ using $R^\top R = I$; taking $y = x$ gives $\|Rx\|^2 = \|x\|^2$. For the cross product identity, use the general fact $(Ax) \times (Ay) = \det(A)\, A^{-\top}(x \times y)$ for invertible A (a standard identity, provable by checking it on the standard basis and using multilinearity/antisymmetry of both sides), and specialise to $A = R$: $R^{-\top} = R$ (since $R^\top = R^{-1}$) and $\det R = 1$. □

Proposition 3.17 (Dimension of $SO(3)$)**.** *$SO(3)$ is a 3-dimensional manifold.*

Proof. The constraint $R^\top R = I$ is a system of equations in the 9 entries of R; the matrix $R^\top R$ is automatically symmetric, so this constraint has only $\binom{3+1}{2} = 6$ independent scalar components (the entries on and above the diagonal of a 3×3 symmetric matrix). One checks (via the implicit function theorem, using that the differential of $R \mapsto R^\top R$ has full rank 6 at every $R \in O(3)$) that this cuts out a smooth $9 - 6 = 3$-dimensional manifold, of which $SO(3)$ is one (open-and-closed, hence a full-dimensional) connected component; the extra condition $\det R = 1$ removes no further dimension, it only selects a component. We take the rank computation as a standard fact from differential topology and do not reprove it in full generality. □

Three degrees of freedom is exactly the number we expect: an orientation in 3-dimensional space can be described by, for instance, two angles fixing an axis on the sphere S^2 (a 2-dimensional choice) and one further angle of rotation about that axis, and this count recurs throughout the rest of this chapter and Chapter 3 onward.

3.3.2 The Lie algebra $\mathfrak{so}(3)$ and the hat/vee maps

Proposition 3.18 ($\mathfrak{so}(3)$ is the skew-symmetric matrices)**.** $T_I SO(3) = \{\Omega \in \mathbb{R}^{3\times 3} : \Omega^\top = -\Omega\} =: \mathfrak{so}(3)$.

Proof. Let $\gamma(\tau)$ be a smooth curve in $SO(3)$ with $\gamma(0) = I$. Since $\gamma(\tau)^\top \gamma(\tau) = I$ for all τ, differentiating at $\tau = 0$ using the product rule gives $\dot\gamma(0)^\top I + I^\top \dot\gamma(0) = 0$, i.e. $\dot\gamma(0)^\top = -\dot\gamma(0)$: every tangent vector at I is skew-symmetric. Conversely, the space of skew-symmetric 3×3 matrices is 3-dimensional (three free entries, above the diagonal, with the rest determined by antisymmetry and a zero diagonal), matching Theorem 3.17, so this necessary condition is also sufficient (both sides are 3-dimensional subspaces, one containing the other, hence equal). □

Definition 3.19 (Hat and vee operators; restated precisely)**.**

$$(\cdot)^\wedge : \mathbb{R}^3 \to \mathfrak{so}(3), \qquad \widehat{\omega} = \begin{pmatrix} 0 & -\omega_3 & \omega_2 \\ \omega_3 & 0 & -\omega_1 \\ -\omega_2 & \omega_1 & 0 \end{pmatrix}, \qquad \omega = (\omega_1, \omega_2, \omega_3)^\top,$$

and $(\cdot)^\vee : \mathfrak{so}(3) \to \mathbb{R}^3$ is its (linear, bijective) inverse.

Proposition 3.20 (Hat map intertwines cross product and matrix commutator)**.** *For $\omega, v \in \mathbb{R}^3$: $\widehat{\omega}\, v = \omega \times v$. Consequently, for $\omega_1, \omega_2 \in \mathbb{R}^3$, the matrix commutator satisfies $[\widehat{\omega_1}, \widehat{\omega_2}] := \widehat{\omega_1}\widehat{\omega_2} - \widehat{\omega_2}\widehat{\omega_1} = \widehat{\omega_1 \times \omega_2}$.*

Proof. Direct computation: $\widehat{\omega}v = (\omega_2 v_3 - \omega_3 v_2,\ \omega_3 v_1 - \omega_1 v_3,\ \omega_1 v_2 - \omega_2 v_1)^\top = \omega \times v$ by the definition of the cross product. The commutator identity follows by applying both sides to an arbitrary v and using the bilinearity and antisymmetry of $\times$ together with the vector identity $a \times (b \times v) - b \times (a \times v) = (a \times b) \times v$ (itself checkable directly, or via the BAC-CAB rule $a \times (b \times c) = b(a \cdot c) - c(a \cdot b)$). □

Code correspondence

Mathematical object: $(\cdot)^\wedge : \mathbb{R}^3 \to \mathfrak{so}(3)$, $(\cdot)^\vee : \mathfrak{so}(3) \to \mathbb{R}^3$
Implementation: `so3_utils.py`::`hat`, `vee` (identical in `SigmaDock/` and `SigmaFlow_Development/`)
Tensor shape: `[..., 3]` ↔ `[..., 3, 3]`
Interpretation: `hat` builds the skew-symmetric matrix by constructing only the strict upper triangle and then anti-symmetrising (`hat_v + -hat_v.transpose(-1,-2)`); `vee` inverts it by reading off $(-A_{23}, A_{13}, -A_{12})$ and only *prints* a warning (rather than raising) if the input is not skew-symmetric within a loose tolerance — this is not a hard safety check, and a caller that accidentally passes a non-skew matrix will not be stopped by `vee` itself.

3.3.3 The exponential map: Rodrigues' formula

Motivation. We now want an explicit, closed-form recipe for $\exp_I : \mathfrak{so}(3) \to SO(3)$ (Theorem 3.7 specialised to $G = SO(3)$, $x = I$): given a small rotation $\widehat{\omega} \in \mathfrak{so}(3)$, what is the actual rotation matrix obtained by "following the corresponding geodesic for unit time"? On a Lie group, this question has a beautifully uniform answer.

Proposition 3.21 (Geodesics through the identity of a bi-invariant Lie group are one-parameter subgroups)**.** *Equip $SO(3)$ with the metric $g_I(\Omega_1, \Omega_2) = \frac{1}{2}\operatorname{tr}(\Omega_1^\top \Omega_2)$ at the identity, extended to every other point by left translation (Theorem 3.14); one checks this metric is also invariant under* right *translation (*bi-invariant*), which is a special, convenient feature of compact groups such as $SO(3)$ and is what makes the following true. The geodesic through I with initial velocity $\widehat{\omega} \in \mathfrak{so}(3)$ is $\gamma(\tau) = \exp(\tau\widehat{\omega})$, the matrix exponential.*

We do not prove Theorem 3.21 in full generality (it is a standard fact about bi-invariant metrics on Lie groups, following from the observation that one-parameter subgroups $\tau \mapsto \exp(\tau\Omega)$ of a bi-invariant Lie group have constant speed and locally minimise length; see, e.g., do Carmo, *Riemannian Geometry*, 1992, Ch. 11); we use it only to justify that $\exp_I = \exp$, the ordinary matrix exponential $\exp(A) = \sum_{k=0}^{\infty} A^k/k!$, restricted to $\mathfrak{so}(3)$. What remains is to evaluate this infinite series in closed form.

Theorem 3.22 (Rodrigues' rotation formula). *For $\omega \in \mathbb{R}^3$ with $\theta = \|\omega\|$,*

$$\exp(\widehat{\omega}) = I + \frac{\sin\theta}{\theta}\,\widehat{\omega} + \frac{1-\cos\theta}{\theta^2}\,\widehat{\omega}^{\,2} \qquad (\theta \neq 0),$$

and $\exp(\widehat{\omega}) = I$ when $\theta = 0$ (continuously extending the formula, since both fractions have removable singularities at $\theta = 0$). The resulting matrix is the rotation by angle θ about the axis ω/θ, in the right-hand sense.

Proof. Write $n = \omega/\theta$ (unit vector) so $\widehat{\omega} = \theta\widehat{n}$. The key algebraic fact is the following closed recursion for powers of $\widehat{n}$, provable directly from Theorem 3.20 (with n a unit vector) via the BAC-CAB identity:

$$\widehat{n}^{\,2} = nn^\top - I, \qquad \widehat{n}^{\,3} = \widehat{n}\,\widehat{n}^{\,2} = \widehat{n}(nn^\top - I) = -\widehat{n},$$

using $\widehat{n}\,n = n \times n = 0$ (so $\widehat{n}\,nn^\top = 0$). Hence $\widehat{n}^{\,3} = -\widehat{n}$, so every higher power reduces: $\widehat{n}^4 = -\widehat{n}^2$, $\widehat{n}^5 = \widehat{n}$, etc., with period 4 up to sign, i.e. odd powers are $\pm\widehat{n}$ and even powers (beyond 0) are $\pm\widehat{n}^2$. Substituting into the exponential series and grouping by parity,

$$\begin{aligned}\exp(\theta\widehat{n}) &= I + \Big(\theta - \frac{\theta^3}{3!} + \frac{\theta^5}{5!} - \cdots\Big)\widehat{n} + \Big(\frac{\theta^2}{2!} - \frac{\theta^4}{4!} + \frac{\theta^6}{6!} - \cdots\Big)\widehat{n}^{\,2} \\ &= I + \sin(\theta)\,\widehat{n} + \big(1 - \cos(\theta)\big)\,\widehat{n}^{\,2},\end{aligned}$$

recognising the Taylor series of sin and $1-\cos$. Substituting back $\widehat{n} = \widehat{\omega}/\theta$ gives the claimed formula. That the result is a rotation by θ about axis n can be checked directly: $\exp(\theta\widehat{n})\,n = n$ (since $\widehat{n}n = 0$, so both correction terms vanish on n), so n is fixed, and restricted to the plane orthogonal to n the formula reduces (by direct computation using $\widehat{n}^2 v = -v$ for $v \perp n$) to the standard 2×2 rotation-by-θ matrix in an orthonormal basis of that plane. □

Code correspondence

Mathematical object: $\exp : \mathfrak{so}(3) \to SO(3)$, $\mathrm{Exp} : \mathbb{R}^3 \to SO(3)$
Implementation: `so3_utils.py`::`exp`, `Exp`
Tensor shape: `[..., 3, 3]` → `[..., 3, 3]` (`exp`); `[..., 3]` → `[..., 3, 3]` (`Exp`, defined as `exp(hat(v))`)
Interpretation: Neither codebase uses the closed-form Rodrigues formula of Theorem 3.22 directly. `exp` is implemented as a literal call to `torch.matrix_exp`, PyTorch's generic (Padé-approximation-based) matrix exponential, with a CPU round-trip fallback on Apple-Silicon (MPS) backends where `torch.matrix_exp` is unsupported. This is mathematically equivalent to Theorem 3.22 (both compute the same matrix exponential; $\mathfrak{so}(3)$'s exponential series converges absolutely for every finite θ, so there is no issue of principle), but it is worth recording explicitly as a discrepancy between how this document derives the map and how the shipped code evaluates it: Theorem 3.22's closed form is what makes the derivations of Section 3.3.4 and Chapters 5 and 10 tractable by hand, but the executed program never evaluates $\sin\theta/\theta$ directly at this step.

3.3.4 The logarithmic map, and its numerically dangerous points

Motivation. $\log : SO(3) \to \mathfrak{so}(3)$ (near I; more on the domain of validity below) is the local inverse of exp, and it is what lets us measure "how different are two rotations" as a single tangent vector, exactly the operation flow matching needs to build an interpolant between a source rotation and a target rotation (Section 11.3).

Theorem 3.23 (Logarithm of a rotation matrix). *For $R \in SO(3)$ with rotation angle $\theta = \arccos\big(\frac{\operatorname{tr}(R)-1}{2}\big) \in [0,\pi]$ and $\theta \notin \{0,\pi\}$,*

$$\log(R) = \frac{\theta}{2\sin\theta}\,(R - R^\top) \;\in\; \mathfrak{so}(3).$$

Proof. By Theorem 3.22, $R = I + \sin\theta\,\widehat{n} + (1-\cos\theta)\widehat{n}^{\,2}$ for the (unknown) axis n and angle θ of R. Since $\widehat{n}$ is skew-symmetric and $\widehat{n}^{\,2} = nn^\top - I$ is symmetric, the skew-symmetric part of R is exactly $\frac{1}{2}(R - R^\top) = \sin\theta\,\widehat{n}$. Solving for $\widehat{n}$ (valid whenever $\sin\theta \neq 0$, i.e. $\theta \notin \{0,\pi\}$) gives $\widehat{n} = (R - R^\top)/(2\sin\theta)$, and hence $\log(R) = \theta\widehat{n} = \dfrac{\theta}{2\sin\theta}(R - R^\top)$. The formula for θ itself follows by taking the trace of Rodrigues' formula: $\operatorname{tr}(R) = 3 + 0 + (1-\cos\theta)\operatorname{tr}(\widehat{n}^{\,2}) = 3 + (1-\cos\theta)(-2) = 1 + 2\cos\theta$, using $\operatorname{tr}(\widehat{n}^{\,2}) = \operatorname{tr}(nn^\top - I) = \|n\|^2 - 3 = -2$; solving gives $\cos\theta = (\operatorname{tr}(R) - 1)/2$. □

The two excluded points $\theta = 0$ and $\theta = \pi$ are not edge cases that can be ignored: $\theta = 0$ (the identity rotation) is reached with probability zero under any continuous sampling scheme but is approached arbitrarily closely whenever two rotations being compared are nearly equal, which happens *routinely* near convergence of an iterative sampler; and $\theta = \pi$ (a half-turn) has positive probability density under the uniform distribution on $SO(3)$ derived in Theorem 3.29 below, so it *will* be sampled during training and inference at a non-negligible rate. Both require separate formulas.

Proposition 3.24 (Small-angle behaviour of log). *As $\theta \to 0$,*

$$\log(R) = \frac{1}{2}\Big(1 + \frac{\theta^2}{6} + O(\theta^4)\Big)(R - R^\top).$$

Proof. Taylor-expand $\theta/(2\sin\theta) = \frac{1}{2}\big(1 - \theta^2/6 + O(\theta^4)\big)^{-1} = \frac{1}{2}\big(1 + \theta^2/6 + O(\theta^4)\big)$, using $\sin\theta = \theta - \theta^3/6 + O(\theta^5)$. □

Proposition 3.25 (Behaviour of log near $\theta = \pi$). *As $\theta \to \pi$, $R \to 2nn^\top - I$ for the rotation axis n, so $\frac{1}{2}(R+I) \to nn^\top$; consequently $|n_i| = \sqrt{\big(\frac{1}{2}(R+I)\big)_{ii}}$, with relative signs fixed by comparing the off-diagonal entries of $\frac{1}{2}(R+I) = nn^\top$ against $n_i n_j$. The overall sign of n is* genuinely undetermined*: at $\theta = \pi$ exactly,*

$$\exp(\pi\widehat{n}) \;=\; 2nn^\top - I \;=\; 2(-n)(-n)^\top - I \;=\; \exp\big(\pi\widehat{(-n)}\big),$$

so πn and $-\pi n$ are two distinct tangent vectors mapping to the same rotation. Both are legitimate values of log(R)*, and both have the same length π; the map* log *is therefore two-valued exactly on the set of half-turns, which is the antipodal set of Theorem 3.8(b).*

Proof. At $\theta = \pi$: $\sin\theta = 0$, $\cos\theta = -1$, so Rodrigues' formula gives $R = I + 0 + 2\widehat{n}^{\,2} = I + 2(nn^\top - I) = 2nn^\top - I$, hence $\frac{1}{2}(R+I) = nn^\top$, a rank-one symmetric matrix with unit-norm columns along $\pm n$. Reading off $|n_i|$ from the diagonal and signs from the off-diagonal entries is then direct linear algebra on the outer product $nn^\top$. □

Caution 3.26 (Do not read this as $\widehat{n} = \widehat{-n}$). It is tempting to compress the last statement to "the sign does not matter because $\widehat{n} = \widehat{-n}$". That equation is *false*: the hat map is linear, so $\widehat{-n} = -\widehat{n}$. What is true is the weaker and more specific statement above — that the two *exponentials* coincide *at* $\theta = \pi$. Away from π the sign of n matters and cannot be chosen freely.

Code correspondence

Mathematical object: $\log : SO(3) \to \mathfrak{so}(3)$ near a generic rotation; its numerically safe extension to $\theta \to 0$ and $\theta \to \pi$
Implementation: `so3_utils.py`::`rotation_vector_from_matrix` (called by `Log`, whose output is then `hat`-lifted by `log`)
Tensor shape: `[..., 3, 3]` $\to$ `[..., 3]` (vee-map convention: the vector form $v = \theta n$, not the $\mathfrak{so}(3)$ matrix)
Interpretation: the code computes *all three* regimes — generic (Theorem 3.23), near-zero (Theorem 3.24, using the exact Taylor truncation derived above), and near-π (Theorem 3.25, with sign recovered from whichever row of $\frac{1}{2}(R+I)$ has the largest norm) — for *every* input, and blends them with continuous 0/1 masks built from `torch.isclose` (tolerance atol $= 10^{-8}$ near $\theta = 0$, the much looser atol $= 10^{-2}$ near $\theta = \pi$), rather than a hard `if`/`else` branch. This is a deliberate autodiff-friendly pattern: computing all three branches unconditionally avoids `NaN`-producing gradients from a branch that was not selected but would otherwise still be backpropagated through, at the cost of some redundant computation.

Caution 3.27 (The angle-clamp bug, and why it mattered only for flow matching). The function that computes $\theta = \arccos\left(\frac{\mathrm{tr}(R)-1}{2}\right)$ (`Omega` in `so3_utils.py`) must clamp its argument to $[-1, 1]$ before calling `arccos`, since floating-point round-off can push $\frac{\mathrm{tr}(R)-1}{2}$ fractionally outside this interval even for a valid $R \in SO(3)$. The original SigmaDock code clamps to $[-0.99, 0.99]$, i.e. $\theta \in [\arccos(0.99), \arccos(-0.99)] \approx [8.1°, 172°]$ — *any true angle beyond* $172°$ *is silently rounded down to* $172°$. For diffusion this is close to harmless: θ enters only through the smooth score function of Section 6.3, degrading gracefully under a slightly wrong clamp. For flow matching it is not harmless: the geodesic interpolant of Section 11.3 needs log to be an *exact* inverse of exp (an exact round trip $\exp(\log(R)) = R$), which the clamp breaks for every pair whose relative angle leaves $[8.1°, 171.9°]$.

How often is that? *Not* $(180 - 172)/180 \approx 4.4\%$: the relative angle between two independent Haar-random rotations is not uniform on $[0, \pi]$ but has density $p(\theta) = (1 - \cos\theta)/\pi$ (Theorem 3.29), which puts most of its mass at large angles. Integrating,

$$\mathbb{P}(\theta > 171.9°) = \frac{\pi - \left(\theta_0 - \sin\theta_0\right)}{\pi}\bigg|_{\theta_0 = 171.9°} \approx 9.0\%, \qquad \mathbb{P}(\theta < 8.1°) \approx 0.015\%.$$

So the damage is essentially one-sided: about **one rotation pair in eleven** is silently truncated at the upper end, while the lower end is irrelevant. (A 400,000-sample Monte Carlo check reproduces 9.01% and 0.018%, and en route confirms the median relative angle 132.4° and mean 126.6° quoted elsewhere in this document.) The frequently-quoted "$\approx 9\%$" is therefore the right order of magnitude, but the uniform-angle reasoning that appears to produce it is wrong twice over — an incorrect density and a spurious doubling — and would give the wrong answer for any other clamp value. SigmaFlow's `so3_utils.py` narrows the clamp to $[-1+10^{-7}, 1-10^{-7}]$, i.e. $\theta \in [0.026°, 179.97°]$, purely for `arccos`'s numerical domain, and documents the reason for the change directly in the source (`so3_flow_matcher.py`, inline comment). This document treats the fix as settled and uses the corrected clamp implicitly wherever θ is discussed.

3.3.5 $SO(3)$ as a metric space, and Brownian motion's invariant measure

We record two facts about $SO(3)$ that are used repeatedly from Chapter 5 onward and are best derived once, here, in isolation from any generative-modelling context.

Proposition 3.28 (Bi-invariant Riemannian distance on $SO(3)$)**.** *With the metric of Theorem 3.21, the Riemannian distance between $R_1, R_2 \in SO(3)$ is $d(R_1, R_2) = \|\log(R_1^\top R_2)\|_g = \theta$, the rotation angle of the relative rotation $R_1^\top R_2$.*

Proof sketch. By bi-invariance, $d(R_1, R_2) = d(I, R_1^\top R_2)$ (left-translate by $R_1^{-1} = R_1^\top$), and $d(I, R)$ is the length of the geodesic from I to R, which by Theorem 3.21 is $\exp(\tau\widehat{\omega})$ for the ω with $\exp(\widehat{\omega}) = R$, $\|\omega\|$ minimal — this is exactly $\theta = \|\log(R)^\vee\|$ under the metric normalisation of Theorem 3.21 (chosen precisely so that the identification $\mathfrak{so}(3) \cong \mathbb{R}^3$ of Theorem 3.19 is a metric isometry, i.e. so that $\|\widehat{\omega}\|_g = \|\omega\|_2$ — a direct check: $g_I(\widehat{\omega}, \widehat{\omega}) = \frac{1}{2}\operatorname{tr}(\widehat{\omega}^\top\widehat{\omega}) = \frac{1}{2}\operatorname{tr}(-\widehat{\omega}^2) = \|\omega\|_2^2$ by a short computation using Theorem 3.20). □

Proposition 3.29 (The Haar measure on $SO(3)$, in axis–angle coordinates)**.** *The measure on $SO(3)$ invariant under both left and right multiplication (the* Haar measure*, unique up to normalisation on a compact group; existence/uniqueness is a classical fact we do not reprove — see, e.g., Folland,* A Course in Abstract Harmonic Analysis*, 1995, Ch. 2), when $SO(3)$ is parameterised by axis $n \in S^2$ and angle $\theta \in [0, \pi]$ via $R = \exp(\theta\widehat{n})$, has density, with respect to $d\theta \times$ (uniform measure on S^2), proportional to $1 - \cos\theta$.*

Proof sketch. This is the statement that the Riemannian volume form of Theorem 3.21, expressed in axis–angle (geodesic normal) coordinates, has Jacobian $\propto (1 - \cos\theta)$ relative to the flat product measure $d\theta \times d\Omega_{S^2}$; the computation is a standard but slightly involved exercise in Riemannian normal coordinates on a compact Lie group (the Jacobian of $\exp_I$ at $\theta\widehat{n}$, for a bi-invariant metric, is $\prod_k \left(\frac{2\sin(\lambda_k\theta/2)}{\lambda_k\theta}\right)$ over the eigenvalues λ_k of $\operatorname{ad}_{\widehat{n}}$; for $\mathfrak{so}(3)$ these are $0, \pm i$, and evaluating the product reduces, after simplifying the complex-conjugate pair, to $(1-\cos\theta)/(\theta^2/2) \cdot (\text{const})$ times the flat measure), which we do not carry out in full here and instead take as a standard fact (see, e.g., Chirikjian, *Stochastic Models, Information Theory, and Lie Groups*, Vol. 2, 2012, §10.2). We record the consequence that matters practically: to sample $R \sim \text{Haar}(SO(3))$, sample n uniformly on S^2 and, independently, sample θ from the density $\propto 1 - \cos\theta$ on $[0, \pi]$. □

Code correspondence

Mathematical object: $R \sim \text{Haar}(SO(3))$
Implementation: `so3_utils.py::sample_uniform`
Tensor shape: returns `[N, 3, 3]`
Interpretation: builds a 1000-point grid on $[0, \pi]$, numerically integrates the density $\propto 1 - \cos\theta$ of Theorem 3.29 into a CDF, draws θ by inverse-CDF sampling via linear interpolation on this grid (`np.interp`), draws an axis by normalising a standard Gaussian vector in $\mathbb{R}^3$ (a standard, exact way to sample uniformly on S^2: the direction of an isotropic Gaussian is rotation-invariant in distribution, since the Gaussian density $\propto e^{-\|x\|^2/2}$ depends on x only through $\|x\|$), and returns $\exp(\theta\widehat{n})$. This sampling routine runs on `numpy`'s global random state, *not* through `torch`'s seeded generator — a genuine reproducibility subtlety: seeding `torch.manual_seed` alone does not fix the axes or angles drawn by this function.

3.4 Left and right trivialisation, resolved for this document

We can now make Theorem 3.14's warning fully concrete for $SO(3)$, since this is a point at which, as flagged in the introduction to this document, the existing thesis draft and the

SigmaFlow source code do not visibly agree, and it must be resolved by direct derivation rather than by trusting either source.

Definition 3.30 (Body-frame and spatial-frame angular velocity)**.** Let $R(\tau)$ be a smooth curve in $SO(3)$. Its *body-frame* angular velocity is $\omega^{\mathrm{bd}}(\tau) \in \mathbb{R}^3$ with

$$\dot{R}(\tau) = R(\tau)\,\widehat{\omega^{\mathrm{bd}}(\tau)}, \qquad \text{equivalently} \qquad \widehat{\omega^{\mathrm{bd}}} = R^{-1}\dot{R};$$

its *spatial-frame* (or *world-frame*) angular velocity is $\omega^{\mathrm{sp}}(\tau) \in \mathbb{R}^3$ with

$$\dot{R}(\tau) = \widehat{\omega^{\mathrm{sp}}(\tau)}\,R(\tau), \qquad \text{equivalently} \qquad \widehat{\omega^{\mathrm{sp}}} = \dot{R}\,R^{-1}.$$

In the language of Theorem 3.14, ω^{bd} is the **left-trivialised** and ω^{sp} the **right-trivialised** velocity, because $\dot{R} = R\,\widehat{\omega}$ is exactly $dL_R|_e(\widehat{\omega})$ and $\dot{R} = \widehat{\omega}\,R$ is exactly $dR_R|_e(\widehat{\omega})$.

Caution 3.31 (The words "left" and "right" are a trap; "body" and "spatial" are not). Two independent naming conflicts meet here, and getting either backwards silently transposes a rotation.

First, some authors name the trivialisation after *which side of R the hat matrix sits on* rather than after which translation map produces it, which inverts the labels relative to Theorem 3.30. The attribution fixed above is the standard Lie-theoretic one and is forced by Theorem 3.14: left translation L_R acts as $S \mapsto RS$, so $dL_R|_e(\xi) = R\xi$, so a tangent vector expressed by left translation is $\dot{R} = R\widehat{\omega}$ — the *body* frame. An earlier draft of this document had these two labels interchanged; the formulas were unaffected, but the names were.

Second, the SigmaDock source comments use "left-invariant" as a synonym for *world* frame (Section 9.8), i.e. with the opposite attribution to the one fixed here. That comment is quoted verbatim where it occurs, and is *not* silently corrected — but the reader should know that the code's label and this document's label disagree, while the underlying object (which side of R the multiplication happens on) is unambiguous and is what every reconstruction in Chapters 9 and 12 is checked against.

Practical rule for reading the rest of this document: trust $\omega^{\mathrm{bd}} = R^{-1}\dot{R}$ and $\omega^{\mathrm{sp}} = \dot{R}R^{-1}$, and treat the words "left" and "right" as mnemonics only.

By definition, $\widehat{\omega^{\mathrm{sp}}} = \dot{R}R^{\top} = \dot{R}R^{-1}$ and $\widehat{\omega^{\mathrm{bd}}} = R^{\top}\dot{R} = R^{-1}\dot{R}$, and they are related by $\omega^{\mathrm{sp}} = R\omega^{\mathrm{bd}}$ (apply Theorem 3.20-style conjugation: $R\widehat{\omega^{\mathrm{bd}}}R^{\top} = \widehat{R\omega^{\mathrm{bd}}}$, a direct consequence of Theorem 3.16's cross-product identity), i.e. the two differ by *rotating the vector itself by R*. They are equal only when R commutes with the relevant tangent direction, generally not the case.

Proposition 3.32 (The geodesic interpolant is naturally left-trivialised (body-frame))**.** *Let $R(\tau) = R_0 \exp(\tau\Xi)$ for fixed $R_0 \in SO(3)$, $\Xi \in \mathfrak{so}(3)$ (this is exactly the geodesic construction of Theorem 11.5 below). Then $\dot{R}(\tau) = R(\tau)\,\Xi$, i.e. the curve has* constant *left-trivialised (body-frame) angular velocity $\omega^{\mathrm{bd}} = \Xi^{\vee}$, in the sense of Theorem 3.30.*

Proof. Differentiate $R(\tau) = R_0 \exp(\tau\Xi)$ directly: $\dot{R}(\tau) = R_0 \exp(\tau\Xi)\,\Xi = R(\tau)\,\Xi$, using that Ξ commutes with $\exp(\tau\Xi)$ (both are polynomials/power series in Ξ). This matches the first case of Theorem 3.30 exactly, with $\omega^{\mathrm{bd}}(\tau) \equiv \Xi^{\vee}$, constant in τ. □

Proposition 3.33 (The corresponding one-step update is a right multiplication)**.** *If a numerical scheme advances R by one Euler step using a body-frame (left-trivialised) velocity $v \in \mathbb{R}^3$ over a time increment dt, the update that remains exactly on $SO(3)$, consistent with Theorem 3.32, is $R_{\mathrm{new}} = R\exp(dt\,\widehat{v})$ (multiplying the exponential on the* right*); the analogous world-frame (right-trivialised) update would instead be $R_{\mathrm{new}} = \exp(dt\,\widehat{v})\,R$.*

Theorems 3.32 and 3.33 settle the question for this document: wherever SigmaFlow's geodesic interpolant (Section 11.3) or its Euler integrator (Section 12.8) is discussed, the angular velocity supplied by, and returned to, the network is the **left-trivialised (body-frame)** angular velocity, consistent with the SigmaFlow source code's own comment (`so3_flow_matcher.py`) and with the update rule $R \mapsto R\exp(dt\,\widehat{v})$ used by both `euler_step` and `expmap` in the actual implementation (both right-multiply the exponential onto R, so — once Theorem 3.30's labelling convention is fixed and applied uniformly — the two are in fact mutually consistent, and the apparent conflict flagged by the code-research pass resolves to a terminology mismatch between the source comment's word choice and the existing thesis draft's word choice, not a mathematical inconsistency in either). Chapter 9 performs the same check, independently, for SigmaDock's diffusion construction, since diffusion's Brownian-motion increments (Section 6.3) are built from `expmap`/`tangent_gaussian`, the same right-multiplying primitives, and the two chapters must therefore use the *same* trivialisation convention for their comparison in Chapter 12 to be meaningful.

Caution 3.34. This resolution is a derivation carried out for this document; it should not be taken to mean the original published SigmaDock description necessarily uses matching language, only that the code that actually runs is self-consistent under Theorem 3.30. Where the distinction between ω^{sp} and ω^{bd} affects a formula materially (as opposed to being a bookkeeping choice with no observable consequence, e.g. because a subsequent step immediately re-expresses the result in world frame), this document states which one is meant at the point of use, rather than relying on the reader to track it globally.

3.5 The pose group $SE(3)$ and the product manifold $SE(3)^N$

Definition 3.35 (Special Euclidean group)**.** $SE(3)$ is the group of rigid motions of $\mathbb{R}^3$: as a set, $SE(3) \cong SO(3) \times \mathbb{R}^3$, with element $T = (R, x)$ acting on $y \in \mathbb{R}^3$ by $T \cdot y = Ry + x$, and group operation

$$(R_1, x_1) \circ (R_2, x_2) = (R_1 R_2,\ R_1 x_2 + x_1)$$

(composition of affine maps: apply T_2 then T_1).

Proposition 3.36. *$SE(3)$ is a 6-dimensional Lie group, with identity $(I, 0)$ and inverse $(R, x)^{-1} = (R^\top, -R^\top x)$.*

Proof. Immediate from Theorem 3.35: $\dim SE(3) = \dim SO(3) + \dim \mathbb{R}^3 = 3 + 3 = 6$ as a manifold (a product of manifolds, Theorem 3.1, has dimension the sum of the factors' dimensions); associativity and the group axioms follow from associativity of composition of affine maps; smoothness of the group operations is immediate since Theorem 3.35's formula is polynomial in (R_1, x_1, R_2, x_2). □

Metric choice. $SE(3)$ admits more than one natural Riemannian metric — one may couple the rotational and translational blocks (e.g. scaling one relative to the other by a length parameter reflecting the physical size of the body being moved) or keep them fully decoupled. We fix, following Yim et al. (2023, cited precisely in Chapter 12 where this choice is used), the *product metric*

$$g_{(R,x)}\big((\dot{R}_1, \dot{x}_1), (\dot{R}_2, \dot{x}_2)\big) = \langle \dot{R}_1, \dot{R}_2 \rangle_{SO(3)} + \langle \dot{x}_1, \dot{x}_2 \rangle_{\mathbb{R}^3}. \tag{3.1}$$

Proposition 3.37 ($SE(3)$ with the product metric is a Riemannian product)**.** *With (3.1), the exponential and logarithmic maps, geodesics, and (once defined in Chapter 5) Brownian motion on $SE(3)$ all factorise componentwise:*

$$\exp_{(R,x)}(\dot{R},\dot{x}) = \big(\exp_R(\dot{R}),\ x+\dot{x}\big), \qquad \log_{(R,x)}(R',x') = \big(\log_R(R'),\ x'-x\big).$$

Proof sketch. A Riemannian product $(\mathcal{M}_1 \times \mathcal{M}_2, g_1 \oplus g_2)$ has geodesics exactly the curves (γ_1,γ_2) with γ_i a geodesic of $(\mathcal{M}_i, g_i)$ (since the geodesic equation, derived from the metric via the Euler–Lagrange equations for the length functional, decouples exactly when the metric does, as (3.1) does by construction), and $\mathbb{R}^3$ with the flat Euclidean metric has $\exp_x(v) = x+v$, $\log_x(y) = y-x$ trivially. Combining these two facts with $SO(3)$'s own maps (Sections 3.3.3 and 3.3.4) gives the claim. □

Theorem 3.37 is, computationally, the single most consequential fact in this chapter: it licenses treating translation and rotation *completely independently* in every later construction (the conditional probability paths of Chapters 5 and 10, the network's two output heads of Chapters 9 and 12), with no cross term anywhere, provided the metric (3.1) — rather than some other, coupled metric — is the one actually in force. Chapter 9 verifies, directly against the code, that this decoupled treatment is indeed what is implemented (no coupling term appears in either `SE3Diffuser`/`SE3_FlowMatcher`, both of which are pure componentwise dispatchers, per the code-research findings recorded there).

Definition 3.38 (The pose manifold of N rigid fragments)**.** The state space of a docking model that represents a ligand as N independently posable rigid fragments (Section 9.1) is the product manifold

$$\mathcal{M}_N = SE(3)^N, \qquad \mathbf{T} = \big(R^{(1)}, x^{(1)}, \ldots, R^{(N)}, x^{(N)}\big),$$

again with the product metric, extending (3.1) fragmentwise.

By the same reasoning as Theorem 3.37 applied N times, every construction on $\mathcal{M}_N$ factorises over fragments *and* over rotation/translation within each fragment: the entire generative model, seen purely geometrically, is $2N$ independent copies of the two constructions this document develops in full generality — one Euclidean ($\mathbb{R}^3$, for each fragment's translation) and one $SO(3)$ (for each fragment's rotation) — coupled *only* through the neural network that predicts the vector field/score, never through the geometry itself. This is exactly the content of the "inter-fragment correlations enter only through the learned model" design point discussed in Section 9.1.

3.6 The torsion space $\mathbb{T}^k$

The remaining internal degree of freedom identified in Section 1.2, the dihedral angle, lives on a space we have not yet defined formally.

Definition 3.39 (Circle group and torus)**.** $\mathbb{T} := \mathbb{R}/2\pi\mathbb{Z}$, the quotient of $\mathbb{R}$ by the equivalence $\phi \sim \phi + 2\pi m$ ($m \in \mathbb{Z}$), with group operation addition mod 2π; equivalently, $\mathbb{T} \cong S^1 = \{(\cos\phi, \sin\phi) : \phi \in \mathbb{R}\} \subset \mathbb{R}^2$ via $\phi \mapsto (\cos\phi, \sin\phi)$, an isomorphism of Lie groups (the group law on the right is complex multiplication of unit-modulus complex numbers, matching addition of angles). $\mathbb{T}^k$ is the k-fold direct product, an abelian Lie group of dimension k.

Being abelian, $\mathbb{T}^k$'s Lie algebra is simply $\mathbb{R}^k$ with the identity map as exponential ($\exp(\phi) = \phi \bmod 2\pi$ in each coordinate), and the natural (bi-invariant, since the group is abelian and hence trivially so) metric is flat Euclidean — *locally*. The one genuinely new phenomenon $\mathbb{T}^k$ introduces, absent from $\mathbb{R}^k$, is that *differences* of angles are only defined up to the 2π-periodicity, and there are two equally valid representatives differing by a full turn; the convention used throughout is to take the *shortest* representative.

Definition 3.40 (Periodic (shortest-path) angular difference)**.** For $\phi_1, \phi_2 \in \mathbb{T}$, the geodesic distance is $d_{\mathbb{T}}(\phi_1, \phi_2) = \left|\left((\phi_2 - \phi_1 + \pi) \bmod 2\pi\right) - \pi\right| \in [0, \pi]$, and the logarithmic map at ϕ_1 is the signed version, $\log_{\phi_1}(\phi_2) = \left((\phi_2 - \phi_1 + \pi) \bmod 2\pi\right) - \pi \in (-\pi, \pi]$, the unique representative of $\phi_2 - \phi_1 \pmod{2\pi}$ lying in $(-\pi, \pi]$.

Remark 3.41. SigmaDock and SigmaFlow, as reconstructed in Chapters 9 and 12 from the codebase, do **not** generate torsion angles as an explicit degree of freedom at all: the fragment-based parameterisation of Section 9.1 treats each rigid fragment's *internal* torsions as fixed at their reference-conformer values (Section 9.2), and only the N copies of $SE(3)$ — never an explicit $\mathbb{T}^k$ factor — are generated. This chapter nonetheless develops $\mathbb{T}^k$ in full, for two reasons stated precisely and not merely asserted: first, understanding *why* SigmaDock avoids torsional generation requires the volume-form argument of Section 9.1, which itself requires comparing the torus construction against the fragment construction on equal footing; second, the torsion angle ϕ_{ijkl} of Section 1.2 is exactly the periodic dihedral variable this section formalises, and it reappears, undiffused and ungenerated but still present as a fixed input, throughout Chapters 9 and 12's discussion of what a rigid fragment template actually is.

Remark 3.42 (Parity and chirality). We noted in Theorem 3.15 that $O(3) \supsetneq SO(3)$ additionally contains orientation-*reversing* maps (reflections, $\det = -1$). Molecules are, in general, chiral: a molecule and its mirror image can be distinct chemical species (different smell, different biological activity, etc.), so a docking model must not be invariant under reflection — a network that could not distinguish a ligand from its mirror image would place both equally well into a pocket that in reality binds only one of them. This is why Chapter 8 tracks a *parity* label alongside the degree ℓ once it introduces representations of $O(3)$ rather than only of $SO(3)$: a rotationally-equivariant-only architecture that additionally happened to be reflection-invariant would be a strictly worse inductive bias for this task, not a neutral simplification.

Chapter 4

Steerable Representations: Spherical Harmonics, Wigner Matrices, and Clebsch–Gordan Coefficients

Chapter 3 equipped us with $SO(3)$ itself; this chapter equips us with the machinery to build *features that transform predictably under* $SO(3)$ — scalars, ordinary 3-vectors, and their higher-degree analogues — and to combine such features while preserving that predictability. This is not optional scaffolding for the neural network of Chapter 8: it *is* the architecture, in the precise sense that once the objects of this chapter are defined, the network's layers will turn out to be almost forced, as Chapter 8 shows in detail. Everything here is representation theory of $SO(3)$, developed only as far as needed and always tied back to 3-dimensional physical intuition (directions, angular momentum) rather than presented abstractly first.

4.1 The problem this chapter exists to solve

Before any representation theory, we should be completely clear about the problem. It is a physical statement, not a mathematical one, and everything technical in this chapter is downstream of it.

4.1.1 Rotating the whole system changes nothing physical

Take a protein pocket with a ligand docked in it. Now pick up the entire assembly — protein *and* ligand together — and rotate it by some $Q \in SO(3)$ about the origin. Every atom moves: $x_i \mapsto Qx_i$ for all i, protein and ligand alike.

Has anything physical changed? No. The binding energy is the same. Every interatomic distance is the same. Whether the pose is correct is the same. We have merely chosen to look at the same physical configuration from a different angle. There is no privileged orientation of space; the laws of chemistry do not know which way is "up".

That single observation constrains what our model is allowed to be, and the constraint has two distinct flavours depending on *what kind of thing* the model outputs.

4.1.2 Two different kinds of "unchanged"

Some quantities should not change at all. The binding affinity of a pose is a number. Rotate the system and you must get the same number. A quantity like this is called *invariant*.

Other quantities must change, in lockstep. Suppose the model outputs, for a ligand fragment, the direction it should be nudged in — a vector $v \in \mathbb{R}^3$. Now rotate the whole input by Q. The correct answer is no longer v: the fragment now sits in a rotated pocket, and the direction it should move in is the rotated direction Qv. If the model still returned v, it would be pointing the wrong way. A quantity like this is called *equivariant*: it must *co-vary* with the input.

The two are not competing options; they are two cases of one principle. The principle is: *the model must not depend on our arbitrary choice of coordinate frame.* Which case applies depends on whether the output is the kind of object that has a direction.

Quantity	Behaviour under a global rotation Q	Type
Atomic number, formal charge	unchanged	invariant
Interatomic distance $\|x_i - x_j\|$	unchanged	invariant
Binding affinity, energy	unchanged	invariant
RMSD to a reference pose	unchanged	invariant
Atom position x_i	$x_i \mapsto Qx_i$	equivariant
Relative vector $x_{ij} = x_j - x_i$	$x_{ij} \mapsto Qx_{ij}$	equivariant
Force on an atom	$F_i \mapsto QF_i$	equivariant
Translational velocity output v	$v \mapsto Qv$	equivariant
Angular velocity output ω	$\omega \mapsto Q\omega$	equivariant
Fragment orientation R	$R \mapsto QR$	equivariant

Table 4.1: Invariant and equivariant quantities in docking. Note that distance is invariant while the vector it is the length of is equivariant: taking the norm is precisely an operation that *converts* equivariant data into invariant data. This will turn out to be the fundamental move in every equivariant architecture.

Caution 4.1 (Invariance is not "better" than equivariance). A common first reaction is: "invariance sounds safer — can we not just make everything invariant?" No, and the reason is decisive for docking. A model whose *every* internal quantity is invariant can only ever output invariant quantities, because there is no way to manufacture a direction out of numbers that carry none. Such a model can predict a binding *affinity*. It cannot predict a binding *pose*: it has no mechanism to say "move that way".

SigmaDock and SigmaFlow must output $\omega, v \in \mathbb{R}^3$ per fragment (Equation (7.1)). Those are equivariant objects. So the network is *required* to carry directional information internally, and the entire difficulty of the architecture is carrying it *correctly*.

4.1.3 Why an ordinary network does not do this by itself

Suppose we simply feed the raw coordinates $x_1, \ldots, x_N \in \mathbb{R}^3$ into a multilayer perceptron. Concatenate them into one vector of length $3N$ and apply $h \mapsto \sigma(Wh + b)$.

Rotate the input. The new input is $Qx_1, \ldots, Qx_N$. Is the output the rotated old output? Almost never. Two independent reasons:

1. **The linear map does not commute with the rotation.** For the first layer to be equivariant we would need $W(Q \oplus \cdots \oplus Q)h = (Q \oplus \cdots \oplus Q)Wh$ for *every* $Q \in SO(3)$, i.e. W must commute with the entire rotation group acting block-wise. Generic weight matrices do not; the set that does is a very thin subspace, and nothing in ordinary training pushes W into it.

2. **The nonlinearity destroys it even if the linear part survived.** Applying ReLU componentwise to a 3-vector is not compatible with rotation: $\mathrm{ReLU}(Qv) \neq Q\,\mathrm{ReLU}(v)$ in general. We return to this with an explicit counterexample in Theorem 4.14.

So equivariance is not something a generic architecture has and can lose. It is something a generic architecture *lacks* and must be built to have.

4.1.4 Why data augmentation is not a substitute

The obvious cheap fix is to train on many randomly rotated copies of each example, hoping the network learns the symmetry. This helps, and it is what many models do, but it is strictly weaker than building the symmetry in. Four reasons, in increasing order of importance for us:

1. **It only ever holds approximately.** Augmentation encourages equivariance on the data distribution; it guarantees nothing off it, and nothing exactly. An architecturally equivariant model satisfies $f(Q \cdot x) = Q \cdot f(x)$ for every Q and every x, exactly, at initialisation, before any training.

2. **It spends capacity.** The network must devote parameters to memorising "these $|SO(3)|$-many inputs are the same input". Those parameters are not available for chemistry. Equivalently: the effective size of the training set is reduced, because the model must relearn each example in every orientation.

3. **It is a 3-parameter group.** Augmenting over a 3-dimensional continuous group by sampling is far less effective than augmenting over, say, a discrete flip. Coverage is thin.

4. **The error mode is invisible and pose-shaped.** A non-equivariant docking model can produce a pose that is correct up to an unnoticed frame error — and a systematically mis-rotated pose is exactly the failure mode that is hardest to detect from aggregate loss curves and most damaging to RMSD. This document's own development history contains an instance of precisely this class of bug (Section 12.4), found only by a targeted diagnostic.

Why this matters for SigmaFlow

SigmaFlow replaces SigmaDock's generative engine and keeps its backbone. The backbone is exactly the machinery of this chapter and Chapter 8. So this material is *not* part of what the thesis changes — but it is part of what the thesis must not *break*. When we later verify that a modification preserves equivariance (Theorem 7.1, and the per-quantity table in Section 12.4), the property being checked is the one defined below in Theorem 4.9.

4.2 Group actions: how a group moves things

Section 3.2 defined a group as a set with a composition law. That is a statement about the group *in isolation*. What we actually need is the group *doing something* to another set — rotations acting on points, on molecules, on features. That is a group action.

The problem. We have $SO(3)$, and we have several different kinds of object: single points in $\mathbb{R}^3$; whole molecules, which are tuples of points; and later, abstract feature vectors inside a network. "Rotate it" means something different in each case. We need one piece of vocabulary that covers all of them and states precisely what the common structure is.

Definition 4.2 (Group action)**.** Let G be a group and X a set. A (left) *action* of G on X is a map

$$\cdot : G \times X \to X, \qquad (g, x) \mapsto g \cdot x,$$

satisfying two axioms:

(A1) **Identity.** $e \cdot x = x$ for all $x \in X$, where $e \in G$ is the identity.
(A2) **Compatibility.** $g_1 \cdot (g_2 \cdot x) = (g_1 g_2) \cdot x$ for all $g_1, g_2 \in G$, $x \in X$.

Both axioms say something one would want to be true and which is easy to violate by accident. (A1) says "doing nothing does nothing". (A2) says "rotating by g_2 and then by g_1 is the same as rotating once by the composite $g_1 g_2$" — the action must respect the group's own composition law, which is the whole reason we insisted G was a group in the first place.

Caution 4.3 ($g \cdot x$ need not be matrix multiplication). The notation $g \cdot x$ is deliberately neutral. In our first example it *is* a matrix times a vector. In the next one it is a matrix applied to each of N vectors. Later, when X is a space of *functions* on the sphere, the action will be $(Q \cdot f)(\hat{x}) = f(Q^{-1}\hat{x})$ — note the inverse, which is forced by (A2) and is a classic source of sign-and-direction errors (Theorem 4.24). And when $X = SO(3)$ itself, the action is left multiplication $Q \cdot R = QR$, where the "point" being acted on is itself a rotation. Writing all of these as $g \cdot x$ is what lets one definition of equivariance cover all of them.

Example 4.4 (Rotations acting on the plane)**.** $G = SO(2)$, $X = \mathbb{R}^2$, and $g \cdot x = R_\vartheta x$ with $R_\vartheta = \left(\begin{smallmatrix} \cos\vartheta & -\sin\vartheta \\ \sin\vartheta & \cos\vartheta \end{smallmatrix}\right)$. (A1): $R_0 = I$. (A2): $R_{\vartheta_1} R_{\vartheta_2} = R_{\vartheta_1 + \vartheta_2}$, so rotating twice is rotating by the sum — exactly the group law of $SO(2)$. This is the smallest example in which the axioms are not vacuous.

Example 4.5 (Rotations acting on $\mathbb{R}^3$)**.** $G = SO(3)$, $X = \mathbb{R}^3$, $Q \cdot x = Qx$. Axioms are immediate from matrix associativity and $Ix = x$. This action is what the word "rotation" means operationally.

Example 4.6 (Rotations acting on a whole molecule)**.** This is the one we actually use. Let $X = (\mathbb{R}^3)^N = \{(x_1, \dots, x_N)\}$, the set of all configurations of N atoms, and define

$$Q \cdot (x_1, \dots, x_N) := (Qx_1, \dots, Qx_N),$$

i.e. the *same* Q applied to every atom simultaneously. Checking the axioms is componentwise and immediate.

Two things are worth noticing. First, this is a genuinely different action from "rotate each atom by its own rotation" — the latter is an action of the much bigger group $SO(3)^N$, and is not a symmetry of chemistry (it would tear the molecule apart). The physical symmetry is the *diagonal* action of a single $SO(3)$. Second, the action does not change N, the atom types, or the bonds: it moves only positions. Those other data are along for the ride, which is exactly why they end up as $\ell = 0$ features later.

Remark 4.7 (Translations, and why we largely ignore them). The full symmetry of free space is $SE(3)$: rotations *and* translations. Translations are handled by a trick so standard it is easy to miss: the architecture never sees absolute positions, only *relative* vectors $x_{ij} = x_j - x_i$, which satisfy $(x_j + a) - (x_i + a) = x_j - x_i$ and are therefore translation-invariant automatically. Once positions enter only through differences, translation symmetry is exact and free, and the remaining, genuinely difficult symmetry is $SO(3)$. That is why this chapter is about $SO(3)$ and not $SE(3)$; the translational half is discharged by Section 7.4's choice of inputs, and SigmaDock additionally centres each pocket at the origin (Section 9.1).

4.3 Invariance and equivariance, formally

We can now state the property we want a network to have. The subtlety worth slowing down for is that a function has *two* sets — a domain and a codomain — and the group acts on each of them, possibly differently.

Definition 4.8 (Invariance)**.** Let G act on X. A function $f : X \to Y$ is *G-invariant* if

$$f(g \cdot x) \;=\; f(x) \qquad \text{for all } g \in G,\ x \in X.$$

No action on Y is needed or used.

Definition 4.9 (Equivariance)**.** Let G act on X *and* on Y (the two actions may be different; we write both as $\cdot$). A function $f : X \to Y$ is *G-equivariant* if

$$f(g \cdot x) \;=\; g \cdot f(x) \qquad \text{for all } g \in G,\ x \in X.$$

Read the equation slowly, because both sides describe a two-step recipe and the content of the definition is that the two recipes agree:

Left-hand side	first *transform the input*, then apply the model
Right-hand side	first apply the model, then *transform the output*

Equivalently, the following square commutes:

$$\begin{array}{ccc} X & \xrightarrow{f} & Y \\ \big\downarrow{\scriptstyle g\cdot} & & \big\downarrow{\scriptstyle g\cdot} \\ X & \xrightarrow{f} & Y \end{array}$$

"Commutes" means: both paths from the top-left to the bottom-right give the same answer. Go right then down, or down then right — same result.

Remark 4.10 (Invariance is a special case of equivariance). Take the action on Y to be trivial, $g \cdot y := y$ for all g. Then Theorem 4.9 reads $f(g \cdot x) = f(x)$, which is Theorem 4.8. So there is really only one concept, applied with two different choices of what the group does to the output. This is worth internalising, because it is why the same representation-theoretic machinery handles both, and why $\ell = 0$ features (Theorem 4.19) are exactly the invariant ones.

Example 4.11 (Distance is invariant, the vector is equivariant)**.** Let $X = (\mathbb{R}^3)^2$ with the diagonal action of Theorem 4.6.

- $f(x_1, x_2) = \|x_2 - x_1\| \in \mathbb{R}$. Then $f(Qx_1, Qx_2) = \|Qx_2 - Qx_1\| = \|Q(x_2 - x_1)\| = \|x_2 - x_1\| = f(x_1, x_2)$, using Theorem 3.16. **Invariant.**
- $f(x_1, x_2) = x_2 - x_1 \in \mathbb{R}^3$, with $Y = \mathbb{R}^3$ carrying the action $Q \cdot y = Qy$. Then $f(Qx_1, Qx_2) = Qx_2 - Qx_1 = Q(x_2 - x_1) = Q \cdot f(x_1, x_2)$. **Equivariant.**

The second computation used nothing but linearity of Q. That is the prototype of every equivariance proof in this document: push the group element through the formula and check it comes out the other side.

Example 4.12 (A function that is neither)**.** Let $f(x_1, x_2) = (x_2 - x_1)_z$, the z-component of the relative vector. Rotating by a Q that tilts the z-axis changes this number, so f is not invariant; and $\mathbb{R}$ carries no rotation action making it equivariant either. Reading off a fixed Cartesian component is exactly the kind of innocuous-looking operation that silently breaks equivariance, and it is worth having a name for the failure: it *privileges a coordinate axis*, and there is no privileged axis.

Proposition 4.13 (Compositions inherit equivariance)**.** *If $f : X \to Y$ and $h : Y \to Z$ are both G-equivariant, so is $h \circ f : X \to Z$. If f is equivariant and h is invariant, then $h \circ f$ is invariant.*

Proof. $(h \circ f)(g \cdot x) = h\big(f(g \cdot x)\big) = h\big(g \cdot f(x)\big) = g \cdot h\big(f(x)\big)$, using equivariance of f then of h. For the second claim, replace the last step by invariance of h. □

Example 4.14 (Componentwise ReLU is not equivariant)**.** This is the counterexample promised in Section 4.1, and it is worth doing with actual numbers because it is the single most common way a well-intentioned architecture silently loses equivariance.

Let $f : \mathbb{R}^3 \to \mathbb{R}^3$ be $f(v) = \mathrm{ReLU}(v) = (\max(v_1, 0), \max(v_2, 0), \max(v_3, 0))$, with $SO(3)$ acting by $Q \cdot v = Qv$ on both sides. Take

$$v = \begin{pmatrix} 1 \\ -1 \\ 0 \end{pmatrix}, \qquad Q = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad (\text{a } +90^\circ \text{ rotation about the } z\text{-axis}).$$

Then $Qv = (1, 1, 0)^\top$, so

$$f(Q \cdot v) = \mathrm{ReLU}\begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix}, \qquad Q \cdot f(v) = Q\,\mathrm{ReLU}\begin{pmatrix} 1 \\ -1 \\ 0 \end{pmatrix} = Q\begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}.$$

These differ. Equivariance fails, and it fails badly — the two answers are not even close.

Why it fails is more instructive than *that* it fails. ReLU acts on each Cartesian component separately, so it treats the coordinate axes as meaningful. But the decomposition of a vector into (v_1, v_2, v_3) is an artefact of the frame we chose; a rotation mixes those components, and any operation that treats them as independent is comparing things that are not comparable. This is the same failure mode as Theorem 4.11's "read off the z-component", in a different costume.

The repair, developed in Section 4.10, is to apply nonlinearities only to quantities that carry no direction — e.g. to $\|v\|$, which is invariant, and then rescale v by the result: $v \mapsto \sigma(\|v\|)\, v$ is equivariant, because $\sigma(\|Qv\|)\, Qv = \sigma(\|v\|)\, Qv = Q\big(\sigma(\|v\|)v\big)$.

Theorem 4.13 is small but it is the reason the whole enterprise is feasible: we do not have to verify equivariance of a 20-layer network as one monolithic claim. We verify it *once per layer*

type, and composition does the rest. Every architectural decision in Chapter 8 is, in the end, a decision about which layer types are on the permitted list.

Why this matters for SigmaFlow

Equation (7.1) says the network maps a pose to a pair (ω, v) of 3-vectors per fragment. In the language just introduced: the domain carries the diagonal $SO(3)$ action on atom positions, the codomain carries the action $(\omega, v) \mapsto (Q\omega, Qv)$, and the network must be equivariant between them. Every layer we are allowed to use is a layer for which we can prove that statement.

4.4 Why representation theory, and not something simpler

We now have the goal (Theorem 4.9) but not yet the means. Here is the gap.

To say that a network layer is equivariant, we must be able to say how the group acts on its *inputs* and on its *outputs*. For the very first layer, the input is a set of positions, and we know the action: $x \mapsto Qx$. For the very last layer, the output is (ω, v), and we know that action too. But in between sit *hidden features* — vectors of numbers with no a priori geometric meaning, produced by the previous layer. Nothing tells us how a rotation acts on those.

And yet we must decide, because Theorem 4.9 is meaningless without an action on both sides. The resolution is to turn this from a discovery problem into a *design* problem:

> We will not *find out* how hidden features transform. We will *declare* how they transform, choosing the rule as part of the architecture, and then build only layers that respect the declaration.

A choice of "how a vector space of features transforms under G" is precisely what representation theory formalises. Two constraints narrow the choice almost completely:

1. **The rule must be linear.** If rotating the input scrambled features nonlinearly, no linear layer could ever commute with it. Linearity is what makes the bookkeeping tractable, and it costs nothing, because all the actions we actually need are linear.
2. **The rule must respect composition.** Applying the rule for Q_1 after the rule for Q_2 must equal the rule for $Q_1 Q_2$ — otherwise it is not consistent with how rotations compose, i.e. with axiom (A2) of Theorem 4.2.

A map assigning to each group element a linear transformation, compatibly with composition, is exactly Theorem 4.15 below. Representation theory then answers the follow-up question that makes the whole design finite: *which such rules are there?* For $SO(3)$ the answer is remarkably clean — a single infinite list indexed by $\ell = 0, 1, 2, \ldots$, from which every other rule is assembled (Theorem 4.18). Designing an equivariant architecture then reduces to choosing how many features of each type ℓ to carry, and using only operations that map declared types to declared types.

4.5 Representations of $SO(3)$

Motivation. A scalar (say, an atom's partial charge) does not change when the whole molecule is rotated: it is the same number before and after. A 3-vector attached to an atom (say, the direction to its nearest neighbour) *does* change: rotating the molecule by Q rotates the vector by Q too. Both are examples of the same general phenomenon — a feature associated with a rotation-dependent transformation rule — and we now formalise the full family of such transformation rules that $SO(3)$ admits.

Definition 4.15 (Representation)**.** A (finite-dimensional, real) *representation* of $SO(3)$ is a smooth map $D : SO(3) \to GL(V)$, for some real vector space V, satisfying $D(Q_1Q_2) = D(Q_1)D(Q_2)$ for all $Q_1, Q_2 \in SO(3)$ (a group homomorphism into the invertible linear maps of V).

The homomorphism property is exactly what makes D a consistent transformation rule: rotating first by Q_2 then by Q_1 has the same effect on a feature as rotating once by the composite Q_1Q_2, matching how physical rotations actually compose.

Example 4.16. $V = \mathbb{R}$, $D(Q) = 1$ for all Q (the *trivial* representation): this is how scalars transform. $V = \mathbb{R}^3$, $D(Q) = Q$ (the *standard* or *defining* representation): this is how ordinary vectors transform. Both are representations of $SO(3)$ in the sense of Theorem 4.15; direct sums (block-diagonal concatenations) and tensor products of representations are again representations (immediate from the definition, checking the homomorphism property component/factor-wise).

Definition 4.17 (Irreducible representation)**.** A representation D on V is *irreducible* (an *irrep*) if V has no proper, nonzero subspace $W \subsetneq V$ (other than $\{0\}$ and V itself) with $D(Q)W \subseteq W$ for every $Q \in SO(3)$.

Irreducibility is the representation-theoretic analogue of "cannot be decomposed further": if V *did* have such an invariant subspace W, the action of $SO(3)$ on V would really just be two smaller, independent actions (on W and, in a suitable sense, on a complement), and nothing would be lost by treating them separately. The classification of all irreps of $SO(3)$ is the single structural fact this chapter is building towards, and it is exactly what tells the network designer of Chapter 8 which "types" of feature exist at all.

Theorem 4.18 (Classification of the irreps of $SO(3)$)**.** *The (real, finite-dimensional) irreducible representations of $SO(3)$ are indexed by a non-negative integer $\ell = 0, 1, 2, \ldots$, called the* degree*; the irrep of degree ℓ acts on a vector space $V_\ell \cong \mathbb{R}^{2\ell+1}$, and every finite-dimensional representation of $SO(3)$ decomposes as a direct sum of copies of these.*

We do not prove Theorem 4.18 from scratch (it is a classical result, most naturally proved via the representation theory of the associated complex Lie algebra $\mathfrak{so}(3, \mathbb{C}) \cong \mathfrak{su}(2)$ and its weight theory, requiring substantially more Lie-algebra machinery than the rest of this document uses; see, e.g., Hall, *Lie Groups, Lie Algebras, and Representations*, 2nd ed., 2015, Ch. 4). We instead *construct* the degree-ℓ irrep concretely and explicitly in Section 4.6, by exhibiting V_ℓ as a specific, checkable space of functions on the sphere and verifying irreducibility is unnecessary for our purposes once the construction is in hand — what we need is the transformation rule (4.1), the dimension count $2\ell + 1$, and the tensor-product decomposition of Theorem 4.34, all of which Sections 4.6 and 4.9 derive directly.

We record the two cases familiar from Section 2.5 onward once and for all in the new language.

Example 4.19 ($\ell = 0$ and $\ell = 1$)**.** $V_0 \cong \mathbb{R}$ with $D^0(Q) = 1$: type-0 features are scalars/invariants. $V_1 \cong \mathbb{R}^3$ with $D^1(Q) = Q$ (the standard representation): type-1 features are ordinary 3-vectors. Every feature encountered so far in this document — atomic charges, distances (type 0); relative position vectors, the translational and rotational velocity outputs of (7.1) (type 1) — is one of these two cases; $\ell \geq 2$ is new content, introduced precisely because it lets the network represent *angular* patterns (e.g. "the neighbours lie in a ring around this axis") that neither scalars nor single vectors can express, as Chapter 8 discusses concretely.

4.6 Spherical harmonics

Motivation. We want an explicit, computable realisation of each degree-ℓ irrep — not just the abstract knowledge that it exists and has dimension $2\ell+1$, but an actual formula that can be evaluated on a direction $\hat{x} \in S^2$ and fed into a neural network. Spherical harmonics provide exactly this, and the cleanest way to see why they arise is by analogy with a construction the reader may already know: Fourier series on the circle.

Intuition: from the circle to the sphere. A periodic function on $\mathbb{T}$ (Theorem 3.39) decomposes into Fourier modes $e^{im\phi}$, $m \in \mathbb{Z}$; the mode $e^{im\phi}$ transforms under rotation of the circle by α (i.e. $\phi \mapsto \phi + \alpha$) by an overall phase, $e^{im(\phi+\alpha)} = e^{im\alpha}e^{im\phi}$ — a 1-dimensional representation of the rotation group of the circle, $SO(2)$, one for each integer m. Spherical harmonics are the exact analogue for functions on the sphere S^2 and the rotation group $SO(3)$: instead of a single integer m indexing 1-dimensional pieces, we get a degree ℓ indexing $(2\ell+1)$-dimensional pieces (the "m" index survives too, now ranging over $-\ell, \ldots, \ell$ *within* a fixed ℓ, and mixing among themselves under a rotation, rather than each being separately invariant as on the circle — exactly the passage from the abelian $SO(2)$ to the non-abelian $SO(3)$).

Definition 4.20 (Spherical harmonics via harmonic homogeneous polynomials)**.** For $\ell \geq 0$, let $\mathcal{H}_\ell$ be the space of *harmonic* (satisfying Laplace's equation $\Delta p = 0$) *homogeneous polynomials* of degree ℓ in three real variables (x_1, x_2, x_3). The (real) *spherical harmonics of degree* ℓ are the restrictions of $\mathcal{H}_\ell$ to the unit sphere S^2.

Proposition 4.21 (Dimension count)**.** $\dim \mathcal{H}_\ell = 2\ell + 1$.

Proof. The space of *all* homogeneous degree-ℓ polynomials in 3 variables has dimension $\binom{\ell+2}{2}$ (the number of monomials $x_1^a x_2^b x_3^c$ with $a+b+c = \ell$). The Laplacian Δ maps this space onto the space of homogeneous degree-$(\ell-2)$ polynomials (surjectivity is a short, standard computation: given a target degree-$(\ell-2)$ polynomial q, one directly exhibits a degree-ℓ polynomial p with $\Delta p = q$, e.g. by induction on ℓ using that $\Delta(\|x\|^2 q) = 2(2\ell+1)q + \|x\|^2 \Delta q$ for homogeneous q of degree $\ell-2$, which for $\ell \geq 2$ can be solved for q in terms of a suitable preimage), and its kernel is exactly $\mathcal{H}_\ell$. By rank–nullity,

$$\dim \mathcal{H}_\ell = \binom{\ell+2}{2} - \binom{\ell}{2} = \frac{(\ell+2)(\ell+1)}{2} - \frac{\ell(\ell-1)}{2} = 2\ell + 1.$$

□

Example 4.22 ($\ell = 0, 1$)**.** $\mathcal{H}_0 = \operatorname{span}\{1\}$ (constants; $\Delta 1 = 0$ trivially), restricting to the constant function on S^2, matching $\dim = 1 = 2(0) + 1$. $\mathcal{H}_1 = \operatorname{span}\{x_1, x_2, x_3\}$ (linear functions are automatically harmonic, since Δ of a linear function is 0), restricting to the three coordinate functions on S^2, matching $\dim = 3 = 2(1) + 1$; note that restricted to S^2, these are literally the components of the unit vector $\hat{x}$ itself.

Proposition 4.23 (Transformation rule of spherical harmonics)**.** *Let $Q \in SO(3)$ act on functions $f : S^2 \to \mathbb{R}$ by $(Q \cdot f)(\hat{x}) = f(Q^\top \hat{x})$ (the natural action: the rotated function evaluated at $\hat{x}$ equals the original function evaluated at the pre-rotation direction). Then this action preserves $\mathcal{H}_\ell$ (restricted to S^2) for each ℓ; consequently, in any basis $Y^\ell_{-\ell}, \dots, Y^\ell_\ell$ of degree-ℓ spherical harmonics, there is a matrix $D^\ell(Q) \in \mathbb{R}^{(2\ell+1)\times(2\ell+1)}$ with*

$$\boxed{Y^\ell(Q\hat{x}) \;=\; D^\ell(Q)\, Y^\ell(\hat{x})} \qquad \textit{i.e.} \qquad Y^\ell_m(Q\hat{x}) = \sum_{m'} D^\ell(Q)_{mm'}\, Y^\ell_{m'}(\hat{x}), \tag{4.1}$$

and D^ℓ is a representation of $SO(3)$ in the sense of Theorem 4.15, of dimension $2\ell+1$, matching Theorem 4.18's degree-ℓ irrep. In the standard orthonormal *real basis, $D^\ell(Q)$ is an orthogonal matrix, so $D^\ell(Q)^{-1} = D^\ell(Q)^\top = D^\ell(Q^\top)$.*

Caution 4.24 (Rotating the point versus rotating the function: the inverse). (4.1) is stated with Q applied to the *point* $\hat{x}$. One also frequently meets the companion statement with Q applied to the *function*. The two differ by an inverse, and conflating them is a standard and hard-to-see error. Substituting $\hat{x} \mapsto Q^\top \hat{x}$ into (4.1) gives

$$Y^\ell(\hat{x}) = D^\ell(Q)\, Y^\ell(Q^\top \hat{x}) \qquad \Longleftrightarrow \qquad Y^\ell(Q^\top \hat{x}) \;=\; D^\ell(Q)^{-1}\, Y^\ell(\hat{x}) \;=\; D^\ell(Q^\top)\, Y^\ell(\hat{x}),$$

not $D^\ell(Q)Y^\ell(\hat{x})$. A numerical check settles it immediately: for $\ell = 1$ with $Y^1(\hat{x}) = \hat{x}$, the claim $Y^1(Q^\top \hat{x}) = D^1(Q)Y^1(\hat{x})$ reads $Q^\top \hat{x} = Q\hat{x}$, which holds only for symmetric Q.

The reason the inverse appears is Theorem 4.3: when a group acts on a space of *functions*, the action must be $(Q \cdot f)(\hat{x}) := f(Q^{-1}\hat{x})$. Without the inverse, axiom (A2) of Theorem 4.2 fails — one would get $(Q_1 \cdot (Q_2 \cdot f)) = (Q_2 Q_1) \cdot f$, the composition in the wrong order. The inverse is what restores $(Q_1 Q_2) \cdot f = Q_1 \cdot (Q_2 \cdot f)$.

Convention fixed for this document: we always use the boxed form, $Y^\ell(Q\hat{x}) = D^\ell(Q)Y^\ell(\hat{x})$, i.e. D^ℓ is the matrix that implements a rotation *of the direction argument*. This is the convention of `e3nn` and hence of EquiformerV2, so no translation is needed when the code is read in Chapter 8. It is consistent with $D^1(Q) = Q$ (Theorem 4.19); a document using the opposite convention would have $D^1(Q) = Q^\top$ there, and every subsequent formula would carry a transpose.

Proof. If $p \in \mathcal{H}_\ell$ (a harmonic homogeneous degree-ℓ polynomial on $\mathbb{R}^3$), then $p \circ Q^\top$ (i.e. $x \mapsto p(Q^\top x)$) is again homogeneous of degree ℓ (composing with a linear map preserves homogeneity), and harmonic (the Laplacian is invariant under orthogonal changes of coordinates: $\Delta(p \circ Q^\top) = (\Delta p) \circ Q^\top$ for Q orthogonal, a standard fact provable by the chain rule and $Q^\top Q = I$), so $p \circ Q^\top \in \mathcal{H}_\ell$ too: the action preserves $\mathcal{H}_\ell$. Since the action is linear on the finite-dimensional space $\mathcal{H}_\ell$, expressing $Q \cdot Y^\ell_m$ in the chosen basis gives exactly a matrix $D^\ell(Q)$ as claimed, and $D^\ell(Q_1 Q_2) = D^\ell(Q_1)D^\ell(Q_2)$ follows because the action of Q on functions is itself a group action (i.e. $(Q_1 Q_2) \cdot f = Q_1 \cdot (Q_2 \cdot f)$, a direct check from the definition), which is exactly the homomorphism property required by Theorem 4.15. □

We do not additionally re-derive *irreducibility* of D^ℓ here (it follows from the classical theory cited after Theorem 4.18, and the identification $V_\ell \cong \mathcal{H}_\ell$ is the standard one); what Theorem 4.23 already gives us — an explicit, computable, correctly-transforming basis for every degree ℓ — is everything the rest of this document needs.

Code correspondence

Mathematical object: $Y^\ell : S^2 \to \mathbb{R}^{2\ell+1}$, degree-$\ell$ real spherical harmonics; the equivariant edge embedding $\big(\psi(d_{ij}), Y^\ell(\hat{x}_{ij})\big)_{\ell \le L}$
Implementation: `net/edge_rot_mat.py` (per-edge frame alignment, see Chapter 8) together with the `e3nn` library's spherical-harmonics routines, called from within the $SO(2)$-convolution machinery of Section 8.7 rather than as an explicit, separately-visible tensor-product step (a structural point this document flags precisely, not glossed over, in Section 8.7)
Interpretation: in EquiformerV2, $L = \texttt{lmax} = 3$ by default (Chapter 8), so the code works with degrees $\ell \in \{0, 1, 2, 3\}$, i.e. steerable features of dimension $1 + 3 + 5 + 7 = 16$ per channel.

4.6.1 Explicit low-degree formulas, and the connection to familiar operations

We record the degree $0, 1, 2$ spherical harmonics explicitly (up to the normalisation convention, which varies across sources and is immaterial to every argument in this document, since it only rescales each ℓ-block by a constant that a subsequent learned linear layer can absorb):

$$Y^0 = 1, \qquad Y^1 = (x_1, x_2, x_3) = \hat{x}, \qquad Y^2 \propto \big(x_1x_2,\ x_2x_3,\ 2x_3^2 - x_1^2 - x_2^2,\ x_3x_1,\ x_1^2 - x_2^2\big),$$

the last being (up to normalisation and a choice of basis) the five independent components of the traceless symmetric matrix $\hat{x}\hat{x}^\top - \frac{1}{3}I$, i.e. exactly a (traceless) quadrupole moment of the direction $\hat{x}$. This connects the abstract degree ℓ directly to physical/geometric intuition: $\ell = 0$ answers "is there something here" (an unsigned scalar); $\ell = 1$ answers "in what single direction" (a vector, capable of representing one preferred direction); $\ell = 2$ answers "along what *axis*, ignoring sign" (a nematic/quadrupolar pattern, e.g. distinguishing a planar ring of neighbours in the xy-plane from an isotropic distribution, something no single vector can encode, since the ring itself has no preferred in-plane direction and Y^1 averaged over the ring would cancel to zero while Y^2 would not); higher ℓ resolve correspondingly finer angular structure.

4.7 Wigner-D matrices

Theorem 4.23 already *is* the definition of the Wigner-D matrix: $D^\ell(Q)$ is, by construction, the matrix realising $SO(3)$'s action on degree-ℓ spherical harmonics in a chosen basis. We isolate the properties of D^ℓ that are used repeatedly.

Proposition 4.25 (Basic properties of Wigner-D matrices)**.** *For every ℓ: (i) $D^\ell(I) = I_{2\ell+1}$; (ii) $D^\ell(Q_1Q_2) = D^\ell(Q_1)D^\ell(Q_2)$ (Theorem 4.23); (iii) $D^\ell(Q)$ is orthogonal, $D^\ell(Q)^\top D^\ell(Q) = I$ (since the action of Theorem 4.23 preserves the natural $L^2(S^2)$ inner product, under which a suitable choice of real spherical-harmonic basis is orthonormal); (iv) $D^0(Q) = 1$ for all Q (the trivial representation, as in Theorem 4.19); $D^1(Q) = Q$ itself, in the standard Cartesian basis $Y^1 = \hat{x}$ of Theorem 4.22.*

Proof. (i) and (ii) restate Theorem 4.23. (iii): the action $f \mapsto f \circ Q^\top$ preserves the rotation-invariant surface measure on S^2, hence preserves the $L^2(S^2)$ inner product $\langle f, g\rangle = \int_{S^2} fg\, d\Omega$; choosing Y^ℓ_m orthonormal in this inner product (always possible by Gram–Schmidt within the finite-dimensional space $\mathcal{H}_\ell|_{S^2}$) makes $D^\ell(Q)$ an isometry of $\mathbb{R}^{2\ell+1}$ with the standard

inner product, i.e. orthogonal. (iv): $D^0(Q) = 1$ since $Y^0 \equiv 1$ is fixed pointwise by every rotation of its argument; $D^1(Q) = Q$ is immediate from Equation (4.1) with $Y^1(\hat{x}) = \hat{x}$: $Y^1(Q\hat{x}) = Q\hat{x} = Q\,Y^1(\hat{x})$. □

Theorem 4.25(iv) is worth dwelling on: it says that $SO(3)$'s *defining* representation (rotation matrices themselves, which is how we have described $SO(3)$ throughout Chapter 3) is nothing but the degree-1 Wigner matrix. Every subsequent degree $\ell \geq 2$ is a genuinely new object, not reducible to ordinary matrix multiplication by a rotation matrix, and this is exactly why Chapter 8's architecture cannot simply reuse the familiar 3×3 rotation-matrix machinery once $\ell \geq 2$ features are involved — it needs the general $D^\ell(Q)$.

Code correspondence

Mathematical object: $D^\ell(Q) \in \mathbb{R}^{(2\ell+1)\times(2\ell+1)}$, the per-edge rotation applied to steerable features
Implementation: `net/wigner.py` (function assembling D^ℓ block-diagonally from Euler angles, via `e3nn`), called from `net/edge_rot_mat.py`::`RotationToWignerDMatrix`
Tensor shape: one D^ℓ per edge, per degree $\ell \leq L$, assembled into a single block-diagonal matrix of size $\sum_{\ell=0}^{L}(2\ell+1)$ acting on the full multi-degree feature at once
Interpretation: every edge in the molecular graph gets its own per-edge frame (Chapter 8, Section 8.3), built from the edge direction $\hat{x}_{ij}$; the corresponding Wigner-D matrix rotates a neighbour's steerable feature *into* that edge-aligned frame before any further processing, and the inverse rotates the result back — exactly the mechanism, detailed in Chapter 8, by which EquiformerV2 reduces full $SO(3)$-equivariant processing to a computationally cheaper $SO(2)$-equivariant one.

4.8 The anatomy of a steerable feature

We now have the list of available feature *types* ($\ell = 0, 1, 2, \dots$) and, for each, the matrix $D^\ell(Q)$ that says how it rotates. What we do not yet have is a picture of what a feature actually looks like inside a network. This section supplies it, because every shape in Chapter 8 and every tensor in the code is an instance of it.

4.8.1 One feature, four indices

The central object is

$$\boxed{h^{(\ell)}_{i,c,m}}$$

a single real number carrying four indices. Each index means something different, and — this is the point — *they behave completely differently under rotation.* Taking them one at a time:

Index	Range	Meaning	Behaviour under a rotation Q
i	$1, \dots, N$	node (atom)	**unchanged**: rotating the molecule does not renumber atoms
ℓ	$0, 1, \dots, L$	irrep type	**unchanged**: a type-1 feature stays type-1
c	$1, \dots, C_\ell$	channel	**unchanged**: channels never mix under rotation
m	$-\ell, \dots, +\ell$	component within the irrep	**mixed**: this is the only index a rotation touches

Everything about equivariant architectures follows from the last row. A rotation acts on exactly one of the four axes. Concretely, for fixed i, ℓ and c, the $(2\ell+1)$ numbers

$$h_{i,c}^{(\ell)} := \big(h_{i,c,-\ell}^{(\ell)}, \dots, h_{i,c,+\ell}^{(\ell)}\big)^\top \in \mathbb{R}^{2\ell+1}$$

form one *irreducible block*, and it transforms as

$$h_{i,c}^{(\ell)} \longmapsto D^\ell(Q)\, h_{i,c}^{(\ell)} \qquad \text{for every } i \text{ and every } c \text{ separately.} \tag{4.2}$$

The same matrix $D^\ell(Q)$ is applied to every channel, independently. No information passes between channels, between nodes, or between degrees when the molecule is rotated.

4.8.2 Why 16 channels of $\ell = 1$ is not a 48-dimensional vector

This is the distinction worth being pedantic about, because both objects contain the same count of real numbers and they are nonetheless mathematically different.

Suppose $C_1 = 16$ channels of type $\ell = 1$. The total number of stored reals is $16\times(2\cdot1+1) = 48$. Now compare two candidate transformation rules for that block of 48 numbers:

A generic 48-vector	**16 channels of $\ell = 1$**
transforms by *some* 48×48 matrix	transforms by $I_{16} \otimes D^1(Q)$
$48^2 = 2304$ free entries	determined by the single 3×3 matrix Q
any two of the 48 slots may mix	only slots within one channel mix
no notion of "direction"	each channel *is* a direction in $\mathbb{R}^3$

The block-diagonal form is worth writing out. Stack the channels into one column $h_i^{(1)} \in \mathbb{R}^{48}$ in the order $(c=1, m=-1), (c=1, m=0), (c=1, m=+1), (c=2, m=-1), \dots$. Then

$$h_i^{(1)} \longmapsto \big(I_{16} \otimes D^1(Q)\big) h_i^{(1)} = \begin{pmatrix} D^1(Q) & & \\ & \ddots & \\ & & D^1(Q) \end{pmatrix} h_i^{(1)}, \tag{4.3}$$

16 identical 3×3 blocks down the diagonal and zeros elsewhere. The notation $I_C \otimes D^\ell(Q)$ is just a compact name for this block-diagonal matrix: the Kronecker product $A \otimes B$ replaces each entry a_{jk} of A by the block $a_{jk}B$, so $I_{16} \otimes D^1(Q)$ is 16 copies of $D^1(Q)$ on the diagonal. Read it as: "do nothing to the channel index, apply $D^1(Q)$ to the m index".

Caution 4.26 (The ordering convention is a real choice). Whether the 48 numbers are laid out channel-major (c slowest, m fastest, as above, giving $I_C \otimes D^\ell$) or m-major (m slowest, c fastest, giving $D^\ell \otimes I_C$) is an implementation decision, not mathematics. Both describe the same transformation; the matrices differ by a permutation. Getting it wrong produces a tensor of the right *shape* that is silently wrong, which is why Chapter 8 records the actual memory layout used by EquiformerV2 rather than assuming one. Nothing in *this* chapter depends on the choice.

4.8.3 What the channel index is for

A natural objection: "if $\ell = 1$ means 'a vector', why do we need 16 of them? A point in space only has one direction to it."

The answer is that a channel is not a coordinate; it is a *learned feature that happens to be directional*. Consider what a single atom might usefully want to represent:

- the direction towards the densest nearby region of protein,
- the direction along which its ring system extends,
- the direction in which a hydrogen-bond donor points,
- the direction it is currently being pushed by the flow field.

Each of these is a separate piece of information, and each is directional — so each needs its own $\ell = 1$ channel. Sixteen channels means the atom carries sixteen independent directional facts about itself. They are all 3-vectors, they all rotate the same way, and they mean different things.

Where the CNN analogy holds, and where it breaks. In a convolutional network, "16 channels" likewise means 16 independent learned quantities per spatial location, and a 1×1 convolution mixes them freely. That much carries over: a linear layer here may also mix the 16 channels freely, with a learned 16×16 (or $C_{\text{out}} \times 16$) matrix.

The analogy breaks in one specific way. In a CNN the channel index is the *only* feature index, so a linear layer may mix everything with everything. Here there is a second index, m, and mixing across it is forbidden — a layer that combined $h_{c=1,m=0}$ with $h_{c=2,m=+1}$ using arbitrary weights would not commute with $I_C \otimes D^\ell(Q)$ and would destroy equivariance. So:

> **Mix channels freely; never mix m by hand.** Whatever happens to the m index must be done by an operation that representation theory certifies — which, as Section 4.9 shows, essentially means the Clebsch–Gordan product.

That sentence is the informal content of Schur's lemma (Theorem 4.46), which we prove in Section 4.10 once the tensor-product machinery is in place.

4.8.4 A concrete feature specification

Architectures specify their feature content as a list of (multiplicity, degree) pairs, written in `e3nn` notation as e.g.

```
"32x0e + 16x1o + 8x2e".
```

Read: 32 channels of $\ell = 0$, 16 of $\ell = 1$, 8 of $\ell = 2$. (The trailing `e/o` is *parity*, relevant for $O(3)$ rather than $SO(3)$; we discuss it only where it matters, in Theorem 3.42, and it plays no role in this chapter.) The total stored numbers per node are

$$32 \times 1 \;+\; 16 \times 3 \;+\; 8 \times 5 \;=\; 32 + 48 + 40 \;=\; 120,$$

and under a rotation these 120 numbers transform by the block-diagonal matrix

$$\underbrace{I_{32} \otimes D^0(Q)}_{32\times 32,\ =I_{32}} \;\oplus\; \underbrace{I_{16} \otimes D^1(Q)}_{48\times 48} \;\oplus\; \underbrace{I_{8} \otimes D^2(Q)}_{40\times 40}.$$

Note that the 32 scalar channels are genuinely untouched ($D^0(Q) = 1$), which is the precise sense in which $\ell = 0$ features are the invariant ones.

Why this matters for SigmaFlow

Two things you will need when reading code.
First, *shapes*. A node-feature tensor in EquiformerV2 is not `[N, F]` for a flat feature count F; it is indexed by node, channel and m, with the degrees stacked. When Chapter 8 reports a shape like `[N, (L+1)², C]`, the middle axis is the m-index of *all* degrees concatenated — because $\sum_{\ell=0}^{L}(2\ell+1) = (L+1)^2$. Recognising which axis is m and which is c is the difference between reading the code and guessing at it.
Second, *where the output comes from*. SigmaFlow's network must emit $\omega, v \in \mathbb{R}^3$ per fragment. Those are $\ell = 1$ objects. So the final readout takes $\ell = 1$ channels and reduces them to one channel — it cannot manufacture them from $\ell = 0$ scalars, and it cannot read them off $\ell = 2$ features without a Clebsch–Gordan step. The architecture's `lmax` must therefore be at least 1, and the reason is this paragraph, not a hyperparameter sweep.

4.9 Tensor products and Clebsch–Gordan coefficients

Motivation. Message passing (Section 7.4) needs to *combine* two features — e.g. a neighbour's current steerable feature $h_j^{\ell_1}$ and the spherical-harmonic embedding $Y^{\ell_2}(\hat{x}_{ij})$ of the edge direction — into a new feature. For ordinary (type-0, scalar) features, "combine" just means multiply. For steerable features of possibly different, positive degree, an arbitrary bilinear combination will *not* in general transform correctly under rotation; we need the *specific* bilinear combinations that do.

4.9.1 Starting from two ordinary vectors

Before any ℓ's, take the most familiar case: two ordinary 3-vectors $u, v \in \mathbb{R}^3$ attached to an atom. Under a global rotation they transform as $u \mapsto Qu$ and $v \mapsto Qv$. We want to build something new out of them. The question is what is available.

Two products we already know. There are two standard ways to multiply vectors in $\mathbb{R}^3$, and both turn out to be equivariant — for reasons we can check in one line each.

Dot product. $u^\top v \in \mathbb{R}$, a single number. Under rotation,

$$(Qu)^\top (Qv) \;=\; u^\top Q^\top Q v \;=\; u^\top v,$$

using $Q^\top Q = I$. The output does not change: this is an **invariant**, i.e. an $\ell = 0$ output. One number.

Cross product. $u \times v \in \mathbb{R}^3$. Under rotation, by Theorem 3.16,

$$(Qu) \times (Qv) \;=\; Q(u \times v).$$

The output rotates like a vector: an $\ell = 1$ output. Three numbers.

The question that forces tensor products. We have made 1 number and 3 numbers out of a pair of vectors. But a pair of vectors is $3 + 3 = 6$ input numbers, and *all products* $u_i v_j$ number $3 \times 3 = 9$. Neither the dot product nor the cross product uses all nine; together they account for $1 + 3 = 4$. So:

Where are the other five?

Answering that question is the whole content of this section, and the answer — an object with 5 components, which is $2\ell + 1$ for $\ell = 2$ — is where higher-degree features come from. They are not an exotic addition to the theory; they are the part of an ordinary product of two vectors that dot and cross throw away.

4.9.2 The outer product and how it rotates

Definition 4.27 (Outer product)**.** For $u, v \in \mathbb{R}^3$, their *outer product* is the 3×3 matrix

$$T \;=\; u \otimes v \;:=\; uv^\top, \qquad T_{jk} = u_j v_k.$$

As a list of numbers it has 9 components: every product of one component of u with one component of v. It retains strictly more information than $u^\top v$ or $u \times v$ — indeed, it retains everything about the pair that is bilinear.

Proposition 4.28 (Transformation of the outer product)**.** *If $u \mapsto Qu$ and $v \mapsto Qv$, then*

$$T \;=\; uv^\top \;\longmapsto\; (Qu)(Qv)^\top \;=\; Q\,uv^\top\,Q^\top \;=\; QTQ^\top.$$

Proof. $(Qu)(Qv)^\top = Qu\,v^\top Q^\top$, since $(Qv)^\top = v^\top Q^\top$. □

So the outer product carries the rotation on *both sides*: $T \mapsto QTQ^\top$. Written with indices, using the summation convention, $T'_{jk} = Q_{ja} Q_{kb} T_{ab}$ — two copies of Q, one per index. Viewing T instead as a single column of 9 numbers, this reads

$$\mathrm{vec}(T) \;\longmapsto\; (Q \otimes Q)\,\mathrm{vec}(T),$$

where $Q \otimes Q$ is the 9×9 Kronecker product. That 9×9 matrix is what Theorem 4.30 below calls the tensor product representation $D^1 \otimes D^1$.

Caution 4.29 (Three different "products", kept apart).

Operation	Output	Dimension	Rotation rule
$u^\top v$ (dot)	scalar	1	unchanged
$u \times v$ (cross)	vector	3	$\mapsto Q(u \times v)$
$u \otimes v$ (outer/tensor)	matrix	9	$\mapsto Q(uv^\top)Q^\top$

The tensor product is the only one of the three that loses nothing; the other two are what you get after discarding part of it. That is the sense in which the tensor product is the *primitive* operation and dot/cross are derived.

Definition 4.30 (Tensor product representation)**.** Given representations D^{ℓ_1} on V_{ℓ_1} and D^{ℓ_2} on V_{ℓ_2}, the *tensor product representation* acts on $V_{\ell_1} \otimes V_{\ell_2}$ by $(D^{\ell_1} \otimes D^{\ell_2})(Q)(u \otimes v) = D^{\ell_1}(Q)u \otimes D^{\ell_2}(Q)v$. Its dimension is $(2\ell_1 + 1)(2\ell_2 + 1)$.

Why this is a representation. The homomorphism property is inherited factor-wise: $(D^{\ell_1} \otimes D^{\ell_2})(Q_1Q_2)(u \otimes v) = D^{\ell_1}(Q_1Q_2)u \otimes D^{\ell_2}(Q_1Q_2)v = D^{\ell_1}(Q_1)D^{\ell_1}(Q_2)u \otimes D^{\ell_2}(Q_1)D^{\ell_2}(Q_2)v$, which is $(D^{\ell_1} \otimes D^{\ell_2})(Q_1)$ applied to $(D^{\ell_1} \otimes D^{\ell_2})(Q_2)(u \otimes v)$. □

4.9.3 Decomposing $1 \otimes 1$ completely

We now answer the "where are the other five?" question in full. The tool is a decomposition of 3×3 matrices that is pure linear algebra, and which turns out to be exactly the representation-theoretic decomposition.

Step 1: split any matrix into three pieces. For *any* $T \in \mathbb{R}^{3\times3}$ write

$$T \;=\; \underbrace{\frac{\operatorname{tr} T}{3} I}_{S} \;+\; \underbrace{\frac{T - T^\top}{2}}_{A} \;+\; \underbrace{\Big(\frac{T + T^\top}{2} - \frac{\operatorname{tr} T}{3} I\Big)}_{Z}. \tag{4.4}$$

That this sums back to T is immediate: the two halves $\frac{T-T^\top}{2} + \frac{T+T^\top}{2}$ already give T, and the $\frac{\operatorname{tr} T}{3} I$ terms cancel. The three pieces are the *isotropic* part, the *antisymmetric* part, and the *symmetric traceless* part.

Step 2: count dimensions.

Piece	Constraint	Free parameters	Count
$S = \lambda I$	one scalar λ	λ	1
A, antisymmetric	$A^\top = -A$	strict upper triangle	3
Z, symmetric traceless	$Z^\top = Z$, $\operatorname{tr} Z = 0$	$\frac{3\cdot4}{2} - 1$	5
			9

The symmetric 3×3 matrices form a space of dimension $\binom{3+1}{2} = 6$; imposing the single linear condition $\operatorname{tr} Z = 0$ removes one, leaving $6 - 1 = 5$. And $1 + 3 + 5 = 9 = \dim(\mathbb{R}^{3\times3})$: the split (4.4) is a direct-sum decomposition of the whole space, with no overlap and nothing missing.

Step 3: each piece is closed under $T \mapsto QTQ^\top$. This is the crucial step — it is what makes the split *representation-theoretic* rather than merely algebraic.

Proposition 4.31 (The three pieces are invariant subspaces)**.** *Let $T \mapsto T' = QTQ^\top$ for $Q \in SO(3)$. Then*

(i) $\operatorname{tr} T' = \operatorname{tr} T$*; hence the isotropic part maps to the isotropic part, and its single parameter is unchanged.*

(ii) If $A^\top = -A$ then $(QAQ^\top)^\top = -QAQ^\top$: antisymmetric matrices map to antisymmetric matrices.

(iii) If $Z^\top = Z$ and $\operatorname{tr} Z = 0$ then $QZQ^\top$ is again symmetric and traceless.

Proof. (i) $\operatorname{tr}(QTQ^\top) = \operatorname{tr}(TQ^\top Q) = \operatorname{tr} T$ by cyclicity of the trace and $Q^\top Q = I$. (ii) $(QAQ^\top)^\top = QA^\top Q^\top = -QAQ^\top$. (iii) symmetry is the same computation with $+$; tracelessness is (i). □

So the 9-dimensional space splits into three subspaces of dimensions $1, 3, 5$, each of which the rotation action maps into itself. By Theorem 4.17, this says the 9-dimensional representation is *reducible* — and it exhibits the pieces explicitly.

Step 4: identify each piece as a known irrep. *The isotropic part is $\ell = 0$.* Its content is the single number $\operatorname{tr} T$, and by (i) that number is unchanged by rotation. An invariant one-dimensional feature is precisely the trivial representation (Theorem 4.19), i.e. $\ell = 0$. For $T = uv^\top$ we have $\operatorname{tr} T = \sum_j u_j v_j = u^\top v$: **the isotropic part is the dot product**, recovered exactly.

The antisymmetric part is $\ell = 1$. A 3×3 antisymmetric matrix is determined by three numbers, and Theorem 3.19 gives the canonical identification: $A = \widehat{a}$ for a unique $a = A^\vee \in \mathbb{R}^3$. The question is whether a transforms as a vector. It does, and the reason is the conjugation identity already proved in Section 3.3.2:

$$Q\,\widehat{a}\,Q^\top \;=\; \widehat{Qa} \qquad \Longrightarrow \qquad \big(QAQ^\top\big)^\vee \;=\; Q\,A^\vee.$$

That is exactly the $\ell = 1$ transformation rule $a \mapsto Qa$. So the antisymmetric part *is* a vector in disguise, and the disguise is the hat map.

For $T = uv^\top$ we can be fully explicit. With our hat/vee convention (Theorem 3.19),

$$\big(T - T^\top\big)^\vee \;=\; -(u \times v), \qquad \text{so} \qquad A^\vee = \Big(\frac{T - T^\top}{2}\Big)^\vee = -\tfrac{1}{2}\,(u \times v). \tag{4.5}$$

The antisymmetric part is the cross product, up to the factor $-\frac{1}{2}$.

Caution 4.32 (The sign in (4.5) is convention-dependent). The minus sign is not a typo and not universal: it is forced by the vee convention of Theorem 3.19, under which $\widehat{\omega} v = \omega \times v$. Checking one component settles it: with $T = uv^\top$, the $(3,2)$ entry of $T - T^\top$ is $u_3 v_2 - u_2 v_3 = -(u \times v)_1$, and $(\cdot)^\vee$ reads that entry off as the first component. Texts that define $T = vu^\top$, or that use the opposite vee sign, obtain $+\frac{1}{2}$. The *content* — that the antisymmetric part is the cross product up to a nonzero constant — is convention-independent; only the constant is not. Since every use in this document passes through a learned weight (Section 4.9.8), the constant is absorbed and never matters numerically; it matters only if one tries to compare formulas across sources.

The symmetric traceless part is $\ell = 2$. This is the five-dimensional piece we were missing. Three arguments identify it:

(a) Its dimension is $5 = 2 \cdot 2 + 1$, matching $\ell = 2$ and no other irrep.
(b) It is invariant under the action (Theorem 4.31(iii)), so it is *some* representation of dimension 5.
(c) By Theorem 4.18, every representation is a sum of irreps; the only ways to build dimension 5 from $\{1, 3, 5, 7, \dots\}$ are 5, or $3+1+1$, or 1×5. One checks it is not further reducible — equivalently, it contains no rotation-invariant direction — so it is the single irrep V_2.

Intuitively, a symmetric traceless matrix encodes an *axis with a magnitude but no arrow*: its eigenvectors give preferred directions, but Z and the "flipped" configuration are not distinguished the way a vector distinguishes a from $-a$. This is exactly the kind of object

needed to express "the neighbours lie in a plane" or "this bond is aligned with that ring" — statements about orientation that are not statements about a direction. Physically, the moment-of-inertia tensor and the quadrupole moment are $\ell = 2$ objects for the same reason.

Conclusion. Putting the four steps together,

$$\boxed{\underbrace{V_1 \otimes V_1}_{\dim 9} \cong \underbrace{V_0}_{\dim 1} \oplus \underbrace{V_1}_{\dim 3} \oplus \underbrace{V_2}_{\dim 5}} \qquad \text{i.e.} \qquad 1 \otimes 1 = 0 \oplus 1 \oplus 2, \tag{4.6}$$

and the three summands are, concretely, the dot product, the cross product, and the symmetric traceless part. The five "missing" numbers from Section 4.9.1 are the $\ell = 2$ component.

Remark 4.33 (This is what "reducible" means, made concrete). Theorem 4.17 defined irreducibility abstractly: no proper nonzero invariant subspace. We have now *exhibited* three such subspaces inside $V_1 \otimes V_1$. The tensor product representation is therefore reducible, and the Clebsch–Gordan decomposition is nothing more mysterious than the systematic version of the split (4.4): it names the invariant subspaces and supplies the change of basis onto them. Everything below is this example, generalised.

This is again a representation of $SO(3)$ (immediate check of the homomorphism property), but — for $\ell_1, \ell_2 \geq 1$ — it is generally *not* irreducible: $\dim(V_{\ell_1} \otimes V_{\ell_2}) = (2\ell_1 + 1)(2\ell_2 + 1)$, which for e.g. $\ell_1 = \ell_2 = 1$ is 9, not matching any single irrep dimension $2\ell + 1$ (odd numbers $1, 3, 5, 7, \ldots$, and $9 = 2(4) + 1$ *is* odd, but we will see explicitly that the 9-dimensional space splits into *three* pieces, of dimensions $1 + 3 + 5 = 9$, not one piece of dimension 9). By Theorem 4.18, every representation — including this one — decomposes as a direct sum of irreps; the following theorem says precisely which ones appear.

Theorem 4.34 (Clebsch–Gordan decomposition)**.**

$$V_{\ell_1} \otimes V_{\ell_2} \cong \bigoplus_{\ell = |\ell_1 - \ell_2|}^{\ell_1 + \ell_2} V_\ell .$$

Proof sketch, and a dimension check. We do not derive Theorem 4.34 from first principles (the standard proof uses the weight/character theory of $\mathfrak{so}(3, \mathbb{C}) \cong \mathfrak{su}(2)$ cited after Theorem 4.18: the weights of $V_{\ell_1} \otimes V_{\ell_2}$ are all sums $m_1 + m_2$ with $m_i \in \{-\ell_i, \ldots, \ell_i\}$, and grouping these sums by which range of consecutive integers they fill out recovers exactly the stated range of ℓ; see Hall (2015), Thm. 4.44 for $\mathfrak{su}(2)$, transported to $SO(3)$ via the double cover). We record instead the dimension identity that this decomposition must, and does, satisfy, as a consistency check the reader can verify unaided:

$$\sum_{\ell = |\ell_1 - \ell_2|}^{\ell_1 + \ell_2} (2\ell + 1) = (2\ell_1 + 1)(2\ell_2 + 1),$$

provable by writing the left side as an arithmetic series (first term $2|\ell_1 - \ell_2| + 1$, last term $2(\ell_1 + \ell_2) + 1$, number of terms $\ell_1 + \ell_2 - |\ell_1 - \ell_2| + 1$, which for $\ell_1 \geq \ell_2$ equals $2\ell_2 + 1$) and simplifying; for $\ell_1 = \ell_2 = 1$ this gives $1 + 3 + 5 = 9 = 3 \times 3$, confirming the example above. □

4.9.4 The selection rule as a type system

Theorem 4.34 deserves to be read not as a formula but as a *rule about what is allowed*. It says: if you combine a feature of type ℓ_1 with a feature of type ℓ_2, the result can contain components of type ℓ only for

$$|\ell_1 - \ell_2| \;\le\; \ell \;\le\; \ell_1 + \ell_2,$$

and each such ℓ appears exactly once. Every other type is simply unavailable: there is no rotation-respecting bilinear map producing it.

$\ell_1 \otimes \ell_2$	allowed ℓ	dimension check	note
$0 \otimes 0$	0	$1 \cdot 1 = 1$	scalar times scalar
$0 \otimes \ell$	ℓ only	$1 \cdot (2\ell+1) = 2\ell+1$	scaling by an invariant
$1 \otimes 1$	$0, 1, 2$	$3 \cdot 3 = 1+3+5 = 9$	Section 4.9.3
$1 \otimes 2$	$1, 2, 3$	$3 \cdot 5 = 3+5+7 = 15$	
$2 \otimes 2$	$0, 1, 2, 3, 4$	$5 \cdot 5 = 1+3+5+7+9 = 25$	
$1 \otimes 3$	$2, 3, 4$	$3 \cdot 7 = 5+7+9 = 21$	note $\ell = 0, 1$ *not* allowed

The row $1 \otimes 3$ is instructive: the lower limit $|\ell_1 - \ell_2| = 2$ excludes $\ell = 0$ and $\ell = 1$ entirely. You cannot make a scalar out of a vector and an $\ell = 3$ feature. Not "it is hard"; there is no such bilinear map.

A useful way to think about it

Representation theory here behaves like a **type system** for geometric features. Each feature carries a type ℓ; the tensor product is the only multiplication; and the selection rule is the typing rule stating which output types a given pair of input types can produce. A layer that claims to produce an ℓ outside the allowed range is not inaccurate — it is ill-typed, and cannot be equivariant.

Two consequences worth noting. *Composability*: because each allowed ℓ appears exactly once, there is no ambiguity about "which" $\ell = 1$ output is meant. *Reachability*: starting from $\ell = 0$ and $\ell = 1$ features only, repeated tensor products can reach every ℓ — e.g. $1 \otimes 1 \ni 2$, then $1 \otimes 2 \ni 3$, and so on — which is why an architecture with a finite `lmax` is a genuine modelling choice rather than a mathematical necessity.

4.9.5 Clebsch–Gordan coefficients are a change of basis

We have established *that* $V_{\ell_1} \otimes V_{\ell_2}$ splits. What remains is to write the splitting down in coordinates, and that is all the Clebsch–Gordan coefficients are. There is nothing else in them.

Two bases for one space. The space $V_{\ell_1} \otimes V_{\ell_2}$ has dimension $(2\ell_1+1)(2\ell_2+1)$, and we have two natural bases for it.

The product basis. Take a basis $\{e^{\ell_1}_{m_1}\}$ of V_{ℓ_1} and $\{e^{\ell_2}_{m_2}\}$ of V_{ℓ_2} and form all pairs:

$$e^{\ell_1}_{m_1} \otimes e^{\ell_2}_{m_2}, \qquad m_1 = -\ell_1, \dots, \ell_1, \quad m_2 = -\ell_2, \dots, \ell_2.$$

This is the basis in which the raw product is easy to compute — coordinates are just the products $u_{m_1} v_{m_2}$ — but in which the decomposition is invisible.

The irreducible (coupled) basis. Alternatively, take a basis adapted to the decomposition Theorem 4.34: for each allowed ℓ, a basis $\{f^\ell_m\}$, $m = -\ell, \dots, \ell$, of the corresponding subspace. Counting, $\sum_\ell (2\ell+1) = (2\ell_1+1)(2\ell_2+1)$: the same dimension, as it must be. This is the basis in which the decomposition is manifest but the product is not.

The coefficients are the transition matrix. Two bases of one finite-dimensional space are related by an invertible change-of-basis matrix. Its entries are the Clebsch–Gordan coefficients:

$$f^\ell_m \;=\; \sum_{m_1,m_2} C^{(\ell,m)}_{(\ell_1,m_1)(\ell_2,m_2)}\, e^{\ell_1}_{m_1} \otimes e^{\ell_2}_{m_2}.$$

That is the entire conceptual content. They are not a physical constant, not something learned, and not something the network chooses: they are the numerical entries of a change of basis, fixed once $SO(3)$ and the basis conventions are fixed.

Remark 4.35 (We have already computed one, by hand). The split (4.4) *is* a change of basis from the product basis of $V_1 \otimes V_1$ (the nine entries $T_{jk} = u_j v_k$) to a basis adapted to $V_0 \oplus V_1 \oplus V_2$ (one trace coordinate, three antisymmetric coordinates, five symmetric-traceless coordinates). The Clebsch–Gordan coefficients for $\ell_1 = \ell_2 = 1$ are precisely the 9×9 matrix implementing that split, written in the m-basis rather than the Cartesian jk-basis.

So the reader who followed Section 4.9.3 has already done a Clebsch–Gordan decomposition; the general theory adds systematic bookkeeping and a convention for the m-labels, not a new idea.

Definition 4.36 (Clebsch–Gordan coefficients and tensor product)**.** The isomorphism of Theorem 4.34, written out in coordinates, defines real numbers $C^{(\ell,m)}_{(\ell_1,m_1)(\ell_2,m_2)}$ (the *Clebsch–Gordan coefficients*, uniquely determined up to an overall sign/phase convention per (ℓ_1,ℓ_2,ℓ) triple by orthonormality of the bases involved) such that, for $u \in V_{\ell_1}$, $v \in V_{\ell_2}$, the degree-ℓ component of their *Clebsch–Gordan tensor product* is

$$(u \otimes_{\mathrm{cg}} v)^\ell_m = \sum_{m_1=-\ell_1}^{\ell_1} \sum_{m_2=-\ell_2}^{\ell_2} C^{(\ell,m)}_{(\ell_1,m_1)(\ell_2,m_2)}\, u^{\ell_1}_{m_1} v^{\ell_2}_{m_2}, \qquad |\ell_1-\ell_2| \le \ell \le \ell_1+\ell_2. \tag{4.7}$$

4.9.6 Reading (4.7) index by index

The formula is short and every symbol in it matters. Unpacked:

Symbol	Lives in	Meaning
$u^{\ell_1}_{m_1}$	$\mathbb{R}$	one component of the first input; the input as a whole is $u^{\ell_1} \in \mathbb{R}^{2\ell_1+1}$
$y^{\ell_2}_{m_2}$	$\mathbb{R}$	one component of the second input, $y^{\ell_2} \in \mathbb{R}^{2\ell_2+1}$
m_1	$\{-\ell_1, \dots, \ell_1\}$	summed over: $2\ell_1 + 1$ values
m_2	$\{-\ell_2, \dots, \ell_2\}$	summed over: $2\ell_2 + 1$ values
ℓ	one allowed value	**fixed**, not summed: we compute one output type at a time
m	$\{-\ell, \dots, \ell\}$	**free**: the formula is evaluated once per m
$C^{(\ell,m)}_{(\ell_1,m_1)(\ell_2,m_2)}$	$\mathbb{R}$	a fixed number, known in advance

So the computation is: *for each* of the $2\ell+1$ output slots m, sum over all $(2\ell_1+1)(2\ell_2+1)$ input pairs, weighting each product $u_{m_1} y_{m_2}$ by the corresponding coefficient. In shapes,

$$\underbrace{u^{\ell_1}}_{[2\ell_1+1]}\ ,\quad \underbrace{y^{\ell_2}}_{[2\ell_2+1]} \xrightarrow{C^{(\ell)} \text{ of shape } [2\ell+1,\ 2\ell_1+1,\ 2\ell_2+1]} \underbrace{z^{\ell}}_{[2\ell+1]}\ .$$

It is a bilinear contraction: linear in u for fixed y, linear in y for fixed u, and the coefficient array is a fixed three-index tensor.

Two points that are easy to misread. First, ℓ does not appear on the right-hand side of (4.7) except inside C — choosing a different allowed ℓ means choosing a different coefficient array and producing a differently-sized output from the *same* inputs. Second, most of the coefficients are zero: $C^{(\ell,m)}_{(\ell_1,m_1)(\ell_2,m_2)}$ vanishes unless $m = m_1 + m_2$, so the double sum really has at most $\min(2\ell_1, 2\ell_2) + 1$ nonzero terms, not $(2\ell_1+1)(2\ell_2+1)$. That sparsity is what makes the operation affordable, and it is the structure EquiformerV2 exploits further (Section 8.4).

Example 4.37 (The simplest case: $0 \otimes \ell \to \ell$)**.** Let $\ell_1 = 0$, so u^0 is a single number c and $m_1 = 0$ only. The selection rule permits $\ell = \ell_2$ alone. The coefficients reduce to $C^{(\ell_2,m)}_{(0,0)(\ell_2,m_2)} = \delta_{mm_2}$, and (4.7) collapses to

$$z^{\ell_2}_m = \sum_{m_2} \delta_{mm_2}\, c\, y^{\ell_2}_{m_2} = c\, y^{\ell_2}_m, \qquad \text{i.e.} \qquad z^{\ell_2} = c\, y^{\ell_2}.$$

Multiplying a steerable feature by an invariant scalar. Equivariance is obvious here — $D^{\ell_2}(Q)(cy) = c\, D^{\ell_2}(Q)y$ — and this is worth noticing because it is the mechanism by which *every* learned scalar weight in the network acts on higher-degree features.

Example 4.38 (The invariant output of $1 \otimes 1$, and why it is the dot product)**.** Take $\ell_1 = \ell_2 = 1$ and $\ell = 0$. The output has $2 \cdot 0 + 1 = 1$ component, and $m = 0$ forces $m_2 = -m_1$ by the $m = m_1 + m_2$ rule, so the double sum has only three surviving terms:

$$z^0_0 = C^{(0,0)}_{(1,-1)(1,1)} u_{-1} y_1 + C^{(0,0)}_{(1,0)(1,0)} u_0 y_0 + C^{(0,0)}_{(1,1)(1,-1)} u_1 y_{-1}.$$

In the standard convention the three coefficients are $\frac{1}{\sqrt{3}}(1, -1, 1)$ up to an overall sign; in the *real* basis used by `e3nn`, where Y^1 is proportional to the Cartesian components, the same object becomes simply

$$z^0 \ \propto\ u^\top y,$$

the ordinary dot product. This is not a coincidence to be memorised: by Section 4.9.3 the $\ell = 0$ piece of $u \otimes y$ *is* the trace of $uy^\top$, which *is* $u^\top y$. The Clebsch–Gordan coefficients are the coordinates of that identification.

The proportionality constant depends on the normalisation convention (Theorem 4.39) and is not worth tracking, because in a network the result is immediately multiplied by a learned weight.

Caution 4.39 (Normalisation, sign, and basis conventions). Three independent conventions affect the numerical values of the coefficients, and sources differ on all three:

(i) **Real versus complex basis.** Physics texts almost always use complex spherical harmonics and complex C's (Condon–Shortley phase); `e3nn`, Equiformer and hence SigmaDock use a *real* orthonormal basis. The two are related by a fixed unitary change of basis per ℓ. The decomposition (4.6) — which ℓ appear, and with what multiplicity — is identical either way; only the numbers differ.

(ii) **Normalisation.** One may normalise so that the coupled basis is orthonormal, or so that the tensor product preserves feature variance (`e3nn` offers several `normalization` modes). This rescales each ℓ-block by a constant.

(iii) **Overall sign per (ℓ_1, ℓ_2, ℓ) triple.** Fixed arbitrarily by a phase convention.

None of these affects anything in this document, for a reason worth stating plainly: every Clebsch–Gordan product in the architecture is immediately followed by a learned linear weight (Section 4.9.8), which absorbs any per-(ℓ_1, ℓ_2, ℓ) constant. Conventions matter when comparing formulas across papers, and when writing a custom kernel; they do not matter for understanding what the layer computes.

4.9.7 Why the result is equivariant

We now prove the property that justifies the whole construction. First the vocabulary.

Definition 4.40 (Intertwiner)**.** Let D_V and D_W be representations of G on V and W. A linear map $\Phi : V \to W$ is an *intertwiner* (or *equivariant linear map*) if

$$\Phi\big(D_V(Q)\, x\big) \;=\; D_W(Q)\, \Phi(x) \qquad \text{for all } Q \in G,\ x \in V,$$

equivalently $\Phi \circ D_V(Q) = D_W(Q) \circ \Phi$ as linear maps.

An intertwiner is exactly Theorem 4.9 specialised to a *linear* map between two spaces each carrying a representation. The name says what it does: it interleaves the two group actions so that they agree.

Proposition 4.41 (The Clebsch–Gordan product is equivariant)**.** *Write $\Phi_\ell : V_{\ell_1} \otimes V_{\ell_2} \to V_\ell$ for the linear map defined by the coefficients, i.e. $\Phi_\ell(u \otimes y)_m = \sum_{m_1, m_2} C^{(\ell, m)}_{(\ell_1, m_1)(\ell_2, m_2)} u_{m_1} y_{m_2}$. Then Φ_ℓ is an intertwiner between the tensor product representation and D^ℓ, and consequently, for all $Q \in SO(3)$,*

$$\big(D^{\ell_1}(Q)u\big) \otimes_{\mathrm{cg}} \big(D^{\ell_2}(Q)y\big) \;=\; D^\ell(Q)\big(u \otimes_{\mathrm{cg}} y\big)$$

in each degree-ℓ component.

Proof. By Theorem 4.34 there is an isomorphism of representations

$$\Psi :\; V_{\ell_1} \otimes V_{\ell_2} \;\xrightarrow{\;\cong\;}\; \bigoplus_{\ell'} V_{\ell'},$$

meaning Ψ is a linear bijection satisfying $\Psi \circ (D^{\ell_1} \otimes D^{\ell_2})(Q) = \big(\bigoplus_{\ell'} D^{\ell'}(Q)\big) \circ \Psi$ for every Q — this identity *is* what "isomorphism of representations" means, and it is the content of the decomposition, not an extra assumption.

Let $\pi_\ell : \bigoplus_{\ell'} V_{\ell'} \to V_\ell$ be the projection onto the ℓ-summand. A projection onto a summand commutes with a block-diagonal action by construction: $\pi_\ell \circ \big(\bigoplus_{\ell'} D^{\ell'}(Q)\big) = D^{\ell}(Q) \circ \pi_\ell$. Now $\Phi_\ell = \pi_\ell \circ \Psi$ by Theorem 4.36 (the coefficients are exactly the matrix entries of this composite in the chosen bases), so for any $w \in V_{\ell_1} \otimes V_{\ell_2}$,

$$\Phi_\ell\big((D^{\ell_1}\otimes D^{\ell_2})(Q)\, w\big) = \pi_\ell \Psi\big((D^{\ell_1}\otimes D^{\ell_2})(Q) w\big) = \pi_\ell\Big(\big(\textstyle\bigoplus_{\ell'} D^{\ell'}(Q)\big)\Psi w\Big) = D^{\ell}(Q)\, \pi_\ell \Psi w = D^{\ell}(Q)\, \Phi_\ell(w).$$

Finally specialise to $w = u \otimes y$ and use $(D^{\ell_1} \otimes D^{\ell_2})(Q)(u \otimes y) = D^{\ell_1}(Q)u \otimes D^{\ell_2}(Q)y$ (Theorem 4.30). □

Remark 4.42 (Sanity check on the example we computed by hand). For $\ell_1 = \ell_2 = 1$ the proposition specialises to three statements we have already verified directly in Section 4.9.3: $\ell = 0$ gives $(Qu)^\top(Qy) = u^\top y$; $\ell = 1$ gives $(Qu) \times (Qy) = Q(u \times y)$; $\ell = 2$ gives that $Z \mapsto QZQ^\top$ preserves the symmetric traceless space (Theorem 4.31(iii)). So the general proof and the concrete example agree, as they must.

Theorem 4.41 is the single fact that makes (4.7) the "$\times$" of equivariant deep learning: it is — up to the choice of basis fixing the specific numbers $C^{(\ell,m)}_{(\ell_1,m_1)(\ell_2,m_2)}$, a choice we do not need to make explicit anywhere in this document, since every use of (4.7) in Chapter 8 treats them as fixed, precomputed constants — the *unique* (up to scale, by Schur's lemma, Theorem 4.46 below) bilinear map $V_{\ell_1} \times V_{\ell_2} \to V_\ell$ respecting the rotation action, and it is therefore the only mechanism by which two steerable features can be combined into a third without breaking equivariance.

4.9.8 From the mathematical product to a network layer

Everything so far is representation theory: given *one* feature of type ℓ_1 and *one* of type ℓ_2, there is an essentially unique equivariant way to produce a type-ℓ output. A neural network layer is more than that, and it is worth being precise about which parts are fixed by mathematics and which are learned. There are three distinct objects, often all called "tensor product":

Object	What it is	Free parameters
Tensor product $u \otimes y$	the raw bilinear object, $(2\ell_1 + 1)(2\ell_2 + 1)$ numbers	none
CG projection $\otimes_{\text{cg}}$	projection onto one allowed ℓ, (4.7)	none — fixed by symmetry
Tensor product *layer*	the above, applied across channels, with learned mixing	the weights W below

Adding channels. Suppose the first input has C_1 channels of type ℓ_1, written $x^{(\ell_1)}_{c_1,m_1}$, and the second has C_2 channels of type ℓ_2. For a chosen output type ℓ and C_{out} output channels,

the layer computes

$$z^{(\ell)}_{c,m} \;=\; \sum_{c_1=1}^{C_1}\sum_{c_2=1}^{C_2} W^{(\ell_1,\ell_2,\ell)}_{c,c_1,c_2} \underbrace{\sum_{m_1,m_2} C^{(\ell,m)}_{(\ell_1,m_1)(\ell_2,m_2)}\, x^{(\ell_1)}_{c_1,m_1}\, y^{(\ell_2)}_{c_2,m_2}}_{\text{the CG product of channel pair } (c_1,c_2)}. \tag{4.8}$$

Read from the inside out: for every pair of input channels, compute the Clebsch–Gordan product (a $(2\ell+1)$-vector); then take a learned linear combination of those C_1C_2 vectors to produce each of the C_{out} output channels.

Note carefully where the two index groups sit. The learned weight W_{c,c_1,c_2} carries **no m-index**. It cannot: an m-dependent weight would mix components within an irrep by hand, which Section 4.8 already flagged as forbidden and which Theorem 4.46 will forbid formally. Conversely the coefficients C carry **no channel index**: they are the same for every channel pair. The two index sets are disjoint, and that separation is exactly what makes (4.8) equivariant.

Proposition 4.43 (The tensor product layer is equivariant)**.** *For any weights W, the map defined by (4.8) satisfies $z^{(\ell)}_c \mapsto D^\ell(Q) z^{(\ell)}_c$ whenever $x^{(\ell_1)}_{c_1} \mapsto D^{\ell_1}(Q) x^{(\ell_1)}_{c_1}$ and $y^{(\ell_2)}_{c_2} \mapsto D^{\ell_2}(Q) y^{(\ell_2)}_{c_2}$ for all channels.*

Proof. Fix c. By Theorem 4.41 each inner bracket transforms by $D^\ell(Q)$. The outer sum is a fixed linear combination with m-independent coefficients, and $D^\ell(Q)$ is linear, so it passes through the sum: $\sum_{c_1c_2} W_{c,c_1,c_2}\, D^\ell(Q)(\cdots) = D^\ell(Q) \sum_{c_1c_2} W_{c,c_1,c_2}(\cdots)$. Note the proof works for *arbitrary* W — equivariance is structural, not a property the training has to preserve. □

Theorem 4.43 is worth pausing on: it says the network is equivariant *at initialisation, for every weight setting, exactly*. Training cannot break it, numerical noise cannot erode it, and no augmentation is needed to encourage it. That is the payoff for all the machinery.

Geometry is fixed; channels are learned

The single most useful summary of this section:

Fixed by symmetry	Learned from data
which ℓ may appear ($\lvert\ell_1-\ell_2\rvert \le \ell \le \ell_1+\ell_2$)	how strongly each allowed ℓ contributes
how the m-components couple (the C's)	how channels mix (the W's)
that the result is equivariant	what the features mean

The network never learns *how to rotate*. It learns *what to compute*, within a space of operations all of which rotate correctly by construction.

One nuance to carry forward: this clean split is exact for the plain tensor-product layer above. EquiformerV2 reorganises the computation (Section 8.4) for efficiency, and in doing so the learned weights appear in a different place — but the principle survives, and Chapter 8 says exactly how.

4.9.9 Where this meets geometry: the edge direction

We can now see why this machinery is the right tool for a molecular graph, which is the point of the whole chapter.

Consider computing a message from atom j to atom i. Two ingredients are available:

- the neighbour's current feature $h_j^{(\ell_1)}$ — *chemistry*: what kind of atom, in what environment, with what learned properties;
- the edge direction $\hat{x}_{ij} = x_{ij}/\|x_{ij}\| \in S^2$ — *geometry*: where the neighbour is.

The direction is not yet a steerable feature, but Section 4.6 made it one: $Y^{\ell_2}(\hat{x}_{ij}) \in \mathbb{R}^{2\ell_2+1}$ transforms by $D^{\ell_2}(Q)$ (Equation (4.1)). So both ingredients are steerable features, and the Clebsch–Gordan product is precisely the operation that combines them:

$$h_j^{(\ell_1)} \otimes_{\text{cg}} Y^{\ell_2}(\hat{x}_{ij}) \longrightarrow \text{message components of every allowed type } \ell.$$

Chemistry meets geometry, and the result is guaranteed to rotate correctly.

Schematically, the pipeline that Chapter 8 will build is:

$$\begin{array}{c} h_j^{(\ell_1)},\ x_{ij} \implies \text{split } x_{ij} \text{ into } \|x_{ij}\| \text{ and } \hat{x}_{ij} \\ \Downarrow \\ Y^{\ell_2}(\hat{x}_{ij}) \text{ (angular, steerable)} \quad \text{and} \quad \text{RBF}(\|x_{ij}\|) \text{ (radial, invariant)} \\ \Downarrow \\ \text{CG product, with weights modulated by the radial features} \\ \Downarrow \\ \text{equivariant message } m_{ij}^{(\ell)} \end{array}$$

Why the radius and the direction are handled separately — and why that separation is forced rather than convenient — is the subject of Section 8.2. For now the essential point is that the box in the middle is (4.8), and we have just built it.

Why this matters for SigmaFlow

Feature shapes. (4.8) is the operation whose tensor shapes you will meet in the code. Recognising which axis is c and which is m (Theorem 4.26) is the difference between reading `so2_ops.py` and guessing.

Why tensor products at all. They are the *only* mechanism by which the network's chemical features learn about the geometry of the pocket. Without them the architecture could pass around scalars and distances, and Theorem 7.14 shows that is provably insufficient to determine a pose.

Why Clebsch–Gordan specifically. Because the output type is not a design choice the network could get wrong: it is determined by symmetry. This is what makes the equivariance of SigmaDock and SigmaFlow a *theorem* about the architecture rather than a hope about the training.

Why none of this changes when diffusion becomes flow matching. Everything in this chapter concerns the *backbone* — how features are represented and combined. The generative engine decides what the network is asked to predict (a score, or a velocity) and how samples are produced (an SDE, or an ODE). It does not touch ℓ, m, the Clebsch–Gordan coefficients, or (4.8). That separation is precisely what makes SigmaFlow a controlled single-variable change relative to SigmaDock, and it is why this chapter is shared, unmodified, by both halves of the document.

Example 4.44 (Familiar special cases)**.** $\ell_1 = \ell_2 = 1$, $\ell = 0$: the degree-0 (invariant) component of $u \otimes_{\text{cg}} v$ for $u, v \in \mathbb{R}^3$ is (up to normalisation) the ordinary dot product $\langle u, v \rangle$ — the unique way to combine two vectors into a rotation-invariant scalar, matching Theorem 7.14's later discussion of what distance-only networks can express. $\ell_1 = \ell_2 = 1$, $\ell = 1$: the degree-1 component is (up to normalisation) the cross product $u \times v$ — matching Theorem 3.20's appearance of the cross product as an $\mathfrak{so}(3)$-valued (equivalently, since $\mathfrak{so}(3) \cong \mathbb{R}^3$, degree-1) object. $\ell_1 = 0$: tensoring with a scalar is just scalar multiplication, $(c \otimes_{\text{cg}} v)^\ell = cv$ (only $\ell = \ell_2$ appears, since $|\ell_1 - \ell_2| = \ell_1 + \ell_2 = \ell_2$ when $\ell_1 = 0$), recovering ordinary scaling of a steerable feature by an invariant weight — the operation that, per Theorem 4.46 below, is the *only* linear operation available on a single feature and hence the mechanism by which every learned scalar weight in Chapter 8's network acts on higher-degree channels.

4.10 Consequences for equivariant linear maps and nonlinearities

4.10.1 The question

We have features organised as irreps. A standard neural network would now flatten everything into one long vector and apply $y = Wh + b$. Suppose a node carries

$$h^{(0)} \in \mathbb{R}^4, \qquad h^{(1)} \in \mathbb{R}^{2\times 3}, \qquad h^{(2)} \in \mathbb{R}^{1\times 5},$$

i.e. four scalar channels, two vector channels, one $\ell = 2$ channel: $4 + 6 + 5 = 15$ numbers in total. The naive layer would learn a 15×15 matrix, 225 parameters, and mix everything with everything.

Which of those 225 parameters may we actually use?

The answer, derived below, is: $16 + 4 + 1 = 21$. The remaining 204 are not merely unhelpful — switching any of them on breaks equivariance. This section explains where the number 21 comes from.

The intuition first. A weight matrix is a fixed array of numbers, chosen once and then held constant while the molecule may be presented in any orientation. Anything that a fixed array can "know" about direction is a lie: it was chosen in *some* frame, and there is no preferred frame. So a legal weight may combine quantities that carry no direction (channels of the same type), but it may not do anything that implicitly names a direction — and mixing an $\ell = 0$ with an $\ell = 1$ component, or one m-slot with another, does exactly that.

4.10.2 The condition a linear layer must satisfy

Make that precise. Let the input transform under a representation ρ_{in} and the output under ρ_{out}, and let W be the layer. Equivariance (Theorem 4.9) demands $W(\rho_{\text{in}}(R)h) = \rho_{\text{out}}(R)(Wh)$ for all h, i.e. as an identity between matrices,

$$\boxed{W\,\rho_{\text{in}}(R) \;=\; \rho_{\text{out}}(R)\,W \qquad \text{for every } R \in SO(3).} \tag{4.9}$$

Read the two sides:

$$\begin{array}{ll} W\rho_{\text{in}}(R)h & \text{rotate the input, } \textit{then} \text{ apply the layer} \\ \rho_{\text{out}}(R)Wh & \text{apply the layer, } \textit{then} \text{ rotate the output} \end{array}$$

Equation (4.9) says these agree for every rotation and every input. This is precisely Theorem 4.40: **an equivariant linear layer is an intertwiner**, and the design problem is to describe the space of intertwiners — because that space *is* the space of admissible weight matrices.

Note what kind of constraint this is. It is not a soft preference; it is a system of linear equations on the entries of W, one for each R. Since $SO(3)$ is 3-dimensional there are infinitely many such equations, and the solution space is correspondingly small.

4.10.3 Three cases, worked by hand

Before invoking any general theorem, let us solve (4.9) in the three cases that matter.

Case 1: scalar → scalar. Here $\rho_{\text{in}} = \rho_{\text{out}} = D^0$, and $D^0(R) = 1$ for every R. So (4.9) reads $W \cdot 1 = 1 \cdot W$, which is *no constraint at all*. Any matrix works. With C_{in} scalar channels in and C_{out} out, the full $C_{\text{out}} \times C_{\text{in}}$ matrix is free. Scalars behave exactly like ordinary neural-network features, which is why $\ell = 0$ channels are the "easy" part of every equivariant architecture.

Case 2: vector → vector. Now $\rho_{\text{in}} = \rho_{\text{out}} = D^1$ and $D^1(R) = R$, so we must find every 3×3 matrix W with

$$WR = RW \qquad \text{for } \textit{every } R \in SO(3).$$

Claim: only $W = \lambda I$. Here is an elementary argument that does not require any representation theory.

Let $R_z(\vartheta)$ be rotation about the z-axis. Commuting with all of these forces W to preserve each eigenspace of R_z; over $\mathbb{R}$ the concrete consequence is that W must map the z-axis to itself and act on the xy-plane as a scaled rotation. So

$$W = \begin{pmatrix} a & -b & 0 \\ b & a & 0 \\ 0 & 0 & c \end{pmatrix}.$$

Now impose commutation with rotation about the x-axis as well. By the same reasoning with the roles of the axes permuted, W must also have that form with respect to x, which forces $c = a$ and $b = 0$. Repeating with the y-axis leaves no freedom: $W = aI$.

The geometric reading is more memorable than the computation. W is a fixed linear map that must look the same from every viewpoint. A map that stretched more along one direction than another would let you *recover* that direction from W — and W is a constant, fixed before we knew the molecule's orientation. The only maps with no preferred direction are uniform scalings.

Case 3: scalar → vector. Now $\rho_{\text{in}} = D^0$, $\rho_{\text{out}} = D^1$, so W is a 3×1 matrix, i.e. a fixed vector $w \in \mathbb{R}^3$, and (4.9) reads

$$w = w \cdot 1 = W\rho_{\text{in}}(R) = \rho_{\text{out}}(R)W = Rw \qquad \text{for every } R \in SO(3).$$

A vector fixed by *every* rotation must be $w = 0$: any nonzero w has some rotation moving it. So the only equivariant linear map from scalars to vectors is the zero map.

Again the intuition is the point: a nonzero w would be a direction manufactured out of a number that carries none. It would single out an axis in space, and nothing in the input justifies that axis.

Caution 4.45 (This is why you cannot "just add a linear layer"). Case 3 generalises: there is no nonzero equivariant linear map between different degrees at all. A network therefore *cannot* create an $\ell = 1$ feature out of $\ell = 0$ features by any linear means, no matter how many layers are stacked. Since the output of SigmaFlow's network is an $\ell = 1$ object (Equation (7.1)), directional information must enter from the geometry — and the only route is the tensor product of Section 4.9. This is the structural reason the architecture cannot be simplified into "scalars plus an MLP".

4.10.4 Schur's lemma

The three cases above are instances of one theorem. Informally:

> **Between irreducible representations there are almost no equivariant linear maps.** If the two irreps are different, there are none but zero. If they are the same, there is only one, up to a scale factor.

That is a strong statement, and its strength is exactly what makes equivariant architecture design tractable: instead of searching a huge space of weight matrices, one is handed a very short list of options.

Proposition 4.46 (Schur's lemma, and the form of equivariant linear maps)**.** *Let $W : V_{\ell_1} \to V_{\ell_2}$ be linear and satisfy $W\,D^{\ell_1}(Q) = D^{\ell_2}(Q)\,W$ for all $Q \in SO(3)$ (an* intertwiner*). If $\ell_1 \neq \ell_2$, then $W = 0$. If $\ell_1 = \ell_2 = \ell$, then $W = cI$ for some scalar $c \in \mathbb{R}$.*

Proof. *Case $\ell_1 \neq \ell_2$.* Consider $\ker W \subseteq V_{\ell_1}$. If $x \in \ker W$ then $W D^{\ell_1}(Q)x = D^{\ell_2}(Q)Wx = 0$, so $D^{\ell_1}(Q)x \in \ker W$ too: the kernel is an invariant subspace. Since V_{ℓ_1} is irreducible, $\ker W$ is $\{0\}$ or all of V_{ℓ_1}. The same argument applied to $\operatorname{im} W \subseteq V_{\ell_2}$ shows the image is $\{0\}$ or all of V_{ℓ_2}. If $W \neq 0$ then $\ker W = \{0\}$ and $\operatorname{im} W = V_{\ell_2}$, i.e. W is an isomorphism — impossible when $\dim V_{\ell_1} = 2\ell_1 + 1 \neq 2\ell_2 + 1 = \dim V_{\ell_2}$. Hence $W = 0$.

Case $\ell_1 = \ell_2 = \ell$. Over $\mathbb{C}$ one argues that an eigenvalue λ of W exists and $W - \lambda I$ is again an intertwiner with nontrivial kernel, hence zero by the first part. Over $\mathbb{R}$ this step is not immediate, because a real matrix need not have a real eigenvalue, and in general the commutant of a real irrep is $\mathbb{R}$, $\mathbb{C}$ or $\mathbb{H}$ (real, complex or quaternionic type). The conclusion $W = cI$ with $c \in \mathbb{R}$ therefore requires knowing that the real irreps V_ℓ of $SO(3)$ are of **real type**, i.e. that their complexifications stay irreducible. That is true, and it is part of the classical classification cited after Theorem 4.18 (Fulton & Harris, *Representation Theory*, 1991, §1.2 and §3; Bröcker & tom Dieck, *Representations of Compact Lie Groups*, 1985, Ch. II). Given real type, the commutant is $\mathbb{R}I$ and the statement follows. □

Caution 4.47 (The real-versus-complex step is not a formality). It is tempting to quote "Schur's lemma: intertwiners between equivalent irreps are multiples of the identity" and move on. That sentence is true over $\mathbb{C}$ and *not* true over $\mathbb{R}$ in general. The standard counterexample is $SO(2)$ acting on $\mathbb{R}^2$ by rotation: this real representation is irreducible (no real line is preserved by all rotations), yet its commutant is two-dimensional — every

rotation matrix commutes with every other, so $W = \left(\begin{smallmatrix} a & -b \\ b & a \end{smallmatrix}\right)$ is an intertwiner for all a, b. That commutant is a copy of $\mathbb{C}$, not $\mathbb{R}$; $SO(2)$'s two-dimensional real irreps are of *complex* type.

So the conclusion we use genuinely depends on a property of $SO(3)$ and would fail for other groups. Since the whole architecture rests on it, it is worth checking rather than assuming. A direct numerical check settles it: solving the linear system (4.9) for $\ell_1 = \ell_2 = 1$ and for $\ell_1 = \ell_2 = 2$ yields a solution space of dimension exactly 1 in each case, spanned by the identity; for $\ell_1 = 1, \ell_2 = 2$ (and conversely) the solution space is $\{0\}$. Those are the four numbers $1, 1, 0, 0$ that Theorem 4.46 predicts.

Proposition 4.48 (Consequence for multi-channel equivariant linear layers)**.** *Let $h = \bigoplus_{\ell,c} h^{\ell,c}$ be a steerable feature with C_ℓ channels of each degree $\ell \le L$ (Theorem 4.54 below), and let $W : \bigoplus_{\ell,c} V_\ell \to \bigoplus_{\ell,c'} V_\ell$ be linear and equivariant (commuting with the diagonal action $\bigoplus_{\ell,c} D^\ell(Q)$ on both sides). Then W acts as $h^{\ell,c} \mapsto \sum_{c'} W^\ell_{cc'} h^{\ell,c'}$ for scalar weights $W^\ell_{cc'} \in \mathbb{R}$: W mixes channels only* within *a fixed degree ℓ, never across degrees.*

Proof. Write W blockwise, indexed by (source degree, source channel) and (target degree, target channel); each block $W_{(\ell,c)\to(\ell',c')} : V_\ell \to V_{\ell'}$ is itself an intertwiner (since W commutes with the full diagonal action, and one may test equivariance against elements supported in a single source/target block by linearity), so Theorem 4.46 applies to each block separately: it vanishes unless $\ell = \ell'$, and is a scalar multiple of the identity when $\ell = \ell'$. Collecting the surviving (same-degree) blocks gives exactly the stated form. □

4.10.5 What the layer actually looks like

Theorem 4.48 is best absorbed in coordinates. With C_{in} input channels and C_{out} output channels of degree ℓ, the layer is

$$y^{(\ell)}_{c,m} \;=\; \sum_{c'=1}^{C_{\text{in}}} A^{(\ell)}_{cc'}\, h^{(\ell)}_{c',m}, \qquad A^{(\ell)} \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}}}. \tag{4.10}$$

The index m appears on both sides untouched: it is a spectator. The learned matrix $A^{(\ell)}$ carries no m-index and acts identically on every one of the $2\ell + 1$ slots.

In Kronecker form, acting on the stacked vector,

$$W^{(\ell)} \;=\; A^{(\ell)} \otimes I_{2\ell+1}. \tag{4.11}$$

Compare this with how the *features* transform, $\rho^{(\ell)}(R) = I_C \otimes D^\ell(R)$ from (4.3). The two commute for a reason that is now visible at a glance:

$$\big(A \otimes I_{2\ell+1}\big)\big(I_{C_{\text{in}}} \otimes D^\ell(R)\big) = A \otimes D^\ell(R) = \big(I_{C_{\text{out}}} \otimes D^\ell(R)\big)\big(A \otimes I_{2\ell+1}\big),$$

using the mixed-product rule $(A \otimes B)(C \otimes D) = (AC) \otimes (BD)$. **Each factor acts on a different index and they simply pass through one another.** The learned part touches only c; the rotation touches only m; neither sees the other.

That single line is, in the end, the whole of Section 4.10.

Two axes, two owners

Axis	Owned by	May be manipulated by
channel c	the network	learned weights, freely
component m	the symmetry	only $D^\ell(R)$ and Clebsch–Gordan

The channel axis carries *learned feature semantics*: what the feature means. The m axis carries *geometric structure*: how the feature points. Keeping them separate is what equivariance amounts to in practice, and mixing them is the single most common way to break it.

4.10.6 Bias terms

A standard linear layer is $y = Wh + b$. We have discussed W; what about b?

The bias is a *constant*, added regardless of the input, so equivariance requires $\rho_{\text{out}}(R)\, b = b$ for every R: the bias must be a fixed point of the output representation.

- $\ell = 0$**:** $D^0(R) = 1$, so the condition reads $b = b$. Any bias is allowed. Scalar channels may have biases exactly as in an ordinary network.
- $\ell \geq 1$**:** the condition reads $D^\ell(R)b = b$ for all R, i.e. b is a rotation-invariant element of V_ℓ. Since V_ℓ is irreducible and nontrivial, the set of such fixed points is $\{0\}$ — otherwise the line $\mathbb{R}b$ would be a proper nonzero invariant subspace, contradicting irreducibility. Hence $b^{(\ell)} = 0$ is forced.

So:

$$b^{(0)} \text{ free}, \qquad b^{(\ell)} = 0 \;\text{ for } \ell \geq 1.$$

The reasoning is the same as Case 3 of Section 4.10.3: a nonzero constant vector would name a direction, and there is no direction to name. Libraries enforce this by construction — `e3nn`'s linear layer accepts a bias only on scalar irreps — and a layer that appeared to have biases on $\ell \geq 1$ would be a bug, not a design choice.

4.10.7 Why linear layers are not enough

Everything so far is linear, and a composition of linear maps is linear. A network built only from (4.10) would be an (equivariant) linear function of its input, which is far too weak: it could not represent the dependence of a docking score on the shape of a pocket any more than a linear regression could.

So we need nonlinearities. And here the difficulty of the equivariant setting reappears in a new place:

> **An ordinary elementwise nonlinearity does not respect the irrep structure.**

Theorem 4.14 already showed this with explicit numbers: for $v = (1, -1, 0)^\top$ and a 90° rotation about z, $\text{ReLU}(Rv) = (1, 1, 0)^\top$ while $R\,\text{ReLU}(v) = (0, 1, 0)^\top$. The reason bears

repeating, because it also tells us what *will* work: ReLU treats the three slots of an $\ell = 1$ feature as three independent numbers, but a rotation mixes them. Any operation that acts slot-by-slot is implicitly asserting that the slots have separate meaning, and they do not.

The repairs all follow one principle:

> **Do the nonlinear thing to something invariant; use the result to scale something equivariant.**

We now examine the three standard realisations of that principle.

4.10.8 Nonlinearity 1: scalars are unrestricted

For $\ell = 0$ features there is no problem at all. Since $D^0(R) = 1$, an $\ell = 0$ feature s satisfies $s \mapsto s$ under rotation, so for any function $\sigma : \mathbb{R} \to \mathbb{R}$,

$$\sigma(s) \;\longmapsto\; \sigma(s).$$

The output is again invariant, hence again a legitimate $\ell = 0$ feature. ReLU, SiLU, GELU, layer normalisation, softmax over channels — all are available on scalar channels, unchanged from ordinary deep learning.

This is more useful than it sounds, because it means the network can perform arbitrary nonlinear computation, *provided* it does so on the invariant part of its state. The remaining two mechanisms are about transporting that nonlinearity back onto the higher-ℓ channels.

4.10.9 Nonlinearity 2: norm nonlinearities

For $\ell \geq 1$ we cannot touch the components. But we can compute an invariant from them.

Proposition 4.49 (The norm of an irrep feature is invariant)**.** *If $D^\ell(R)$ is orthogonal — which it is in the standard real orthonormal basis, by Theorem 4.25 — then $\|D^\ell(R)v\| = \|v\|$ for every $v \in V_\ell$.*

Proof. $\|Dv\|^2 = v^\top D^\top D v = v^\top v = \|v\|^2$. □

So $\|v\|$ is a scalar we are allowed to feed to any nonlinearity. Define

$$f(v) \;=\; \alpha\big(\|v\|\big)\, v, \tag{4.12}$$

for any function $\alpha : \mathbb{R}_{\geq 0} \to \mathbb{R}$ (e.g. $\alpha(r) = \sigma(r)/(r + \varepsilon)$, which turns (4.12) into "rescale v to have length $\sigma(\|v\|)$").

Proposition 4.50 (Norm nonlinearities are equivariant)**.** $f\big(D^\ell(R)v\big) = D^\ell(R)\, f(v)$.

Proof.

$$f\big(D^\ell(R)v\big) = \alpha\big(\|D^\ell(R)v\|\big)\, D^\ell(R)v \overset{\text{(a)}}{=} \alpha\big(\|v\|\big)\, D^\ell(R)v \overset{\text{(b)}}{=} D^\ell(R)\big(\alpha(\|v\|)\, v\big) = D^\ell(R) f(v),$$

where (a) is Theorem 4.49 and (b) is linearity of $D^\ell(R)$ together with the fact that $\alpha(\|v\|)$ is a *scalar*, which therefore commutes with the matrix. □

Remark 4.51 (What norm nonlinearities cannot do). The direction of v is untouched: (4.12) can only rescale, never reorient. In particular $f(v)$ is always parallel to v (or antiparallel, if $\alpha < 0$). A network built only from linear layers and norm nonlinearities can therefore change how strongly a direction is expressed but cannot change *which* direction — reorientation must come from the tensor product. A second, practical drawback is the singularity at $v = 0$, which is why implementations carry an ε and why the derivative near the origin needs care.

4.10.10 Nonlinearity 3: gating

Gating is the variant Equiformer actually relies on, and it is a small generalisation of the previous idea: instead of deriving the scalar from v's own norm, derive it from a *separate scalar channel*, which the network is free to compute however it likes.

The mechanism, in one line. Let $g \in \mathbb{R}$ be an invariant scalar and $v \in V_\ell$ a feature. Then $v \mapsto g\,v$ is equivariant:

$$g\,D^\ell(R)v \;=\; D^\ell(R)\,(g\,v),$$

because a scalar commutes with a matrix. That is the entire argument. Its force comes from where g is allowed to come from: *anywhere*, as long as it is invariant. In particular g may be the output of a deep, thoroughly nonlinear MLP acting on the node's scalar channels.

As a network block. A gated layer is organised as follows. Suppose we want C_ℓ output channels of degree ℓ. The preceding equivariant layer is asked to produce

- C_ℓ channels of degree ℓ: the features to be gated, $h_c^{(\ell)} \in \mathbb{R}^{2\ell+1}$;
- C_ℓ *extra* scalar channels: the gate pre-activations, $s_c \in \mathbb{R}$, one per output channel;
- plus whatever ordinary scalar channels the block also outputs.

Then

$$g_c = \sigma(s_c) \in \mathbb{R}, \qquad h_{c,m}^{(\ell),\mathrm{out}} = g_c\, h_{c,m}^{(\ell)}, \tag{4.13}$$

with σ any scalar nonlinearity (sigmoid, SiLU, ...). Shapes: $s \in \mathbb{R}^{C_\ell}$, $h^{(\ell)} \in \mathbb{R}^{C_\ell \times (2\ell+1)}$, and the product broadcasts g along the m-axis — again, m is a spectator.

Why this is strictly more expressive than a norm nonlinearity. In (4.12) the scaling factor is a function of that one feature's own magnitude. In (4.13) the gate can depend on *all* scalar information available at the node — atom type, local density, time embedding, anything — so the network can learn to suppress or amplify a directional channel based on chemical context rather than only on its own size. Both are equivariant; gating simply has a richer input.

Caution 4.52 (A gate is not an attention weight). Both are invariant scalars multiplying equivariant quantities, so they look alike in a formula. They occupy different roles:

	Gate	Attention weight
Indexed by	channel c (and node i)	*edge* (i, j)
Purpose	modulate feature channels	weight neighbours in a sum
Normalised?	no	yes, across j (softmax)
Appears in	the pointwise nonlinearity	the aggregation step

Keeping the two apart matters when reading Section 8.7, where both occur within a few lines of each other.

4.10.11 The three admissible operations, and what each is for

We can now state the design space completely. An equivariant network is assembled from exactly three kinds of operation, and each does something the others cannot:

Operation	Can do	Cannot do
Equivariant linear (4.10)	mix channels within one ℓ; change channel count	create or mix degrees; introduce nonlinearity
Tensor product (4.8)	couple two degrees; *create* new degrees ℓ within the selection rule	act on a single feature alone
Gate / norm nonlinearity (4.13)	introduce nonlinearity; modulate by context	change ℓ; reorient a feature

Each row is a genuine limitation, and together they are exhaustive: any equivariant layer is built from these. Read the middle column downwards and you get a recipe — couple geometry into features (tensor product), remix the resulting channels (linear), apply context-dependent nonlinearity (gate), repeat. That is, in outline, an Equiformer block.

4.10.12 A small block, end to end

Concretely, take a node carrying 4 scalar channels and 2 vector channels, and build a layer producing 8 scalars and 5 vectors.

Input. $h^{(0)} \in \mathbb{R}^4$, $h^{(1)} \in \mathbb{R}^{2\times 3}$. Total numbers: $4 + 6 = 10$.

Equivariant linear. Two independent matrices,

$$A^{(0)} \in \mathbb{R}^{(8+5)\times 4}, \qquad A^{(1)} \in \mathbb{R}^{5\times 2}.$$

The $\ell = 0$ matrix produces 13 scalar outputs: 8 ordinary scalars plus 5 gate pre-activations, one for each intended vector channel. The $\ell = 1$ matrix produces 5 vector channels by mixing the 2 input vector channels. Parameter count: $13 \cdot 4 + 5 \cdot 2 = 52 + 10 = 62$, plus a bias $b^{(0)} \in \mathbb{R}^{13}$ — and *no* bias on the $\ell = 1$ part.

Note what is absent: there is no matrix from the 4 scalars to the 5 vectors, and none from the 2 vectors to the scalars. By Theorem 4.45 those blocks are forced to zero. Of the $\big(8 \cdot 1 + 5 \cdot 3\big) \times \big(4 \cdot 1 + 2 \cdot 3\big) = 23 \times 10 = 230$ entries a naive dense layer would have, only 62 are admissible.

Gate. Split the 13 scalars into 8 outputs and 5 gates $s \in \mathbb{R}^5$; set $g = \sigma(s) \in \mathbb{R}^5$ and

$$h^{(1),\mathrm{out}}_{c,m} = g_c \, h^{(1)}_{c,m}, \qquad c = 1, \dots, 5, \quad m = 1, 2, 3.$$

Output. 8 scalars and 5 vectors, i.e. $8 + 15 = 23$ numbers, transforming under $I_8 \oplus (I_5 \otimes D^1(R))$.

What is still missing. Nothing in this block created an $\ell = 2$ feature, and nothing could have: linear layers preserve degree and gates preserve degree. Higher degrees enter only through the tensor product with the spherical harmonics of an edge — which is the message-passing step, and the subject of Section 8.1.

4.10.13 Summary in one formula

Collecting (4.3) and (4.11): if a node's feature space is $\bigoplus_\ell \big(\mathbb{R}^{C_\ell} \otimes V_\ell\big)$, then the rotation acts by

$$\rho(R) \;=\; \bigoplus_{\ell} \Big(I_{C_\ell} \otimes D^\ell(R) \Big),$$

and the most general equivariant linear layer is

$$W \;=\; \bigoplus_{\ell} \Big(A^{(\ell)} \otimes I_{2\ell+1} \Big), \qquad A^{(\ell)} \in \mathbb{R}^{C_\ell^{\text{out}} \times C_\ell^{\text{in}}} \text{ arbitrary.}$$

Block-diagonal in ℓ; free in c; identity in m. Everything in this section is a consequence of those three words.

Remark 4.53 (A preview: parity, and $O(3)$ versus $SO(3)$). Implementations label irreps not by ℓ alone but by ℓ together with a *parity*, written `0e`, `1o`, `2e` and so on (`e` = even, `o` = odd). This extends the group from $SO(3)$ to $O(3)$ by including the inversion $x \mapsto -x$: an even feature is unchanged by inversion, an odd one flips sign. Ordinary position vectors are odd; angular momenta and cross products are even (they are *pseudo*vectors).

Nothing in this chapter changes: parity refines the bookkeeping and adds a second selection rule (parities multiply), but the ℓ-structure, Schur's lemma and the layer forms above are unaffected. We take parity up only where it actually bites — molecular chirality, where the distinction between $SO(3)$ and $O(3)$ is chemically real — in Theorem 3.42.

Why this matters for SigmaFlow

Linear mixing. The backbone can learn arbitrary new combinations of same-type channels — new chemical semantics — without any risk to geometry. This is where most of the parameters live, and it is why the architecture can be wide without becoming fragile.
Gating. It is what makes the network nonlinear at all on its directional channels. Without it, all the $\ell \geq 1$ processing would be linear, and the model could not represent the strongly nonlinear dependence of a pose on its environment.
Tensor products remain essential. Section 4.10.11 makes precise why they cannot be dispensed with: they are the only operation that can create a degree that was not already present. Since the network's output is $\ell = 1$ and its raw chemical inputs are $\ell = 0$, at least one tensor product with the geometry is unavoidable.
Independent of the generative engine. All three operations are statements about how features may be transformed. They say nothing about what the network is asked to predict. Replacing SigmaDock's score target with SigmaFlow's velocity target changes the loss and the sampler; it does not touch a single one of the constraints in this section. That is precisely why the backbone is shared, and why the SigmaFlow conversion could be confined to five files (Chapter 12).

Definition 4.54 (Steerable feature; restated)**.** A *steerable feature* (of a network node or edge) is a direct sum $h = \bigoplus_{\ell=0}^{L} \bigoplus_{c=1}^{C_\ell} h^{\ell,c}$, $h^{\ell,c} \in V_\ell \cong \mathbb{R}^{2\ell+1}$, transforming under $Q \in SO(3)$

by $h \mapsto \big(\bigoplus_{\ell,c} D^\ell(Q)\big)h$ — i.e. every one of the C_ℓ channels at degree ℓ transforms by the *same* $D^\ell(Q)$, independently of the others.

Theorem 4.48 is, in words, the entire content of "an equivariant linear layer has one learnable scalar per (source channel, target channel, shared degree) triple, and cannot move information between degrees": every degree ℓ is processed by its *own*, otherwise completely ordinary, linear layer, exactly the pattern the `SO3_LinearV2` code correspondence in Chapter 8 will confirm directly. Since linear maps can never mix degrees, and since pointwise nonlinearities (e.g. ReLU applied entrywise) do *not*, in general, commute with $D^\ell(Q)$ for $\ell \geq 1$ (a direction-dependent nonlinearity applied to a rotated vector is generally not the same as rotating the nonlinearly-transformed vector), the Clebsch–Gordan product (4.7) of this chapter is the *only* remaining mechanism, besides invariant-gated rescaling (Chapter 8's gate nonlinearity), by which an equivariant network can create new degrees of information or genuinely nonlinearly combine two existing steerable features — and this is precisely why the tensor product, rather than being one architectural ingredient among many, is unavoidable, as promised at the end of Section 7.4.

Part III

Diffusion Models

Chapter 5

Diffusion Models in $\mathbb{R}^d$

We now build the first of the two generative-modelling machineries identified in Section 2.5: a way to construct a probability path $(p_t)_{t\in[0,1]}$ implicitly, as the law of a stochastic process, and to sample from $p_1 = p_{\text{data}}$ by simulating that process. Everything in this chapter is developed in $\mathbb{R}^d$ first; the manifold (in particular $SO(3)$) generalisation is Chapter 6, and SigmaDock's actual use of both is reconstructed in full, with code correspondence, in Chapter 9.

5.1 Denoising score matching

Section 2.3 showed that knowing $\nabla_x \log p_{\text{data}}$ would suffice for sampling, via Langevin dynamics, but left open how to *learn* this score from samples alone. The direct approach — fit a network $s_\theta(x) \approx \nabla_x \log p_{\text{data}}(x)$ by minimising $\mathbb{E}_{x\sim p_{\text{data}}}\big[\|s_\theta(x) - \nabla_x \log p_{\text{data}}(x)\|^2\big]$ — is circular: the target $\nabla_x \log p_{\text{data}}$ is exactly the unknown quantity. The resolution, exactly analogous in structure to Theorem 2.3's role in Section 2.5, is to introduce a *noised* version of the data whose score *is* computable in closed form.

Definition 5.1 (Gaussian perturbation kernel)**.** For $\sigma > 0$, let $p_\sigma(x \mid z) = \mathcal{N}(x \mid z, \sigma^2 I_d)$, and let $p_\sigma(x) = \int p_\sigma(x \mid z)\, p_{\text{data}}(z)\, dz$ be the corresponding *noised marginal* (Theorem 2.3, with $p_\sigma(\cdot \mid z)$ playing the role of the conditional density there).

By Theorem 2.10, the *conditional* score is available in closed form:

$$\nabla_x \log p_\sigma(x \mid z) = -\frac{x - z}{\sigma^2}. \tag{5.1}$$

What we actually want, to run Langevin dynamics targeting p_σ rather than p_{data} directly (an approximation we return to and justify via the noise ladder / SDE construction of Section 5.2), is the *marginal* score $\nabla_x \log p_\sigma(x)$, which is *not* available in closed form, since it requires the (intractable) integral defining p_σ. The following theorem is the resolution, and its proof is the prototype for every "conditional loss has the same minimiser as marginal loss" argument in this document, including Theorem 10.8 in Chapter 10.

Theorem 5.2 (Denoising score matching)**.** *Define*

$$\mathcal{L}_{\text{ESM}}(\theta) = \mathbb{E}_{x\sim p_\sigma}\Big[\big\|s_\theta(x) - \nabla_x \log p_\sigma(x)\big\|^2\Big] \quad \textit{(explicit/marginal score matching)},$$

$$\mathcal{L}_{\text{DSM}}(\theta) = \mathbb{E}_{z\sim p_{\text{data}},\, x\sim p_\sigma(\cdot\mid z)}\Big[\big\|s_\theta(x) - \nabla_x \log p_\sigma(x \mid z)\big\|^2\Big] \quad \textit{(denoising score matching)}. \tag{5.2}$$

Then $\mathcal{L}_{\mathrm{ESM}}(\theta) = \mathcal{L}_{\mathrm{DSM}}(\theta) + C$ with C independent of θ; in particular the two losses have identical gradients in θ and the same minimisers.

Proof. Expand the square in $\mathcal{L}_{\mathrm{ESM}}$ under the marginal $p_\sigma(x)$:

$$\mathcal{L}_{\mathrm{ESM}}(\theta) = \mathbb{E}_{x\sim p_\sigma}\big[\|s_\theta(x)\|^2\big] - 2\,\mathbb{E}_{x\sim p_\sigma}\big[\langle s_\theta(x), \nabla_x \log p_\sigma(x)\rangle\big] + \mathbb{E}_{x\sim p_\sigma}\big[\|\nabla_x \log p_\sigma(x)\|^2\big].$$

The third term does not depend on θ; call it C_1. The first term already matches $\mathcal{L}_{\mathrm{DSM}}$'s first term, since $x \sim p_\sigma$ (marginally) is the same distribution as x obtained by first drawing $z \sim p_{\mathrm{data}}$ then $x \sim p_\sigma(\cdot \mid z)$ (Theorem 2.3), so $\mathbb{E}_{x\sim p_\sigma}[\|s_\theta(x)\|^2] = \mathbb{E}_{z,x}[\|s_\theta(x)\|^2]$. It remains to handle the cross term. Using $p_\sigma(x)\nabla_x \log p_\sigma(x) = \nabla_x p_\sigma(x) = \int \nabla_x p_\sigma(x \mid z)\, p_{\mathrm{data}}(z)\, dz = \int p_\sigma(x \mid z)\nabla_x \log p_\sigma(x \mid z)\, p_{\mathrm{data}}(z)\, dz$ (differentiating under the integral sign, and using $\nabla_x p_\sigma(x \mid z) = p_\sigma(x \mid z)\nabla_x \log p_\sigma(x \mid z)$),

$$\begin{aligned}
\mathbb{E}_{x\sim p_\sigma}\big[\langle s_\theta(x), \nabla_x \log p_\sigma(x)\rangle\big] &= \int \langle s_\theta(x), \nabla_x \log p_\sigma(x)\rangle\, p_\sigma(x)\, dx \\
&= \iint \langle s_\theta(x), \nabla_x \log p_\sigma(x \mid z)\rangle\, p_\sigma(x \mid z)\, p_{\mathrm{data}}(z)\, dz\, dx \\
&= \mathbb{E}_{z\sim p_{\mathrm{data}},\, x\sim p_\sigma(\cdot\mid z)}\big[\langle s_\theta(x), \nabla_x \log p_\sigma(x \mid z)\rangle\big],
\end{aligned}$$

which is exactly $\mathcal{L}_{\mathrm{DSM}}$'s cross term. Collecting terms, $\mathcal{L}_{\mathrm{ESM}}(\theta) = \mathcal{L}_{\mathrm{DSM}}(\theta) + C_1 - C_2$, where $C_2 = \mathbb{E}_{z,x}[\|\nabla_x \log p_\sigma(x \mid z)\|^2]$ is also θ-independent, giving $C = C_1 - C_2$. □

The practical content of Theorem 5.2: to train a score network, draw a clean data point $z \sim p_{\mathrm{data}}$ (a sample from the training set), draw a noised version $x = z + \sigma\varepsilon$, $\varepsilon \sim \mathcal{N}(0, I_d)$ (sampling from $p_\sigma(\cdot \mid z)$ directly, rather than needing the marginal), and regress $s_\theta(x)$ against the *known*, closed-form target $-\varepsilon/\sigma$ (substituting $x - z = \sigma\varepsilon$ into (5.1)). No integral over p_{data} or p_σ is ever evaluated.

5.2 The SDE perspective, and why a single noise level is not enough

A single σ is not sufficient for high-quality sampling: for large σ, $p_\sigma \approx p_{\mathrm{init}}$ is easy to sample from via Langevin dynamics (the perturbation dominates, and the resulting density is close to an isotropic Gaussian, which mixes quickly under Langevin dynamics) but far from p_{data}, and vice versa for small σ. Song and Ermon's original resolution (Song & Ermon, *Generative Modeling by Estimating Gradients of the Data Distribution*, NeurIPS 2019) trains a single, noise-conditional network $s_\theta(x, \sigma)$ across a decreasing ladder of noise levels $\sigma_1 > \cdots > \sigma_N$ and runs annealed Langevin dynamics, decreasing σ over the course of sampling. Song et al. (*Score-Based Generative Modeling through Stochastic Differential Equations*, ICLR 2021) subsequently observed that this discrete ladder is the time-discretisation of a single, continuous-time stochastic process, and it is this continuous formulation, not the discrete ladder, that both SigmaDock's actual implementation and this document use throughout.

Write the *noising* process in the reversed time variable $s = 1 - t$ (Theorem 0.2), so it runs from data ($s = 0$) to noise ($s = 1$):

$$dY_s = f(Y_s, s)\, ds + g(s)\, dW_s, \qquad Y_0 \sim p_{\mathrm{data}}, \tag{5.3}$$

with drift $f : \mathbb{R}^d \times [0,1] \to \mathbb{R}^d$, diffusion coefficient $g : [0,1] \to \mathbb{R}_{\geq 0}$, and W a standard Wiener process (Theorem 2.11). Write p_t for the law of Y_{1-t} (i.e. time is relabelled to our global generation-time convention, so p_0 is the law of Y_1, the fully-noised end, and p_1 is the law of $Y_0 = p_{\mathrm{data}}$).

Example 5.3 (Variance-exploding (VE) SDE). $f \equiv 0$, $g(s)^2 = \frac{d}{ds}\sigma(s)^2$ for an increasing schedule $\sigma(s)$. Then (5.3) integrates in closed form to $Y_s = Y_0 + \sigma(s)\varepsilon$, $\varepsilon \sim \mathcal{N}(0, I_d)$ (a direct check: this is a driftless SDE whose solution is Brownian motion time-changed by $\sigma(s)^2$, and its marginal variance at time s is exactly $\sigma(s)^2$ by Itô isometry, which we do not derive here but which matches the perturbation kernel of Theorem 5.1 with $\sigma = \sigma(s)$) — exactly recovering the noise ladder of Section 5.1 as the time slices $\sigma(s_i) = \sigma_i$ of a single continuous process. This is the SDE family used, as Chapter 9 confirms directly against the code, by SigmaDock's *rotational* diffusion process.

Example 5.4 (Variance-preserving (VP) SDE). $f(x, s) = -\frac{1}{2}\beta(s)x$, $g(s) = \sqrt{\beta(s)}$, for a noise-rate schedule $\beta(s) \geq 0$. Solving the resulting (linear, hence exactly solvable) SDE gives

$$Y_s = \alpha(s)\, Y_0 + \sqrt{1 - \alpha(s)^2}\, \varepsilon, \qquad \alpha(s) = \exp\Big(-\tfrac{1}{2}\int_0^s \beta(r)\, dr\Big),$$

i.e. $p_s(\cdot \mid Y_0) = \mathcal{N}\big(\alpha(s)Y_0,\ (1-\alpha(s)^2)I_d\big)$ — unlike the VE case, the *signal* Y_0 is itself attenuated, and the total variance $\alpha(s)^2 \cdot \mathrm{Var}(Y_0) + (1-\alpha(s)^2)$ stays bounded (indeed, if $\mathrm{Var}(Y_0) = 1$, stays exactly at 1) as $s \to 1$, rather than exploding — hence the name. This is the SDE family used, as Chapter 9 again confirms directly, by SigmaDock's *translational* diffusion process; the exact solution just derived is this document's own re-derivation of the closed-form perturbation kernel that the code computes via `R3Diffuser`'s `integrated_beta_t`, not a claim quoted from elsewhere.

We record, once, the general fact that both examples instantiate, since it is exactly Theorem 5.1's Gaussian-perturbation-kernel construction generalised to an SDE with (possibly time- and state-dependent) linear drift.

Proposition 5.5 (Perturbation kernel of a linear SDE). *If $f(x, s) = -\frac{1}{2}\beta(s)x$ (including $\beta \equiv 0$, the VE case) and $g(s)^2 = \frac{d}{ds}\sigma(s)^2$ is otherwise free, the solution of (5.3) has Gaussian conditional law $p_s(\cdot \mid Y_0) = \mathcal{N}\big(\alpha(s)Y_0, \beta(s)^2 I_d\big)$ for schedules $\alpha(s), \beta(s)$ determined by f, g (linear SDEs are exactly solvable; we do not re-derive the general variation-of-constants formula here, having exhibited it explicitly for the two cases used by this document in Theorems 5.3 and 5.4).*

5.2.1 Score, weighting, and the training objective in continuous time

Given a schedule as in Theorem 5.5, the conditional score is again available in closed form by Theorem 2.10, and the continuous-time denoising score matching objective (obtained from the single-noise-level objective (5.2) by letting the fixed σ become the time-dependent $\beta(s)$ of Theorem 5.5 and averaging over $s \sim \mathcal{U}(0, 1)$ with weight $\lambda(s)$ — the literal continuous-time analogue of Theorem 5.2 applied at every noise level simultaneously) is

$$\mathcal{L}(\theta) = \mathbb{E}_{s\sim\mathcal{U}(0,1),\, Y_0\sim p_{\mathrm{data}},\, Y_s\sim p_s(\cdot|Y_0)}\Big[\lambda(s)\big\|s_\theta(Y_s, s) - \nabla_{Y_s}\log p_s(Y_s \mid Y_0)\big\|^2\Big], \tag{5.4}$$

for a positive weighting function $\lambda(s)$. The role of λ is purely practical, not conceptual: by Theorem 5.2 (applied noise-level by noise-level, which is valid since the theorem's proof never uses anything special about a fixed σ, only that x is drawn from *some* fixed conditional given z), *any* positive $\lambda(s)$ gives a loss with the same minimiser, $s_\theta(x, s) = \nabla_x \log p_s(x)$ — but the target's own scale, $\|\nabla_x \log p_s(x \mid z)\| = \|\varepsilon\|/\beta(s)$ in Theorem 5.5's notation, varies enormously across s (it blows up as $\beta(s) \to 0$, i.e. as $s \to 0$, towards clean data), so an unweighted loss lets the large-noise (easy, large-target) terms dominate the gradient at

the expense of the small-noise (hard, small-target) terms that determine fine sample detail. The standard choice, $\lambda(s) = \beta(s)^2$ (i.e. $\lambda(s) = 1/\mathbb{E}\|\nabla_x \log p_s(x \mid z)\|^2$ up to the constant d, since $\mathbb{E}\|\varepsilon/\beta(s)\|^2 = d/\beta(s)^2$ for $\varepsilon \sim \mathcal{N}(0, I_d)$), exactly cancels this scale variation, and reappears, generalised to $SO(3)$, as the `score_scaling` construction reconstructed precisely in Section 6.2.2.

5.2.2 The reverse-time SDE

Equation (5.3) destroys structure (runs from data to noise); generation needs the reverse. The following classical theorem is what licenses training a score network and then *simulating backwards* to generate samples.

Theorem 5.6 (Anderson's reverse-time SDE)**.** *Let $(Y_s)_{s\in[0,1]}$ solve (5.3) with marginal laws $q_s = \mathrm{Law}(Y_s)$. Under mild regularity conditions (existence and smoothness of q_s for all s, and suitable growth conditions on f, g; see Anderson,* Reverse-Time Diffusion Equation Models, *Stochastic Processes and their Applications, 1982), the time-reversed process $\widetilde{Y}_s := Y_{1-s}$ solves the SDE*

$$d\widetilde{Y}_s = \Big[-f(\widetilde{Y}_s, 1-s) + g(1-s)^2\, \nabla_x \log q_{1-s}(\widetilde{Y}_s)\Big]\, ds + g(1-s)\, d\overline{W}_s,$$

where $\overline{W}$ is a Wiener process for the reversed filtration (a technical point about which σ-algebra the increments are independent of, which does not affect how the SDE is simulated and which we do not elaborate on further).

We do not reprove Theorem 5.6 (it is a statement about how the Fokker–Planck equation governing q_s's time evolution transforms under time reversal, requiring Itô calculus we have not developed; see the cited reference, or Haussmann & Pardoux, *Time Reversal of Diffusions*, Annals of Probability, 1986, for the technically complete statement). We translate it into our generation-time convention. The reversed process $\widetilde{Y}$ already runs *forward* in its own time argument, so no second reversal is needed — the translation is a pure renaming. Set

$$X_t \;:=\; \widetilde{Y}_t \;=\; Y_{1-t}, \qquad \text{hence} \qquad p_t \;=\; \mathrm{Law}(X_t) \;=\; q_{1-t},$$

which gives $X_0 = Y_1 \sim p_{\text{init}}$ (the fully-noised end) and $X_1 = Y_0 \sim p_{\text{data}}$, as our convention demands. Renaming s to t in Theorem 5.6's SDE and substituting $q_{1-t} = p_t$,

$$dX_t = \Big[-f(X_t, 1-t) + g(1-t)^2 \nabla_x \log p_t(X_t)\Big]\, dt + g(1-t)\, d\overline{W}_t, \qquad X_0 \sim p_{\text{init}}, \tag{5.5}$$

integrated *forward* in t from 0 to 1. This is the SDE that generation simulates, and it requires exactly one previously-undetermined quantity: the score $\nabla_x \log p_t(x)$, precisely what (5.4) trains a network to approximate. Substituting $s_\theta \approx \nabla_x \log p_t$ into (5.5) and discretising by Euler–Maruyama (the direct generalisation of (2.2)'s update rule to a state- and time-dependent drift/diffusion) gives the sampling algorithm; Chapter 9 exhibits the exact discretisation SigmaDock's code uses, which matches this recipe precisely for the translational (VP) component.

5.3 The probability flow ODE, and the bridge to flow matching

Theorem 5.7 (Probability flow ODE)**.** *Under the same conditions as Theorem 5.6, the deterministic ODE*

$$\frac{dX_t}{dt} = u_t(X_t) := -f(X_t, 1-t) + \tfrac{1}{2} g(1-t)^2\, \nabla_x \log p_t(X_t) \tag{5.6}$$

has the same *marginal laws* $(p_t)_{t\in[0,1]}$ *as* (5.5) *(and hence as the original forward process* (5.3)*, relabelled).*

Proof sketch. Both (5.5) and (5.6) are constructed so that their associated Fokker–Planck / continuity equations (the PDE governing how a density evolves under the dynamics; for an SDE $dX = b(X,t)dt + g(t)dW$, this is $\partial_t p_t = -\nabla\cdot(b\,p_t) + \frac{1}{2}g(t)^2\Delta p_t$, and for an ODE $dX/dt = u_t(X)$ it is the driftless-diffusion special case $\partial_t p_t = -\nabla\cdot(u_t p_t)$, which is exactly Theorem 10.3 of Chapter 10, proved there in full) coincide: substituting $b = -f(\cdot, 1-t) + g(1-t)^2\nabla\log p_t$ into the SDE's Fokker–Planck equation and using $\nabla\cdot(p_t\nabla\log p_t) = \nabla\cdot(\nabla p_t) = \Delta p_t$ shows the diffusion term $\frac{1}{2}g(1-t)^2\Delta p_t$ exactly cancels against half of the drift's contribution, leaving the ODE's continuity equation with u_t as in (5.6). We do not carry out this cancellation in full generality here (it requires the Fokker–Planck equation, which Chapter 10 develops in the driftless case needed there), but note it is a standard, mechanical computation once Theorem 10.3 is available — see, e.g., Song et al. (2021, Appendix D.1) for the complete derivation in this SDE setting. □

Theorem 5.7 is exactly the content of Section 5.3's claim, made precise: *every* score-based diffusion model secretly defines a deterministic flow with the same marginals, whose velocity field (5.6) is an explicit, invertible function of the score. Chapter 10 develops the flow-matching framework as though starting from scratch, and Section 5.3 (there) proves the general form of this correspondence; what Theorem 5.7 establishes here is only the specific fact, needed to state Chapter 9's Remark on "items requiring verification", that SigmaDock's own sampler *could* in principle be run as a deterministic ODE (setting the SDE's injected noise to zero) with unchanged marginals — and Chapter 9 confirms, directly from the code, that this is in fact the *default* configuration SigmaDock ships with (`conf/sampling/base.yaml`'s default `noise_scale:0.0`), narrowing the "stochastic vs. deterministic" distinction between the two methods considerably.

Chapter 6

Diffusion Models on $SO(3)$

Chapter 5 developed diffusion entirely in $\mathbb{R}^d$; by Theorem 3.37, this already suffices for the *translational* component of a pose. This chapter develops the rotational analogue: what does "add Gaussian noise" mean on $SO(3)$, where addition is undefined, and what replaces the closed-form Gaussian kernel of Theorem 5.1?

6.1 Brownian motion on a Riemannian manifold

Motivation. In $\mathbb{R}^d$, Brownian motion can be built either as the limit of a random walk with i.i.d. Gaussian increments, or as the unique (in law) continuous process with independent Gaussian increments of variance proportional to elapsed time. On a curved manifold, "increment" must be reinterpreted: an increment is a step in a *tangent space*, which must then be mapped back onto the manifold.

Definition 6.1 (Manifold Brownian motion, via a random-walk limit)**.** Fix $x_0 \in \mathcal{M}$ and a step size $\Delta\tau > 0$. Define a discrete process $x_0, x_{\Delta\tau}, x_{2\Delta\tau}, \dots$ by

$$x_{(k+1)\Delta\tau} = \exp_{x_{k\Delta\tau}}\big(\sqrt{\Delta\tau}\,\xi_k\big), \qquad \xi_k \overset{\text{iid}}{\sim} \mathcal{N}\big(0, I_{\dim\mathcal{M}}\big) \text{ in } T_{x_{k\Delta\tau}}\mathcal{M}$$

(a Gaussian tangent vector sampled in the tangent space at the current point, using an orthonormal-basis identification $T_x\mathcal{M} \cong \mathbb{R}^{\dim\mathcal{M}}$ via the metric g_x, then pushed back onto $\mathcal{M}$ by the exponential map). As $\Delta\tau \to 0$, this process converges (in an appropriate sense; we do not make this precise, taking it as the operational definition throughout this document, following the treatment standard in the geometric-diffusion-model literature, e.g. Leach et al., *Denoising Diffusion Probabilistic Models on SO(3) for Rotational Alignment*, ICLR Workshop 2022, and De Bortoli et al., *Riemannian Score-Based Generative Modelling*, NeurIPS 2022) to a continuous-time process $B^{\mathcal{M}}_\tau$, *Brownian motion on* $\mathcal{M}$.

Code correspondence

Mathematical object: one step of $B^{SO(3)}_\tau$: sample a Gaussian tangent vector and push it onto the manifold
Implementation: `so3_utils.py`::`tangent_gaussian` (samples $\widehat{\varepsilon}$, $\varepsilon \sim \mathcal{N}(0, I_3)$, and left-multiplies by R: returns $R\widehat{\varepsilon}$, a left-trivialised (body-frame) — Section 3.4, note the tangent vector's own components are drawn in the *same* $\mathbb{R}^3$ regardless of R, then transported by right-multiplication, i.e. this is exactly Theorem 3.33's convention — Gaussian tangent vector at R) composed with `so3_utils.py`::`expmap` (the map $\exp_R$)

Tensor shape: `[..., 3, 3]` → `[..., 3, 3]`
Interpretation: `expmap`, per Section 3.3.3's code correspondence, computes $\exp_R(v)$ via `torch.matrix_exp` after re-symmetrising the skew part for numerical stability, rather than Rodrigues' closed form; this is the code-level realisation of one step of Theorem 6.1's random walk, used directly inside SigmaDock's reverse-time sampler (Section 6.2.2).

Proposition 6.2 (Stationary distribution of $SO(3)$ Brownian motion)**.** *$B_\tau^{SO(3)}$'s law converges, as $\tau \to \infty$, to the Haar (uniform) measure of Theorem 3.29, independently of the starting point x_0.*

Proof sketch. Brownian motion on a compact Riemannian manifold is reversible with respect to, and converges to, the Riemannian volume measure — the manifold analogue of Theorem 2.13's statement that $\mathbb{R}^d$ Langevin dynamics converges to $p \propto e^{-U}$, specialised to the driftless case $U \equiv 0$ (constant potential, i.e. pure diffusion with no preferred point), whose stationary measure is simply the volume measure itself; on $SO(3)$ this volume measure is (proportional to, hence equal to after normalisation) the Haar measure, since Haar measure *is* the (unique, up to normalisation) bi-invariant, and in particular the Riemannian-volume-induced, measure. We do not reprove the general convergence statement (it again requires the manifold Fokker–Planck / heat equation, whose generator is the Laplace–Beltrami operator, Theorem 6.3 below), but note it is exactly why $p_{\text{init}} = \mathcal{U}(SO(3))$, not some other distribution, is the correct rotational prior for a diffusion model whose forward process runs Brownian motion for a long enough time — Section 9.6 confirms this is precisely what the code uses. □

This is the promised structural point from Section 3.1: unlike $\mathbb{R}^d$ Brownian motion, whose stationary behaviour is "variance $\to \infty$", a *compact* manifold's Brownian motion has a genuine equilibrium distribution determined entirely by the manifold's geometry, and $SO(3)$'s compactness (it is a closed, bounded subset of $\mathbb{R}^{3\times3}$) is what makes the variance-exploding schedule of Theorem 5.3 converge to something useful rather than literally exploding without bound, as it would in $\mathbb{R}^d$.

Definition 6.3 (Laplace–Beltrami operator, informal)**.** The *Laplace–Beltrami operator* $\Delta_{\mathcal{M}}$ is the natural generalisation of the Euclidean Laplacian $\Delta = \sum_i \partial_i^2$ to a Riemannian manifold, defined so that it is the infinitesimal generator of Brownian motion on $\mathcal{M}$ (Theorem 6.1) in exactly the sense that Δ is the generator of ordinary $\mathbb{R}^d$ Brownian motion: the transition density $k_\tau(x, y)$ of $B_\tau^{\mathcal{M}}$ (started at x) solves the *heat equation* $\partial_\tau k_\tau = \Delta_{\mathcal{M}} k_\tau$, exactly as the ordinary Gaussian kernel solves $\partial_\tau k_\tau = \Delta k_\tau$ in $\mathbb{R}^d$. We do not need $\Delta_{\mathcal{M}}$'s coordinate formula anywhere in this document; only the existence and role of the heat kernel it generates, constructed explicitly for $SO(3)$ in Section 6.2 below.

6.2 The heat kernel on $SO(3)$: IGSO(3)

Motivation. Theorem 5.1's Gaussian perturbation kernel $p_\sigma(x \mid z) = \mathcal{N}(x \mid z, \sigma^2 I)$ is, in $\mathbb{R}^d$-specific language, exactly the *heat kernel*: the transition density of Brownian motion after time σ^2. We now want its $SO(3)$-analogue: the transition density $k_{\sigma^2}(R_0, R)$ of $SO(3)$-Brownian motion (Theorem 6.1) run for "time" σ^2 starting at R_0.

Theorem 6.4 (The isotropic Gaussian distribution on $SO(3)$)**.** *The heat kernel on $SO(3)$, called* $\text{IGSO}(3)(R_0, \sigma^2)$*, depends on R_0, R only through the relative-rotation angle $\omega = \Omega(R_0^\top R)$*

of Theorem 3.23 (bi-invariance of the metric, hence of the Laplace–Beltrami operator and its heat kernel, under simultaneous left/right translation), and its density with respect to the Haar measure admits the spectral (Peter–Weyl) expansion

$$f_\sigma(\omega) = \sum_{\ell=0}^{\infty} (2\ell+1)\,\chi_\ell(\omega)\,\exp\!\Big(-\tfrac{1}{2}\ell(\ell+1)\sigma^2\Big), \qquad \chi_\ell(\omega) = \frac{\sin\big((\ell+\frac{1}{2})\omega\big)}{\sin(\omega/2)}, \tag{6.1}$$

where χ_ℓ is the character of the degree-ℓ irrep of $SO(3)$ (Theorem 4.18).

Proof sketch. This is the Peter–Weyl expansion of the heat kernel of a compact Lie group: the heat kernel, viewed as a function on the group (using bi-invariance to reduce to a function of one variable via $k_{\sigma^2}(R_0, R) = k_{\sigma^2}(I, R_0^\top R) =: f_\sigma(\omega)$), decomposes in the orthogonal basis of *matrix coefficients* of the irreducible representations D^ℓ (the Peter–Weyl theorem: $L^2(G)$ for a compact group G decomposes as a direct sum, over irreps, of matrix-coefficient spaces), each an eigenfunction of the Laplace–Beltrami operator with eigenvalue $-\ell(\ell+1)$ (the Casimir eigenvalue of the degree-ℓ irrep of $SO(3)$, a standard fact from the representation theory cited after Theorem 4.18), and the coefficient $e^{-\ell(\ell+1)\sigma^2/2}$ is exactly the corresponding solution $e^{\lambda\tau}$ of the heat equation's eigenvalue ODE with $\lambda = -\ell(\ell+1)/2$, $\tau = \sigma^2$; summing the trace of D^ℓ over the class-function structure of a bi-invariant kernel (i.e. summing each matrix-coefficient block's diagonal, which is exactly the character χ_ℓ) and using the classical closed form for $SO(3)$'s degree-ℓ character, $\chi_\ell(\omega) = \sin((\ell+\frac{1}{2})\omega)/\sin(\omega/2)$ (itself derivable from the eigenvalues of $D^\ell(\exp(\theta\widehat{n}))$, which are $\{e^{im\theta} : m = -\ell, \dots, \ell\}$ in a suitable complex basis, summing as a geometric series), gives (6.1). We take the Peter–Weyl decomposition and the Casimir eigenvalue computation as classical facts and do not reprove them (see Chirikjian, 2012, cited earlier, §10.3, for the complete derivation). □

Remark 6.5 (The series must be truncated in practice). Equation (6.1) is an infinite sum; Section 6.2.2 below records exactly how the actual SigmaDock implementation truncates, tabulates, and interpolates it, since this is a genuine engineering component of the method, not an afterthought — and, as Section 11.3 in Chapter 10 shows, it is precisely this machinery that flow matching eliminates entirely.

Proposition 6.6 (The angle marginal density)**.** *The density of $\omega = \Omega(R_0^\top R)$ alone (as opposed to the full density on $SO(3)$) is $\frac{1-\cos\omega}{\pi}\,f_\sigma(\omega)$: the IGSO(3) density f_σ times the same Haar-measure Jacobian $1-\cos\omega$ that appeared in Theorem 3.29 (up to the same normalisation constant), since $\mathrm{IGSO}(3)(R_0, \sigma^2)$, in the limit $\sigma \to \infty$, must reduce to the Haar measure of Theorem 3.29 by Theorem 6.2, and this is consistent with (6.1) since every term with $\ell \geq 1$ decays to 0 as $\sigma \to \infty$, leaving only the $\ell = 0$ term $f_\infty(\omega) = 1 \cdot \chi_0(\omega) \cdot 1 = 1$ (using $\chi_0(\omega) = \sin(\omega/2)/\sin(\omega/2) = 1$), recovering exactly Theorem 3.29's density $\propto 1 - \cos\omega$.*

6.2.1 The score of the IGSO(3) kernel

Proposition 6.7 (Riemannian score of IGSO(3))**.** *Under the right-trivialised (world-frame) identification of Theorem 3.30 at R, the score of $p_\sigma(R \mid R_0) := \mathrm{IGSO}(3)(R_0, \sigma^2)$-density at R, as a tangent vector in $T_R SO(3) \cong \mathbb{R}^3$, is*

$$\nabla_R \log p_\sigma(R \mid R_0) = \frac{v}{\|v\|} \cdot \frac{\partial}{\partial\omega} \log f_\sigma(\omega)\Big|_{\omega=\|v\|}, \qquad v = \log(R_0^\top R)^\vee \in \mathbb{R}^3. \tag{6.2}$$

Proof sketch. Since (Theorem 6.4) $p_\sigma(R \mid R_0) = f_\sigma(\omega)$ depends on R only through the scalar $\omega = \Omega(R_0^\top R)$, the chain rule gives $\nabla_R \log p_\sigma = f_\sigma'(\omega)/f_\sigma(\omega) \cdot \nabla_R \omega$ (the Riemannian gradient

of the composite scalar function). The gradient of the angle function $\omega(R) = \Omega(R_0^\top R)$, as a function on the manifold, points in the direction of the tangent vector that most rapidly increases ω — exactly the unit tangent vector along the geodesic away from R_0, which by Theorem 3.28 is $v/\|v\| = \log(R_0^\top R)^\vee/\omega$ — with magnitude 1 (since ω itself is arc length along that geodesic, by Theorem 3.28 again, and the gradient of arc length along its own geodesic has unit norm, the direct manifold analogue of $\nabla_x\|x\| = x/\|x\|$ in $\mathbb{R}^d$). Combining gives (6.2). □

Code correspondence

Mathematical object: $\nabla_R \log p_\sigma(R \mid R_0)$
Implementation: `so3_diffuser.py`::`SO3Diffuser.score`
Tensor shape: `[..., 3, 3]` (world-frame element of $\mathfrak{so}(3)$, i.e. the code returns $R \cdot \widehat{v} \cdot (\partial_\omega \log f_\sigma/\omega)$ rather than the bare $\mathbb{R}^3$ vector, matching the world-frame identification of Theorem 3.30)
Interpretation: $\partial_\omega \log f_\sigma$ is *not* recomputed by differentiating the tabulated density table at score-evaluation time; it is evaluated *live* from the analytic term-by-term derivative of the truncated series (6.1) (function `d_log_f_d_omega`, Section 6.2.2), for the exact, continuous $\sigma(t)$ of the query, not a value snapped to the precomputed grid.

6.2.2 Numerical implementation: truncation, tabulation, and caching

We now record, precisely and only from the code (not reconstructed analytically), how SigmaDock evaluates the objects of the preceding two subsections in practice, since Theorem 6.5 promised this level of detail and it is exactly the machinery Section 11.3 will show flow matching does not need.

Code correspondence

Mathematical object: the truncated series $f_\sigma(\omega) = \sum_{\ell=0}^{L-1}(2\ell+1)\chi_\ell(\omega)e^{-\ell(\ell+1)\sigma^2/2}$ of Theorem 6.4, and its analytic derivative $\partial_\omega f_\sigma$, $\partial_\omega \log f_\sigma$
Implementation: `so3_diffuser.py`, module-level functions `igso3_expansion` (the truncated sum, default $L = 1000$ terms, i.e. degrees $\ell = 0, \ldots, 999$), `igso3_density` (multiplies in the angle-marginal Jacobian $1 - \cos\omega$ of Theorem 6.6), `d_f_igso3_d_omega` (the term-by-term derivative, via the quotient rule applied to each $\chi_\ell(\omega) = \sin((\ell+\frac{1}{2})\omega)/\sin(\omega/2)$), `d_log_f_d_omega`
Interpretation: `d_log_f_d_omega` additionally hard-branches (via boolean masking, not the soft `isclose`-blend pattern of Section 3.3.4) between the literal ratio $\partial_\omega f/f$ and the small-noise approximation $-\omega/\sigma^2$ whenever $\sigma^2 < 0.3$, since the literal ratio becomes numerically ill-conditioned at small σ (both numerator and denominator of the truncated series become extremely small simultaneously). We have not independently re-derived that $-\omega/\sigma^2$ is the correct $\sigma \to 0$ limit of $\partial_\omega \log f_\sigma$ from (6.1) (it is plausible — as $\sigma \to 0$, $\mathrm{IGSO}(3)(R_0, \sigma^2)$ should concentrate at R_0 and locally resemble a flat Gaussian, whose log-density derivative in the radial direction is exactly $-\omega/\sigma^2$ by Theorem 2.10's formula specialised to one radial coordinate — but we record this as the code's own documented claim, flagged per this document's stated policy on uncertainty, not as a result independently verified here).

Code correspondence

Mathematical object: the full apparatus of caching, tabulation, and inverse-CDF sampling used at both training and inference time

Implementation: `so3_diffuser.py::SO3Diffuser.__init__`, `sample_igso3_angle`, `sample`, `sample_ref`
Interpretation: at construction time, the diffuser builds a 1000×2000 grid: 1000 noise levels σ_i (linearly spaced in $t \in [0,1]$, then mapped through the schedule $\sigma(t) = \log\big(te^{\sigma_{\max}} + (1-t)e^{\sigma_{\min}}\big)$, the *logarithmic* schedule) $\times$ 2000 angles ω_j (linearly spaced in $(0,\pi]$, deliberately excluding $\omega = 0$ to avoid the $\sin(\omega/2) = 0$ singularity visible in (6.1)), and evaluates $f_{\sigma_i}(\omega_j)$ (via `igso3_density`, with `marginal=True`, i.e. including the Jacobian of Theorem 6.6) and $\partial_\omega \log f_{\sigma_i}(\omega_j)$ (via `d_log_f_d_omega`) at every grid point, once, and caches both tables to disk, together with a row-wise cumulative-sum CDF table (a Riemann-sum numerical integral of the density table, safe-normalised by its own final value clamped away from zero). The cache directory name encodes every hyperparameter of the grid (matching the `eps_1000_omega_2000_min_sigma_..._schedule_logarithmic`-style paths visible in this project's own training logs). *Sampling* an angle at a queried t (`sample_igso3_angle`) looks up the nearest tabulated σ_i row via `torch.bucketize`, draws $u \sim \mathrm{Unif}(0,1)$, locates u's bracketing interval in that row's cached CDF, and *linearly interpolates* between the two bracketing grid angles — piecewise-linear inverse-CDF sampling on a fixed 2000-point grid, not exact analytic inversion. `sample_ref` (the prior sampler, i.e. $R \sim p_{\text{init}}$) calls `sample` at $t = 1$, the schedule's maximum-noise end — an *approximation* to the exact Haar-uniform prior of Theorem 6.2, exact only in the limit $\sigma_{\max} \to \infty$, not literally the closed-form Haar sampler of Theorem 3.29 (that routine, `sample_uniform`, is a separate function, used instead by *SigmaFlow*'s prior, as recorded in Section 11.3). By contrast, the *score itself* (Theorem 6.7's code correspondence, immediately above) is **not** read from this cached table at evaluation time — it is recomputed live via `d_log_f_d_omega` at the query's exact, continuous (ω, σ); the cached `score_norms` table exists only to build the training loss-weight $\lambda(t)$ (`score_scaling`, matching (5.4)'s $\lambda(s)$ in this document's notation), computed once at construction time as a weighted second moment, $\lambda_i^{-1/2} = \sqrt{\mathbb{E}_\omega[(\partial_\omega \log f_{\sigma_i})^2]/3}$ (the division by 3 normalising a full-vector squared-norm expectation down to a per-coordinate one), over the angle marginal at noise level σ_i.

6.3 Riemannian denoising score matching and the reverse-time SDE on $SO(3)$

With Theorem 6.7 in hand, the entire machinery of Chapter 5 transfers to $SO(3)$ by the single substitution "Euclidean gradient $\to$ Riemannian gradient", exactly as Theorem 3.4's generalisation of $\mathbb{R}^d$ calculus to manifolds promised at the start of Chapter 3.

Theorem 6.8 (Riemannian denoising score matching)**.** *Define, analogously to Theorem 5.2,*

$$\mathcal{L}_{\text{RDSM}}(\theta) = \mathbb{E}_{t,\, R_1 \sim p_{\text{data}},\, R_t \sim p_t(\cdot \mid R_1)} \Big[\lambda_t \big\| s_\theta(R_t, t) - \nabla_{R_t} \log p_t(R_t \mid R_1) \big\|_g^2 \Big],$$

with $p_t(\cdot \mid R_1) = \mathrm{IGSO}(3)(R_1, \sigma(1-t)^2)$ (Theorem 0.2's convention: R_1 is data, so the conditioning is on the $t = 1$ endpoint, and the noise level increases as t decreases, i.e. as we move away from data towards $t = 0$). Then, exactly as in Theorem 5.2's proof (which uses nothing about the ambient space beyond the existence of the gradient/score and the marginalisation identity of Theorem 2.3, both of which hold verbatim on a Riemannian manifold with ∇_x replaced by the Riemannian gradient and dx by the Riemannian volume element), minimising $\mathcal{L}_{\text{RDSM}}$ has the same minimiser, $s_\theta(R,t) \approx \nabla_R \log p_t(R)$, as the (intractable) marginal Riemannian score matching loss.

We do not repeat the proof (it is, verbatim, the computation of Theorem 5.2, with every Euclidean integral replaced by an integral against the Riemannian volume measure, valid since none of the manipulations used — integration by parts under the gradient, Fubini, linearity of expectation — depend on the ambient space being flat).

Sampling then simulates the manifold reverse-time SDE, the direct $SO(3)$-analogue of (5.5):

$$dR_t = g(1-t)^2 \nabla_R \log p_t(R_t)\, dt + g(1-t)\, d\overline{B}_t^{SO(3)}, \qquad R_0 \sim p_{\text{init}}, \tag{6.3}$$

(no drift term f, since Theorem 5.3's VE-type schedule, which SigmaDock's rotational component uses per the code correspondence of Section 6.2.2, has $f \equiv 0$), discretised as a *geodesic random walk*: at each step, take one Theorem 6.1-style Brownian increment, scaled and drifted by the (learned, substituted) score, and map it onto $SO(3)$ via $\exp_{R_t}$ — guaranteeing, by construction, that the discretised trajectory never leaves $SO(3)$, for any step size, unlike a naive Euclidean discretisation of a constrained system.

Code correspondence

Mathematical object: one Euler–Maruyama step of (6.3)
Implementation: `so3_diffuser.py::SO3Diffuser.reverse`
Tensor shape: `[..., 3, 3]` → `[..., 3, 3]`
Interpretation: computes `perturb`= $g_t^2 \cdot \text{score}_t \cdot dt + \text{noise\_scale} \cdot g_t \cdot \sqrt{dt} \cdot z$ with z =`tangent_gaussian`(R_t) (Section 6.1's code correspondence) and g_t =`diffusion_coef`(t), then returns R_{t-dt} =`expmap`$(R_t, \text{perturb})$ — exactly (6.3)'s Euler–Maruyama discretisation, with the manifold-respecting update $\exp_{R_t}(\cdot)$ in place of the naive Euclidean $R_t + (\cdot)$. An optional `noise_scale` parameter (default, per `conf/sampling/base.yaml`, exactly 0.0 — see Chapter 9) rescales the injected noise term z independently of the score-drift term, allowing the sampler to be run anywhere between fully stochastic (`noise_scale=1`) and fully deterministic (`noise_scale=0`, in which case (6.3)'s SDE degenerates to Theorem 5.7's probability flow ODE, specialised to $SO(3)$ by the same cancellation argument, restated but not re-derived here).

This completes the diffusion side of the theory. Chapter 9 assembles Chapters 5 and 6 into the full SigmaDock model — combining the VP translational process of Theorem 5.4 and the VE rotational process of this chapter via the product-manifold structure of Theorem 3.37, and connecting every formula derived here to the exact network architecture (Chapter 8) that predicts the required scores.

Part IV

Architecture

Chapter 7

From Sets to Geometry: Attention, Message Passing, and the Equivariance Obstruction

Chapters 5 and 6 reduced "generate a pose" to "learn a score/vector-field network", but said nothing about what that network's *input* looks like or how it must be built. This chapter develops that architecture from the ground up, assuming only that the reader knows what a matrix multiplication and a feed-forward layer are; by its end we will have derived, not merely stated, why an architecture built from scalars and distances alone *cannot* do the job, which is exactly the gap Chapter 8 closes.

7.1 The network's signature

Consolidating Chapters 5 and 6: the object to be learned, whether called a score (s_θ) or (in Chapter 10) a velocity (u_θ), is a function

$$u_\theta : \mathcal{M}_N \times [0,1] \longrightarrow T\mathcal{M}_N, \qquad u_\theta(\mathbf{T}, t) = \big(\omega_\theta^{(i)}, v_\theta^{(i)}\big)_{i=1}^{N} \in (\mathbb{R}^3 \times \mathbb{R}^3)^N, \tag{7.1}$$

where, by the left/right-trivialisation identification fixed in Section 3.4, $\omega_\theta^{(i)}$ is fragment i's rotational output and $v_\theta^{(i)}$ its translational output, each an element of $\mathbb{R}^3$ per fragment (never a full 3×3 matrix; the tangent-space identification of Chapter 3 is precisely what collapses "an infinitesimal rotation" to "three numbers"). The input $\mathbf{T} \in \mathcal{M}_N$ is not, however, presented to the network in this abstract group-element form; it is presented as the concrete Cartesian coordinates $\mathbf{x} = \varphi(\mathbf{T}) \in \mathbb{R}^{n\times3}$ of every atom (protein and ligand) obtained by applying each fragment's current rigid motion to its reference-conformer template (Section 9.7.1 makes this map φ precise), plus each atom's discrete chemical identity, together with t. The network, therefore, is a map from an *unordered, geometric* collection of labelled points in $\mathbb{R}^3$ to a per-fragment vector-pair output, and the rest of this chapter and the next derive what such a map is allowed to look like.

7.2 Two structural requirements

Permutation equivariance. Atoms in $\mathbf{x}$ carry no canonical order; they are a *set* (indexed arbitrarily for storage), not a sequence. Reindexing the input must permute the corre-

sponding output identically and change nothing else.

$SE(3)$**-equivariance.** The physical process being modelled (a ligand binding a pocket) does not depend on the pose of the laboratory reference frame. Applying a rigid motion to the entire input (protein and ligand together) must transform the predicted vector field exactly as Theorem 7.1 below makes precise — *not* leave it unchanged, and *not* change it arbitrarily.

Definition 7.1 (Group action, invariance, equivariance)**.** Let a group G act on a set (or manifold) $\mathcal{X}$ via maps $\Phi_Q : \mathcal{X} \to \mathcal{X}$, $Q \in G$, satisfying $\Phi_{Q_1 Q_2} = \Phi_{Q_1} \circ \Phi_{Q_2}$ (a group action, the natural generalisation of a linear representation, Theorem 4.15, to a possibly nonlinear space). A function $\rho : \mathcal{X} \to \mathbb{R}$ is *G-invariant* if $\rho(\Phi_Q(x)) = \rho(x)$ for all Q, x. A function $u : \mathcal{X} \to T\mathcal{X}$ (with G also acting on the tangent bundle by the differential $d\Phi_Q$) is *G-equivariant* if $u(\Phi_Q(x)) = d\Phi_Q|_x \, u(x)$ for all Q, x: transform-then-evaluate equals evaluate-then-transform.

For the permutation group $G = S_n$ acting on $\mathbf{x} \in \mathbb{R}^{n\times 3}$ by reindexing, and for the rigid-motion group $SE(3)$ acting by $\Phi_{(Q,b)}(\mathbf{x})_i = Q\mathbf{x}_i + b$, Theorem 7.1 specialises to exactly the two requirements stated informally above; for the rotational part of (7.1), using the world/body-frame bookkeeping fixed in Section 3.4 (equivariance of the *body-frame* rotational output under a *world-frame* rotation of the whole scene means the body-frame output is left *unchanged*, since it is expressed relative to the fragment's own, co-rotating frame — an important, easily-misstated point, discussed again with the network's actual output convention in Section 8.9).

Proposition 7.2 (Why equivariance, not merely invariance, is the right constraint)**.** *Let G act on $\mathcal{M}$ by isometries (maps preserving the Riemannian metric g; both the permutation action on a graph's node set and the rigid-motion action on $\mathcal{M}_N$ satisfy this), let p_0 be G-invariant, and let u_θ be G-equivariant. Then every marginal p_t^θ of the flow $\frac{d}{dt}X_t = u_\theta(X_t, t)$, $X_0 \sim p_0$ (Chapter 10; the analogous statement for the diffusion SDEs of Chapters 5 and 6 holds by the same argument, stated precisely in Theorem 9.7), is G-invariant; in particular the generated data distribution p_1^θ is.*

Proof. Let $Y_t := \Phi_Q(X_t)$. Since Φ_Q is a fixed diffeomorphism, $\dot{Y}_t = d\Phi_Q|_{X_t}\dot{X}_t = d\Phi_Q|_{X_t} \, u_\theta(X_t, t) = u_\theta(\Phi_Q(X_t), t) = u_\theta(Y_t, t)$, using equivariance of u_θ in the last step: Y_t solves the *same* ODE as X_t. Since $Y_0 = \Phi_Q(X_0)$ has law $\Phi_{Q\#}p_0 = p_0$ (G-invariance of p_0), and solutions of an ODE are uniquely determined by their initial condition (under standard regularity, e.g. Lipschitz u_θ; a classical existence/uniqueness fact for ODEs we take as given), Y_t and X_t have the *same* law for every t. But $Y_t = \Phi_Q(X_t)$ by definition, so $\mathrm{Law}(X_t) = \mathrm{Law}(\Phi_Q(X_t)) = \Phi_{Q\#}\mathrm{Law}(X_t)$: $p_t^\theta := \mathrm{Law}(X_t)$ is G-invariant. □

This is the reason equivariance is not a stylistic preference: it is a *hard, exact* guarantee — for *every* value of the learned parameters θ, at *every* point during training, not only at convergence — that the generative model respects the symmetries of the physical problem, obtained purely from an architectural constraint rather than needing to be learned from data (data augmentation could *approximately* teach a non-equivariant network the same symmetry, at the cost of a strictly larger effective hypothesis space to search and no exact guarantee at any finite amount of training).

7.3 Attention, and the permutation-equivariance half of the problem

Definition 7.3 (Scaled dot-product self-attention; restated)**.** For tokens $X = (x_1, \dots, x_n)^\top \in \mathbb{R}^{n\times d}$ and projections $W_Q, W_K \in \mathbb{R}^{d\times d_k}$, $W_V \in \mathbb{R}^{d\times d_v}$, set $Q = XW_Q, K = XW_K, V = XW_V$ and

$$\mathrm{Attn}(X) = AV, \qquad A = \mathrm{softmax}\Big(\frac{QK^\top}{\sqrt{d_k}}\Big) \in \mathbb{R}^{n\times n}$$

(row-wise softmax, so $A_{ij} \geq 0$, $\sum_j A_{ij} = 1$).

Row i of $\mathrm{Attn}(X)$ is a convex combination $\sum_j A_{ij} v_j$ of value vectors, weighted by the learned compatibility $\langle q_i, k_j\rangle$ between tokens i and j; the $1/\sqrt{d_k}$ scaling keeps logits at unit scale for i.i.d. unit-variance q, k entries, preventing early softmax saturation. Multiple parallel *heads* (independent W_Q, W_K, W_V triples, concatenated and mixed by a further W_O) and a pre-normalised residual block,

$$X' = X + \mathrm{Attn}(\mathrm{LN}(X)), \qquad X'' = X' + \mathrm{MLP}(\mathrm{LN}(X')), \tag{7.2}$$

complete the ordinary Transformer layer.

Proposition 7.4 (Permutation equivariance of attention)**.** *For a permutation matrix $P \in \mathbb{R}^{n\times n}$:* $\mathrm{Attn}(PX) = P\,\mathrm{Attn}(X)$*, and* (7.2) *inherits this property.*

Proof. $Q, K, V \mapsto PQ, PK, PV$ (the projections act on the feature axis, so permuting rows of X permutes rows of Q, K, V identically). Hence $QK^\top \mapsto PQK^\top P^\top$; since row-wise softmax acts identically on each row regardless of row order, $A \mapsto PAP^\top$. Then $AV \mapsto PAP^\top PV = PAV$ using $P^\top P = I$. LN and MLP act identically, tokenwise, on every row, hence commute with P. □

Attention thus solves permutation equivariance exactly *because* it treats its input as an (index-labelled but otherwise symmetric) set: the formula (7.3) never references the token index i except through x_i itself. Two further observations connect attention to the constructions of the next section: first, (7.3) *is* message passing (Theorem 7.13 below) on the *complete* graph, with learned edge weight A_{ij}; second, its $\mathcal{O}(n^2 d)$ cost, from computing $QK^\top$ densely, is why in practice attention (and the equivariant version developed below) is restricted to a sparse graph — a radius or k-nearest-neighbour graph — rather than the complete graph, recovering tractability while keeping the learned, data-dependent weighting.

What (7.3) does *not* provide is any use of geometry: $\langle q_i, k_j\rangle$ depends on the tokens' *features*, never on where in $\mathbb{R}^3$ the corresponding atoms sit. Concatenating raw coordinates $x_i \in \mathbb{R}^3$ into the feature vector would inject geometry but destroy $SE(3)$-equivariance immediately (the resulting logits, and hence A, would change under a global rotation, since nothing in (7.3) constrains how a rotated coordinate enters the dot product). Geometry must instead enter through *relationships between atoms* — the subject of the next section.

7.4 Geometric graphs and message passing

7.4.1 An ordinary GNN, with no geometry at all

Start from the version with no symmetry considerations whatsoever, so that we can see exactly what has to be added.

A graph neural network keeps a feature vector $h_i \in \mathbb{R}^d$ at every node and updates it by looking at neighbours. One round of *message passing* is two steps:

$$m_{ij} = M\big(h_i,\, h_j,\, e_{ij}\big) \qquad \text{compute a message along each edge,} \tag{7.3}$$

$$h_i' = U\Big(h_i,\, \textstyle\sum_{j \in \mathcal{N}(i)} m_{ij}\Big) \qquad \text{aggregate and update.} \tag{7.4}$$

Here M and U are learned functions (typically MLPs), e_{ij} is an optional edge feature, and $\mathcal{N}(i)$ is the set of neighbours of i. Stacking T such rounds lets information travel T hops through the graph.

For a molecule the identification is immediate:

Graph object	Molecular meaning
node i	atom
h_i	learned chemical state: element, charge, hybridisation, ...
edge (i, j)	a pair of atoms considered to interact
e_{ij}	bond type, or nothing
$x_i \in \mathbb{R}^3$	the atom's position

Everything in (7.3)–(7.4) is standard deep learning. No representation theory has appeared, and none is needed — because no geometry has appeared either.

7.4.2 The question: where is the 3D structure?

Look at what the update actually consumes: h_i, h_j, e_{ij}. Atom types and bond types. The positions x_i appear nowhere.

The consequence is severe. Two conformers of the same molecule — the same atoms, the same bonds, folded into completely different shapes — have *identical* graphs in the sense of (7.3). A plain GNN cannot tell them apart, in principle, no matter how it is trained. For docking this is fatal: the entire task is to distinguish spatial arrangements.

So positions must enter. The question is how, and that question is delicate for a reason we can now state precisely:

> Geometry must enter in a form that carries *real spatial information* but does not smuggle in a *choice of coordinate frame*.

7.4.3 Why absolute positions are the wrong input

The naive fix is to append x_i to h_i and carry on. It fails on the first symmetry we listed in Section 4.1.

Translate the whole system by $a \in \mathbb{R}^3$, so $x_i \mapsto x_i + a$ for every atom. Nothing physical changed — the molecule is the same molecule, we moved the origin. But every input to the network changed, so every output changes too. The network would have to learn, from data, that all of $\mathbb{R}^3$ worth of translations are irrelevant. That is the augmentation problem of Section 4.1 in its purest form, and now over a *non-compact* group, where sampling coverage is hopeless.

The fix, and why it works. Use differences instead of positions:

$$\boxed{x_{ij} \;:=\; x_j - x_i \;\in\; \mathbb{R}^3} \tag{7.5}$$

Proposition 7.5 (The edge vector is translation-invariant and rotation-equivariant)**.** *Under a global rigid motion $x \mapsto Qx + a$ with $Q \in SO(3)$, $a \in \mathbb{R}^3$:*

$$x_{ij} \;\longmapsto\; (Qx_j + a) - (Qx_i + a) \;=\; Q(x_j - x_i) \;=\; Q\,x_{ij}.$$

Proof. The two copies of a cancel; Q factors out by linearity. □

Read the conclusion carefully, because it contains two different statements:

- the translation a has **disappeared** — x_{ij} is translation-*invariant*, and translation symmetry is now exact and free, requiring no further work anywhere in the architecture;
- the rotation Q has **survived** — x_{ij} is rotation-*equivariant*, an $\ell = 1$ object, and rotation symmetry is therefore still a live constraint.

This asymmetry is why the rest of this document is about $SO(3)$ and barely mentions translations again.

Caution 7.6 (Edge orientation, fixed once). We use $x_{ij} = x_j - x_i$ throughout: the vector *from i to j*, i.e. "where is my neighbour, seen from me". The message m_{ij} correspondingly flows from j into i.

The opposite convention $x_j - x_i \rightsquigarrow x_i - x_j$ is equally defensible and used by some sources, and the choice is *not* harmless: $x_{ji} = -x_{ij}$, and by Theorem 7.7 below the spherical harmonics satisfy $Y^\ell(-\hat{u}) = (-1)^\ell Y^\ell(\hat{u})$. So reversing the edge flips the sign of every *odd-ℓ* angular feature while leaving even ones alone. A model built with one convention and evaluated with the other is not merely mismatched — it is systematically wrong on exactly the $\ell = 1$ channels that carry the directional output.

Proposition 7.7 (Parity of the spherical harmonics)**.** *For $\hat{u} \in S^2$ and every degree ℓ, $Y^\ell(-\hat{u}) = (-1)^\ell\, Y^\ell(\hat{u})$.*

Proof. By Theorem 4.20, Y^ℓ is the restriction to S^2 of a homogeneous polynomial of degree ℓ, and a homogeneous polynomial of degree ℓ satisfies $p(-x) = (-1)^\ell p(x)$. □

7.4.4 Splitting an edge into radius and direction

We now have one geometric object per edge, $x_{ij} \in \mathbb{R}^3$. The next move is the one that organises the entire architecture: *split it.*

$$x_{ij} \;=\; \underbrace{d_{ij}}_{\text{how far}} \cdot \underbrace{\hat{x}_{ij}}_{\text{which way}}, \qquad d_{ij} = \|x_{ij}\| \in \mathbb{R}_{>0}, \qquad \hat{x}_{ij} = \frac{x_{ij}}{d_{ij}} \in S^2. \tag{7.6}$$

Why split at all? Because the two halves have *different symmetry types*, and therefore must be processed by different machinery:

Piece	Lives in	Under Q	Consequence
d_{ij}	$\mathbb{R}_{>0}$	unchanged	$\ell = 0$: ordinary neural operations are legal
$\hat{x}_{ij}$	S^2	$\mapsto Q\hat{x}_{ij}$	equivariant: needs steerable treatment

The invariance of the distance is Theorem 3.16: $\|Qx_{ij}\| = \|x_{ij}\|$. That one line is what licenses everything in Section 7.4.5.

Caution 7.8 (The split degenerates at $d_{ij} = 0$). $\hat{x}_{ij}$ is undefined when $x_i = x_j$. In a molecular graph two distinct atoms never coincide, so this is not a modelling problem — but it is a numerical one, because $\hat{x}_{ij}$ and its derivatives blow up as $d_{ij} \to 0$, and self-edges $i = j$ would trigger it exactly. Implementations therefore exclude self-loops from the radius graph, or add an ε to the denominator. Chapter 13 records what SigmaDock actually does.

The organising principle

Not everything in an equivariant network needs representation theory. *Invariant scalars may be processed by ordinary neural operations* — MLPs, ReLU, layer norm, anything. Representation theory is needed only where directional information lives. The radial/angular split is the device that separates the two, so that each can be handled with the cheapest sufficient tool. Nearly every design decision in Chapter 8 is an instance of this principle.

7.4.5 The radial half: from a distance to a feature vector

Since d_{ij} is invariant, any function of it is invariant, and we may use a plain MLP. So why do molecular models almost always embed the distance first, into a vector $\phi(d_{ij}) \in \mathbb{R}^K$, instead of feeding the single number 3.71 directly?

This is an architectural choice, not a requirement. The reasons it is usually made:

1. **Locality of response.** A network often needs to react differently to 1.4 Å (a covalent bond), 2.8 Å (a hydrogen bond) and 5 Å (weak contact). A single scalar input forces an MLP to carve these regimes out of one monotone coordinate; a basis of localised bumps hands it the regimes directly, and each output weight can specialise.
2. **Conditioning.** Raw distances live on a scale of a few Ångström; basis outputs are $O(1)$ and bounded, which behaves better under standard initialisation.
3. **Smoothness control.** The basis width sets how fast the model *can* vary with distance, acting as an architectural prior against fitting sharp, spurious distance dependence.
4. **A natural place for the cutoff.** Multiplying the basis by an envelope (Section 7.4.6) makes every downstream quantity go smoothly to zero at the graph boundary.

Definition 7.9 (Radial basis embedding)**.** A *radial basis* is a map $\phi : \mathbb{R}_{\geq 0} \to \mathbb{R}^K$, $\phi(d) = (\phi_1(d), \dots, \phi_K(d))$. Two common choices:

$$\text{Gaussian:}\quad \phi_k(d) = \exp\!\big(-\gamma\,(d - \mu_k)^2\big), \qquad \text{Bessel:}\quad \phi_k(d) = \sqrt{\tfrac{2}{r_c}}\,\frac{\sin(k\pi d/r_c)}{d},$$

with centres $\mu_1 < \cdots < \mu_K$ spanning the interaction range, width γ, and cutoff r_c.

Shapes. Input $d_{ij} \in \mathbb{R}$ (one number per edge); output $\phi(d_{ij}) \in \mathbb{R}^K$. For a graph with E edges the batched tensor is $[E, K]$.

Invariance. Each ϕ_k is a function of an invariant, hence invariant: $\phi_k(\|Qx_{ij}\|) = \phi_k(\|x_{ij}\|)$. The whole vector $\phi(d_{ij})$ is therefore a collection of K independent $\ell = 0$ features, and may be fed to an ordinary MLP, $\psi(d_{ij}) = \mathrm{MLP}\big(\phi(d_{ij})\big)$, whose output is again invariant.

Example 7.10 (Three Gaussian centres)**.** Take $K = 3$, centres $\mu = (1.5, 3.0, 4.5)\,\text{Å}$ and $\gamma = 1$. Then

$$\phi(1.6) \approx (0.99, 0.13, 0.0003), \qquad \phi(3.1) \approx (0.10, 0.99, 0.14), \qquad \phi(4.4) \approx (0.0003, 0.14, 0.99).$$

Each distance activates mainly one coordinate, and neighbouring coordinates overlap smoothly so that the representation is continuous in d. A downstream linear layer can now assign an independent weight to "short", "medium" and "long" contacts — which is exactly what one would have had to *learn* to extract from the raw number.

7.4.6 The cutoff

Molecular graphs are built with a radius cutoff, $\mathcal{E} = \{(i, j) : d_{ij} < r_c\}$. Three reasons: *physics* (interactions decay, and beyond a few Ångström the influence is negligible), *cost* (a complete graph on N atoms has N^2 edges; a radius graph has $O(N)$ for fixed density), and *node degree* (bounded neighbourhoods keep the aggregation well-scaled).

But a hard cutoff introduces a discontinuity: an atom drifting across r_c makes its message appear or vanish abruptly. That is undesirable for two concrete reasons.

First, the model's output becomes discontinuous in the atom positions. For a *generative* model this is worse than cosmetic: the sampler integrates a vector field along a trajectory, and a field with jump discontinuities is not something an ODE solver can integrate reliably.

Second, gradients are ill-behaved near the boundary.

The remedy is an envelope $\chi : [0, r_c] \to [0, 1]$ with $\chi(r_c) = 0$ (and usually $\chi'(r_c) = 0$), multiplied into the radial features: $\psi(d)\,\chi(d)$. A common choice is the polynomial envelope $\chi(d) = 1 - 6u^5 + 15u^4 - 10u^3$ with $u = d/r_c$, which satisfies $\chi(0) = 1$, $\chi(r_c) = 0$ and $\chi'(0) = \chi'(r_c) = 0$. The message then fades to zero smoothly as a neighbour leaves the cutoff, and nothing jumps.

7.4.7 The angular half: why spherical harmonics

Now the harder piece: $\hat{x}_{ij} \in S^2$.

We cannot simply pass the three components of $\hat{x}_{ij}$ through an MLP as if they were scalars — that is precisely the error Theorem 4.14 rules out. We need to turn a direction into features whose transformation law we know.

We already have the tool. Section 4.6 constructed, for each degree ℓ, a map $Y^\ell : S^2 \to \mathbb{R}^{2\ell+1}$ with

$$Y^\ell(Q\hat{u}) = D^\ell(Q)\, Y^\ell(\hat{u})$$

(Equation (4.1), in this document's fixed convention). Applying it to the edge direction gives

$$Y^\ell(\hat{x}_{ij}) \in \mathbb{R}^{2\ell+1}, \qquad Y^\ell(\hat{x}_{ij}) \longmapsto D^\ell(Q)\, Y^\ell(\hat{x}_{ij}) \ \text{ under a global rotation,}$$

because $\hat{x}_{ij} \mapsto Q\hat{x}_{ij}$ by Theorem 7.5.

The point of spherical harmonics, stated plainly

They are not merely a convenient basis for functions on a sphere. They are the map that converts a raw geometric direction — an object the network cannot use — into features of *exactly the types* $\ell = 0, 1, 2, \ldots$ *that the network's algebra is built around.* Before Y^ℓ, the edge direction is data of unknown type. After Y^ℓ, it is a steerable feature that can enter a Clebsch–Gordan product like any other.

What the different degrees carry. For a *single* direction $\hat{u}$:

- $\ell = 0$: Y^0 is constant. It carries *no* directional information at all — the same value for every edge.
- $\ell = 1$: $Y^1(\hat{u}) \propto \hat{u}$. First-order direction: which way.
- $\ell = 2$: quadratic in the components. It distinguishes $\hat{u}$ from $-\hat{u}$ not at all ($(-1)^2 = +1$, Theorem 7.7) but resolves *axis* information — alignment with a line rather than a ray.
- higher ℓ: finer angular resolution, in the same sense that higher Fourier modes resolve finer detail on a circle.

The useful mental model is angular resolution. Truncating at $L = \ell_{\max}$ is like low-pass filtering the angular dependence: the network can represent angular patterns up to that frequency and no finer. Raising L increases expressive power, but the cost grows quickly — the number of components per node is $\sum_{\ell \le L}(2\ell+1) = (L+1)^2$, and the number of Clebsch–Gordan paths grows faster still. Whether a larger L helps is an empirical question and depends on how much genuine angular structure the task has; it is not a free improvement. EquiformerV2's design is in large part a response to this cost (Section 8.4).

7.4.8 Recombining radius and direction

We now have, per edge, an invariant radial vector $\psi(d_{ij}) \in \mathbb{R}^K$ and a family of steerable angular features $Y^\ell(\hat{x}_{ij}) \in \mathbb{R}^{2\ell+1}$. The natural way to put them back together is

$$e_{ij}^{(\ell)} \;=\; f^{(\ell)}\big(d_{ij}\big)\; Y^\ell(\hat{x}_{ij}) \;\in\; \mathbb{R}^{2\ell+1}, \tag{7.7}$$

where $f^{(\ell)}$ is any learned scalar function of the distance (in practice a channel of $\mathrm{MLP}(\phi(d_{ij}))$).

Why this is legal: $f^{(\ell)}(d_{ij})$ is an invariant *scalar*, and a scalar commutes with $D^\ell(Q)$. So

$$f^{(\ell)}(d_{ij})\, D^\ell(Q)\, Y^\ell(\hat{x}_{ij}) = D^\ell(Q)\Big(f^{(\ell)}(d_{ij})\, Y^\ell(\hat{x}_{ij})\Big),$$

i.e. $e_{ij}^{(\ell)}$ is again a degree-ℓ feature. This is the gating argument of Section 4.10.10 reappearing in a geometric costume: an invariant modulating an equivariant.

Remark 7.11 (The split is not a trick; it is the geometry of $\mathbb{R}^3$). Writing a function of a 3-vector as "radial part times angular part" is exactly what separation of variables in spherical coordinates does: $f(x) = \sum_{\ell m} f_{\ell m}(r)\, Y_m^\ell(\hat{x})$, with the spherical harmonics as the angular basis. Equation (7.7) is one term of such an expansion with a learned radial coefficient. So the architecture is not imposing an arbitrary factorisation on the data — it is using the factorisation that the rotation group itself induces on functions of $\mathbb{R}^3$. We do not develop the full harmonic analysis; the point is only that the split is natural rather than expedient.

Definition 7.12 (Geometric graph; restated)**.** A *geometric graph* is $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \{h_i\}, \{x_i\})$: nodes $i \in \mathcal{V}$ with $SE(3)$-invariant features $h_i \in \mathbb{R}^d$ (atom type, charge, fragment/entity type,

time embedding of t) and positions $x_i \in \mathbb{R}^3$; edges $\mathcal{E} \subseteq \mathcal{V}\times\mathcal{V}$. Write $x_{ij} = x_j - x_i$, $d_{ij} = \|x_{ij}\|$; we take $\mathcal{E}$ to be a radius graph, $\{(i,j) : d_{ij} < r_{\text{cut}}\}$, for some fixed or per-edge-type cutoff r_{cut}.

Definition 7.13 (Message passing layer; restated)**.** A message passing layer computes

$$m_{ij} = \phi_m(h_i, h_j, e_{ij}), \qquad m_i = \bigoplus_{j\in\mathcal{N}(i)} m_{ij}, \qquad h_i' = \phi_u(h_i, m_i), \tag{7.8}$$

for learned maps ϕ_m, ϕ_u, edge attributes e_{ij}, and a permutation-invariant aggregator $\bigoplus$ (sum or mean over j).

Since $\bigoplus$ depends only on the *multiset* $\{m_{ij}\}$, not on any ordering of $\mathcal{N}(i)$, (7.8) is permutation equivariant by the same argument as Theorem 7.4 (indeed Theorem 7.3 is the special case $\mathcal{N}(i) = \mathcal{V}$ for all i, $\bigoplus$ a learned convex combination). Geometry enters through e_{ij}; the invariant (and hence, per Section 7.5 below, safe-but-limited) choice expands the distance in a radial basis,

$$e_{ij} = \big(\psi_1(d_{ij}), \dots, \psi_K(d_{ij})\big), \qquad \psi_k(d) = \exp\!\big(-\gamma(d-\mu_k)^2\big)\,\chi(d), \tag{7.9}$$

with basis centres μ_k and a smooth cutoff envelope χ vanishing (along with enough derivatives to avoid a kink) at $d = r_{\text{cut}}$: an edge that is about to leave the radius graph as atoms move contributes a message that continuously shrinks to zero *before* it is dropped from $\mathcal{E}$ entirely, rather than discontinuously disappearing — without this, the learned vector field would be discontinuous in $\mathbf{x}$ exactly at the cutoff radius, with predictable consequences for the stability of the ODE/SDE integrators of Chapters 5 and 10.

Code correspondence

Mathematical object: the radial basis $\psi_k(d)\chi(d)$ of (7.9)
Implementation: `net/encoder.py`::`BesselBasis`, `PolynomialCutoff`
Interpretation: the actual basis used is the (MACE-style) Bessel form $\psi_k(d) = \sqrt{2/r_{\max}}\,\sin(k\pi d_{\text{scaled}}/r_{\max})/(d_{\text{scaled}}+\varepsilon)$ (trainable frequencies $k\pi/r_{\max}$, not fixed Gaussian centres as in (7.9) — this document presents the Gaussian form first purely because it is the more elementary starting point, and records the Bessel form actually used as a documented substitution, mathematically serving an identical role: an orthogonal-ish family of radial basis functions), multiplied by a degree-8 polynomial envelope

$$\chi(d) = \Big[1 - \tfrac{(p+1)(p+2)}{2}\Big(\tfrac{d}{r_{\max}}\Big)^p + p(p+2)\Big(\tfrac{d}{r_{\max}}\Big)^{p+1} - \tfrac{p(p+1)}{2}\Big(\tfrac{d}{r_{\max}}\Big)^{p+2}\Big]\mathbf{1}[d < r_{\max}], \quad p = 8,$$

constructed exactly so that χ and its first two derivatives vanish at $d = r_{\max}$ (a standard DimeNet/MACE-style envelope), confirming the smooth-cutoff argument above at the level of the actual, exact formula used. **Exact cutoff radii used**, per `oracle.py`: protein–ligand interaction edges (`inter_complex`) and ligand–ligand cross-fragment edges (`inter_fragments`), both 4.0 Å; Cα–Cα protein-virtual edges, 18 Å; ligand-virtual–protein-virtual edges, 15 Å — these are distinct from, and should not be confused with, the separate pocket-definition cutoffs (`data.py`: `pocket_distance_cutoff`= 6.0, `pocket_virtual_cutoff`= 15.0) governing which protein atoms are extracted into the graph *at all*, upstream of any of the above edge-construction cutoffs.

7.5 The obstruction: invariant features cannot output a vector

We can now state, precisely, why (7.8) as given is insufficient for (7.1)'s task.

Proposition 7.14 (Invariant features cannot produce an equivariant vector output)**.** *Suppose every node feature h_i (initial and, by induction through (7.8), at every subsequent layer, since ϕ_m, ϕ_u are ordinary functions of $SE(3)$-invariant inputs h_i, h_j, e_{ij}) is $SE(3)$-invariant, and a readout $v_i = \rho(h_i) \in \mathbb{R}^3$ is defined as a function of h_i alone. If v_i is required to be $SE(3)$-equivariant in the sense of (7.10) below, then $v_i \equiv 0$.*

$$v_i\big(\Phi_{(Q,b)}(\mathbf{x})\big) = Q\, v_i(\mathbf{x}) \qquad \forall (Q,b) \in SE(3). \tag{7.10}$$

Proof. By hypothesis, $h_i(\Phi_{(Q,b)}(\mathbf{x})) = h_i(\mathbf{x})$ for every (Q,b) (invariance), so $v_i(\Phi_{(Q,b)}(\mathbf{x})) = \rho(h_i(\Phi_{(Q,b)}(\mathbf{x}))) = \rho(h_i(\mathbf{x})) = v_i(\mathbf{x})$: the left side of (7.10) equals $v_i(\mathbf{x})$ identically, so (7.10) forces $Qv_i(\mathbf{x}) = v_i(\mathbf{x})$ for *every* $Q \in SO(3)$. The only vector in $\mathbb{R}^3$ fixed by every rotation is 0 (if $v \neq 0$, take Q a rotation by 180° about an axis perpendicular to v: $Qv = -v \neq v$). □

This is the obstruction promised at the end of Chapter 3's motivation: a purely distance-based (invariant-feature) message passing network can predict *any scalar* (an energy, an affinity, a confidence score) but literally cannot, for any choice of ϕ_m, ϕ_u, ρ, output the translational or rotational velocity (7.1) requires. Invariance is too strong a symmetry constraint for a vector-valued task; equivariance (a strictly weaker requirement, since invariance is the special case "equivariant with respect to the trivial representation", Theorem 4.19) is needed instead, and providing it is the entire content of Chapter 8.

7.6 A minimal repair, and why it is not enough

Before building the full steerable machinery, it is instructive to see the cheapest possible fix, since its specific limitations motivate exactly the representation-theoretic apparatus of Chapter 4.

Definition 7.15 (EGNN-style equivariant readout)**.**

$$v_i = \sum_{j \in \mathcal{N}(i)} \phi_v(h_i, h_j, d_{ij})\, x_{ij}, \qquad \phi_v : \text{ scalar-valued, built from invariant inputs only.}$$

Proposition 7.16. *Theorem 7.15's v_i satisfies (7.10).*

Proof. Under $\mathbf{x} \mapsto Q\mathbf{x} + b$: $x_{ij} \mapsto Qx_{ij}$ (translations cancel in a difference of two points; the term "$+b$" subtracted from itself), and $d_{ij} = \|x_{ij}\|$ is unchanged (Theorem 3.16), so $\phi_v(h_i, h_j, d_{ij})$, depending only on invariants, is unchanged too. Hence $v_i \mapsto \sum_j \phi_v(\dots)\, Qx_{ij} = Q \sum_j \phi_v(\dots) x_{ij} = Qv_i$. □

This construction — known as EGNN (E(n)-Equivariant Graph Neural Networks; Satorras et al., ICML 2021) — is genuinely equivariant and genuinely cheap, but its expressive limitation is structural, not merely empirical, and worth stating precisely rather than only qualitatively. Theorem 7.15's v_i is a *scalar-weighted sum of degree-1 vectors*: in the language of Chapter 4, it uses only the $\ell_1 = 0$ (scalar weight ϕ_v) $\otimes_{\text{cg}}$ $\ell_2 = 1$ (vector x_{ij}) $\to \ell = 1$ case of Theorem 4.44, and *no other Clebsch–Gordan path at all.* Two concrete consequences follow directly from Theorem 4.48's degree-mixing constraint and Theorem 4.34's decomposition rule, not from vague "expressivity" intuition:

1. **Angular patterns among three or more neighbours are invisible.** A ring of neighbours arranged symmetrically around node i, all at the same distance and evenly spaced in angle, produces $\sum_j x_{ij} \approx 0$ by symmetry (the vectors cancel), so Theorem 7.15's readout is insensitive to the ring's orientation, even though that orientation is physically meaningful (e.g. the plane of an aromatic ring relative to a binding pocket wall); Theorem 4.22's explicit degree-2 harmonic $\propto 2x_3^2 - x_1^2 - x_2^2$ (a quadrupole) does *not* cancel under the same symmetric averaging, and this is precisely the sense in which $\ell \geq 2$ features carry information that $\ell \leq 1$ features cannot.

2. **Axial (rotation-vector) outputs get the wrong reflection parity unless tracked explicitly.** $\omega_\theta^{(i)} \in \mathfrak{so}(3) \cong \mathbb{R}^3$ is an *axial* vector (Theorem 3.42): under a reflection, $\widehat{\omega} \mapsto -\det(Q)\, Q\widehat{\omega}Q^\top$ for $Q \in O(3)$ picks up an extra sign relative to an ordinary (*polar*) vector like x_{ij} under the same reflection, a fact provable directly from Theorem 3.20's cross-product identity together with the general transformation rule $(Ax) \times (Ay) = \det(A)A^{-\top}(x \times y)$ quoted in the proof of Theorem 3.16, now with $A = Q \in O(3) \setminus SO(3)$ so $\det Q = -1$. A construction of exactly the form of Theorem 7.15, built only from polar vectors x_{ij}, assigns $\omega_\theta^{(i)}$ the *polar* parity by default, which is simply the wrong object unless the reflection sign is tracked and corrected elsewhere — an easy silent error, and the reason Chapter 8 (and, per Theorem 3.42, the discussion of molecular chirality) carries a parity label alongside ℓ.

What is needed, and what Chapter 4 already built, is: features of every angular degree ℓ, not only 0 and 1; and a way to combine two such features, of *any* degrees, into a third, without leaving the class of equivariant objects — exactly the Clebsch–Gordan tensor product (4.7). We assemble that now.

7.7 The steerable message

Everything is in place. We have, per edge, an invariant radial part and a steerable angular part; per node, steerable features; and, from Section 4.9, the one legal way to multiply two steerable objects. Assembling them gives the central equation of equivariant message passing.

7.7.1 The equation

$$\boxed{m_{ij}^{(\ell)} \;=\; \sum_{\ell_1,\ell_2} w^{(\ell_1,\ell_2,\ell)}(d_{ij}) \left(h_j^{(\ell_1)} \;\otimes_{\mathrm{cg}}\; Y^{\ell_2}(\hat{x}_{ij})\right)^{(\ell)}} \tag{7.11}$$

where the sum runs over pairs (ℓ_1, ℓ_2) for which ℓ is allowed, i.e. $|\ell_1 - \ell_2| \leq \ell \leq \ell_1 + \ell_2$ (Section 4.9.4).

Taken slowly, piece by piece:

$h_j^{(\ell_1)}$ — **what the neighbour knows.** The current state of atom j: a steerable feature of degree ℓ_1, living in $\mathbb{R}^{2\ell_1+1}$ per channel. It encodes chemistry — element, environment, whatever the previous layers accumulated — with no reference to where j sits relative to i.

$Y^{\ell_2}(\hat{x}_{ij})$ — **where the neighbour is.** The edge direction, converted by Section 7.4.7 into a degree-ℓ_2 feature in $\mathbb{R}^{2\ell_2+1}$. It encodes geometry, with no reference to what kind of atom j is.

$\otimes_{\text{cg}}$ — **the only legal way to multiply them.** Chemistry and geometry are features of different degrees; the Clebsch–Gordan product is the unique (up to scale) bilinear map combining them equivariantly, and the superscript (ℓ) selects one allowed output degree. This is where the two kinds of information actually meet.

$w^{(\ell_1,\ell_2,\ell)}(d_{ij})$ — **how much, at this distance.** A learned scalar, computed from the radial features $\phi(d_{ij})$ of Section 7.4.5 by an ordinary MLP. Two questions about it deserve answers.

Why may it be an ordinary scalar? Because d_{ij} is invariant (Theorem 3.16), so w is invariant, and an invariant scalar multiplying an equivariant feature preserves equivariance (Section 4.10.10). No representation theory is needed for w itself.

Why should it depend on distance at all? Because the strength of an interaction does. A covalent neighbour at 1.4 Å and a loose contact at 5 Å should not contribute equally, and the network should be able to learn a different distance profile for each coupling path $(\ell_1, \ell_2) \to \ell$ — e.g. to let a short-range path dominate the $\ell = 2$ output while a long-range one dominates $\ell = 0$. Making w a function of d_{ij}, with a separate output channel per path, is what grants that freedom.

$m_{ij}^{(\ell)}$ — **the result.** A degree-ℓ feature in $\mathbb{R}^{2\ell+1}$, attached to the edge, ready to be delivered to node i.

What the message says

A plain GNN message says:

> "Atom j has these properties."

A steerable message says:

> "Atom j has these properties, *and it is over there, this far away.*"

The difficulty is entirely in the words "over there". A direction cannot be appended as three ordinary numbers without breaking rotation equivariance. Passing it through Y^{ℓ_2} converts it into an object of known type, and the Clebsch–Gordan product then couples it to the chemistry in the one way that keeps the whole message co-rotating with the molecule.

7.7.2 Two cases worth doing by hand

Example 7.17 (Scalar node feature: $0 \otimes 1 \to 1$)**.** Suppose j carries only a scalar, $h_j^{(0)} \in \mathbb{R}$, and we use $\ell_2 = 1$. The selection rule allows only $\ell = 1$, and by Theorem 4.37 the Clebsch–Gordan product with a scalar is ordinary multiplication:

$$m_{ij}^{(1)} \;=\; w(d_{ij})\, h_j^{(0)}\, Y^1(\hat{x}_{ij}) \;\propto\; \underbrace{w(d_{ij})\, h_j^{(0)}}_{\text{a number}} \cdot \underbrace{\hat{x}_{ij}}_{\text{a direction}} .$$

Interpretation: *a scalar at j becomes a vector at i, pointing along the edge, with magnitude set by the scalar and the distance.* If the scalar channel has learned to encode "how strongly I should push", the message is literally a push in the direction of the neighbour.

This is worth dwelling on because it is the minimal mechanism by which directional information is created from non-directional information — which, by Theorem 4.45, no linear layer

could do. The direction did not come from the features; it came from the geometry, and the tensor product is what let them meet. It is also, structurally, exactly the EGNN readout of Theorem 7.15: EGNN is the special case of (7.11) with $\ell_1 = 0$, $\ell_2 = 1$ and nothing else.

Example 7.18 (Vector node feature: $1 \otimes 1 \to 0 \oplus 1 \oplus 2$)**.** Now let j carry a vector feature $h_j^{(1)} \in \mathbb{R}^3$, still with $\ell_2 = 1$. Now *three* output degrees are available, and by Section 4.9.3 we know exactly what each is:

Output	Concretely	What it detects
$\ell = 0$	$\propto h_j^{(1)} \cdot \hat{x}_{ij}$	how much j's vector points *along* the edge
$\ell = 1$	$\propto h_j^{(1)} \times \hat{x}_{ij}$	the component *perpendicular* to the edge
$\ell = 2$	sym. traceless part	the *alignment* of j's vector with the edge axis

A single pair of inputs produces three qualitatively different, and simultaneously available, geometric readings. The $\ell = 0$ output is an invariant that node i can feed to any MLP; the $\ell = 1$ output is a new direction; the $\ell = 2$ output is orientation information that neither of the others can express.

This is the payoff for Section 4.9.3. The decomposition $1 \otimes 1 = 0 \oplus 1 \oplus 2$ was, at the time, an exercise in splitting a 3×3 matrix. Here it is a statement about what a molecule's geometry can tell a network.

7.7.3 Aggregation over neighbours

Each edge produces a message; node i must combine them:

$$m_i^{(\ell)} \;=\; \sum_{j \in \mathcal{N}(i)} m_{ij}^{(\ell)}. \tag{7.12}$$

Proposition 7.19 (Sum aggregation is equivariant)**.** *If each* $m_{ij}^{(\ell)} \mapsto D^\ell(Q)\, m_{ij}^{(\ell)}$*, then* $m_i^{(\ell)} \mapsto D^\ell(Q)\, m_i^{(\ell)}$*.*

Proof.

$$\sum_j D^\ell(Q)\, m_{ij}^{(\ell)} \;=\; D^\ell(Q) \sum_j m_{ij}^{(\ell)} \;=\; D^\ell(Q)\, m_i^{(\ell)},$$

by linearity of $D^\ell(Q)$. □

The proof is one line, but the hypothesis is what matters: *all summands must live in the same representation space.* Adding an $\ell = 1$ message to an $\ell = 2$ message is not merely bad practice — the sum has no transformation law at all. This is why implementations keep degrees in separate buffers and sum within each.

Note also what is *not* required: nothing about $\mathcal{N}(i)$, its size, or the order of summation.

Caution 7.20 (Two different symmetries, easily conflated). Equation (7.12) is doing two independent jobs.

Spatial symmetry ($SO(3)$). Handled by Theorem 7.19: the sum commutes with $D^\ell(Q)$ because $D^\ell(Q)$ is linear.

Permutation symmetry (S_n). The neighbours of i form a *set*; they have no canonical order, and the answer must not depend on one. A sum is permutation-invariant because addition is commutative.

These are different groups acting on different things — rotations act on $\mathbb{R}^3$, permutations act on node labels — and a construction can respect one and violate the other. Concatenating neighbour messages in index order, for instance, is perfectly rotation-equivariant and badly permutation-dependent. Sum (or mean, or max over invariants) is the standard way to secure the second; Theorem 7.4 treats it in its own right.

7.7.4 Self-interaction and residual connections

A node should not forget itself. The update combines the aggregated message with the node's own state:

$$h_i^{(\ell),\text{new}} \;=\; W_{\text{self}}^{(\ell)}\, h_i^{(\ell)} \;+\; m_i^{(\ell)}, \tag{7.13}$$

where $W_{\text{self}}^{(\ell)}$ is an equivariant linear layer (Equation (4.10)), i.e. channel mixing within degree ℓ.

Equivariance is immediate and worth stating as a general principle:

Proposition 7.21 (Sums of equivariant maps of the same type are equivariant)**.** *If f, g both map into V_ℓ and both satisfy $f(Q \cdot x) = D^\ell(Q) f(x)$, then so does $f + g$. In particular, with $g = \mathrm{id}$, the residual connection $h \mapsto h + f(h)$ is equivariant whenever f is and f's output has the same degree as h.*

Proof. $(f+g)(Q\cdot x) = D^\ell(Q)f(x) + D^\ell(Q)g(x) = D^\ell(Q)\big((f+g)(x)\big)$. □

The residual case is what makes deep equivariant transformers possible: every block may be written $h \mapsto h + \mathrm{Block}(h)$, and Theorem 7.21 guarantees the skip connection costs nothing in symmetry — *provided* the block's output degrees match its input degrees, which is why architectures fix an irrep specification and keep it constant across blocks.

Remark 7.22 (Normalisation: a preview). Deep networks need normalisation, and the naive version is unavailable here: computing a mean and variance across all m-components of an $\ell \geq 1$ feature and subtracting the mean would treat the components as interchangeable numbers — the same error as componentwise ReLU. Equivariant normalisation must instead act on invariants (e.g. rescale each irrep block by a function of its norm, leaving direction untouched, in the spirit of Section 4.10.9). EquiformerV2 has a specific construction; we defer it to Chapter 8 rather than sketch it here.

7.7.5 The pipeline, end to end

Collecting everything:

$$
\begin{array}{c}
x_i,\, x_j \in \mathbb{R}^3 \implies x_{ij} = x_j - x_i \in \mathbb{R}^3 \quad \text{(translation-invariant, Theorem 7.5)} \\
\Downarrow \\
x_{ij} = d_{ij}\,\hat{x}_{ij} \qquad d_{ij} \in \mathbb{R}_{>0}\ (\ell = 0), \qquad \hat{x}_{ij} \in S^2 \ \text{(equivariant)} \\
\Downarrow \\
\phi(d_{ij}) \in \mathbb{R}^K \ \text{(radial, invariant)} \qquad Y^{\ell_2}(\hat{x}_{ij}) \in \mathbb{R}^{2\ell_2+1} \ \text{(angular, steerable)} \\
\Downarrow \\
h_j^{(\ell_1)} \otimes_{\text{cg}} Y^{\ell_2}(\hat{x}_{ij}) \qquad \text{allowed } \ell \in \{|\ell_1 - \ell_2|, \ldots, \ell_1 + \ell_2\} \\
\Downarrow \\
m_{ij}^{(\ell)} = \textstyle\sum_{\ell_1 \ell_2} w^{(\ell_1,\ell_2,\ell)}(d_{ij})\,\big(\cdots\big)^{(\ell)} \in \mathbb{R}^{2\ell+1} \\
\Downarrow \\
m_i^{(\ell)} = \textstyle\sum_{j \in \mathcal{N}(i)} m_{ij}^{(\ell)} \qquad \text{(Theorem 7.19)} \\
\Downarrow \\
h_i^{(\ell),\text{new}} = W_{\text{self}}^{(\ell)} h_i^{(\ell)} + m_i^{(\ell)}
\end{array}
$$

7.7.6 A small worked example

Three atoms i, j, k; features 1 channel of $\ell = 0$ and 1 channel of $\ell = 1$ per node; $L = \ell_{\max} = 1$; $K = 3$ radial basis functions.

Object	Shape (per edge / node)	Type
x_i	[3]	position (not a feature)
$x_{ij} = x_j - x_i$	[3]	$\ell = 1$
$d_{ij} = \|x_{ij}\|$	[1]	$\ell = 0$
$\hat{x}_{ij}$	[3]	on S^2
$\phi(d_{ij})$	[3]	$\ell = 0$ (three basis values)
$Y^0(\hat{x}_{ij})$	[1]	$\ell = 0$ (constant)
$Y^1(\hat{x}_{ij})$	[3]	$\ell = 1$
$h_j^{(0)}$	[1, 1] ($c \times m$)	$\ell = 0$
$h_j^{(1)}$	[1, 3]	$\ell = 1$

Which coupling paths exist? With inputs of degree $\ell_1 \in \{0, 1\}$, edge harmonics of degree $\ell_2 \in \{0, 1\}$, and outputs truncated at $\ell \le 1$:

(ℓ_1, ℓ_2)	allowed ℓ	kept ($\ell \le 1$)
$(0, 0)$	0	0
$(0, 1)$	1	1
$(1, 0)$	1	1
$(1, 1)$	0, 1, 2	0, 1 (the $\ell = 2$ path is discarded by the truncation)

So five surviving paths, each with its own learned radial weight $w^{(\ell_1,\ell_2,\ell)}(d_{ij})$ — five scalar functions of distance, i.e. a radial MLP $\mathbb{R}^3 \to \mathbb{R}^5$.

Messages. For edge $i \leftarrow j$:

$$m_{ij}^{(0)} = w_{(0,0,0)}\, h_j^{(0)} Y^0 + w_{(1,1,0)}\big(h_j^{(1)} \cdot \hat{x}_{ij}\big) \in \mathbb{R}^1,$$

$$m_{ij}^{(1)} = w_{(0,1,1)}\, h_j^{(0)}\, Y^1(\hat{x}_{ij}) + w_{(1,0,1)}\, h_j^{(1)} + w_{(1,1,1)}\big(h_j^{(1)} \times \hat{x}_{ij}\big) \in \mathbb{R}^3,$$

suppressing normalisation constants. Every term on the right of the first line is a scalar; every term on the second is a 3-vector. Nothing else could appear.

Aggregation and update. $m_i^{(\ell)} = m_{ij}^{(\ell)} + m_{ik}^{(\ell)}$, then (7.13). Output shapes are unchanged: $[1, 1]$ and $[1, 3]$ per node, ready for the next block.

Sanity check. Rotate all three positions by Q and translate by a. Then d_{ij}, ϕ, and all w's are unchanged; $\hat{x}_{ij} \mapsto Q\hat{x}_{ij}$; $Y^1 \mapsto QY^1$; so $m_i^{(0)}$ is unchanged and $m_i^{(1)} \mapsto Qm_i^{(1)}$ — exactly as required. (Simulating this numerically on three random atoms reproduces the identity to machine precision, $\sim 10^{-16}$; it is a worthwhile five-line check when implementing.)

7.7.7 What is fixed and what is learned

Fixed by geometry and symmetry	Learned from data
$x_{ij} = x_j - x_i$	the radial MLP producing $w(d)$
d_{ij}, $\hat{x}_{ij}$	channel-mixing matrices $A^{(\ell)}$
the spherical harmonics Y^ℓ	self-interaction weights
which coupling paths exist	how strongly each path is used
the Clebsch–Gordan coefficients	gate pre-activations
all transformation laws	(later) attention weights

Note where the boundary falls. The network never learns *which* couplings are permitted — symmetry decides that — but it does learn how much to use each permitted one, and at what distances. That is a much smaller, much better-posed learning problem than "figure out three-dimensional geometry from data".

7.7.8 Why this deserves the name "steerable"

The word is not decoration. A feature is *steerable* when its behaviour under the group is known in advance and given by an explicit matrix: rotate the input, and the feature transforms by $D^\ell(Q)$ — one can "steer" it without recomputing it from scratch. Every quantity in (7.11) is of this kind, and every operation used (spherical harmonics, Clebsch–Gordan product, scalar multiplication by an invariant, summation, equivariant linear layers) maps steerable inputs to steerable outputs with a known law.

That closure property is what makes the equivariance of the whole network a *theorem* obtained by induction over layers, rather than a property to be measured after training.

7.7.9 The question this leaves open

Look again at (7.12). Every neighbour contributes with the same structural weight; the only way a neighbour can matter more is through $w(d_{ij})$, which sees *distance alone*. Two neighbours at the same distance are treated identically regardless of what they are or what the rest of the environment looks like.

But chemistry is not like that. A hydrogen-bond donor at 3 Å and a hydrophobic carbon at 3 Å should not contribute equally, and how much each matters may depend on the rest of the pocket.

> **What if the model could learn, per edge and in context, how much each neighbour should contribute?**

That is attention, and it is where Chapter 8 begins. Note the constraint it must satisfy, already derivable from what we have: an attention weight multiplies an equivariant message, so by Section 4.10.10 it must be an *invariant* scalar. Everything in Equiformer's attention mechanism follows from getting an expressive, context-dependent, invariant number out of a pair of steerable features.

Why this matters for SigmaFlow

Relative geometry. Docking depends on where the ligand sits *relative* to the pocket, never on the global frame. Building the architecture on x_{ij} rather than x_i makes translation symmetry exact and free, and reduces the whole symmetry problem to $SO(3)$.

The radial/angular split. Distance and direction have different symmetry types, so they get different machinery: an ordinary MLP for the invariant half, spherical harmonics for the equivariant half. Recognising this split is what makes `gaussian_rbf.py` and `so3.py` look like two halves of one idea rather than two unrelated files.

Tensor-product messages. They are how chemical features learn about geometry at all. Theorem 7.14 proves a scalar-only network cannot output a pose; (7.11) is the repair.

Aggregation. Each atom integrates geometrically structured information from its neighbourhood, with spatial equivariance and permutation invariance secured by different arguments (Theorem 7.20) — both of which must hold.

Independent of the generative engine. Nothing in this section mentions scores, velocities, noise schedules or ODEs. It is entirely about how a network may represent and propagate geometric information. Swapping SigmaDock's diffusion for SigmaFlow's flow matching changes what the final $\ell = 1$ readout *means* and how it is trained — not one line of the message construction above. That is the precise sense in which the backbone is shared.

Chapter 8 now assembles this into a complete, code-verified architecture.

Chapter 8

Equiformer and EquiformerV2

This chapter is deliberately the slowest and most granular in the document, per the stated goal of enabling a reader to open the actual network source (`net/` in `SigmaFlow_Development/src/sigmadock/`) and follow it line by line afterward. We assume from the reader only Chapters 3 and 4 (manifolds, $SO(3)$, spherical harmonics, Wigner-D, Clebsch–Gordan) and Chapter 7 (attention, message passing, the invariance obstruction); nothing else from earlier chapters is needed to follow the architecture itself, though its role as the score/vector-field network of Chapters 5 and 6 is what it is *for*.

The architecture implemented is EquiformerV2 (Liao et al., *EquiformerV2: Improved Equivariant Transformer for Scaling to Higher-Degree Representations*, ICLR 2024), building on the original Equiformer (Liao & Smidt, ICLR 2023) and the eSCN reduction (Passaro & Zitnick, *Reducing SO(3) Convolutions to SO(2) for Efficient Equivariant GNNs*, ICML 2023). We derive the ideas in the order that makes them motivated, not necessarily the historical order, and flag explicitly, in Section 8.10, the one place where the actual shipped configuration is a genuine *hybrid* of the two papers' recipes rather than a pure implementation of either.

8.1 From scalar messages to steerable messages

Motivation. Theorem 7.13's message $m_{ij} = \phi_m(h_i, h_j, e_{ij})$ was, implicitly, scalar-valued (or, at best, per Theorem 7.15, a scalar times a fixed direction vector). We now let *every* object in sight — node features, messages, edge embeddings — be a full steerable feature in the sense of Theorem 4.54: a direct sum over degrees $\ell = 0, \dots, L$ (EquiformerV2's default, confirmed directly from the constructor defaults in `net/model.py`, is $L = \texttt{lmax} = 3$), each degree carrying some number C_ℓ of channels.

Definition 8.1 (Steerable node/edge feature, restated for the network)**.** A node i's feature is $h_i = \bigoplus_{\ell=0}^{L} \bigoplus_{c=1}^{C_\ell} h_i^{\ell,c} \in \bigoplus_\ell (\mathbb{R}^{2\ell+1})^{C_\ell}$, transforming, under a rotation Q of the whole scene, by $h_i \mapsto \big(\bigoplus_{\ell,c} D^\ell(Q)\big) h_i$ (Theorem 4.54). EquiformerV2's default channel counts are uniform across degree, $C_\ell \equiv C = \texttt{sphere\_channels} = 256$ (constructor default, `net/model.py`).

The embedding step (Section 8.8 below) initialises every node's $\ell = 0$ channels from ordinary categorical/continuous input features (Section 7.5's invariant quantities are exactly the correct content for a type-0 channel) and leaves every $\ell \geq 1$ channel at exactly zero; the

network's job, across its layers, is to populate the higher-degree channels with genuinely angular information extracted from the geometry, and finally to read a 3-vector out of the $\ell = 1$ channels as the network's geometric output (Section 8.9).

8.2 Embedding geometry: spherical harmonics on edges

Motivation. Chapter 4 constructed, for each degree ℓ, an explicit map $Y^\ell : S^2 \to \mathbb{R}^{2\ell+1}$ satisfying the equivariance law (4.1). This is exactly the tool needed to lift a *geometric* quantity — the direction $\hat{x}_{ij} = x_{ij}/d_{ij}$ of an edge — into a *steerable feature* that can be combined, via the Clebsch–Gordan product, with a node's existing steerable feature.

Definition 8.2 (Equivariant edge embedding)**.** Each edge (i, j) is embedded as the pair $\big(\psi(d_{ij})\chi(d_{ij}),\ Y^\ell(\hat{x}_{ij})_{\ell \le L}\big)$: an invariant radial part (Equation (7.9), code-verified form recorded in Section 7.4's code correspondence) and an equivariant angular part, one $(2\ell+1)$-dimensional block per degree up to L.

Recall from Theorem 4.22 that $Y^0 \equiv$ const and (up to normalisation) $Y^1(\hat{x}) = \hat{x}$; substituting $L = 0$ or $L = 1$ into Theorem 8.2 exactly recovers Section 7.4's distance-only GNN or Theorem 7.15's EGNN readout respectively: the steerable-feature construction is a strict generalisation, not a different framework, of everything built so far.

8.3 The eSCN reduction: aligning every edge to its own frame

Motivation, and the problem eSCN solves. A naive implementation of steerable message passing would, at every edge, form a full Clebsch–Gordan tensor product (4.7) between the neighbour's feature h_j (up to degree L) and the edge's spherical harmonic $Y^{\ell'}(\hat{x}_{ij})$ (up to degree L), for every pair of degrees (ℓ, ℓ') — a computation whose cost grows steeply with L, since the number of $(\ell_1, \ell_2) \to \ell$ paths in Theorem 4.34 grows roughly cubically in L (each pair (ℓ_1, ℓ_2) contributes up to $\min(2\ell_1, 2\ell_2) + 1$ output degrees). EquiformerV2's central algorithmic contribution, inherited from eSCN, is an exact reformulation that avoids ever explicitly instantiating a Clebsch–Gordan coefficient table.

Proposition 8.3 (Rotating an edge onto a fixed axis reduces $SO(3)$ to $SO(2)$)**.** *Let $\hat{x}_{ij} \in S^2$ be an edge direction, and let $Q_{ij} \in SO(3)$ be* any *rotation with $Q_{ij}\hat{x}_{ij} = e_1$ (the first standard basis vector; such Q_{ij} always exists, and is unique up to an arbitrary further rotation about e_1 itself, since $\{Q : Qe_1 = e_1\} \cong SO(2)$). Working in the rotated frame (applying $D^\ell(Q_{ij})$ to every steerable feature, per Equation (4.1)), the edge direction itself becomes the* fixed *vector e_1, and* every *subsequent operation that only needs to be equivariant with respect to rotations fixing the edge direction — rather than all of $SO(3)$ — may therefore use the much smaller symmetry group $SO(2)$ (rotations about e_1) instead of the full $SO(3)$, since $\hat{x}_{ij}$ itself no longer transforms (it is a fixed constant, e_1, in this frame) and only the* other *steerable quantities (the rotated node features) need to respect the residual $SO(2)$ symmetry about the now-fixed axis.*

Justification, not a formal theorem. This is a change-of-frame argument, not a new mathematical fact requiring proof beyond Equation (4.1) itself: given any $SO(3)$-equivariant operation F acting on (steerable feature, edge direction) pairs, define $\widetilde{F}(h, e_1) := D^\ell(Q_{ij})F\big(D^\ell(Q_{ij})^{-1}h, \hat{x}_{ij}\big)$; by construction $\widetilde{F}$ agrees with F after transporting through the fixed rotation Q_{ij}, and since

its second argument is now always the fixed vector e_1, $\widetilde{F}$ need only respect rotations fixing e_1 (the residual $SO(2)$) to guarantee F itself was $SO(3)$-equivariant — the constraint on the *first* argument's transformation law has been reduced from a full $SO(3)$ representation to an $SO(2)$ representation, at the cost of applying $D^\ell(Q_{ij})$ and its inverse once per edge. □

This is the eSCN insight stated precisely: *per-edge*, rotate into a frame where the edge direction is a fixed axis, do all the expensive feature-mixing work under the much cheaper $SO(2)$-equivariance constraint (an abelian group, whose irreps are 1-dimensional — ordinary complex, equivalently 2×2-real, numbers indexed by a single integer m, rather than $SO(3)$'s $(2\ell+1)$-dimensional irreps requiring the full Clebsch–Gordan machinery), then rotate back.

Code correspondence

Mathematical object: the per-edge rotation Q_{ij} of Theorem 8.3 and its Wigner-D realisation
Implementation: `net/edge_rot_mat.py`::`init_edge_rot_mat` (builds the orthonormal 3×3 frame: first axis $=\hat{x}_{ij}$; second axis chosen, per-edge, as the least-aligned of several candidate reference vectors, to avoid a near-degenerate cross product when constructing the third axis; near-zero-length edges, $\|x_{ij}\| < 10^{-5}$, fall back to a fixed default direction, logging a warning) composed with `net/wigner.py` (assembling the full block-diagonal Wigner-D matrix, via `e3nn`'s `o3.wigner_D` applied to the Euler angles of Q_{ij}, extracted via `o3.xyz_to_angles`)
Tensor shape: one $Q_{ij} \in \mathbb{R}^{3\times 3}$, and one block-diagonal Wigner-D matrix of size $\sum_{\ell=0}^{L}(2\ell+1) = \sum_{\ell=0}^{3}(2\ell+1) = 16$ (for the default $L=3$), *per edge*
Interpretation: every neighbour's steerable feature is rotated into its (edge-specific!) aligned frame *before* the message computation of Section 8.4 below, and the resulting message is rotated back *afterward*, via `SO3_Embedding._rotate`/`_rotate_inv` (`transformer_block.py`) — i.e. this per-edge alignment happens *inside* every attention block, not once globally, since a different edge has a different direction and hence a different aligned frame.

8.4 $SO(2)$-equivariant convolution: the actual EquiformerV2 message

What an $SO(2)$-equivariant linear map looks like. By the same Schur's-lemma argument as Theorem 4.46, but for the *abelian* group $SO(2)$ (whose irreps, unlike $SO(3)$'s, are complex 1-dimensional, indexed by an integer $m \in \mathbb{Z}$, with Q_α (rotation by angle α) acting on the degree-m irrep by multiplication by $e^{im\alpha}$): an $SO(2)$-equivariant linear map on the m-indexed components of a steerable feature, once expressed in the aligned frame of Theorem 8.3, may mix channels *within* a fixed order $|m|$ (matching real and imaginary/paired components together) but not across different values of $|m|$, and (since a real degree-ℓ spherical-harmonic component of order m and one of order $-m$ transform as a genuine, non-collapsible complex-conjugate pair under $SO(2)$ for $m \neq 0$) the map on this pair is exactly a 2×2 real matrix realising multiplication by a complex number, while the single $m=0$ component per degree is an ordinary real scalar with no complex structure.

Definition 8.4 ($SO(2)$ convolution, informal)**.** Given a steerable feature h (in the aligned frame), an $SO(2)$-equivariant linear layer computes, for each order $m = 0, 1, \ldots,$ `mmax`: an ordinary real-linear map on the (concatenated, across degrees $\ell \geq |m|$) $m=0$ components; and,

for $m \geq 1$, a complex-linear map $(x_r, x_i) \mapsto (W_1 x_r - W_2 x_i,\ W_1 x_i + W_2 x_r)$ on the (real, imaginary) parts of the m-order components, with W_1, W_2 learned real matrices mixing channels within that order. All weights (W_1, W_2, and the $m = 0$ map's weights) are generated from an ordinary MLP applied to *invariant* scalars only (the radial+chemistry edge embedding), per edge, per edge type.

Code correspondence

Mathematical object: the $SO(2)$-equivariant message layer of Theorem 8.4
Implementation: `net/so2_ops.py`::`SO2_Convolution`, `SO2_m_Convolution`
Interpretation: this is, precisely, the departure from a naive Clebsch–Gordan-tensor-product message $m_{ij} = W_{ij}\big(h_j \otimes_{\mathrm{cg}} Y(\hat{x}_{ij})\big)$ that a first-principles-only reading of Chapter 4 might suggest as the message formula, and it is worth being explicit that the *executed code does not form this Clebsch–Gordan product at all*: after the edge-frame rotation of Section 8.3, the edge direction is the fixed vector e_1, so mixing a node's steerable feature with the (now-fixed, hence not needing to be explicitly multiplied in via CG coefficients) edge direction reduces, by Theorem 8.3, to an $SO(2)$-equivariant linear operation on the node feature alone, and it is *this* operation, not a tensor product, that `SO2_Convolution` implements. Both are, by Theorem 8.3's justification, mathematically legitimate realisations of an equivariant message; the eSCN reduction is a genuine *algorithmic* improvement (removing the CG-path combinatorics entirely, replacing them with ordinary/complex linear layers), not merely a reparameterisation of the same computation, and this is the precise, verifiable sense in which EquiformerV2 "improves on" the original Equiformer's direct tensor-product message construction. The per-edge radial MLPs generating W_1, W_2 have their **bias terms removed** specifically for the transient (protein–ligand interaction) edge types — confirmed directly from the architecture description this monograph is auditing, and consistent with Section 7.4's smooth-cutoff argument: with a bias, a message would not vanish even when the radial features $\psi(d)\chi(d)$ do, at $d \to r_{\mathrm{cut}}$, breaking the continuity property the envelope χ was introduced to guarantee.

8.5 Equivariant nonlinearity: gating and the separable S^2 activation

Why an ordinary nonlinearity breaks equivariance. By Theorem 4.46, the *only* linear equivariant operations are per-degree scalar rescalings; a network built from linear layers alone is itself linear (up to the invariant-scalar-weight structure) and severely limited in expressive power, exactly as an ordinary neural network with no nonlinearity would be. But an entrywise nonlinearity σ applied directly to a degree-$\ell \geq 1$ feature, $h \mapsto \sigma(h)$, does not commute with $D^\ell(Q)$ in general (a direction-dependent nonlinear reshaping of a vector is generally not the same before and after rotating the vector), so it is not equivariant, and cannot be used directly.

Definition 8.5 (Gate nonlinearity)**.** For a steerable feature channel $h^{\ell,c}$ ($\ell \geq 1$) and an associated, separately-computed *invariant* (type-0) scalar $g^{\ell,c}$,

$$h^{\ell,c} \longmapsto \sigma(g^{\ell,c}) \cdot h^{\ell,c},$$

for an ordinary scalar nonlinearity σ (e.g. a sigmoid or SiLU).

Proposition 8.6. *Theorem 8.5's map is equivariant.*

Proof. Under a rotation, $g^{\ell,c} \mapsto g^{\ell,c}$ (invariant, by construction — it is produced by a separate, invariant-only computation path) and $h^{\ell,c} \mapsto D^\ell(Q)h^{\ell,c}$, so $\sigma(g^{\ell,c})h^{\ell,c} \mapsto \sigma(g^{\ell,c})D^\ell(Q)h^{\ell,c} = D^\ell(Q)\big(\sigma(g^{\ell,c})h^{\ell,c}\big)$ using linearity of $D^\ell(Q)$ and that $\sigma(g^{\ell,c})$, an unchanged scalar, commutes past the matrix. □

Geometrically: the gate rescales a steerable feature's *magnitude* nonlinearly (via the invariant scale $\sigma(g)$) while leaving its *direction* (its transformation behaviour) completely untouched — exactly the one degree of freedom (an overall scalar multiplier) that Theorem 4.46 says is available on a single channel without violating equivariance, now made nonlinear in the *scalar* channel that controls it.

Code correspondence

Mathematical object: the equivariant nonlinearity actually used by default
Implementation: `net/activation.py`::`SeparableS2Activation` (config default: `use_gate_act=False`, `use_sep_s2_act=True`)
Interpretation: EquiformerV2's default nonlinearity is *not* literally Theorem 8.5 but a closely related, more expressive construction: the steerable feature is projected onto a discrete grid of points on S^2 (via `e3nn`'s `ToS2Grid`, resolution set by L/`mmax`), an *ordinary*, pointwise nonlinearity is applied at each grid point (legitimate precisely because evaluating a steerable function at a rotated grid point equals evaluating the un-rotated function at the inverse-rotated point — the grid-sampling analogue of Theorem 4.23's transformation law, applied to function values rather than coefficients), and the result is projected back (`FromS2Grid`) into spherical-harmonic coefficients. An additional, separate scalar "gating" path (extra invariant channels produced alongside the main $SO(2)$-convolution output) modulates this activation further, combining both mechanisms. We record this as the precise implemented nonlinearity without re-deriving the grid-sampling equivariance argument in full technical detail (it is a standard construction in the equivariant-networks literature, e.g. Cohen et al., *Spherical CNNs*, ICLR 2018).

8.6 Equivariant linear layers and normalisation

Code correspondence

Mathematical object: the general equivariant linear map of Theorem 4.48
Implementation: `net/so3.py`::`SO3_LinearV2`
Tensor shape: weight tensor `[lmax+1, out_features, in_features]`
Interpretation: implements exactly $h^{\ell,c} \mapsto \sum_{c'} W^{\ell}_{cc'} h^{\ell,c'}$ via `einsum("bmi,moi->bmo", ...)` with the weight broadcast identically across the $2\ell+1$ components m of a given degree ℓ (one weight matrix *per degree*, shared across all m within it, exactly matching Theorem 4.46's statement that an intertwiner is a scalar multiple of the identity *on* V_ℓ, generalised here to multiple channels per degree via Theorem 4.48); and a bias term is added **only to the** $\ell = 0$ **slice** of the output (`out[:,0:1,:] += bias`) — the code's own structural enforcement of the fact, provable directly from Theorem 4.46 (an additive constant is a map from the trivial representation, $\ell = 0$, into the target; by Schur's lemma such a map can only land in the $\ell = 0$ component of the target without violating equivariance, since a nonzero constant vector in a degree-$\ell \geq 1$ channel would not itself be fixed by every rotation, contradicting $D^\ell(Q) \cdot \text{bias} = \text{bias}$ required for a translation-equivariant *additive* bias), that higher-degree channels cannot carry an additive bias at all.

Code correspondence

Mathematical object: equivariant normalisation
Implementation: `net/layer_norm.py` (`norm_type` default `"rms_norm_sh"`)
Interpretation: normalises each degree-ℓ, channel-c block $h^{\ell,c}$ by (an RMS-style estimate of) its own *norm* $\|h^{\ell,c}\|_2$ — an invariant quantity by Theorem 3.16's orthogonality of $D^\ell(Q)$, since $\|D^\ell(Q)v\| = \|v\|$ for orthogonal $D^\ell(Q)$ (Theorem 4.25(iii)) — rather than by any coordinate-wise statistic, which is what makes the normalisation equivariant (dividing every component of $h^{\ell,c}$ by the same invariant scalar $\|h^{\ell,c}\|$ commutes with $D^\ell(Q)$ by the same scalar-multiplication argument as Theorem 8.5's gate). The exact formula implemented (beyond "RMS of the norm, per spherical-harmonic degree") was not read in the code-research pass underlying this document and is flagged, per this document's stated policy, as not independently confirmed beyond its name and role.

8.7 Equivariant attention weights

The constraint attention weights must satisfy. We now assemble messages and weights into the full attention block, and the first thing to determine is what kind of object the attention weight a_{ij} (Theorem 7.3's analogue) is allowed to be.

Proposition 8.7 (Attention weights must be invariant scalars)**.** *Let $h'_i = h_i + \sum_{j\in\mathcal{N}(i)} a_{ij}\, m_{ij}$, with each message m_{ij} steerable of a fixed degree ℓ. If the a_{ij} are $SE(3)$-invariant scalars, this update is equivariant. If a_{ij} were instead permitted to be an arbitrary equivariant (not merely invariant) quantity of matching degree, the sum $\sum_j a_{ij} m_{ij}$ would generally fail to commute with $D^\ell(Q)$, since a_{ij} itself would then also transform under Q, and the product of two degree-ℓ-transforming quantities is not, in general, again degree-ℓ (Theorem 4.34).*

Proof. If a_{ij} is invariant and $m_{ij} \mapsto D^\ell(Q)m_{ij}$: $\sum_j a_{ij} m_{ij} \mapsto \sum_j a_{ij} D^\ell(Q) m_{ij} = D^\ell(Q) \sum_j a_{ij} m_{ij}$, using linearity of $D^\ell(Q)$ and that scalar a_{ij} commutes past it. Adding $h_i \mapsto D^\ell(Q)h_i$ (assuming h_i is also degree-ℓ; otherwise, apply the same argument degree-by-degree in a multi-degree sum) preserves equivariance of h'_i. If a_{ij} instead transformed under Q (e.g. if it were itself a component of a degree-$\ell' > 0$ feature), the product $a_{ij}m_{ij}$ would transform by a Clebsch–Gordan combination of degrees ℓ' and ℓ, generally landing (partially) outside degree ℓ, breaking the claimed degree-ℓ equivariance of the sum. □

This forces, structurally, exactly the design choice Chapter 7 already used implicitly for ordinary attention (Theorem 7.3's softmax output is a scalar) and is why an equivariant network cannot simply reuse a dot-product attention score computed *directly* between two full multi-degree steerable features (which would not, in general, be invariant, unless the specific $\ell \to 0$ Clebsch–Gordan path of Theorem 4.44 is used deliberately) without first projecting onto an invariant quantity.

Code correspondence

Mathematical object: the attention weight a_{ij} of Theorem 8.7
Implementation: `net/transformer_block.py::SO2EquivariantGraphAttention.forward`
Interpretation: a_{ij} is computed **only** from the extra $\ell = 0, m = 0$ scalar output channels of the first $SO(2)$-convolution (Section 8.4) applied to the edge message — i.e. the network allocates a few dedicated invariant "gating" channels alongside its main steerable output, specifically for this purpose — via a LN → SmoothLeakyReLU → learned per-head dot product, then a standard `torch_geometric.utils.softmax` keyed

on the *target* node index (`edge_index[1]`), i.e. a per-target-node softmax over its incoming edges, exactly Theorem 7.3's row-wise softmax specialised to a sparse graph. This is, precisely, the architectural enforcement of Theorem 8.7: there is structurally no other path in the code by which a non-scalar, degree-≥ 1 quantity could enter the attention weight.

8.7.1 Assembling the full attention block

Proposition 8.8 (The full block is equivariant)**.** *Let messages m_{ij} be constructed (via Sections 8.3 and 8.4) to be equivariant, and a_{ij} invariant scalars (Theorem 8.7). Then $h'_i = h_i + \sum_{j \in \mathcal{N}(i)} a_{ij}\, m_{ij}$ is equivariant, degree-by-degree.*

This is exactly Theorem 8.7's proof, restated as the conclusion for the assembled block. In code, the full per-layer pipeline (`TransBlockV2`, `net/transformer_block.py`) is: (1) concatenate source and target node features; (2) rotate into the per-edge frame (Section 8.3); (3) first $SO(2)$-convolution (Section 8.4), also producing the invariant gating channels used for attention weights; (4) equivariant nonlinearity (Section 8.5); (5) second $SO(2)$-convolution; (6) compute a_{ij} from the gating channels (Section 8.7); (7) rotate messages back out of the edge frame; (8) scatter-sum $a_{ij}m_{ij}$ into target nodes (an `index_add_` along `edge_index[1]`, the permutation-equivariant aggregator of Theorem 7.13); (9) final equivariant linear projection (`SO3_LinearV2`, Section 8.6); embedded, as in Equation (7.2), in a pre-normalised residual structure alongside a per-node equivariant feed-forward block.

8.8 Node embedding: turning chemistry into a steerable feature

Before any message passing, every node needs an initial steerable feature (Section 8.1); by Section 7.5's argument in reverse, only the $\ell = 0$ (invariant) channels can be populated directly from ordinary categorical/continuous chemistry, since nothing in the raw per-atom input (an atom type, a formal charge, a time value) carries any directional information to seed a higher-degree channel with.

Code correspondence

Mathematical object: the initial (type-0-only) node embedding
Implementation: `net/encoder.py`::`AtomDiffusionEncoder`, called once per node-entity type from `net/model.py`::`compute_node_embedding`
Interpretation: for each of the (default) **ten** categorical atom features (`atom_feature_dims`= $(54, 5, 3, 4, 4, 7, 2, 2, 4, 3)$, `config.py` — cardinalities only; the exact chemical identity of each of the ten slots, e.g. which one is atomic number versus hybridisation, was not independently confirmed against the preprocessing code in the pass underlying this document and is left as an open item, not asserted), a separate `nn.Embedding` table (with one extra row for an explicit "unknown/out-of-range" category) produces a vector; these ten vectors are combined not by concatenation-then-MLP but — in the configuration actually used, `linear_aggregate=False` at every call site — by **summing them and dividing by** $\sqrt{10}$, a variance-preserving sum (if the ten embeddings were independent with unit variance per coordinate, the sum's variance would be $10\times$ larger, and dividing by $\sqrt{10}$ restores unit variance — the same normalisation principle as the $1/\sqrt{d_k}$ scaling of Theorem 7.3's attention logits). Continuous auxiliary

features (e.g. protein residue/ESM embeddings, where used) are instead passed through a small MLP and *added*. The time embedding t_{emb} — a `sinusoidal` embedding (default `t_emb_dim=32`, `t_emb_scale=10000.0`; the exact functional form was not independently re-derived from the code in this pass, but the name and parameters strongly suggest the standard Transformer construction $\big(\sin(t/10000^{2k/d}), \cos(t/10000^{2k/d})\big)_k$, flagged here as plausible but not code-confirmed) — is concatenated (not summed) onto the aggregated categorical embedding and passed through one further linear layer (`time_projector`). **Ligand-fragment virtual nodes** (one per fragment, per Section 9.7 below) do *not* get an independent embedding at all: their $\ell = 0$ feature is the *mean* (a `scatter`, `reduce="mean"`) of the already-computed embeddings of every real atom belonging to that fragment, propagated along dedicated virtual-to-atom (`ligand_v2a`) edges — i.e. a fragment's virtual node is, by construction, exactly $SE(3)$-invariant-feature-wise a summary of its members, before any message passing layer has even run.

8.9 Reading out the vector field

After `num_layers` $= 6$ (default) blocks, each atom carries a full steerable feature $h_i = \bigoplus_{\ell=0}^{3} \bigoplus_{c=1}^{256} h_i^{\ell,c}$. The network's geometric output, per (7.1), is a single 3-vector per *atom* (not yet per fragment; the reduction to per-fragment force/torque via Newton–Euler mechanics is a separate step, reconstructed precisely in Chapter 9), read off from the $\ell = 1$ channels by one further equivariant linear layer.

Code correspondence

Mathematical object: the per-atom pseudo-force readout
Implementation: `net/model.py::EquiformerV2.forward`, final `force_block` (an `SO2EquivariantGraphAttention` with a single output channel), then `forces = force_block_output.embedding.narrow(1, 1, 3)`
Tensor shape: `[N_atoms, 3]`
Interpretation: `narrow(1, 1, 3)` slices the flattened $(L+1)^2 = 16$-length coefficient axis starting at index 1, length 3 — i.e. it selects exactly the three $\ell = 1$ ($m = -1, 0, 1$) coefficients, discarding the $\ell = 0$ coefficient (index 0) and every $\ell \geq 2$ coefficient (indices 4 and up) entirely. By Theorem 4.25(iv), this $\mathbb{R}^3$-valued output transforms under a global rotation Q exactly as $D^1(Q) = Q$, i.e. as an ordinary polar 3-vector in the *world frame* in which the network's input coordinates were expressed — this is precisely the "atom-wise pseudo-force in the world frame" object that Chapter 9's Newton–Euler reduction (net force, torque, per fragment) consumes, and it is only at that later, non-network step that a fragment's rotational output is converted into the *body-frame* (left-trivialised, Section 3.4) angular quantity $\omega_\theta^{(i)} \in \mathfrak{so}(3)$ that (7.1) ultimately requires — the network itself, note carefully, never outputs a body-frame quantity directly; the conversion is a geometric post-processing step external to `EquiformerV2` proper, and Chapter 9 traces it precisely.

8.10 EquiformerV1 vs. EquiformerV2: what actually changed, and what this codebase actually ships

We can now state the V1-versus-V2 comparison precisely, having derived every ingredient involved, rather than merely asserting a version number.

Original Equiformer (V1). Messages are formed by an explicit Clebsch–Gordan tensor product (4.7) between a neighbour's steerable feature and the edge's spherical harmonic embedding (Theorem 8.2), with per-path learned weights; no per-edge frame alignment is used. Attention weights are computed from invariant edge/node scalars via an ordinary MLP → dot product → softmax pipeline, structurally identical to Theorem 7.3's original Transformer recipe, merely applied to steerable rather than plain features.

EquiformerV2 (eSCN-based). Messages are formed via the per-edge frame alignment of Section 8.3 and the $SO(2)$-convolution of Section 8.4, *replacing* the explicit Clebsch–Gordan tensor product with an algorithmically cheaper, exactly equivalent (by Theorem 8.3's justification) construction. V2's own paper additionally proposes an alternative attention-weight mechanism, `use_s2_act_attn`, computing attention directly from grid-sampled (S^2-activation-based) features rather than V1's separate scalar-dot-product path.

Caution 8.9 (This codebase implements a hybrid, not pure V2). The constructor of `EquiformerV2` in this codebase sets `use_s2_act_attn: bool = False` *and enforces this with an explicit assertion* (`assert not self.use_s2_act_attn`, with an inline comment recording that this path is unused). That is: the message mechanism is EquiformerV2's eSCN-based $SO(2)$-convolution (Section 8.4) throughout, but the *attention-weight* computation follows the **original Equiformer(V1) recipe** — separate invariant scalar channels, MLP, per-head dot product, softmax (Section 8.7) — rather than V2's own alternative `use_s2_act_attn=True` path. This is a genuine, verifiable, non-cosmetic architectural fact about the shipped model, not a matter of degree or approximation, and this document states it explicitly rather than describing the backbone as "EquiformerV2" without qualification, as a less careful account might.

8.11 Summary: architecture, hyperparameters, and parameter count

Hyperparameter	Default value (constructor, `net/model.py`)
Maximum degree L (`lmax`)	3
Maximum order M (`mmax`, $SO(2)$-conv truncation)	2
Number of resolutions	1 (single (L, M) pair)
Channels per degree (`sphere_channels`)	256
Number of attention layers (`num_layers`)	6
Attention heads (`num_heads`)	4
Edge/radial embedding dim	64 / 32 basis functions
Polynomial cutoff order	8
Time embedding	sinusoidal, dim 32, scale 10^4
Normalisation	`rms_norm_sh` (per-degree RMS norm)
Radial cutoff function	Bessel

Total learnable parameters, as reported in every training log inspected for this document: **14,979,377**.

Remark 8.10 (What this chapter has, and has not, independently verified). Several fine-grained implementation details — the exact RMS-norm formula, the exact sinusoidal time-embedding formula, the internals of the `S2Activation`/`GateActivation` classes as distinguished from `SeparableS2Activation`, and the precise semantics of each of the ten categorical atom features — were, per this document's stated policy, flagged rather than guessed at in the code-research pass underlying this chapter, because the corresponding source files (`net/layer_norm.py`, `net/timestep_embedder.py`, `net/activation.py` in full, `chem/processing.py`) were not read to completion. Everything else in this chapter — the edge-frame mechanism, the $SO(2)$-convolution message construction, the attention-weight computation and its V1/V2 hybrid status, the equivariant linear layer's degree-block structure, the node embedding's aggregation formula, and the final $\ell = 1$ readout — was confirmed directly against the source.

Part V

SigmaDock

Chapter 9

SigmaDock: Full Reconstruction

Every mathematical ingredient is now available: Chapters 3 and 4 give the geometry, Chapters 5 and 6 give the generative-modelling machinery, and Chapters 7 and 8 give the network. This chapter assembles them into SigmaDock exactly as implemented, tracing every design decision back to either a theorem already proved or a specific line of code, and marking clearly the few points that remain genuinely open (not independently re-derivable from a static reading of the code alone).

9.1 Choosing the state space

Section 1.2 identified three coordinate systems in which a pose could be represented: all $3n$ Cartesian coordinates, the $(k{+}6)$-dimensional torsional parameterisation $\big(SE(3)\times\mathbb{T}^k\big)$, or a fragment-based parameterisation $\big(SE(3)^m\big)$. SigmaDock's central architectural decision is to use the third, and the argument for doing so is a statement about the induced *measure*, not merely about dimensionality — worth deriving carefully, since it is what licenses the product-manifold construction (Theorem 3.37) underlying every diffusion/flow-matching formula in this document.

Definition 9.1 (Pullback volume form)**.** Let $\Omega : \mathcal{Y} \to \mathbb{R}^{n\times 3}$ be a smooth parameterisation map from a parameter space $\mathcal{Y}$ into Cartesian coordinates, with Jacobian $J_\Omega = \partial\Omega/\partial y$ and Gram matrix $G_\Omega = J_\Omega^\top J_\Omega$. The *pullback volume form* induced on $\mathcal{Y}$ is $d\mu_\Omega(y) = \sqrt{\det G_\Omega(y)}\, dy$ — the Riemannian volume form of the metric on $\mathcal{Y}$ obtained by declaring Ω a local isometry onto its image with the ambient Euclidean metric.

Theorem 9.2 (Torsional entanglement)**.** *Let $\psi : SE(3)\times\mathbb{T}^k \to \mathbb{R}^{n\times 3}$ be the torsional parameterisation of a ligand with k rotatable bonds (global rigid motion composed with dihedral-angle-driven internal reconstruction), and let $\varphi : SE(3)^m \to \mathbb{R}^{n\times 3}$ be the fragment parameterisation with m pairwise-disjoint rigid fragments. Then, generically (for a molecular topology in which some rotatable bond's rotation displaces atoms that a different rotatable bond also displaces — true for essentially every branched or chain topology with more than one rotatable bond), G_ψ is non-diagonal and $d\mu_\psi$ is not a product measure over the individual torsion coordinates. By contrast, J_φ is block-diagonal (one block per fragment, since moving fragment i's rigid motion changes no coordinate belonging to fragment $j \neq i$), so $\det G_\varphi = \prod_{i=1}^m \det(J_i^\top J_i)$ and $d\mu_\varphi$ is a product measure over fragments.*

Proof. Fragment case. By definition of φ, atom a's position depends only on the rigid motion of the fragment containing a: $\partial\varphi_a/\partial(\text{fragment } j\text{'s coordinates}) = 0$ whenever $a \notin$ fragment j.

Hence J_φ, with rows grouped by atom and columns grouped by fragment, is block-diagonal (a given row has nonzero entries only in the column-block of its own fragment), and for a block-diagonal matrix $J = \bigoplus_i J_i$, $J^\top J = \bigoplus_i J_i^\top J_i$ is also block-diagonal, with det of a block-diagonal matrix the product of the blocks' determinants.

Torsional case. Let ϕ_i, ϕ_j be two rotatable-bond angles whose associated atom-displacement fields overlap (an atom a moved by rotating about bond i is also moved by rotating about bond j — true whenever the two bonds lie on a common path through the molecular graph with atom a downstream of both, which occurs for any chain or branched topology with ≥ 2 non-independent rotatable bonds). Then $(G_\psi)_{ij} = \sum_a (\partial_{\phi_i}\psi_a)\cdot(\partial_{\phi_j}\psi_a)$ is generically nonzero, since it is a sum, over the atoms moved by *both* torsions, of dot products of two generically-non-orthogonal displacement vectors — there is no structural reason for this sum to vanish identically across all molecular geometries, and it generically does not. Since $(G_\psi)_{ij} \neq 0$ for some $i \neq j$, G_ψ is not diagonal, and $\det G_\psi$ does not factor as a product over the individual coordinates ϕ_i (a non-diagonal Gram matrix's determinant is not, in general, the product of its diagonal entries), so $d\mu_\psi(\boldsymbol{\phi})$ cannot be written as $\prod_i d\mu_i(\phi_i)$ for single-coordinate measures $d\mu_i$. □

The mechanism, made concrete: the Jacobian column for torsion ϕ_i is the Cartesian velocity field generated by rotating everything *downstream* of bond i about that bond's axis, and in a chain or branched molecule this field moves *many* atoms, with the sets of atoms moved by different torsions typically overlapping. Three *practical* consequences follow directly from Theorem 9.2, each of which would separately motivate avoiding the torsional parameterisation for a diffusion or flow-matching model specifically (as opposed to, e.g., a search-based method, for which the measure's factorisation is less directly load-bearing):

1. **Independent conditional noise does not stay independent.** A forward noising kernel that factorises as a product over torsion coordinates — exactly the kind of kernel Chapter 5 builds, one independent Gaussian/heat-kernel perturbation per coordinate — maps, under ψ, to a correlated, anisotropic perturbation of the Cartesian positions the network actually observes (since ψ's Jacobian mixes the torsion coordinates together non-trivially, per Theorem 9.2). The network must then implicitly learn to invert this metric-induced coupling, an unnecessarily hard learning problem compared to Theorem 3.37's clean, coupling-free product structure on $SE(3)^m$.

2. **Global and internal motion do not decouple.** A torsional increment generically shifts the fragment's centre of mass ($\sum_a m_a \partial_{\phi_i}\psi_a \neq 0$ in general, since the atoms downstream of bond i do not, in general, have their displacements sum to zero) and induces a net torque, so the $\mathbb{T}^k$ and $SE(3)$ factors of the torsional parameterisation are not, geometrically, independent either — any diffusion or flow constructed to treat them as independent (again, the natural, simple construction) is therefore only approximately correct, exactly (an infinitesimal-step) in the limit, not exactly at any finite step size.

3. **The dihedral definition itself has a gauge ambiguity.** A dihedral angle ϕ_{ABCD} does not, by itself, specify *which side* of the bond BC is held fixed while the other rotates; a change $\Delta\phi$ can be realised by rotating the A-side, the D-side, or any split between the two, and these give physically different (though dihedral-angle-equal) intermediate trajectories. Any concrete implementation must fix a convention, and a learned score/vector-field network must be trained consistently with whichever convention sampling uses — a consistency requirement with no natural enforcement mechanism, unlike the fragment parameterisation, where no such choice ever arises (each fragment simply *is* a specified rigid body).

Fragmentation dissolves all three simultaneously, *because* of Theorem 9.2's block-diagonal Jacobian: moving one fragment's rigid motion changes no other fragment's coordinates at all, so independent per-fragment noise *stays* independent under φ, global and (now nonexistent, at the fragment level) internal motion trivially coincide, and there is no gauge ambiguity in specifying a rigid motion. The price is dimensional: $6m$ degrees of freedom against the torsional parameterisation's $k+6$, and $m > k+1-m$ generically (fragmenting at every rotatable bond, without merging, gives $\hat{m} = k+1$ fragments, so $6\hat{m} = 6k+6$ against $k+6$ — *more* parameters than the torsional route, not fewer), which is exactly the deficit Section 9.3 below addresses.

Caution 9.3 (Status of this argument). Theorem 9.2 and its practical consequences are this document's own reconstruction and elaboration of the reasoning given in the thesis draft this monograph audits (`Texte/main.tex`); we have verified the mathematical content — the block-diagonal-Jacobian argument for fragments, and the generic-non-diagonality argument for torsions — independently, by direct derivation, and found it sound, but we have *not* located a specific published SigmaDock source stating the theorem in this exact form, and it is presented here as a rigorous, self-contained argument for a design choice the codebase clearly makes (fragment-based, not torsional, generation), rather than as a verbatim quotation from a paper.

9.2 The conformational manifold: what rigidity costs

Treating each fragment as perfectly rigid is itself an approximation, and its cost should be quantified, not merely assumed acceptable. In vacuum, the accessible conformations of a ligand (fixed molecular topology) concentrate, under a Boltzmann distribution at physiological temperature, near a manifold $\mathcal{M}_c = \{x_c : g(x_c) \approx 0\}$ where g's components softly encode bond lengths ($\|r_{AB}\| - d_0$, d_0 the equilibrium bond length) and bond angles ($\hat{u} \cdot \hat{v} - \cos\tau_0$), *excluding* dihedrals (which are, by contrast, the ligand's soft/free coordinates, exactly the ones Section 1.2 identified as free rotatable-bond degrees of freedom). $\mathcal{M}_c$ is sampled, in practice, by conformer-generation software (RDKit's ETKDGv3 algorithm) rather than by literally simulating the Boltzmann distribution.

The question relevant to a fragment-based (rigid-template) docking model is whether a bound (experimentally observed) pose is well approximated by rigidly repositioning *some* sample from $\mathcal{M}_c$: writing $\psi(\boldsymbol{\phi})$ for the dihedral-to-Cartesian reconstruction map and $\Psi(p, R, \boldsymbol{\phi}) = (p, R)\cdot\psi(\boldsymbol{\phi})$ for the composite (rigid motion applied to an internally-reconstructed conformer), the relevant quantity is

$$\min_{p\in\mathbb{R}^3,\, R\in SO(3),\, \boldsymbol{\phi}\in\mathbb{T}^k} \mathrm{RMSD}\big(x_b,\, \Psi(p, R, \boldsymbol{\phi})\big), \qquad x_b \sim \pi_{\mathcal{M}_b}$$

(the bound-pose distribution). This document does not re-derive this empirical figure from scratch (it is a dataset-dependent, measured quantity, not a theorem); we record it, as reported in the thesis draft this monograph audits, and flag it as an empirical claim requiring the cited dataset (Astex Diverse Set) to independently reproduce: the best of five ETKDGv3 conformers aligns to the bound pose with mean RMSD 0.21 Å (and 0.39 Å for a single conformer), well under the 2 Å success threshold of Theorem 1.5. The consequence for this document: the roughly 0.2 Å gap between a rigid-fragment reconstruction and the true bound pose is an **error floor** inherited by *any* model built on this fragmentation, including SigmaFlow, and is not attributable to the choice between diffusion and flow matching — it bounds what either method can, even in principle, achieve, and should not be mistaken for a deficiency of one method specifically when comparing the two (Chapter 12).

9.3 FR3D: reducing fragment count

Cutting *every* rotatable bond gives $\hat{m} = k + 1$ fragments and $6\hat{m} = 6k + 6$ degrees of freedom, exceeding the torsional parameterisation's $k+6$ — the naive fragment construction has bought Theorem 9.2's decoupled measure at the cost of *more*, not fewer, dimensions than the alternative it replaces. FR3D (Fragment REduction) recovers some of this cost back by merging adjacent fragments: starting from the maximal $\hat{m}$-fragment decomposition, a stochastic recursive procedure repeatedly merges two fragments across a shared torsional bond, terminating at an irreducible fragment set of size $m < \hat{m}$; empirically (per the audited thesis draft, not independently re-derived here) $m \approx \frac{2}{3}(k + 1)$. Because which bonds get merged is itself stochastic and (per the source description) equiprobable across valid cut-sets, re-running the fragmentation on the same molecule at a different training epoch can yield a genuinely different m-fragment decomposition, functioning simultaneously as a data augmentation scheme.

Caution 9.4. The *exact* merge heuristic and stopping criterion implemented by FR3D were not confirmed line-by-line against `fragmentation.py` in the code-research pass underlying this document (the file is over 1500 lines; only the rotatable-bond-detection and triangulation-mapping functions, described precisely below and in Section 9.4, were read to completion). What *was* confirmed directly is (i) rotatable-bond detection itself (Section 9.3.1 below), (ii) that the file's own top-of-module documentation describes a "recursive merge" procedure, consistent with the description above, and (iii) the resulting dummy-atom bookkeeping (Section 9.3.2 below). The precise stochastic merge algorithm itself is therefore reported here as the audited thesis draft's own description, not as independently re-derived or line-verified fact, and is flagged accordingly.

9.3.1 Rotatable-bond detection, as actually implemented

Code correspondence

Mathematical object: the set of rotatable bonds defining the cut points of Section 1.2
Implementation: `fragmentation.py`::`detect_torsional_bonds`
Interpretation: uses RDKit's `TorsionFingerprints.CalculateTorsionLists` to enumerate torsion quadruples, then filters to bonds that are: not part of any ring, of bond type `SINGLE`, and (governed by an `ignore_conjugated` flag, default `False` per this project's own training logs) optionally excluding conjugated bonds not touching a ring. This is a specific, code-verified criterion, more concrete than a generic "rotatable bond" description: not merely "any single, non-ring bond", but specifically those bonds RDKit's own torsion enumeration recognises as defining a well-formed dihedral quadruple.

9.3.2 Dummy atoms and fragment boundaries

Cutting a bond leaves each resulting fragment with a "dangling" bond at the cut point; SigmaDock represents this by introducing a *dummy atom* on either side, so that a fragment's atom count can exceed the corresponding subset of the original ligand's atom count ($|\mathcal{G}_{\text{ligand}}| \leq \sum_i |\mathcal{G}_{F_i}|$, one extra dummy per retained cut). A dummy atom is retained only if it remains *free* (marks a torsional bond that FR3D's merge procedure did *not* collapse into a single fragment); if FR3D merges the two sides of a torsional bond back together, the corresponding dummy is *pruned*, since retaining it would spuriously freeze a dihedral that — per Section 9.2's reachability argument — must remain free for the rigid-fragment reconstruction to stay close to the true bound-pose distribution.

9.4 Triangulation: an invariant, soft geometric prior at fragment boundaries

Fragmentation is lossy in one specific, quantifiable sense: once two fragments move independently, nothing in $SE(3)^m$ itself prevents the bond angles *across* the cut bond from taking chemically absurd values (Chapter 12's empirical analysis of exactly this failure mode, in the flow-matching setting, is one of the central quantitative results of this project's broader investigation, referenced there). SigmaDock's answer is not to hard-constrain the manifold (which would reintroduce Theorem 9.2's coupling problems) but to *condition the network* on an elementary, exact piece of Euclidean geometry.

Theorem 9.5 (Triangulation determines the boundary bond angles)**.** *Let $\overline{BC}$ be a cut torsional bond, with B in fragment $\mathcal{A} \ni A$ and C in fragment $\mathcal{D} \ni D$ (A, D each bonded to B, C respectively, within their own fragment). Fixing the four intra-fragment/bond lengths $\|A-B\|$, $\|B-C\|$, $\|C-D\|$ and the two* cross-fragment *distances $\|A-C\|$, $\|B-D\|$ determines the two boundary bond angles $\angle(A,B,C)$, $\angle(B,C,D)$ uniquely, while leaving the dihedral ϕ_{ABCD} completely unconstrained.*

Proof. The triple $\{\|A-B\|, \|B-C\|, \|A-C\|\}$ determines the triangle (A,B,C) up to rigid motion (the classical side-side-side congruence criterion), and the law of cosines gives $\cos\angle(A,B,C) = \dfrac{\|A-B\|^2 + \|B-C\|^2 - \|A-C\|^2}{2\|A-B\|\|B-C\|}$, a function of the three fixed lengths alone; identically, $\cos\angle(B,C,D)$ is determined by $\{\|B-C\|, \|C-D\|, \|B-D\|\}$. Neither computation constrains the relative rotation of the two triangles' planes about the shared edge $\overline{BC}$, which is exactly the dihedral ϕ_{ABCD} by Section 1.2's definition. □

The two distances the theorem needs, $\|A-C\|$ and $\|B-D\|$, are manifestly $SE(3)$-invariant (ordinary Euclidean distances, Theorem 3.16), and hence admissible, per Section 7.5's discussion, as ordinary type-0 (scalar) edge features consumable by an equivariant network without any special handling.

Code correspondence

Mathematical object: the triangulation conditioning feature
Implementation: the deviation formula $d_{ij}(t) = \big|\,\|x_i(t) - x_j(t)\| - \|x_i^{\text{ref}} - x_j^{\text{ref}}\|\,\big|$, computed at `net/model.py`::`compute_edge_embedding`'s "boundary" edge branch, using `pos_0` as the reference geometry and `pos_t` as the current (noised/interpolated) geometry; the *atom pairs* entering this formula are enumerated by `fragmentation.py`::`get_triangle_equality_mapping`
Interpretation: the network is given, at every step t, the absolute deviation of each triangulation distance from its equilibrium (reference-conformer) value, so $d_{ij}(t) \to 0$ as $t \to 1$ (approaching data, in this document's generation-time convention) is exactly the signal that "this boundary is currently geometrically wrong, correct it"; no projection or hard equality is ever imposed, only this soft conditioning signal. **More general than Theorem 9.5's minimal statement:** the actual code maps every anchor atom to *all* atoms directly bonded to each of its neighbouring anchors (not merely one representative atom A or D per side), so more than the minimal two cross-fragment distances of the Theorem are typically supplied per cut bond in practice — the Theorem establishes that two such distances *suffice* in principle, not that exactly two are used. An `ignore_triangulation` ablation flag exists (`data.py`), consistent with the audited thesis draft's report of an ablation study in which removing this conditioning costs a docu-

mented 12.8 percentage points of PoseBusters validity (Def. 1.6) — the single largest effect size reported in that study, though we have not independently reproduced this specific number and report it as the audited source's own claim.

Remark 9.6 (Degrees of freedom of the soft-constrained manifold). Were the triangulation distances imposed as *hard* equality constraints (which they are not — Theorem 9.5's feature is a soft conditioning signal, never a projection), the constrained submanifold $\mathcal{M}_{\text{frag}} = \{z \in SE(3)^m : c(z) = 0\}$ would, for a tree-structured fragment graph ($e = m - 1$ cut bonds), have two non-degenerate distance constraints per joint removing five of the six relative degrees of freedom between adjacent fragments (leaving exactly the one free dihedral, as Theorem 9.5's proof shows), so $\dim \mathcal{M}_{\text{frag}} = 6m - 5(m-1) = m + 5$ — interpolating between the torsional parameterisation's $k + 6$ and the unconstrained $SE(3)^m$'s $6m$. Since the actual constraints are soft, sampled trajectories only *concentrate* near $\mathcal{M}_{\text{frag}}$ as $t \to 1$; this dimension count is a useful sanity check on the geometry, not a claim about the trained network's exact behaviour.

9.5 Symmetry: gauge invariance and stochastic $SE(3)$-equivariance

Two distinct symmetry statements matter for SigmaDock, and conflating them (as the broader literature sometimes does) obscures which architectural property is responsible for which guarantee.

Gauge invariance. A rigid fragment's *local* coordinate frame (the coordinate system its reference-conformer template is expressed in, before any rigid motion is applied) has no canonical choice: for any fixed $R_s \in SO(3)$, replacing the local template $\tilde{x}_F$ by $R_s \tilde{x}_F$ and simultaneously replacing the fragment's rotation R_F by $R_F R_s^\top$ describes the *same* global pose, $R_F \tilde{x}_F + p_F = (R_F R_s^\top)(R_s \tilde{x}_F) + p_F$. A correctly constructed loss or sampler must not depend on this arbitrary choice.

Stochastic $SE(3)$-equivariance. The *generated distribution* should satisfy $p_\theta(dz \mid R_0 \cdot \mathcal{G}_{\text{dock}}) = R_0 \cdot p_\theta(dz \mid \mathcal{G}_{\text{dock}})$ for every $R_0 \in SO(3)$: rotating the input pocket rotates the entire predicted pose *distribution* and nothing else about it.

Theorem 9.7 (SigmaDock's symmetry)**.** *SigmaDock's training objective and sampling procedure are invariant to the (arbitrary) choice of fragment local-frame orientation, and the induced kernel $p_\theta(z \mid \mathcal{G}_{\text{dock}})$ is stochastically $SO(3)$-equivariant.*

This is the stochastic-process analogue of Theorem 7.2: there, a deterministic equivariant ODE plus an invariant prior gave an invariant marginal law at every time t; here, the corresponding statement for the *diffusion SDE* (Chapters 5 and 6) is that an $SE(3)$-equivariant score network, together with the $SE(3)$-invariant prior $p_{\text{init}} = \mathcal{N}(0, I) \otimes \mathcal{U}(SO(3)^m)$ established in Section 9.6 below, is sufficient (via the reverse-time SDE of Equations (5.5) and (6.3), whose drift and diffusion coefficients depend on the score in an equivariant way by construction) for stochastic equivariance of the generated law; we do not reprove this SDE-level statement in full technical generality (it requires checking that the reverse-time SDE's law, not merely a deterministic flow's pushforward, transforms correctly, a short but somewhat more technical argument than Theorem 7.2's proof), and refer to the general treatment in De Bortoli et al. (2022, cited in Section 6.1) for the manifold-SDE case. *Gauge invariance* of the loss and sampler, by contrast, is a structural property directly checkable from Section 8.9's observation that the network's backbone consumes only world-frame Cartesian coordinates

$\mathbf{x} = \varphi(\mathbf{T})$ and never the fragment-local decomposition $(\tilde{x}_F, p_F, R_F)$ separately — so its output is, by construction, manifestly independent of any particular choice of R_s, since R_s never appears anywhere in the network's input.

9.6 The forward (noising) process

Remark 9.8 (SigmaDock's own internal time convention, relabelled). SigmaDock's own source code (`r3_diffuser.py`, `so3_diffuser.py`) runs its internal time variable in the *opposite* direction from this document's global convention (Theorem 0.2): its $t = 0$ is clean data ($\alpha_{t=0} = 1$, no noise) and its $t = 1$ is the fully-noised end (confirmed directly, e.g. by `SO3Diffuser.sample_ref` literally evaluating the forward kernel at $t = 1$ to obtain an approximately-prior sample). Every formula in this chapter is stated in *this document's* convention ($t = 0$ noise, $t = 1$ data, matching Theorem 0.2 and, as Section 12.2 confirms, matching SigmaFlow's own native convention exactly), obtained from SigmaDock's native formulas by the substitution $s = 1 - t$ throughout — i.e. s below always denotes SigmaDock's own noising-time variable, and every SigmaDock formula quoted in this chapter has already been relabelled via this substitution before being stated, rather than being transcribed directly in SigmaDock's own time variable.

We can now assemble Chapters 5 and 6's two constructions, componentwise, on the state space $\mathcal{M}_m = SE(3)^m$ fixed by Section 9.1, per Theorem 3.37's licence to do so. Writing $s = 1 - t$ for noising time (Theorem 9.8) and suppressing the fragment index:

Translation: a variance-preserving process (Theorem 5.4), $\beta(s) = \beta_{\min} + s(\beta_{\max} - \beta_{\min})$ (linear schedule), so the conditional law derived in Theorem 5.4 is $p_{s|0}(p^{(s)} \mid p^{\text{data}}) = \mathcal{N}\big(\alpha_s p^{\text{data}}, (1 - \alpha_s^2) I\big)$, $\alpha_s = \exp\big(-\frac{1}{2}\int_0^s \beta(r)\, dr\big)$.

Rotation: a variance-exploding process (Theorem 5.3), with the *logarithmic* schedule $\sigma(s) = \log\big(s e^{\sigma_{\max}} + (1 - s) e^{\sigma_{\min}}\big)$ (confirmed exactly against `so3_diffuser.py` in Section 6.2.2), so the conditional law is $p_{s|0}(R^{(s)} \mid R^{\text{data}}) = \text{IGSO}(3)(R^{\text{data}}, \sigma(s)^2)$ (Theorem 6.4).

By Theorem 3.37's product structure, the forward kernel factorises exactly across the two components (and, trivially, across fragments, by Theorem 3.38), and the stationary distribution — the model's prior — is $p_{\text{init}} = \mathcal{N}(0, I) \otimes \mathcal{U}(SO(3)^m)$: an isotropic Gaussian for translations (Theorem 5.5's VP schedule drives the marginal variance to 1 as $s \to 1$, for a suitably long/steep β schedule) and the Haar-uniform measure for rotations (Theorem 6.2), matching exactly the invariant prior Theorem 9.7 requires.

Remark 9.9 (Coordinate rescaling). Before any of the above is applied, ligand and protein coordinates are centred on the binding-pocket centre of mass and divided by a fixed constant, `dimensional_scale` = 2.7 (`oracle.py`); this same constant additionally rescales *every* graph-construction cutoff radius used throughout Chapter 8 (protein–ligand interaction, Ca–Ca virtual-node, and every other edge type's cutoff is divided by 2.7 before use, per `oracle.py`::`get_edge_specs`) — i.e. this is a single, global Ångstrom-to-model-units rescaling applied uniformly throughout the pipeline, *not* solely a device for matching the translational Gaussian prior's unit variance to the data's spread, as a narrower reading might suggest. Its specific numerical value (2.7) is reported, in the audited thesis draft, as the mean standard deviation (in Å) of fragment centres of mass relative to the pocket centre of mass across the training set — an empirical, dataset-derived constant, not a theoretically derived one.

Training minimises the Riemannian denoising score matching objective of Theorems 5.2 and 6.8, with per-component weights λ_s matching Section 5.2's $\lambda(s) = 1/\mathbb{E}[\|\nabla \log p_{s|0}\|^2]$

recipe: for translation, this reduces to the closed form of Section 5.2's discussion ($\lambda_s \propto 1/(1-\alpha_s^2)$, matching `R3Diffuser.score_weight` exactly); for rotation, this expectation has no closed form and is computed numerically from the tabulated $\partial_\omega \log f_\sigma$ table (Section 6.2.2's `score_scaling` code correspondence). This machinery — the IGSO(3) kernel with its truncated spectral series, the conditional score derived from it, and the tabulated loss weight — is the *entirety* of what is diffusion-specific in SigmaDock; Chapter 12 shows precisely which of these pieces flow matching removes and which it retains unchanged.

9.7 Input graph, featurisation, and backbone

The network never sees the abstract group element $z \in SE(3)^m$ directly; it sees the geometric graph of Theorem 7.12, built from the current Cartesian coordinates $\mathbf{x}^{(t)} = \varphi(z^{(t)})$ (the rigid-fragment reconstruction of Section 9.7.1 below) and the (fixed) protein structure. Chapter 8 already derived this construction in full architectural detail — the categorical atom/edge features and their dimensions (Section 8.8's code correspondence), the radial basis and smooth cutoffs (Section 7.4's code correspondence), the hierarchical virtual-node structure (fragment-level and protein-Cα-level, Section 8.8), and the triangulation edges just derived (Section 9.4) — and none of it is repeated here. We record only the one remaining, purely diffusion-side, distinction: the graph's *topology* splits into a *global* part, immutable along the noising/generation trajectory (intra-fragment bonds, the protein structure, the triangulation and virtual-node edges), and a *transient* part, recomputed at every step from the current, moving positions $\mathbf{x}^{(t)}$ (the protein–ligand interaction radius graph, 4 Å cutoff per Section 7.4's code correspondence): exactly the distinction that made the smooth radial-cutoff envelope χ of (7.9) necessary, since an edge in the transient part can appear or disappear as $\mathbf{x}^{(t)}$ evolves during either training's noising or sampling's ODE/SDE integration.

The backbone is EquiformerV2 exactly as reconstructed in Chapter 8, with the one modification already noted there (Section 8.4's code correspondence): the radial MLPs generating $SO(2)$-convolution weights on transient (protein–ligand interaction) edges have their bias terms removed, which is what makes the smooth-cutoff argument of Section 7.4 hold exactly rather than only approximately.

9.7.1 Fragment reconstruction: the map φ

We finally make precise the map $\varphi : \mathcal{M}_m \to \mathbb{R}^{n\times 3}$, used silently throughout this chapter, that turns an abstract group element $z = (R^{(1)}, p^{(1)}, \dots) \in SE(3)^m$ into the atomic Cartesian coordinates the network actually consumes.

Proposition 9.10 (Rigid per-fragment reconstruction)**.** *Fix a reference conformer $x^{\mathrm{ref}} \in \mathbb{R}^{n\times 3}$ (the rigid-fragment template of Section 9.2, one fixed geometry per training/sampling instance, never itself diffused or flowed), and let p_F^{ref} denote fragment F's centre of mass in that reference geometry. For a target pose $z = (R_F, p_F)_F$, the reconstructed atom position, for atom i in fragment F, is*

$$\varphi(z)_i = R_F\big(x_i^{\mathrm{ref}} - p_F^{\mathrm{ref}}\big) + p_F^{\mathrm{ref}} + \Delta p_F, \qquad \Delta p_F := p_F - p_F^{\mathrm{ref}}, \tag{9.1}$$

i.e. every atom of fragment F is displaced from its reference position by exactly the same rigid motion: rotate about the fragment's own reference centre of mass by R_F, then translate by Δp_F. Protein atoms, having no fragment structure, are passed through unchanged.

Why this is the right formula. This is, by construction, the unique map with two required properties: (i) *rigidity* — for two atoms i, j in the same fragment F, $\|\varphi(z)_i - \varphi(z)_j\| = \|R_F(x_i^{\text{ref}} - x_j^{\text{ref}})\| = \|x_i^{\text{ref}} - x_j^{\text{ref}}\|$ by Theorem 3.16, i.e. every intra-fragment distance — in particular every bond length and bond angle — is preserved *exactly*, for *any* (R_F, p_F), which is exactly the content of Section 9.2's claim that rigidity is enforced structurally, not learned; and (ii) *correct fragment placement* — setting $R_F = I, \Delta p_F = 0$ recovers the reference conformer exactly, and in general the fragment's centre of mass is placed at $p_F^{\text{ref}} + \Delta p_F = p_F$, matching the z-coordinate's own translational component by construction. □

Code correspondence

Mathematical object: φ of Theorem 9.10
Implementation: `denoiser.py::_apply_transformations` (SigmaDock) and, identically in form, `sigma_flow_generator.py::get_transformations_from_rototranslations` (SigmaFlow, Chapter 12)
Tensor shape: `[N, 3]` → `[N, 3]`
Interpretation: both modules implement the identical formula $\texttt{new\_pos}[i] = R_F \cdot (\texttt{pos\_old}[i] - p_F^{\text{ref}}) + p_F^{\text{ref}} + \Delta p_F$, confirming, at the level of this specific, central piece of geometric plumbing, the claim (recorded again in Chapter 12) that SigmaFlow's conversion from SigmaDock leaves this component completely unchanged: only the process that determines *which* $(R_F, \Delta p_F)$ to evaluate φ at, at a given step t, differs between the two methods (Chapters 5 and 6 vs. Chapter 10), never the reconstruction map itself.

9.8 The Newton–Euler prediction head

Section 8.9 established that the network's raw output is a per-atom 3-vector in the world frame, read from the $\ell = 1$ channels. This section derives how that raw output becomes the two score components (7.1) requires — and, in doing so, explains precisely *why* the code names its intermediate quantities "force", "torque", "mass", and "inertia", since this is not decorative language but the literal reuse of classical rigid-body mechanics formulas as an architectural device.

9.8.1 From network output to a physical analogy

Code correspondence

Mathematical object: the raw per-atom network output, interpreted as a noise-prediction (an ϵ-parameterisation target, in the language of Section 5.1: recall $x = z + \sigma\varepsilon$, so predicting ε and predicting the score $-\varepsilon/\sigma$ are related by a fixed, known rescaling)
Implementation: `denoiser.py::_compute_forces`; the network call itself is `epsilon_theta = self.model(batch, t, **model_kwargs)`, followed by `lig_forces = -epsilon_theta.index_select(0, forces_idxs)`
Tensor shape: `[N_ligand_atoms, 3]` (the code's own inline type annotation states `[B x F x A, 3, 3]`, i.e. a full 3×3 matrix per atom rather than a 3-vector; every downstream use — indexing along a single leading dimension, elementwise operations consistent with a 3-vector — is inconsistent with that annotation, and we record it, per this document's stated policy, as a probable stale/incorrect docstring rather than a 3×3-matrix-valued quantity actually being used)
Interpretation: the variable name `lig_forces`, and the sign flip (`-epsilon_theta`), signal the reinterpretation about to happen: **this per-atom vector is about to be**

treated as a literal force acting on that atom, and everything from here to Equation (9.4) below is the consequence of taking that interpretation seriously and applying textbook rigid-body mechanics.

9.8.2 Net force, torque, and inertia: exact classical mechanics

Definition 9.11 (Fragment mass, per this codebase's specific convention)**.** For fragment F with (non-virtual, non-dummy) atoms indexed by $i \in F$, the *mass* is $M_F := |F| = \sum_{i \in F} 1$ — the **atom count**, with every atom assigned unit mass regardless of its chemical identity. This is **not** physical/chemical atomic mass (e.g. carbon is not weighted 12× more than hydrogen); it is a uniform, purely topological quantity.

Code correspondence

Mathematical object: M_F of Theorem 9.11, and the fragment inertia tensor $I_F = \sum_{i \in F}\big(\|r_i\|^2 I_3 - r_i r_i^\top\big)$, $r_i = x_i - p_F$ (position relative to the fragment centre of mass) — the classical inertia tensor of a rigid body composed of point masses, specialised to unit mass per point
Implementation: `denoiser.py`::`get_fragment_mass_inertia`
Interpretation: the assertion `assert M.min() >= 2` (with message `"Fragment mass below 2 detected. Check input data."`) is now precisely explained: it fires whenever a fragment has *fewer than two atoms*, a purely topological/fragmentation degeneracy (a fragment reduced, by FR3D's merging or by the underlying molecule's own structure, to a single heavy atom plus dummies), not a statement about any atom's chemical mass being unusually low — the theory-document text of Chapter 9 must therefore never describe this condition as "fragment mass" in the chemical sense, only in the atom-count sense Theorem 9.11 fixes. The inertia tensor's eigenvalues are additionally floored (via an eigendecomposition, clamping to at least $\max(10^{-8}\,\mathrm{tr}(I)/3,\ 10^{-12})$) before use, protecting the subsequent linear solve (Equation (9.3) below) against near-degenerate (e.g. near-linear, two-to-three-atom) fragments.

Given the per-atom pseudo-forces f_i (the negated network output) and the mass/inertia just defined, the exact classical-mechanics formulas for a rigid body's net force, net torque about its centre of mass, are:

$$F_F = \sum_{i \in F} f_i, \qquad \tau_F = \sum_{i \in F} (x_i - p_F) \times f_i. \tag{9.2}$$

Code correspondence

Mathematical object: (9.2)
Implementation: `denoiser.py`::`linear_mechanics` (a `scatter`/`index_add`-based per-fragment reduction of a per-atom force field, computed at the *current*, time-t fragment centres of mass, i.e. using `coms` = `trans_t`, not the data-time centres)
Interpretation: this is, literally, the textbook definition of net force ($\sum$ of per-particle forces) and net torque about a pivot ($\sum r_i \times f_i$) for a rigid body under a field of per-particle forces — the "pseudo-force"/"Newton–Euler" naming is earned by direct, unmodified reuse of these formulas, with the network's noise prediction standing in for the physical force field.

9.8.3 Newton–Maruyama: one discretised step of rigid-body dynamics

$$dT = \frac{F_F}{M_F} \quad \text{(force over mass = acceleration-like increment)}, \qquad dW = I_F^{-1}\tau_F \quad \text{(Euler's rigid-body equa} \tag{9.3}$$

with $dW \in \mathbb{R}^3$ then lifted to $\mathfrak{so}(3)$ via $\widehat{dW}$ (Theorem 3.19).

Code correspondence

Mathematical object: (9.3)
Implementation: `denoiser.py`::`newton_maruyama` (`dT = force_per_fragment / mass`; `dW_world = torch.linalg.solve(inertia, torque_per_fragment)`, with the inertia additionally Tikhonov-regularised by adding $10^{-8}I$ on top of the eigenvalue floor of Theorem 9.11's code correspondence; then `omega = hat(dW_world)`, clamped entrywise to $[-10^3, 10^3]$ as a loose numerical safety net)
Interpretation: the function name "Newton–Maruyama" fuses Newton's second law (the $dT = F/M$ step) with the Euler–Maruyama discretisation scheme referenced throughout Chapters 5 and 6 (this specific function itself is purely deterministic given force/torque — the stochastic Maruyama noise increment is added *separately*, at the reverse-SDE-integration step of Section 9.10 below, not here). The code's own inline comments assert this quantity is expressed in the "GLOBAL (left-invariant — WORLD) frame", which we read, per Section 3.4's resolved terminology, as the **world-frame (spatial)** angular velocity ω^{sp} of Theorem 3.30 — note that the code's word "left-invariant" carries the opposite attribution to this document's, per Theorem 3.31, while the object meant is unambiguous: no rotation by R_t is applied to dW before or after the linear solve, consistent with this reading, and consistent with I_F itself having been computed at the current *world-frame* positions x_i (not in any fragment-local frame). This is worth flagging explicitly against Section 3.4's resolution for the *flow-matching* side, which found the *opposite* (left-trivialised/body-frame) convention in `so3_flow_matcher.py`: SigmaDock's Newton–Euler head and SigmaFlow's flow-matching update operate in different trivialisation conventions at this specific point, a genuine architectural difference (not necessarily an inconsistency, since each is internally self-consistent within its own module, by Section 3.4's verification for the flow-matching side and the world-frame reading just given here for the diffusion side) that Chapter 12 discusses further when it reconstructs the corresponding flow-matching quantity.

9.8.4 Dressing the raw output into a score: the two branches

The pair $(dT, dW) \in \mathbb{R}^3 \times \mathbb{R}^3$ from (9.3) already has the correct *shape* of a tangent vector on $\mathcal{M}_m$ ((7.1)), but it is not yet a score: Theorem 6.7 and Theorem 2.10 show the true conditional score carries a noise-level-dependent scalar prefactor that this raw mechanical quantity does not yet include. SigmaDock dresses the translational component in one fixed way, and the rotational component via a code-configurable choice of *two* distinct branches.

Translation (single formula, no branching):

$$s_\theta^p = r_3\text{-scaling}(t) \cdot dT, \qquad r_3\text{-scaling}(t) = \texttt{R3Diffuser.score\_scaling}(t), \tag{9.4}$$

matching Theorem 2.10's Gaussian-score form up to the fixed, schedule-determined scalar prefactor of Section 5.2.

Rotation, branch `rot_score_method="space"` (constructor default): treat dW as an *implied final-rotation update* rather than a score directly — form $\widehat{dW}$, rescale by the $SO(3)$-analogue

of the translational scaling (`so3_scaling`, from `SO3Diffuser.score_scaling`), exponentiate ($\Delta R = \exp(\text{scaled } \widehat{dW})$, Section 3.3.3), compose $\widehat{R}_0 := \Delta R \cdot R_t$ (an implied estimate of the clean/data rotation), and *then* evaluate the exact closed-form conditional score (6.2) of Theorem 6.7 at this implied $\widehat{R}_0$:

$$s_\theta^R = \nabla_{R_t} \log p_{t|1}(R_t \mid \widehat{R}_0) \qquad \text{(Equation (6.2), evaluated at } R_1 = \widehat{R}_0). \tag{9.5}$$

Rotation, branch `rot_score_method="score"` (raises a configuration error unless a scaling sub-choice is also specified): treat dW as *directly proportional to* the score itself,

$$s_\theta^R = -\alpha(t) \cdot \widehat{dW} \cdot R_t, \tag{9.6}$$

with the scalar $\alpha(t)$ chosen by a further sub-choice: either `"rms"` (reuse the same `so3_scaling` used by the `"space"` branch) or `"true"` (evaluate $\partial_\omega \log f_{\sigma(t)}(\omega)/\omega$ live, at the network's own implied angle $\omega = \|dW\|$, from Section 6.2.2's `d_log_f_d_omega` routine — i.e. the *exact* closed-form score-scaling of Theorem 6.7 evaluated at whatever angle the network happens to have produced, rather than the RMS-averaged $\sigma(t)$-only scaling of the `"rms"` option).

Caution 9.12 (This document's confirmed configuration). Per the code-research pass underlying this document, the constructor default is `rot_score_method="space"`, `rot_score_scaling="rms"`. Which branch the specific trained checkpoints referenced elsewhere in this project's broader development history actually used at training time was not independently cross-checked against those checkpoints' saved hyperparameters as part of producing this document, and should be verified separately before drawing any conclusion that depends on the specific branch in force.

9.9 The training loss

Code correspondence

Mathematical object: the four-component SigmaDock training loss
Implementation: `denoiser.py`::`compute_losses`
Interpretation: four terms, each a λ-weighted squared error, matching (5.4)'s continuous-time DSM objective pattern:

$$\begin{aligned}
\text{(score-matching, translation)} \quad & \lambda_t^p \left\| s_\theta^p - s_{\text{true}}^p \right\|_2^2, \qquad \lambda_t^p = 1/(\text{T_score_scaling})^2, \\
\text{(score-matching, rotation)} \quad & \lambda_t^R \left\| s_\theta^R - s_{\text{true}}^R \right\|_{\text{Fro}}^2, \qquad \lambda_t^R = 1/(\text{R_score_scaling})^2, \\
\text{(data-space, translation)} \quad & \lambda_t^{p,0} \left\| p_0 - \widehat{p}_0 \right\|_2^2, \qquad \lambda_t^{p,0} = (\alpha_t \cdot \text{T_score_scaling})^2, \\
\text{(data-space, rotation)} \quad & \lambda_t^{R,0} \left\| \log(\widehat{R}_0^\top R_0) \right\|_{\text{Fro}}^2, \qquad \lambda_t^{R,0} = (\text{R_score_scaling})^2,
\end{aligned}$$

returned as a four-key dictionary {`T_score`, `R_score`, `T0`, `R0`}.

Code correspondence

Mathematical object: per-molecule reduction of the (per-fragment) losses above
Implementation: `denoiser.py`::`scaled_fragmented_loss`: `scatter(x, batch_idx, reduce="sum") / num_fragments^fragment_scaling`
Interpretation: `fragment_scaling`$= 1$ divides by the exact fragment count, i.e. averages per molecule regardless of how many fragments it has; `fragment_scaling`$= -1$ (per one code comment site's own docstring) applies *no* normalisation, biasing the loss towards molecules with more fragments; intermediate values interpolate. **Note, per the**

code-research pass: the default value of this hyperparameter differs between two source locations in the codebase (`1.0` in one configuration dataclass, `0.5` in the Lightning module's own constructor default) — almost certainly immaterial in practice if the training script's CLI parsing always threads an explicit value through, but this was not independently confirmed, and is recorded as an open item rather than resolved by assumption.

9.10 Sampling

Generation integrates the reverse-time SDEs of Equations (5.5) and (6.3), componentwise, from a near-$t = 0$ starting time to $t = 1$.

Code correspondence

Mathematical object: the timestep discretisation of the reverse integration
Implementation: `sampling.py`::`sampler`, matching `conf/sampling/base.yaml`'s `diffusion:` block
Interpretation: default `num_steps`= 25, `t_min`= 0.005, `rho`= 3.0, default `discretization="edm"` (the Karras et al. power-law-in-$1/\rho$ schedule, $t_k = \left(t_{\max}^{1/\rho} + \frac{k}{N-1}(t_{\min}^{1/\rho} - t_{\max}^{1/\rho})\right)^{\rho}$, rather than the simpler `"power"` schedule $t_k = \text{linspace}(t_{\max}, t_{\min}, N)^{\rho}$, which is available but not the default here); the timestep array itself walks *downward*, in this module's own internal (noising-time-aligned) convention, from $t_{\max} = 1$ to $t_{\min}$, consistent with Theorem 0.2's warning that SigmaDock's native time convention is reversed relative to this document's global convention (Chapter 12 makes the exact reversal explicit and code-confirmed).

Code correspondence

Mathematical object: one reverse-time step (Euler–Maruyama for translation, geodesic random walk for rotation)
Implementation: `sampling.py`::`sampler`'s loop body, composing `denoiser._compute_forces` → `_compute_fragment_dynamics` → `_predict_fragment_updates` (Newton–Maruyama, Equation (9.3)) → `_compute_scores` (Section 9.8.4's branch) → `denoiser.diffuser.reverse` (dispatching componentwise to `R3Diffuser.reverse`, Theorem 5.4's code correspondence, and `SO3Diffuser.reverse`, Section 6.3's code correspondence) → `_apply_transformations` (rigid-fragment position reconstruction, Section 9.7.1)
Interpretation: exactly one EquiformerV2 forward pass per integration step — the same cost structure Chapters 10 and 12 will show flow matching shares, so any difference in the number of network evaluations needed for comparable sample quality between the two methods is a genuine, comparable efficiency question, examined in this project's broader experimental work rather than in this theory document. **Heun's method is not implemented**, despite `solver: Literal["euler","heun"]` appearing in the function signature: both this sampler and the flow-matching sampler of Chapter 12 contain only a dead `# TODO Heun's method` comment block after the (Euler-only) loop; selecting `solver="heun"` has, per the code-research pass underlying this document, no effect on the executed computation.

Code correspondence

Mathematical object: the stochastic noise-annealing schedule
Implementation: quadratic decay `noise_scales` =

linspace$\big(\texttt{noise_scale}^{1/\texttt{noise_decay}}, 0, \texttt{num_steps}\big)^{\texttt{noise_decay}}$, default `noise_scale`$=0.0$, `noise_decay`$=2.0$ (per `conf/sampling/base.yaml`)
Interpretation: this array rescales *only* the injected Brownian-increment term of the Euler–Maruyama/geodesic-random-walk update (the $\sqrt{dt}\,z$ term), independently of the score-drift term, and decays from `noise_scale` to exactly 0 over the course of sampling. **With the shipped default `noise_scale=0.0`, every injected-noise term is zero at every step**: SigmaDock's *out-of-the-box default sampling configuration is already fully deterministic*, i.e. it already runs, without any code change, the probability-flow-ODE-equivalent trajectory of Theorem 5.7 rather than a genuinely stochastic SDE trajectory. This narrows the "diffusion is stochastic, flow matching is deterministic" framing considerably: the distinction that is architecturally forced for flow matching (Chapter 10 never constructs a noising process at all) is, for this specific configuration of SigmaDock, an *opt-in* choice (setting `noise_scale`>0) rather than an always-active default behaviour.

9.11 Ranking generated poses

SigmaDock draws N_{seeds} (default 40, per the audited thesis draft) independent samples per complex and ranks them by a fixed, learned-model-free heuristic,

$$s_i = -b_i\, p_i^{\beta}, \qquad \beta = 4,$$

where b_i is a Vinardo docking score (a classical, physics-based scoring function, not part of the generative model itself) and $p_i \in [0,1]$ is the fraction of PoseBusters checks (Theorem 1.6) sample i passes. SigmaDock deliberately dispenses with both a learned confidence/ranking model and any post-hoc coordinate-altering energy minimisation — both standard in the DiffDock lineage this document's introduction references — specifically so that reported accuracy can be attributed to the generative model itself rather than to a downstream correction step; Section 12.9 in Chapter 12 returns to whether flow matching's ODE structure offers a principled alternative to this heuristic.

Part VI

Flow Matching

Chapter 10

Continuous Normalizing Flows and Flow Matching in $\mathbb{R}^d$

Section 5.3 showed, as a *consequence* of diffusion theory, that every score-based diffusion model secretly transports p_{init} to p_{data} along a deterministic ODE. This chapter develops the same object — a deterministic flow transporting one distribution to another — as a *starting point* rather than a derived fact, entirely independently of scores, noising processes, or SDEs. The resulting framework, flow matching, is what Chapter 12 substitutes for Chapters 5 and 6 to obtain SigmaFlow.

10.1 Continuous normalizing flows

Definition 10.1 (Time-dependent vector field, flow map)**.** A *time-dependent vector field* is a map $u : \mathbb{R}^d \times [0,1] \to \mathbb{R}^d$. Given u, the associated *flow map* $\phi_t : \mathbb{R}^d \to \mathbb{R}^d$ is defined by solving, for each starting point x,

$$\frac{d}{dt}\phi_t(x) = u_t(\phi_t(x)), \qquad \phi_0(x) = x,$$

i.e. $\phi_t(x)$ is the position at time t of the particle that started at x and has been carried along by the velocity field u ever since. We assume throughout (as is standard, and guaranteed by, e.g., u_t globally Lipschitz in x uniformly in t) that this ODE has a unique solution for every x and every $t \in [0,1]$, so that ϕ_t is well defined and, since distinct trajectories of an autonomous-in-space-at-each-fixed-t ODE cannot cross without violating uniqueness, a diffeomorphism of $\mathbb{R}^d$ for each fixed t.

Definition 10.2 (Continuous normalizing flow)**.** Given u and a source density p_{init}, the *continuous normalizing flow (CNF)* associated with u is the probability path $p_t := (\phi_t)_{\#}p_{\text{init}}$, the pushforward (Theorem 2.4) of p_{init} under the flow map.

Equivalently, and operationally: draw $X_0 \sim p_{\text{init}}$, solve $\dot{X}_t = u_t(X_t)$ forward to $t = 1$; then $X_1 \sim p_1$. This is precisely Theorem 10.1's flow map restated with full definitional care, and it is worth being explicit about what has *not* yet been claimed: nothing so far says $p_1 = p_{\text{data}}$ for any *particular* choice of u, only that *some* probability path p_t is generated by *any* choice of (well-behaved) u_t. The next result is the tool that lets us go the other way: given a *desired* path p_t, characterise which vector fields generate it, so that we may search for u within that characterisation rather than by trial and error.

10.1.1 The continuity equation

Theorem 10.3 (Continuity equation)**.** *Let u be a time-dependent vector field with flow ϕ_t (Theorem 10.1), and let $p_t = (\phi_t)_{\#}p_{\text{init}}$ (Theorem 10.2). Then, wherever p_t is differentiable,*

$$\partial_t p_t(x) + \nabla \cdot \big(u_t(x) p_t(x)\big) = 0, \qquad \nabla \cdot v := \sum_{k=1}^{d} \partial_{x_k} v_k \ \text{(divergence)}. \tag{10.1}$$

Conversely, if a smooth, compactly-supported-density path (p_t) and vector field u_t satisfy (10.1), then u_t generates (p_t) in the sense of Theorem 10.2 (given standard regularity sufficient for existence/uniqueness of the flow, Theorem 10.1).

Proof. (Forward direction.) By Theorem 2.5 applied to the diffeomorphism ϕ_t,

$$p_t(x) = p_{\text{init}}\big(\phi_t^{-1}(x)\big)\,\big|\det D\phi_t^{-1}(x)\big|.$$

Write $J_t(x) := \det D\phi_t(\phi_t^{-1}(x))$, so $\big|\det D\phi_t^{-1}(x)\big| = 1/J_t(x)$ (inverse function theorem, and $J_t > 0$ since ϕ_t is orientation-preserving as the flow of a continuous vector field starting at the identity, $\phi_0 = \mathrm{id}$, $\det D\phi_0 = 1 > 0$, and J_t cannot cross zero continuously without the flow ceasing to be a diffeomorphism). We use *Liouville's formula* for the rate of change of this Jacobian along the flow,

$$\frac{d}{dt} \log J_t(\phi_t(x_0)) = (\nabla \cdot u_t)\big(\phi_t(x_0)\big), \tag{10.2}$$

which we justify as follows: differentiating $J_t(\phi_t(x_0)) = \det D\phi_t(x_0)$ in t, using the matrix identity $\frac{d}{dt}\det A(t) = \det A(t)\,\operatorname{tr}\big(A(t)^{-1}\dot{A}(t)\big)$ (Jacobi's formula, a standard fact of matrix calculus we take as given) with $A(t) = D\phi_t(x_0)$, and $\dot{A}(t) = D\big(\partial_t\phi_t(x_0)\big) = D\big(u_t(\phi_t(x_0))\big) = Du_t(\phi_t(x_0)) \cdot D\phi_t(x_0) = Du_t(\phi_t(x_0))\,A(t)$ (chain rule, differentiating the defining ODE of Theorem 10.1 with respect to the initial condition x_0, and using that spatial and initial-condition derivatives commute for a smooth flow), gives $\operatorname{tr}(A(t)^{-1}\dot{A}(t)) = \operatorname{tr}\big(Du_t(\phi_t(x_0))\big) = (\nabla \cdot u_t)(\phi_t(x_0))$ (the trace of the Jacobian matrix of a vector field is, by definition, its divergence), yielding (10.2) directly.

Now compute $\partial_t p_t(x)$ along a *fixed* x by relating it to the material (Lagrangian) derivative along the flow. Write $q(t) := p_t(\phi_t(x_0))\,J_t(\phi_t(x_0))$ for fixed x_0; since $p_t(\phi_t(x_0)) = p_{\text{init}}(x_0)/J_t(\phi_t(x_0))$ (the change-of-variables formula above, evaluated at $x = \phi_t(x_0)$, using $\phi_t^{-1}(\phi_t(x_0)) = x_0$), we get $q(t) = p_{\text{init}}(x_0)$, constant in t. Differentiating $q(t) = p_t(\phi_t(x_0))J_t(\phi_t(x_0))$ using the product rule and the chain rule (treating $p_t(\phi_t(x_0))$'s time-derivative as $\partial_t p_t(\phi_t(x_0)) + \nabla p_t(\phi_t(x_0)) \cdot u_t(\phi_t(x_0))$, the material derivative), and using (10.2) for $\dot{J}_t(\phi_t(x_0)) = J_t(\phi_t(x_0))(\nabla \cdot u_t)(\phi_t(x_0))$, gives, after dividing by $J_t(\phi_t(x_0)) \neq 0$ and evaluating at the (arbitrary) point $x = \phi_t(x_0)$,

$$0 = \partial_t p_t(x) + \nabla p_t(x) \cdot u_t(x) + p_t(x)\,(\nabla \cdot u_t)(x) = \partial_t p_t(x) + \nabla \cdot \big(p_t(x) u_t(x)\big),$$

using the product rule for divergence, $\nabla \cdot (pu) = \nabla p \cdot u + p\,\nabla \cdot u$, in the last step. This is (10.1).

(Converse direction.) We do not prove the converse in full generality here (it is a uniqueness statement for the linear transport PDE (10.1), following from the method of characteristics: any two solutions of (10.1) with the same initial condition p_0 agree, because (10.1) is exactly the statement that mass is conserved along the characteristics $\dot{x} = u_t(x)$, so the unique solution is obtained by transporting p_0's mass along those characteristics, which is precisely $(\phi_t)_{\#}p_0$); see, e.g., Villani, *Optimal Transport: Old and New*, 2009, §4, or Chen et al., *Neural Ordinary Differential Equations*, NeurIPS 2018, §2 for the specific CNF framing. □

Theorem 10.3 is the ODE analogue of a Fokker–Planck equation (indeed Theorem 5.7's proof sketch used exactly this fact, citing this theorem for the diffusionless special case that appears when the SDE's diffusion term is cancelled by the specific drift construction there), and it is the tool that converts "I want to transport p_{init} to p_{data}" into a concrete, checkable condition on u_t: any (p_t, u_t) pair satisfying (10.1), with $p_0 = p_{\text{init}}$, generates a CNF ending, at $t = 1$, at whatever p_1 the pair specifies.

10.1.2 CNF likelihood

A byproduct of Theorem 10.3's proof is worth recording separately, since Section 12.9 in Chapter 12 needs it.

Corollary 10.4 (Instantaneous change of variables)**.** *Along a trajectory $X_t = \phi_t(X_0)$ of the flow, the log-density evolves by*

$$\frac{d}{dt} \log p_t(X_t) = -(\nabla \cdot u_t)(X_t).$$

Proof. This is exactly (10.2) combined with $p_t(\phi_t(x_0)) = p_{\text{init}}(x_0)/J_t(\phi_t(x_0))$ from the proof of Theorem 10.3: taking log of both sides, $\log p_t(\phi_t(x_0)) = \log p_{\text{init}}(x_0) - \log J_t(\phi_t(x_0))$, and differentiating in t using (10.2) for the second term gives the claim directly. □

Theorem 10.4 says that, given the vector field u_t *and* the initial log-density $\log p_{\text{init}}(X_0)$, the log-density of the sample $X_1 = \phi_1(X_0)$ can be recovered *exactly* by integrating the (scalar!) divergence $-\nabla \cdot u_t$ along the same trajectory the sample itself follows — no separate, learned density network is needed, only the already-learned vector field and one additional scalar ODE run alongside the original one. Evaluating $\nabla \cdot u_t$ exactly requires the trace of a $d \times d$ Jacobian, $\mathcal{O}(d)$ backward passes if done naively; in practice this is estimated stochastically via the *Hutchinson trace estimator*,

$$\nabla \cdot u_t(x) = \operatorname{tr}\big(Du_t(x)\big) = \mathbb{E}_\varepsilon\big[\varepsilon^\top Du_t(x)\varepsilon\big]$$

for any ε with $\mathbb{E}[\varepsilon\varepsilon^\top] = I$ (e.g. $\varepsilon \sim \mathcal{N}(0, I)$ or Rademacher), computable with a single Jacobian-vector product per sample of ε rather than a full Jacobian; we record this identity (a direct consequence of linearity of trace and expectation, $\mathbb{E}[\varepsilon^\top A\varepsilon] = \mathbb{E}[\operatorname{tr}(A\varepsilon\varepsilon^\top)] = \operatorname{tr}(A\mathbb{E}[\varepsilon\varepsilon^\top]) = \operatorname{tr}(A)$) without further development, since Section 12.9 records only whether this machinery is implemented for SigmaFlow, not an independent numerical study of it.

10.2 Conditional flow matching

Theorem 10.3 characterises which u_t generate a *given* path (p_t), but a path connecting an entire distribution p_{init} to an entire distribution p_{data} is generally just as intractable to write down directly as the marginal score of Section 5.1 was. The resolution is, again, Theorem 2.3's conditioning trick, now applied to probability paths instead of static densities — and the reader will notice the entire structure of this section mirrors Section 5.1's denoising-score-matching argument almost verbatim, which is not a coincidence but the content of Section 5.3's "diffusion and flow matching are two parameterisations of one object" claim, now demonstrated by the parallel structure of the two derivations rather than merely asserted.

Definition 10.5 (Conditional probability path, conditional vector field)**.** A *conditional probability path* is a family $p_t(\cdot \mid z)$, $z \in \mathbb{R}^d$, with $p_0(\cdot \mid z) = p_{\text{init}}$ and $p_t(\cdot \mid z) \to \delta_z$ weakly as $t \to 1$ (a Dirac mass at z), for every z. A *conditional vector field* $u_t(\cdot \mid z)$ generates $p_t(\cdot \mid z)$ in the sense of Theorem 10.3, for each fixed z and each $t \in [0, 1)$.

Caution 10.6 (The endpoint $t = 1$ is a limit, not a density). δ_z is a measure, not a density with respect to Lebesgue measure, so strictly speaking $p_1(\cdot \mid z)$ is outside the framework of Theorem 2.1, and Theorem 10.3 — which requires p_t to be differentiable — does not apply at $t = 1$. Everything below is therefore stated on $[0, 1)$ and extended to $t = 1$ by continuity. Two remarks make this harmless, and one makes it practically important.

Harmless: the marginal path is still well defined, since $p_t(x) = \int p_t(x \mid z) p_{\text{data}}(z) dz \to p_{\text{data}}(x)$ as $t \to 1$ in the same weak sense; and the training loss $\mathcal{L}_{\text{CFM}}$ samples $t \sim \mathcal{U}(0, 1)$, so $t = 1$ has probability zero.

Practically important: the conditional field (10.4) has a $1/(1 - t)$ factor that blows up as $t \to 1$. Implementations therefore either use the equivalent endpoint form $z - x_0$ (which is finite everywhere — see the discussion after (10.4)), or replace the Dirac endpoint by a narrow Gaussian $\mathcal{N}(z, \sigma_{\min}^2 I)$ with $\sigma_{\min} > 0$, as in Lipman et al. (2023).

SigmaFlow takes the first route. Its `r3_flow_matcher.py` accepts a $\sigma_{\min}$ parameter, but the parameter is stored and never read (Section 12.3 verifies this against the source), and its default is 0; the endpoint really is a Dirac, and the $1/(1 - t)$ singularity is avoided purely by using the endpoint form of the target. This is legitimate, but it means the sampler must not be integrated to exactly $t = 1$ with the current-point form — a point taken up in Section 12.8.

By Theorem 2.3, the *marginal* path $p_t(x) := \int p_t(x \mid z) p_{\text{data}}(z)\, dz$ automatically satisfies $p_0 = p_{\text{init}}$ (each conditional starts at p_{init}, so their p_{data}-weighted mixture does too) and $p_1 = p_{\text{data}}$ ($p_1(x) = \int \delta_z(x) p_{\text{data}}(z) dz = p_{\text{data}}(x)$). The following is the flow-matching analogue of Theorem 10.7, now proved (main.tex, the audited draft, stated this without proof; we supply one, since it is short and instructive).

Theorem 10.7 (Marginalisation of vector fields)**.** *Let $u_t(\cdot \mid z)$ generate the conditional path $p_t(\cdot \mid z)$ for every z. Then the marginal vector field*

$$u_t(x) := \int u_t(x \mid z)\, \frac{p_t(x \mid z)\, p_{\text{data}}(z)}{p_t(x)}\, dz = \mathbb{E}\big[u_t(x \mid Z) \mid X_t = x\big] \tag{10.3}$$

(a $p_t(x)$-weighted average of the conditional fields, i.e. the posterior expectation of $u_t(x \mid Z)$ given that the marginal path is at x at time t, treating $Z \sim p_{\text{data}}$ as a latent variable and $X_t \sim p_t(\cdot \mid Z)$) generates the marginal path (p_t).

Proof. It suffices, by Theorem 10.3, to check that (p_t, u_t) satisfies the continuity equation (10.1). Each conditional pair satisfies it: $\partial_t p_t(x \mid z) + \nabla \cdot (u_t(x \mid z) p_t(x \mid z)) = 0$ for every z. Multiply by $p_{\text{data}}(z)$ and integrate over z:

$$\partial_t \underbrace{\int p_t(x \mid z) p_{\text{data}}(z)\, dz}_{= p_t(x)} + \nabla \cdot \underbrace{\int u_t(x \mid z) p_t(x \mid z)\, p_{\text{data}}(z)\, dz}_{=: V(x)} = 0$$

(differentiation and the divergence operator both commute with integration over z, a standard application of dominated convergence, which we do not belabour). It remains to check $V(x) = u_t(x) p_t(x)$ with u_t as defined in (10.3): indeed, $V(x) = \int u_t(x \mid z)\, p_t(x \mid z) p_{\text{data}}(z)\, dz = p_t(x) \int u_t(x \mid z) \frac{p_t(x \mid z) p_{\text{data}}(z)}{p_t(x)}\, dz = p_t(x) u_t(x)$, using the definition of $u_t(x)$ verbatim. Hence $\partial_t p_t(x) + \nabla \cdot (u_t(x) p_t(x)) = 0$, i.e. (p_t, u_t) satisfies (10.1), and Theorem 10.3's converse direction gives the claim. □

The marginal field (10.3), like the marginal score of Section 5.1, is an intractable integral (it requires the unknown $p_t(x)$ in its own denominator). The following is the flow-matching analogue of Theorem 5.2, and, again, its proof is structurally identical.

Theorem 10.8 (Conditional flow matching)**.** *Define*

$$\mathcal{L}_{\mathrm{FM}}(\theta) = \mathbb{E}_{t\sim\mathcal{U}(0,1),\, x\sim p_t}\big[\|u_\theta(x,t) - u_t(x)\|^2\big], \qquad \mathcal{L}_{\mathrm{CFM}}(\theta) = \mathbb{E}_{t,\, z\sim p_{\mathrm{data}},\, x\sim p_t(\cdot|z)}\big[\|u_\theta(x,t) - u_t(x\mid z)\|^2\big].$$

Then $\mathcal{L}_{\mathrm{FM}}(\theta) = \mathcal{L}_{\mathrm{CFM}}(\theta) + C$, C independent of θ; the two losses have identical gradients and minimisers.

Proof. Identical in structure to Theorem 5.2's proof, with $\nabla_x \log p_\sigma(\cdot)$ replaced throughout by $u_t(\cdot)$, and the Gaussian perturbation kernel replaced by the general conditional path $p_t(\cdot \mid z)$: expand the square under $x \sim p_t$, the first (quadratic-in-θ) term already matches by Theorem 2.3 applied to the joint law of (Z, X_t); the third (θ-independent) term is absorbed into C; and the cross term is handled by the identity $p_t(x)u_t(x) = \int u_t(x \mid z)p_t(x \mid z)p_{\mathrm{data}}(z)\,dz$ established in Theorem 10.7's proof (specifically the definition of $V(x)$ there), which lets the cross term $\int \langle u_\theta(x,t), u_t(x)\rangle p_t(x)\,dx$ be rewritten as an expectation over (Z, X_t) exactly as in Theorem 5.2's proof. We do not repeat every line. □

Theorem 10.8 is the entire flow-matching recipe: pick *any* conditional path $p_t(\cdot \mid z)$ whose conditional vector field $u_t(\cdot \mid z)$ can be written down in closed form, train u_θ by regressing against that closed-form target on samples (t, z, x) drawn exactly as in $\mathcal{L}_{\mathrm{CFM}}$, and the trained u_θ approximates the marginal field that transports p_{init} to p_{data} — without ever needing to evaluate the intractable integral (10.3).

10.2.1 The linear (conditional optimal transport) path

Proposition 10.9 (The linear path and its vector field)**.** *Let $p_t(\cdot \mid z)$ be the law of*

$$X_t = (1-t)X_0 + tz, \qquad X_0 \sim p_{\mathrm{init}} \text{ drawn independently of } z,$$

i.e. $p_t(\cdot \mid z) = \big(x_0 \mapsto (1-t)x_0 + tz\big)_{\#} p_{\mathrm{init}}$ in the notation of Theorem 2.4. For the Gaussian source $p_{\mathrm{init}} = \mathcal{N}(0, I_d)$ this is explicitly

$$p_t(\cdot \mid z) = \mathcal{N}\big(tz,\ (1-t)^2 I_d\big), \qquad t \in [0, 1),$$

an ordinary linear interpolation between an independently drawn noise sample and the data point z. This path satisfies Theorem 10.5, and its conditional vector field is

$$u_t(x \mid z) = \frac{z - x}{1-t} = z - x_0 \qquad \text{(the second equality valid exactly on the interpolant itself, } x = (1-t)x_0 + tz\text{).} \tag{10.4}$$

Proof. $p_0(\cdot \mid z)$: at $t = 0$, $X_0 \sim p_{\mathrm{init}}$ by construction. $p_1(\cdot \mid z)$: at $t = 1$, $X_1 = z$ deterministically, i.e. $p_1(\cdot \mid z) = \delta_z$. For the vector field, differentiate the interpolant directly: $X_t = (1-t)X_0 + tz \;\Rightarrow\; \dot{X}_t = z - X_0$, constant in t along a fixed sample path; substituting $X_0 = (X_t - tz)/(1-t)$ (inverting the interpolation formula) gives $\dot{X}_t = z - (X_t - tz)/(1-t) = \big((1-t)z - X_t + tz\big)/(1-t) = (z - X_t)/(1-t)$, recovering (10.4)'s first form as a function of the current point $X_t = x$ alone (as required by Theorem 10.5, which needs $u_t(\cdot \mid z)$ as a genuine function on $\mathbb{R}^d$, not merely along one sample path). □

Equation (10.4)'s two forms are algebraically equal (exactly, not merely approximately) when evaluated on the interpolant, but computationally distinct: the first, $(z-x)/(1-t)$, is a function of (x,t,z) alone and is what a network, given only the current state and time, is asked to match at training time; the second, $z-x_0$, is *constant in* t along a fixed pair (x_0,z) and is the more numerically convenient quantity to compute the regression *target* from during training (no division, no singularity as $t\to 1$), since x_0, z are both known exactly at the moment a training example is constructed. This exact distinction — "current-point form", singular at $t\to 1$, versus "constant-in-t endpoint form", used for the actual training target — reappears, verified directly against the code, as precisely the two forms `conditional_probability_path` and `calc_trans_vector_field` realise in SigmaFlow's `r3_flow_matcher.py`, reconstructed exactly in Section 12.3.

Remark 10.10 (On the name "conditional optimal transport"). The linear path is frequently called the "conditional optimal transport" (CondOT) path, since for fixed x_0, z the straight-line interpolant $(1-t)x_0+tz$ is exactly the (constant-speed) geodesic of the flat Euclidean metric between them, and constant-speed geodesics are the building block of Euclidean optimal-transport (Wasserstein) interpolation. We do not develop optimal transport theory in this document; the name is recorded for consistency with the literature this monograph audits and with the terminology used when this path is generalised to a Riemannian manifold in Chapter 11, where "geodesic interpolant" (a notion this document *has* developed, Theorem 3.4) is the more directly justified term and the one used throughout the rest of this document.

10.3 Sampling: integrating the learned ODE

Given a trained $u_\theta \approx u_t$, generation solves $\dot{X}_t = u_\theta(X_t,t)$, $X_0 \sim p_{\text{init}}$, forward to $t=1$, by any numerical ODE solver; the simplest, and the one this document's code correspondences confirm is what both SigmaDock's (nominally deterministic default, Section 9.10) and SigmaFlow's samplers actually execute despite a `"heun"` option appearing in each function's type signature, is the forward Euler step,

$$X_{t+\Delta t} = X_t + \Delta t\, u_\theta(X_t,t).$$

Unlike the diffusion sampler of Equation (6.3), there is no injected noise anywhere in this recipe: the entire stochasticity of the generative model is confined to the single draw $X_0 \sim p_{\text{init}}$, after which the trajectory is completely deterministic given the trained network — the structural fact, not an implementation simplification, that makes the "deterministic vs. stochastic" framing of Section 9.10's noise-annealing discussion an *architecturally forced* property for flow matching specifically, where for SigmaDock (as that section showed) it was, in the shipped default configuration, already true as well, just not forced by the method's structure.

Chapter 11

Riemannian Flow Matching

Chapter 10 developed flow matching entirely in $\mathbb{R}^d$; by Theorem 3.37 this already suffices for translation. This chapter lifts every construction to a Riemannian manifold $\mathcal{M}$, following exactly the same two-step pattern Chapter 6 used for diffusion — state the general manifold construction, then specialise concretely to $SO(3)$ — and, as that chapter did, ends by comparing directly against SigmaFlow's actual source code.

11.1 Vector fields and flows on a manifold

Definition 11.1 (Vector field on a manifold, flow)**.** A (time-dependent) *vector field* on $\mathcal{M}$ is $u : \mathcal{M} \times [0,1] \to T\mathcal{M}$ with $u_t(x) \in T_x\mathcal{M}$ for every x (i.e. a section of the tangent bundle at each time). Its *flow* $\phi_t : \mathcal{M} \to \mathcal{M}$ solves $\frac{d}{dt}\phi_t(x) = u_t(\phi_t(x))$, $\phi_0(x) = x$, exactly as in Theorem 10.1, now understood as a curve *on the manifold* whose velocity at each instant is the prescribed tangent vector.

Everything in Section 10.1 generalises verbatim once "vector field" and "flow" are understood in the sense of Theorem 11.1: a CNF on $\mathcal{M}$ is $(\phi_t)_{\#}p_{\text{init}}$ for p_{init} a distribution on $\mathcal{M}$; a manifold continuity equation $\partial_t p_t + \operatorname{div}_g(u_t p_t) = 0$ holds, with div_g the Riemannian divergence (the manifold generalisation of $\nabla\cdot$, defined so that Theorem 10.3's proof — which used only the change-of-variables formula, itself valid on a manifold with the Riemannian volume form in place of Lebesgue measure, and Liouville's formula, which likewise generalises — carries over without conceptual change); and Theorems 10.7 and 10.8's proofs use nothing beyond linearity of integration and the (now manifold) continuity equation, so they too generalise verbatim, with $\|\cdot\|$ replaced by the Riemannian norm $\|\cdot\|_g$ of Theorem 3.4. We record the two generalised results once, for reference, without repeating their (unchanged) proofs.

Theorem 11.2 (Riemannian conditional flow matching)**.** *Let $p_t(\cdot \mid z)$ be a conditional probability path on $\mathcal{M}$ ($p_0(\cdot \mid z) = p_{\text{init}}$, $p_1(\cdot \mid z) = \delta_z$) with conditional vector field $u_t(\cdot \mid z)$. Then*

$$\mathcal{L}_{\text{RCFM}}(\theta) = \mathbb{E}_{t,\, z\sim p_{\text{data}},\, x\sim p_t(\cdot|z)}\Big[\big\|u_\theta(x,t) - u_t(x \mid z)\big\|_g^2\Big]$$

has the same gradients and minimisers, in θ, as the (intractable) marginal loss $\mathbb{E}_{t,x\sim p_t}[\|u_\theta(x,t) - u_t(x)\|_g^2]$, where $u_t(x) = \mathbb{E}[u_t(x \mid Z) \mid X_t = x]$ as in (10.3).

What remains, exactly as in Chapter 6's treatment of diffusion, is to exhibit a *specific*, computable conditional path and its vector field on $SO(3)$ — and this is where the Riemannian generalisation of exp/log (Theorem 3.7) does all the work, in a construction dramatically simpler than Chapter 6's heat-kernel machinery.

11.2 The geodesic conditional path

Definition 11.3 (Geodesic interpolant)**.** For $x_0, z \in \mathcal{M}$ (assuming, for simplicity, that z lies within the domain where $\log_{x_0}$ is defined — always true when $\mathcal{M} = SO(3)$ except at the single problematic value $\theta = \pi$ discussed already in Section 3.3.4, and handled there), the *geodesic interpolant* between x_0 and z is

$$\gamma_t(x_0, z) := \exp_{x_0}\big(t \log_{x_0}(z)\big).$$

By Theorem 3.4,Theorem 3.7, this traces exactly the (constant-speed) geodesic from x_0 ($t = 0$) to z ($t = 1$), the direct manifold generalisation of $\mathbb{R}^d$'s straight line $(1-t)x_0 + tz$ (indeed, on $\mathcal{M} = \mathbb{R}^d$ with the flat metric, $\exp_x(v) = x + v$, $\log_x(y) = y - x$ exactly, so Theorem 11.3 reduces to $\gamma_t = x_0 + t(z - x_0) = (1-t)x_0 + tz$, recovering Theorem 10.9's linear path precisely as the flat special case).

Proposition 11.4 (Conditional vector field of the geodesic interpolant)**.** *Fixing $x_0 \sim p_{\text{init}}$ and setting $X_t = \gamma_t(x_0, z)$ (Theorem 11.3), the tangent vector $u_t(X_t \mid z) := \dot{X}_t$ satisfies*

$$u_t(X_t \mid z) = \frac{\log_{X_t}(z)}{1-t},$$

generalising Theorem 10.9's Euclidean formula exactly by replacing $z - x$ with $\log_x(z)$.

We give this proposition's proof concretely for $\mathcal{M} = SO(3)$ in Section 11.3 below (the general Riemannian proof requires facts about geodesics under reparameterisation and Jacobi fields that are more naturally handled case-by-case for the one manifold this document actually needs than developed in full generality; the $SO(3)$ computation that follows is a complete, self-contained proof for the case that matters).

11.3 Flow matching on $SO(3)$

Definition 11.5 (Geodesic interpolant on $SO(3)$, restated concretely)**.** For $R_0, R_1 \in SO(3)$, $t \in [0, 1]$,

$$R_t := \exp_{R_0}\big(t \log_{R_0}(R_1)\big) = R_0 \exp\big(t \log(R_0^\top R_1)\big), \tag{11.1}$$

using the body-frame (left-trivialised) exponential and logarithm of Theorem 3.22 (equivalently, per Section 3.4's notation, this expresses the interpolant via the group-log $\log(R_0^\top R_1) \in \mathfrak{so}(3)$, a coordinate-free object not itself labelled left or right, whose trivialisation only becomes a meaningful distinction once we ask for the interpolant's *velocity*, next).

In quaternion language, (11.1) is exactly spherical linear interpolation (SLERP) between R_0 and R_1, and it is the direct $SO(3)$-analogue of the linear path Theorem 10.9, playing the role Chapter 6's IGSO(3) kernel played for diffusion — except that, unlike IGSO(3)'s infinite spectral series, (11.1) is a two-line, exactly-closed-form expression.

Proposition 11.6 (Constant body-frame (left-trivialised) velocity of the geodesic)**.** *Let $\Xi := \log(R_0^\top R_1) \in \mathfrak{so}(3)$ and $R_t = R_0 \exp(t\Xi)$ as in (11.1). Then $\dot{R}_t = R_t\,\Xi$, i.e. the curve has **constant left-trivialised (body-frame)** angular velocity $\omega^{\text{bd}}(t) \equiv \Xi^\vee$ (Theorem 3.30), for every t.*

This is exactly Theorem 3.32 from Chapter 3, restated here in the flow-matching notation; we do not reprove it (the proof — differentiate $R_0 \exp(t\Xi)$ directly, using that Ξ commutes with its own exponential — is identical).

Proposition 11.7 (The current-point form: $SO(3)$ specialisation of Theorem 11.4)**.** $\log(R_t^\top R_1) = \exp\big((1-t)\Xi\big)$, *hence*

$$\frac{\log(R_t^\top R_1)^\vee}{1-t} = \Xi^\vee, \tag{11.2}$$

i.e. the "current-point" form of the conditional vector field (Theorem 11.4's $\log_{X_t}(z)/(1-t)$*, here with* $X_t = R_t$*,* $z = R_1$*) and the "constant-in-*t*, endpoint" form (Theorem 11.6's* $\Xi^\vee$*) are, exactly, the* same *tangent vector at every* t *where both are defined — not two competing formulas, but one quantity computed two different ways.*

Proof. $R_t^\top R_1 = \exp(-t\Xi) R_0^\top R_1 = \exp(-t\Xi)\exp(\Xi) = \exp\big((1-t)\Xi\big)$, using $R_0^\top R_1 = \exp(\Xi)$ (definition of Ξ) and that $\exp(-t\Xi)$ and $\exp(\Xi)$ commute (both are power series in the same matrix Ξ, so $\exp(-t\Xi)\exp(\Xi) = \exp((1-t)\Xi)$ by the standard exponential addition law for commuting matrices, itself a direct consequence of the binomial theorem applied term-by-term to the two power series — valid here specifically because $-t\Xi$ and Ξ commute, not in general for two arbitrary elements of $\mathfrak{so}(3)$). Taking log of both sides (valid for $t \in [0,1)$, away from the $\theta = \pi$ subtlety of Section 3.3.4) gives $\log(R_t^\top R_1) = (1-t)\Xi$, and dividing by $1-t$ gives (11.2). □

This is the exact algebraic identity underlying the code's use of *two* superficially different formulas for what is provably the same mathematical object; Section 12.4 records precisely where each of the two forms of Theorem 11.7 is used computationally, and why the distinction matters numerically (one form is singular as $t \to 1$, the other is not) despite the two being exactly equal wherever both are defined.

Theorem 11.8 (Riemannian CFM loss on $SO(3)$)**.**

$$\mathcal{L}_{\mathrm{RCFM}}(\theta) = \mathbb{E}_{t\sim\mathcal{U}(0,1),\, R_0\sim p_{\mathrm{init}},\, R_1\sim p_{\mathrm{data}}}\Big[\big\|u_\theta(R_t,t) - \Xi^\vee\big\|_2^2\Big], \qquad R_t = R_0\exp(t\Xi),\ \Xi = \log(R_0^\top R_1),$$

has the same minimiser, $u_\theta(R,t) \to u_t(R)$ *(the true marginal field), as the intractable marginal loss, by Theorem 11.2 specialised to* $\mathcal{M} = SO(3)$ *(using Theorem 3.28's bi-invariant metric, under which* $\|\cdot\|_g$ *on* $T_R SO(3) \cong \mathbb{R}^3$ *is the ordinary Euclidean norm, per Theorem 3.28's proof).*

Remark 11.9 (Prior distribution). The natural (and, per Chapter 12's code correspondence, actually used) source distribution for $SO(3)$ flow matching is the exact Haar-uniform measure $\mathcal{U}(SO(3))$ of Theorem 3.29, sampled directly via Theorem 3.29's code correspondence (`sample_uniform`) — *not* the diffusion side's $t = 1$-of-a-VE-schedule approximation to it (Section 6.2.2's `sample_ref`). This is possible *because* flow matching never needs a noising process to define its prior at all: any fixed, sampleable distribution with the right invariance properties (Theorem 7.2) will do, and the exact Haar measure is simply the natural, no-approximation-needed choice, unlike diffusion's necessarily-approximate large-noise limit.

Sampling integrates

$$R_{t+\Delta t} = \exp_{R_t}\big(\Delta t\, u_\theta(R_t,t)\big) = R_t \exp\big(\Delta t\, \widehat{u_\theta(R_t,t)}\big), \tag{11.3}$$

exactly Theorem 3.33's body-frame Euler update, which — by construction, since exp always lands back on $SO(3)$ regardless of step size (Section 3.3.3) — keeps every iterate exactly on $SO(3)$, for any number of steps, not merely approximately for small steps.

Comparing Sections 6.3 and 11.3: flow matching on $SO(3)$ needs no Brownian motion, no IGSO(3) heat kernel or its truncated spectral series, no numerical tabulation/caching (Section 6.2.2), no score estimation, and no reverse-time SDE — only the closed-form geodesic interpolant (11.1), its (equally closed-form) velocity Equation (11.2), and a deterministic ODE. This elimination of an entire, nontrivial piece of numerical machinery is the first of the two arguments motivating the SigmaDock-to-SigmaFlow substitution examined in this project's broader work; the second, a reduction in the number of network evaluations needed at sampling time, is an empirical question addressed in that broader experimental work, not re-derived here.

Remark 11.10 (Numerical instability near $t = 1$). The current-point form (11.2) carries a $(1-t)^{-1}$ factor that is exactly cancelled, algebraically, by Theorem 11.7's identity — but only when evaluated *exactly*, in exact arithmetic. In floating-point arithmetic, the numerator $\log(R_t^\top R_1)$ and the denominator $1-t$ both individually approach 0 as $t \to 1$, so this specific closed form is numerically fragile in exactly this limit, even though the underlying tangent vector it computes is perfectly well-behaved (constant, in fact, by Theorem 11.6). This is a genuine, verified numerical consideration, examined empirically (via oracle-vector-field ablations that isolate this exact mechanism from network-quality effects) in this project's broader experimental investigation, referenced but not re-conducted in this theory document.

11.4 Flow matching on the pose manifold $SE(3)^N$

By Theorem 3.37's Riemannian-product structure, applied to flow matching exactly as it was applied to diffusion in Section 9.6: the translational component uses Theorem 10.9's linear path independently on each fragment's centre of mass, the rotational component uses Section 11.3's geodesic interpolant independently on each fragment's orientation, and no cross term couples them — the geometry factorises completely, with all inter-fragment and rotation–translation coupling introduced *only* by the shared neural network (Chapter 8) that predicts both components jointly from the same input graph, exactly as Section 9.1 established for SigmaDock and as Chapter 12 now confirms, component by component and directly against the code, holds equally for SigmaFlow.

Part VII

SigmaFlow

Chapter 12

SigmaFlow: Full Reconstruction

We now perform, for SigmaFlow, exactly what Chapter 9 did for SigmaDock: assemble the already-derived theory (Chapters 10 and 11) with the specific network of Chapter 8 into a complete, code-verified reconstruction. Because Chapter 9 already derived the state space (Section 9.1), the fragmentation and triangulation machinery (Sections 9.3 and 9.4), the symmetry argument (Section 9.5), the input graph and backbone (Section 9.7), and the fragment-reconstruction map φ (Section 9.7.1) — all of which Table 12.1 below confirms are unchanged — this chapter focuses entirely on what *does* change: the probability path, the training target, the loss, and the sampler.

12.1 What changes, and what does not

Table 12.1: Component-wise comparison, verified against the source in both trees. Rows above the double rule are unchanged; rows below are what this chapter derives.

Component	SigmaDock (Chapter 9)	SigmaFlow
State space $\mathcal{M}_m = SE(3)^m$, product metric	identical (Section 9.1 and theorem 3.37)	
Fragmentation (FR3D), triangulation	identical (Sections 9.3 and 9.4; code diff'd byte-identical, per this project's own auc	
Input graph, featurisation, backbone	identical (Chapter 8 and section 9.7)	
Fragment reconstruction φ	identical formula (Theorem 9.10)	
Prior p_{init}, translation	$\mathcal{N}(0, I)$ (VP schedule's $s \to 1$ limit)	$\mathcal{N}(0, I)$ (exact, by construction)
Prior p_{init}, rotation	$\approx \mathcal{U}(SO(3))$ (VE schedule's $s \to 1$ limit, Section 6.2.2)	$\mathcal{U}(SO(3))$ (exact Haar sampler, Theorem 11.9)
Probability path	VP Gaussian $\oplus$ IGSO(3) heat kernel	linear $\oplus$ geodesic (SLERP) interpolant
Regression target	conditional score $\nabla \log p_{t\|1}$	conditional velocity $u_t(\cdot \mid z)$

Component	SigmaDock	SigmaFlow
Numerical machinery	truncated IGSO(3) series, tabulation, caching (Section 6.2.2)	none — closed form throughout
Output dressing	Section 9.8.4's t-dependent scalar prefactors	**none** (Section 12.5)
Loss terms	four (`T_score,R_score,T0,R0`, Section 9.9)	**two** (`loss_trans,loss_R`, Section 12.6)
Sampler	reverse-time SDE (deterministic by default configuration, Section 9.10)	deterministic ODE (architecturally, not by configuration)
Time convention (internal)	$t = 0$ data, $t = 1$ noise	$t = 0$ noise, $t = 1$ data (**reversed**, Section 12.2)

12.2 A time-convention warning, confirmed at the source level

Theorem 0.2 fixed a global convention for this document ($t = 0$ noise, $t = 1$ data) and warned that SigmaDock's own native convention runs the opposite way internally (Theorem 9.8 in Chapter 9, and Section 9.10's observation that the diffusion sampler's own timestep array decreases from $t_{\max} = 1$ to $t_{\min}$). We record here, as a fact independently confirmed against *both* source trees during the preparation of this document (not merely inferred from exposition), that this is not a convention chosen for pedagogical convenience in this monograph but a genuine, load-bearing difference between the two codebases' own internal time variables:

- `R3Diffuser`/`SO3Diffuser` (SigmaDock): $t = 0$ is clean data ($\alpha_{t=0} = 1$, no noise); $t = 1$ is the fully-noised end (`sample_ref` literally evaluates the forward kernel at $t = 1$).
- `R3_FlowMatcher`/`SO3_FlowMatcher` (SigmaFlow): $t = 0$ is the noise/source end ($X_0 \sim p_{\text{init}}$ drawn internally by `conditional_probability_path`); $t = 1$ is the data end ($X_1 = z$, the training target).

This document's global convention (Theorem 0.2) matches SigmaFlow's own native convention exactly, and every formula attributed to SigmaDock throughout Chapter 9 was, correspondingly, already relabelled via $s = 1 - t$ before being stated (Theorem 9.8 there). Any reader cross-referencing a formula directly against SigmaDock's own source should apply this relabelling explicitly; failing to do so is, per this document's own code-research process, a genuine and easy source of sign errors, exactly as this document's introduction anticipated.

12.3 Translation flow matching, exactly as implemented

Code correspondence

Mathematical object: the source distribution, conditional path, and both forms of the conditional vector field of Theorem 10.9
Implementation: `r3_flow_matcher.py`, class `R3_FlowMatcher` (56 lines total)
Interpretation: `sample_init`(n) = `randn(n,3)` ($X_0 \sim \mathcal{N}(0, I_3)$ per fragment, no rescaling by any $\sigma_{\min}$-type parameter); `conditional_probability_path`(x_1, t) draws x_0 *internally* (unlike the diffusion side's `forward_marginal`, which takes x_0 as an argument — a structural difference reflecting that flow matching's source is drawn once per training

example and never needs to be revisited, whereas diffusion's x_0 is the fixed data point being noised), computes $x_t = (1-t)x_0 + tx_1$ (Theorem 10.9's interpolant) and returns $u_t = x_1 - x_0$, the *constant-in-t, endpoint* form — this is the quantity actually regressed against during training (Section 12.6). `calc_trans_vector_field`$(x_t, x_1, t) = (x_1 - x_t)/(1-t)$ implements the *current-point* form (10.4), singular as $t \to 1$ by construction (the docstring itself notes t must lie in $(0,1)$); this form is **not** used during training (confirmed by tracing every call site in `sigma_flow_generator.py`), only at sampling time (Section 12.8) and for the `use_true_vector_field` oracle debugging path, where x_0 is not naturally available but the current point (x_t, t) is. `euler_step`$(x_t, v_t, dt) = x_t + v_t\,dt$, the plain (non-geodesic, since $\mathbb{R}^3$ is flat) Euler step of Section 10.3, and the module's own inline comment notes this step is *exact* (zero discretisation error, for any step size) specifically because u_t is constant along the true conditional path — a special property of the linear path, not shared by the rotational component's geodesic (Section 12.4) once the network's prediction, rather than the true field, is substituted in. The constructor parameter `sigma_min` is accepted but **provably unused**: it is stored (`self.sigma_min = sigma_min`) and never read anywhere else in the file, with an explicit docstring noting it is "reserved for future noised OT path, currently unused — the linear path below is deterministic".

12.4 Rotation flow matching, exactly as implemented

Code correspondence

Mathematical object: the source distribution, conditional path, and both forms of the conditional vector field of Section 11.3
Implementation: `so3_flow_matcher.py`, class `SO3_FlowMatcher` (63 lines total)
Interpretation: `sample_init`(n) =`so3_utils.sample_uniform`(n), the exact Haar sampler of Theorem 3.29's code correspondence (Theorem 11.9's claim, confirmed directly). The class's own module-level comment states the convention adopted is "right-trivialisation". That *word* does not match this document's attribution (Theorem 3.31: what the code calls right-trivialisation is what we call left-trivialisation), but the *object* does, and the object is what matters: direct inspection of `euler_step`$(R_t, v_t, dt) = R_t \exp(v_t\,dt)$ shows it right-multiplies the exponential onto R_t, which is exactly Theorem 3.33's **body-frame** update. The velocity the network supplies is therefore ω^{bd}, however either side chooses to label it. `conditional_probability_path`(R_1, t) draws R_0 internally, computes $\Delta = R_0^\top R_1$, $\Xi = \log(\Delta) \in \mathfrak{so}(3)$ (Theorem 11.5's notation), sets $u_t = \Xi$ (a $[n,3,3]$ skew-symmetric-matrix-valued, i.e. $\mathfrak{so}(3)$-valued, not yet vee-mapped-to-$\mathbb{R}^3$ quantity — the *constant-in-t* form of Theorem 11.6, and the training target, Section 12.6), and $R_t = R_0 \exp(t\Xi)$ (Equation (11.1), confirmed exactly). `calc_rot_vector_field`$(R_t, R_1, t) = \log(R_t^\top R_1)/(1-t)$ implements the *current-point* form (11.2), algebraically equal to Ξ by Theorem 11.7 but singular in floating-point arithmetic as $t \to 1$ (Theorem 11.10) — again not used during training, only at sampling time and inside the `use_true_vector_field` oracle path. The module additionally carries an explicit inline comment recording exactly Theorem 3.27's finding in this document's own words: the original SigmaDock `Omega` clamp broke exact $\exp \circ \log = \mathrm{id}$ round trips needed by (11.1), motivating the narrower clamp confirmed in Section 3.3.4 — i.e. the bugfix's rationale is documented, first-party, directly in the file that needed it.

Remark 12.1 (The hard-coded $R_1 \equiv I$ simplification). A significant, easily-missed simplification, confirmed directly against `sigma_flow_generator.py`::`get_fragment_com_and_rot`:

the function that computes each fragment's data-time state (trans_1, R_1) from the reference conformer sets the rotational component to the identity, *always*, for every fragment, with an explicit inline comment: "Rotation matrix is identity: it is (and should be) independent in the vector field." Consequently, throughout the actual training and sampling code, $R_1 \equiv I$ is not a general fragment orientation but a fixed constant, and formulas that look like they involve a genuine R_1 — for instance the delta-rotation $\Delta R := R_t\, R_1^\top$ appearing inside Theorem 9.10's reconstruction map when specialised to this codebase — collapse to $\Delta R = R_t$ exactly. This is consistent with, and is the concrete mechanism realising, Section 9.5's gauge-invariance discussion: since the reference conformer's own orientation is arbitrary (Theorem 9.7's gauge freedom), fixing it at the identity is simply one particular, legitimate choice of gauge, not a loss of generality — but a reader auditing the code without this fact stated explicitly could easily mistake R_1 for a nontrivial, data-derived quantity, which it is not.

12.5 No output dressing: the entire diffusion-specific scalar machinery is gone

Code correspondence

Mathematical object: the network's predicted vector field, $(\text{pred\_u\_t\_trans}, \text{pred\_u\_t\_R})$

Implementation: `sigma_flow_generator.py::_compute_vector_field`, quoted here in full (it is short enough to reproduce verbatim, and its brevity is itself the point):

```
def _compute_vector_field(self, sampled, updates, t_batch):
    """
    Interprets the pooled per-fragment force/torque directly as the
    predicted conditional vector field. No time dependent scaling
    needed, unlike the diffusion score.
    """
    pred_u_t_trans = updates["total_force"]
    pred_u_t_R = updates["omega"]
    return {"pred_u_t_trans": pred_u_t_trans, "pred_u_t_R": pred_u_t_R}
```

Interpretation: `updates["total_force"]` and `updates["omega"]` are exactly the raw Newton–Maruyama outputs $dT = F_F/M_F$ and $\widehat{dW} = \widehat{I_F^{-1}\tau_F}$ of (9.3) — *unchanged* from Section 9.8's construction, since the physical Newton–Euler reduction (net force, torque, mass, inertia) is one of the components Table 12.1 lists as unmodified. What is different is that this raw mechanical output is returned *directly* as the predicted vector field, with **no** t-dependent rescaling of the kind Equations (9.4) to (9.6) apply: no `r3_scaling`, no `so3_scaling`, no $\partial_\omega \log f_\sigma/\omega$ dressing, no branch selection (`rot_score_method`/`rot_score_scaling`, whose corresponding config fields were removed entirely from `config.py` in an earlier development pass of this project, since they had become dead code once this function stopped consulting them). The comment's own justification — "no time dependent scaling needed, unlike the diffusion score" — is a genuine mathematical observation, not an oversight: Theorems 10.9 and 11.6 both establish that the flow-matching conditional targets are *constant in* t (unlike the diffusion score, whose scale, per Section 5.2's discussion, varies enormously across noise levels and specifically requires the $\lambda(s)$/scalar-prefactor machinery to keep training well-conditioned) — **but** this observation concerns the *training target's* scale, not necessarily whether the specific quantity a fixed-capacity network can most easily regress onto (raw Newton–Euler out-

put, versus some other, still t-independent, reparameterisation) is optimally chosen; this project's broader experimental investigation (the "time-weighting" variant referenced in this project's development history) tested, and found no benefit from, reintroducing a t-dependent multiplicative reweighting on top of this already-unweighted target, discussed further in Section 12.10.

12.6 The loss function

Code correspondence

Mathematical object: $\mathcal{L}_{\text{trans}}$, $\mathcal{L}_R$ of Theorems 10.8 and 11.8, evaluated at their sampled-path targets
Implementation: `sigma_flow_generator.py`::`compute_losses`
Interpretation:

$$\texttt{loss\_trans} = \|\texttt{pred\_u\_t\_trans} - u_t^{\text{trans}}\|_2^2 \quad \text{(sum over the 3 coordinates, per fragment; } u_t^{\text{trans}} = x_1 - x_0 \text{ o}$$
$$\texttt{loss\_R} = \|\texttt{pred\_u\_t\_R} - u_t^{R}\|_{\text{Fro}}^2 \quad \text{(Frobenius norm}^2 \text{ of the } 3 \times 3 \text{ difference; } u_t^R = \Xi \text{ of Theorem}$$

Both are raw per-fragment sums with **no weighting and no clamping** inside `compute_losses` itself — confirming, precisely, Table 12.1's claim that SigmaFlow's loss has *two* terms where SigmaDock's `compute_losses` (Section 9.9) has *four*: the two "data-space" ($\hat{x}_0$-reconstruction) terms `T0`/`R0` that SigmaDock's diffusion parameterisation computes alongside its score-matching terms have **no analogue at all** in SigmaFlow's loss, confirmed by direct tracing of `trainer.py`::`SigmaLightningModule._shared_step`, whose total-loss combination `total_loss` = `trans_score_weight` · `loss_trans` + `rot_score_weight` · `loss_R` consumes exactly these two keys and no others (the CLI flag names `trans_score_weight`/`rot_score_weight` are themselves a naming holdover from the diffusion codebase this file is shared with; functionally, for the flow-matching path, they are ordinary fixed loss weights on (10.4)/(11.2)'s regression losses, not score-matching weights in Section 5.2's $\lambda(s)$ sense). Per-molecule reduction uses `trainer.py`::`scaled_fragmented_loss`, structurally identical to Section 9.9's code correspondence (shared infrastructure, per Table 12.1).

12.7 The full training step

We now trace `SigmaFlowGenerator.forward` end to end, with every intermediate tensor's shape as commented in the source, assembling every piece derived above into the single computational pipeline a training batch actually executes.

1. **Prepare batch, sample time.** t = `_sample_time`(batch) $\in [\varepsilon_t, 1]^B$, one scalar *per molecule* in the batch (*not* per fragment), via $t = \varepsilon_t + (1 - \varepsilon_t) \cdot \text{Unif}(0, 1)$, ε_t = `HPARAMS.general.epsilon_t` $= 0.01$ — matching Section 5.2's $t \sim \mathcal{U}(0, 1)$ up to the training-time floor ε_t, whose purpose (per an explicit code comment quoted verbatim, judged authoritative and reproduced here rather than re-derived) is distributional consistency between training and sampling, not any numerical-singularity avoidance: "training's `sample_time` never samples t below this; sampling's `t_min` should match it ... not related to the separate $1/(1-t)$ singularity near $t = 1$" — i.e. the network

should never be *queried* at a t value it was never trained on, which is a distributional/generalisation concern entirely distinct from Theorem 11.10's numerical concern about the current-point vector-field formula near $t = 1$.

2. **Get initial (data-time) states.** pos_0, trans_1, R_1, num_fragments = `_get_initial_states`(batch): despite the subscript "0", pos_0 is the (pocket-centred, `dimensional_scale`-rescaled, Theorem 9.9) ground truth/reference atom geometry — the fixed per-fragment *template* x^{ref} of Theorem 9.10, reused at every t, not a draw from the source distribution — while trans_1, R_1 are the per-fragment data-time (centre-of-mass, and, per Theorem 12.1, identically-I) rigid-motion state. t is broadcast from one value per molecule to one value per fragment, $t_{\text{batch}} \in [\varepsilon_t, 1]^{\Sigma F}$ (*all fragments of a given molecule share the same sampled t*, a design choice with no counterpart requirement in the theory of Chapters 10 and 11, which permit — but do not require — an independent t per component; the code's own inline comment flags this exact choice as a documented, unresolved limitation: "better sampling. Uniform makes noises (sigmas) that are too redundant").

3. **Sample the conditional path.** $(\text{trans}_t, R_t, u_t^{\text{trans}}, u_t^R)$ = `_sample_flow`$(\text{trans}_1, R_1, t_{\text{batch}})$, i.e. Sections 12.3 and 12.4's `conditional_probability_path`, dispatched componentwise per Theorem 3.37.

4. **Reconstruct atom positions at time t.** $\text{pos}_t = \varphi(\text{trans}_t, R_t)$ via Theorem 9.10's formula (`_apply_transformations`), and the graph's transient (interaction) edges are recomputed at these new positions (`_update_batch`, Section 9.7's discussion of the global/transient topology split).

5. **Network forward pass.** lig_pseudoforces, forces_idxs = `_compute_forces`(batch, t) — exactly one EquiformerV2 evaluation (Chapter 8 and section 9.8's code correspondence), identical in cost and mechanism to SigmaDock's own single-network-call Newton–Euler pipeline.

6. **Newton–Euler reduction and Newton–Maruyama update.** (F_F, τ_F, M_F, I_F) = `_compute_fragment_dynamics`$(\dots)$ ((9.2)); $(dT, \widehat{dW})$ = `_predict_fragment_updates`$(\dots)$ ((9.3)) — structurally byte-for-byte the same construction as Section 9.8, confirming Table 12.1's claim that this component is unchanged.

7. **Predicted vector field.** (pred_u_t_trans, pred_u_t_R) = `_compute_vector_field`$(\dots)$, Section 12.5's undressed pass-through.

8. **Loss.** Section 12.6's two-term `compute_losses`, weighted and summed by `trainer.py`::SigmaLightningMo
wrapped in the try/except-`AssertionError` and finite-value guards this project's own development history documents in detail (catching Theorem 9.11's "fragment mass < 2" assertion specifically, among other numerical failure modes).

Every step above, except step 8's loss formula and steps 2–3's use of the flow-matching (rather than diffusion) conditional path, is verified, in Table 12.1, to be identical to SigmaDock's corresponding pipeline stage — the precise, exhaustive sense in which this project's conversion of SigmaDock into SigmaFlow is, as claimed throughout this document's audited source material, a *surgical*, minimally-invasive substitution of the probability path and training target alone.

12.8 Sampling

Code correspondence

Mathematical object: the forward ODE integration loop of Sections 10.3 and 11.3
Implementation: `sampling.py`::`sampler` (in `SigmaFlow_Development/src/sigmadock/diff/`)
Interpretation: default `num_steps`= 25, `t_min`= $\varepsilon_t = 0.01$ (matching training's floor, per this chapter's Step 1 discussion above), `rho`= 3.0, default `discretization="power"` (*not* SigmaDock's default `"edm"` — a genuine, code-confirmed default-configuration asymmetry between the two samplers, flagged here rather than assumed identical: any timing/quality comparison between the two must either override one to match the other or explicitly account for this difference), `solver="euler"` (Heun again unimplemented, per Section 9.10's finding, verbatim-identical dead `# TODO` comment block in this file too). The timestep array walks *upward*, $t_{\min} \to t_{\max} = 1$, matching this document's global convention directly (no relabelling needed, unlike SigmaDock's sampler). Each loop iteration: (1) evaluate the closed-form current-point field $u_t(\cdot \mid z)$ (`_compute_true_vector_field`, i.e. Equations (10.4) and (11.2)'s singular-but-exact forms, using the *known* trans_1, R_1 — available here because sampling is being run, in this project's own evaluation methodology, on complexes whose true pose is known, purely for diagnostic loss logging at each step, not because a real blind-docking deployment would have this available); (2) if `use_true_vector_field=False` (the normal, non-diagnostic case), run the network (Steps 5–7 of Section 12.7's list) to get the predicted field instead; (3) Euler step (`SE3_FlowMatcher.euler_step`, dispatching to Sections 12.3 and 12.4's componentwise `euler_step` implementations); (4) reconstruct positions via φ (Theorem 9.10) at the new $(\text{trans}_{t+dt}, R_{t+dt})$. Exactly one network evaluation per step, matching SigmaDock's sampler exactly in this specific respect (Section 9.10).

Remark 12.2 (The `use_true_vector_field` oracle: what Test 1 established). Substituting the closed-form field for the network's prediction at *every* step (`use_true_vector_field=True`) isolates a question this chapter's derivations make precise: does the ODE integration/reconstruction machinery itself (Euler stepping, the φ reconstruction, batching and edge recomputation) introduce error, independent of network quality? By Theorem 11.7's exact algebraic identity and Theorem 10.9's exact linearity (both established rigorously in this chapter, not merely observed empirically), the answer, *in exact arithmetic*, must be no: the oracle field reconstructs the true trajectory exactly, for any number of steps, since both conditional vector fields are (by Section 10.3's remark on the linear path, and Theorem 11.7 for the geodesic path) either exactly integrable by a single Euler step (translation) or exactly consistent with the constant-velocity geodesic the Euler step is discretising (rotation, up to floating-point round-off). This is the theoretical content underlying this project's empirical "Test 1" finding (oracle-vector-field sampling reconstructs the true structure to within floating-point precision, ≤ 0.0002 Å deviation, across a sample of real generated structures) — the theorem, derived here, is what the experiment confirmed, not a separate, independent fact.

12.9 Likelihood and confidence: what is implemented, and what is not

Section 10.1.2 derived, from first principles, that a CNF's log-density along any trajectory evolves by $\frac{d}{dt} \log p_t(X_t) = -(\nabla \cdot u_t)(X_t)$ (Theorem 10.4), estimable via the Hutchinson trace estimator without a separate density network. This is a genuine, well-established capability of the flow-matching framework in general (and one diffusion models do not share as directly,

since the diffusion side's analogous likelihood requires the full probability-flow-ODE construction of Theorem 5.7, an extra derived object rather than something immediately available from the sampler already being run).

Caution 12.3 (Not implemented in this codebase). Per the code-research pass underlying this document, **no code path in either `sampling.py` or `sigma_flow_generator.py` computes $\nabla \cdot u_\theta$, a Hutchinson trace estimate, or any accumulated log-density along a sampled trajectory**; no likelihood or confidence score derived from Theorem 10.4 is produced anywhere in the sampling pipeline reconstructed in Section 12.8. SigmaFlow's actual ranking mechanism, where used, is inherited unchanged from SigmaDock's non-learned heuristic (Section 9.11), per Table 12.1. Theorem 10.4's construction is recorded in this document because it is a genuine, theoretically well-founded *capability* that the flow-matching formulation — unlike the diffusion formulation — would make relatively straightforward to add (a scalar ODE run alongside the existing vector field, using an already-trained u_θ with no retraining required), and because Chapter 1's discussion of confidence estimation as a distinct task from pose generation (Section 1.3) specifically named this as an open, unimplemented direction; it should not be read as a description of current SigmaFlow behaviour.

12.10 A tested, and rejected, loss variant

We record, briefly and for completeness given Section 12.5's discussion of the "no time-dependent scaling needed" design choice, one concrete alternative this project's broader experimental work tested empirically, since a reader who has just followed Section 5.2's discussion of diffusion's $\lambda(s)$ weighting might reasonably ask whether an analogous reweighting should be added to SigmaFlow's (currently unweighted, per Section 12.6) flow-matching loss.

Caution 12.4 (Empirically tested, no benefit found; not part of this document's core theory). A variant multiplying Section 12.6's per-fragment squared errors by a factor $1/(1-t)^2$ (capped near $t = 1$ for numerical stability) was implemented, trained, and evaluated as part of this project's broader experimental investigation. Two findings from that investigation are relevant here, stated precisely rather than left vague: **(i)** this reweighting has *no* grounding in the flow-matching theory developed in Chapters 10 and 11 of this document — unlike diffusion's $\lambda(s)$, which Section 5.2 derives as compensating for the conditional score's genuinely t-varying scale, Section 12.5's constant-in-t conditional targets have no analogous scale variation to compensate for, so this variant is, by this document's own theory, an unmotivated modification rather than a theoretically indicated correction; **(ii)** consistent with point (i), it was found, empirically, to produce no measurable improvement (and if anything a marginal degradation) in the fragment-boundary bond-length metric this project's broader work uses to quantify exactly the failure mode Section 9.4's discussion anticipates. We record this only to close the loop on a natural question the theory of this chapter raises, not as part of SigmaFlow's actual, shipped training recipe.

Part VIII

Synthesis

Chapter 13

Implementation Details, Numerical Stability, and Consistency Checks

This chapter consolidates, in one place, the numerical-implementation details that Chapters 3, 5, 6, 8, 9 and 12 introduced individually, at the point each became relevant, and adds the handful that are more naturally stated only once every other piece of machinery is available. None of the content here is new mathematics; it is an index and a systematic check, in the spirit of Section 13.4's closing consistency pass.

13.1 Every clamp, epsilon, and numerical safeguard in one table

Location	Guard	Purpose (cross-reference)
`Omega` (angle from trace)	clamp$(\cdot, -1+10^{-7}, 1-10^{-7})$ before $\arccos$ (SigmaFlow); $(-0.99, 0.99)$ (original SigmaDock, now recognised as a bug for the flow-matching use case)	domain safety; the wider original clamp additionally, and incorrectly, truncated true angles $> 172°$ (Theorem 3.27)
`rotation_vector_from_matrix`	small-θ blend (`isclose`, atol=10^{-8}) near θ=0; hard/loose blend (atol=10^{-2}) near θ=π	avoids $\sin\theta$=0 division in $\log(R) = \frac{\theta}{2\sin\theta}(R - R^\top)$ (Section 3.3.4)

Location	Guard	Purpose
`d_log_f_d_omega` (IGSO(3) score derivative)	hard branch at $\sigma^2 < 0.3$: literal ratio vs. approximation $-\omega/\sigma^2$	literal $\partial_\omega f/f$ numerically ill-conditioned at small noise (Section 6.2.2); small-noise limit asserted by the code, not independently re-derived here
`get_fragment_mass_inertia`	eigenvalue floor $\max(10^{-8}\,\mathrm{tr}(I)/3, 10^{-12})$, plus $+10^{-8}I$ Tikhonov term before the linear solve	protects `torch.linalg.solve` against near-degenerate (near-linear, 2–3-atom) fragment inertia tensors (Section 9.8)
`newton_maruyama`	`omega`= hat(dW) clamped entrywise to $[-10^3, 10^3]$	loose safety net against a diverging angular-velocity solve, not a tight physical bound
`d_f_igso3_d_omega`, series truncation	fixed truncation at L=1000 terms; angle grid excludes ω=0 exactly	(6.1) is an infinite series (Theorem 6.5); $\sin(\omega/2)$=0 singularity at ω=0 avoided by grid construction, not a runtime guard
`BesselBasis` (radial embedding)	$+\varepsilon$ in denominator $d_{\text{scaled}} + \varepsilon$	avoids 0/0 as an edge length $\to$ 0 (Section 7.4)
`PolynomialCutoff`	indicator $\mathbf{1}[d < r_{\max}]$, degree-8 envelope vanishing (with 2 derivatives) at $r_{\max}$	continuity of messages as an edge crosses the cutoff radius (Section 7.4)
Assorted `vee`, `expmap`	*print*-only warnings (not raised exceptions) on a non-skew-symmetric input	a soft, non-blocking sanity check — callers can silently pass a malformed matrix without the check stopping execution (Theorem 3.19's code correspondence)
`assert M.min()` ≥ 2	hard assertion, caught upstream	fragment mass (atom count, Theorem 9.11) degeneracy, not a chemical-mass concern

13.2 Masking, batching, and scatter conventions

Every per-fragment or per-molecule reduction encountered in Chapters 9 and 12 (net force/torque, mass/inertia, per-molecule loss aggregation) uses the same underlying mechanism, worth stating once explicitly since it is assumed silently throughout: a flat tensor over *all* atoms (or frag-

ments) in a batch, together with an integer index tensor recording which molecule/fragment each row belongs to, reduced via a `scatter`/`index_add` operation keyed on that index. This is the standard `torch_geometric` convention for representing a batch of variable-sized graphs as one large disjoint-union graph rather than as a padded, fixed-size tensor, and it is why, throughout Chapters 9 and 12, a formula written "per fragment" (e.g. (9.2)) is, in the actual tensor implementation, a single vectorised scatter-sum over the entire batch's flattened atom list, not a Python-level loop over fragments. Masking (e.g. excluding virtual/dummy atoms from the physical mass/inertia computation, Section 9.8) is implemented via boolean indexing keyed on the categorical node-entity type (`HPARAMS.get_node_idx`), not via a separate, redundant tensor.

13.3 Floating-point precision

Several routines identified in earlier chapters explicitly upcast to `float64` (or, on Apple-Silicon/MPS backends where `float64` support is limited, round-trip through the CPU) before a numerically sensitive computation and cast back afterward: `Log`, `Omega` (Section 3.3's code correspondences). This reflects that the angle/logarithm computation, involving `arccos` near its domain boundary and ratios like $\theta/\sin\theta$ near $\theta = 0$, is more sensitive to rounding error than the surrounding `float32`-precision network computation that dominates the rest of the pipeline; training and sampling otherwise run at `float32` (`conf/sampling/base.yaml`'s `hardware.precision:32` default).

13.4 Dimensional and consistency checks

As a final, explicit sanity pass — in the spirit this document has maintained throughout, of tying every formula to a concrete tensor shape — we record the shape/transformation-behaviour consistency of the central quantities of Chapter 12, gathered in one place.

Quantity	Shape	Transformation under globa
$\mathrm{pos}_0, \mathrm{pos}_t$ (atom coords)	`[N,3]`	$x \mapsto Qx$ (world-frame, polar
$\mathrm{trans}_1, \mathrm{trans}_t$ (fragment COM)	`[ΣF,3]`	$x \mapsto Qx$ (world-frame, polar
R_1, R_t (fragment orientation)	`[ΣF,3,3]`	$R \mapsto QR$ (world-frame)
u_t^{trans}, pred_u_t_trans	`[ΣF,3]`	$v \mapsto Qv$ (polar, Equation (7
u_t^R, pred_u_t_R	`[ΣF,3,3]`	world-frame $\mathfrak{so}(3)$-valued (p
h_i (steerable node feature)	`[N, 16, 256]` (L=3, `sphere_channels`= 256)	$h^{\ell,c} \mapsto D^\ell(Q)h^{\ell,c}$, degreewis
Network output (pre-slice)	`[N_atoms, 16]`	type-1 slice, indices 1–3, is t
Loss `loss_trans`, `loss_R`	`[ΣF]`	scalar (invariant), per fragm

Every row is either a direct restatement of a code correspondence already given in full in an earlier chapter (cross-referenced there) or an immediate consequence of Theorem 4.25(iv)'s identification $D^1(Q) = Q$; we do not introduce any new claim in this table, only collect what has already been derived, as a single point of reference for verifying a specific tensor's shape or transformation law without re-reading the chapter it first appeared in.

13.5 Summary of open items

In keeping with this document's stated policy of flagging rather than guessing, we collect, in one place, every point flagged elsewhere in this document as not fully, independently verified from the code (each is discussed, with full context, at the cross-referenced location):

- The exact FR3D stochastic merge algorithm and stopping criterion (Section 9.3's caution).
- The precise semantic identity of each of the ten categorical atom feature slots (Section 8.8's code correspondence).
- The exact formula of the `rms_norm_sh` normalisation layer and the sinusoidal time-embedding (Section 8.6, Section 8.8).
- Whether the small-noise limit $-\omega/\sigma^2$ used by `d_log_f_d_omega` has been independently re-derived from (6.1) (it has not, in this document; it is reported as the code's own documented claim, Section 6.2.2).
- Which specific value of `rot_score_method`/`rot_score_scaling` (Section 9.8.4) was used to train the specific checkpoints referenced in this project's broader experimental work, as opposed to the constructor defaults recorded here.
- The `fragment_scaling` default-value discrepancy between two source locations (Section 9.9).

None of these open items affects the mathematical derivations of Chapters 3 to 6, 10 and 11, which are self-contained proofs independent of any specific code detail; they affect only the precision with which a small number of secondary implementation facts are pinned down.

Closing Remarks

This document set out to make one substitution precise: Table 12.1's claim that converting SigmaDock into SigmaFlow replaces a probability path and a regression target while leaving the state space, the fragmentation and triangulation geometry, the input graph, and the entire EquiformerV2 backbone untouched. Making that claim *precise* required building, from elementary analysis and linear algebra upward, the full apparatus of Riemannian manifolds and Lie groups (Chapter 3), the representation theory of $SO(3)$ (Chapter 4), score-based diffusion in Euclidean space and on $SO(3)$ (Chapters 5 and 6), the equivariant message-passing architecture that both methods share (Chapters 7 and 8), and continuous-normalizing-flow theory in its Euclidean and Riemannian forms (Chapters 10 and 11) — at which point the substitution itself (Chapter 12) could be stated as a small, precisely delimited set of changes against the common baseline (Chapter 9) that the preceding chapters established.

A reader who has worked through this document in order should now be able to: state and prove why a fragment-based parameterisation, rather than a torsional one, gives a decoupled noising/flow measure (Theorem 9.2); derive the IGSO(3) heat kernel and its score from the Peter–Weyl decomposition of the Laplace–Beltrami operator on $SO(3)$ (Section 6.2); derive the geodesic flow-matching interpolant on $SO(3)$ and prove the exact algebraic identity between its two computational forms (Theorem 11.7); explain, from Schur's lemma, exactly which operations an equivariant neural network layer is and is not permitted to perform (Theorems 4.46 and 4.48); and, for every one of these constructions, point to the specific file, function, tensor shape, and line of reasoning connecting the mathematics to the program that actually runs on real protein–ligand complexes.

Where this document's own derivations and the codebase's own comments disagreed, or where a detail could not be pinned down from a static reading of the source, this document has said so explicitly rather than resolving the disagreement by assumption — most notably the corrected $SO(3)$-angle clamp (Theorem 3.27), the reversed internal time conventions between the two methods (Section 12.2), the hybrid V1/V2 attention mechanism actually shipped (Theorem 8.9), and the always-identity fragment reference rotation (Theorem 12.1). These are exactly the kind of facts a purely paper-level account of either method would not surface, and locating them is, alongside the from-scratch mathematical development itself, the primary contribution of treating this material as a full monograph rather than a summary.